SOLUTIONS MANUAL

PART 1: CHAPTERS 1-5

Rev 06/05/2017
Rev 20180306

DIGITAL DESIGN

WITH AN INTRODUCTION to the VERILOG HDL, VHDL, and SystemVerilog
Sixth Edition

M. MORRIS MANO
Professor Emeritus
California State University, Los Angeles

MICHAEL D. CILETTI
Professor Emeritus

University of Colorado, Colorado Springs

Note: Solutions to problems requiring HDL code are presented in Verilog and VHDL

CHAPTER 1

1.1 Base-10: 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32
Octal: 16 17 20 21 22 23 24 25 26 27 30 31 32 33 34 35 36 37 40
Hex: 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F 20

Base-10: 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
Base-12 8 9 0A 0B 10 11 12 13 14 15 16 17 18 19 1A 1B 20 21 22 23

24

1.2 **(a)** 32,768 **(b)** 16,777,216 **(c)** 4,294,967,296

1.3 **(a)** 82

(b) 1138

(c) 26154

(d) 168

1.4 16-bit binary: 1111_1111_1111_1111
Decimal equivalent: $2^{16} - 1 = 65{,}535_{10}$
Hexadecimal equivalent: $FFFF_{16}$

1.5 **(a)** 8

(b) 4

(c) 7

1.6 $(x - 3)(x - 6) = x^2 - (6 + 3)x + 6*3 = x^2 - 11x + 22$

Therefore: 6 + 3 = b + 1, so b = 8
Also, $6*3 = (18)_{10} = (22)_8$

1.7 $(155517)_8$

1.8 **(a)** Results of repeated division by 2 (quotients are followed by remainders):

431_{10} = 215(1); 107(1); 53(1); 26(1); 13(0); 6(1) 3(0) 1(1)
Answer: $1111\_1010_2 = FA_{16}$

(b) Results of repeated division by 16:

431_{10} = 26(15); 1(10) (Faster)
Answer: FA = 1111_1010

1.9 **(a)** 17.625

(b) 82.3125

(c) 226.875

(d) 26540.6875

(e) 42.625

1.10 **(a)** $1.10010_2 = 0001.1001_2 = 1.9_{16} = 1 + 9/16 = 1.563_{10}$

(b) $110.010_2 = 0110.0100_2 = 6.4_{16} = 6 + 4/16 = 6.25_{10}$

Reason: 110.010_2 is the same as 1.10010_2 shifted to the left by two places.

1.11 Quotient = 10001

1.12 **(a)** 10000 and 110111

1011 1011

$$\begin{array}{r} +101 \\ \hline 10000 = 16_{10} \end{array} \qquad \begin{array}{r} \times 101 \\ \hline 1011 \\ 1011 \\ \hline 110111 = 55_{10} \end{array}$$

(b) 62_h and 958_h

$$\begin{array}{ll} 2E_h & 0010\_1110 \\ +34_h & 0011\_0100 \\ \hline 62_h & 0110\_0010 = 98_{10} \end{array} \qquad \begin{array}{r} 2E_h \\ \times 34_h \\ \hline B^3 8 \\ 8^2 A \\ \hline 9\ 5\ 8_h = 2392_{10} \end{array}$$

1.13 **(a)** Convert 27.315 to binary:

	Integer Quotient		Remainder	Coefficient
27/2 =	13	+	½	$a_0 = 1$
13/2	6	+	½	$a_1 = 1$
6/2	3	+	0	$a_2 = 0$
3/2	1	+	½	$a_3 = 1$
½	0	+	½	$a_4 = 1$

$27_{10} = 11011_2$

	Integer		Fraction	Coefficient
.315 x 2 =	0	+	.630	$a_{-1} = 0$
.630 x 2 =	1	+	.26	$a_{-2} = 1$
.26 x 2 =	0	+	.52	$a_{-3} = 0$
.52 x 2 =	1	+	.04	$a_{-4} = 1$

$.315_{10} \cong .0101_2 = .25 + .0625 = .3125$

$27.315 \cong 11011.0101_2$

(b) 2/3 ≅ .6666666667

	Integer		Fraction	Coefficient
.6666_6666_67 x 2 =	1	+	.3333_3333_34	$a_{-1} = 1$
.3333333334 x 2 =	0	+	.6666666668	$a_{-2} = 0$
.6666666668 x 2 =	1	+	.3333333336	$a_{-3} = 1$
.3333333336 x 2 =	0	+	.6666666672	$a_{-4} = 0$
.6666666672 x 2 =	1	+	.3333333344	$a_{-5} = 1$
.3333333344 x 2 =	0	+	.6666666688	$a_{-6} = 0$
.6666666688 x 2 =	1	+	.3333333376	$a_{-7} = 1$
.3333333376 x 2 =	0	+	.6666666752	$a_{-8} = 0$

$.6666666667_{10} \cong .10101010_2 = .5 + .125 + .0313 + ..0078 = .6641_{10}$

$.101010102 = .1010\_1010_2 = .AA_{16} = 10/16 + 10/256 = .6641_{10}$ (Same as (b)).

1.14

Number	1's complement	2's complement
11001101	00110010	00110010
01101000	10010111	10011000
10011100	01100011	01100100
10100101	01001100	01001101
10110100	01001011	01001100
01001001	10110110	10110111

1.15

	(a)		(b)
	25,875,036		76,325,800
9s comp:	74,124,963	9s comp:	26,674,199
10s comp:	74,124,964	10s comp:	26,674,200

	(c)		(d)
	25,101,236		00000000
9s comp:	74,898,763	9s comp:	99999999
10s comp:	74,898,764	10s comp:	100000000

1.16

(a) 8's complement of $(1740)_8$ = 6040

(b) $(6040)_8 = (1111100000)_2$

(c) 2's complement of $(1111100000)_2$ = 1111100001

(d) $(1111100001)_2 = (1741)_8$; the resulting octal number is one more than the original octal number.

1.17

(a) 6,473 – 5297 = 1176
5297 → 05297 →94702 (9s comp) → 94703 (10s comp)
6473 – 5297 = 6473 + 94703 = 101,176 (positive)
Magnitude: 1,176
Result: 6,473 – 5297 = 1176

(b) 1,076 – 3,217 = -2,141
3,217 → 96,782 (9s comp) → 96,783 (10s comp)
1,076 – 3,217 = 1,076 + 96,783 = 97,858 (negative)
Magnitude: 2,141
Result: 1,076 – 3,217 = -2,141

(c) 4,361 → 04361 → 95638 (9s comp) → 95639 (10s comp)
2043 – 4361 = 02043 + 95639 = 97682 (Negative)
Magnitude: 2318
Result: 2043 – 6152 = -2318

(d) 745 → 00745 → 99254 (9s comp) → 99255 (10s comp)
1631 -745 = 01631 + 99255 = 0886 (Positive)
Result: 1631 – 745 = 886

1.18 **(a)** – 00111

(**b**) – 011111

(c) 00010

(**d**) 10001111

1.19 +9286 → 009286; +801 → 000801; -9286 → 990714; -801 → 999199

(a) (+9286) + (_801) = 009286 + 000801 = 010087

(b) (+9286) + (-801) = 009286 + 999199 = 008485

(c) (-9286) + (+801) = 990714 + 000801 = 991515

(d) (-9286) + (-801) = 990714 + 999199 = 989913

1.20 +49 → 0_110001 (Needs leading zero extension to indicate + value);
+29 → 0_011101 (Leading 0 indicates + value)
-49 → 1_001110 + 0_000001→ 1_001111
-29 → 1_100011 (sign extension indicates negative value)

(a)(+29) + (-49) = 0_011101 + 1_001111 = 1_101100 (1 indicates negative value.)
Magnitude = 0_010011 + 0_000001 = 0_010100 = 20; Result (+29) + (-49) = -20

(b) (-29) + (+49) = 1_100011 + 0_110001 = 0_010100 (0 indicates positive value)
(-29) + (+49) = +20

(c) Must increase word size by 1 (sign extension) to accomodate overflow of values:
(-29) + (-49) = 11_100011 + 11_001111 = 10_110010 (1 indicates negative result)
Magnitude: 01_001110 = 78_{10}
Result: (-29) + (-49) = -78_{10}

1.21 +9742 → 009742 → 990257 (9's comp) → 990258 (10s) comp
+641 → 000641 → 999358 (9's comp) → 999359 (10s) comp

(a) (+9742) + (+641) → 010383

(b)(+9742) + (-641) →009742 + 999359 = 009101
Result: (+9742) + (-641) = 9101

(c) -9742) + (+641) = 990258 + 000641 = 990899 (negative)
Magnitude: 009101
Result: (-9742) + (641) = -9101

(d) (-9742) + (-641) = 990258 + 999359 = 989617 (Negative)
Magnitude: 10383
Result: (-9742) + (-641) = -10383

1.22 6,514

BCD**:** 0110_0101_0001_0100
ASCII: 0_011_0110__0_011_0101__1_011_0001__1_011_0100
ASCII: 0011_0110__0011_0101__1011_0001__1011_0100

3,274
BCD: 0011_0010_0111_0100
ASCII: 0011_0011_1011_0010_1011_0111_1011_0100

1.23 In BCD, 8124 is represented as: 1000 0001 0010 0100
8127 is represented as: 1000 0001 0010 0111
The BCD addition is:

		1	1	1
	1000	0001	0010	0100
+	1000	0001	0010	0111
1	0110	0110	0100	1011
				+0110
0001	0110	0010	0101	0001
↓	↓	↓	↓	↓
1	6	2	5	6

1.24 **(a)**

Decimal digit	**6 3 1 1 code**
0	0000
1	0001
2	0010
3	0100
4	0110 (or 0101)
5	0111
6	1000
7	1010 (or 1001)
8	1011
9	1100

(b)

Decimal digit	**6 4 2 1 code**
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	1000 (or 0110)
7	1001
8	1010
9	1011

1.25 **(a)** BCD: 0010 0010 0110 0001

(b) Excess-3: 0101 0101 0110 0100

(c) 2421: 1000 1000 0011 0001

(d) 6311: 0010 0010 0100 0001

1.26 9's complement: 7768
2421 code: 0111 0111 1100 1110
1's complement: 1000 1000 0011 0001 same as 1.25 (c).

1.27 For a deck with 52 cards, we need 6 bits ($2^5 = 32 < 52 < 64 = 2^6$). Let the msb's select the suit (e.g., diamonds, hearts, clubs, spades are encoded respectively as 00, 01, 10, and 11. The remaining four bits select the "number" of the card. Example: 0001 (ace) through 1011 (9), plus 101 through 1100 (jack, queen, king). This a jack of spades might be coded as 11_1010. (Note: only 52 out of 64 patterns are used.)

1.28 G (dot) (space) B o o l e

11000111_11101111_01101000_01101110_00100000_11000100_11101111_11100101

1.29 Microsoft

1.30 73 F4 E5 76 E5 4A EF 62 73

73:	0_111_0011	s
F4:	1_111_0100	t
E5:	1_110_0101	e
76:	0_111_0110	v
E5:	1_110_0101	e
4A:	0_100_1010	j
EF:	1_110_1111	o
62:	0_110_0010	b
73:	0_111_0011	s

Even parity

1.31 62 + 32 = 94 printing characters; 34 special characters

1.32 Complement bit 6 (from the right)

1.33 **(a)** 5344 **(b)** 2011 **(c)** 6477 **(d)** VD

1.34 ASCII for decimal digits with even parity:

(0):	00110000	(1):	10110001	(2):	10110010	(3):	00110011
(4):	10110100	(5):	00110101	(6):	00110110	(7):	10110111
(8):	10111000	(9):	00111001				

CHAPTER 2

2.1 **(a)**

x y z	$x + y + z$	$(x + y + z)'$	x'	y'	z'	$x'y'z'$
0 0 0	0	1	1	1	1	1
0 0 1	1	0	1	1	0	0
0 1 0	1	0	1	0	1	0
0 1 1	1	0	1	0	0	0
1 0 0	1	0	0	1	1	0
1 0 1	1	0	0	1	0	0
1 1 0	1	0	0	0	1	0
1 1 1	1	0	0	0	0	0

x y z	(xyz)	$(xyz)'$	x'	y'	z'	$x' + y' + z'$
0 0 0	0	1	1	1	1	1
0 0 1	0	1	1	1	0	1
0 1 0	0	1	1	0	1	1
0 1 1	0	1	1	0	0	1
1 0 0	0	1	0	1	1	1
1 0 1	0	1	0	1	0	1
1 1 0	0	1	0	0	1	1
1 1 1	1	0	0	0	0	0

(b)

x y z	$x + yz$	$(x + y)$	$(x + z)$	$(x + y)(x + z)$
0 0 0	0	0	0	0
0 0 1	0	0	1	0
0 1 0	0	1	0	0
0 1 1	1	1	1	1
1 0 0	1	1	1	1
1 0 1	1	1	1	1
1 1 0	1	1	1	1
1 1 1	1	1	1	1

(c)

x y z	$x(y + z)$	xy	xz	$xy + xz$
0 0 0	0	0	0	0
0 0 1	0	0	0	0
0 1 0	0	0	0	0
0 1 1	0	0	0	0
1 0 0	0	0	0	0
1 0 1	1	0	1	1
1 1 0	1	1	0	1
1 1 1	1	1	1	1

(c)

x y z	x	$y + z$	$x + (y + z)$	$(x + y)$	$(x + y) + z$
0 0 0	0	0	0	0	0
0 0 1	0	1	1	0	1
0 1 0	0	1	1	1	1
0 1 1	0	1	1	1	1
1 0 0	1	0	1	1	1
1 0 1	1	1	1	1	1
1 1 0	1	1	1	1	1
1 1 1	1	1	1	1	1

(d)

x y z	yz	$x(yz)$	xy	$(xy)z$
0 0 0	0	0	0	0
0 0 1	0	0	0	0
0 1 0	0	0	0	0
0 1 1	1	0	0	0
1 0 0	0	0	0	0
1 0 1	0	0	0	0
1 1 0	0	0	1	0
1 1 1	1	1	1	1

2.2 **(a)** $x + y$

(b) y

(c) xy

2.3 **(a)** $xyz + x'y + xyz' = xy + x'y = y$

(b) $x'yz + xz = (x'y + x)z = z(x + x')(x + y) = z(x + y)$

(c) $(x + y)'(x' + y') = x'y'(x' + y') = x'y'$

(d) $xy + x(wz + wz') = x(y + wz + wz') = x(w + y)$

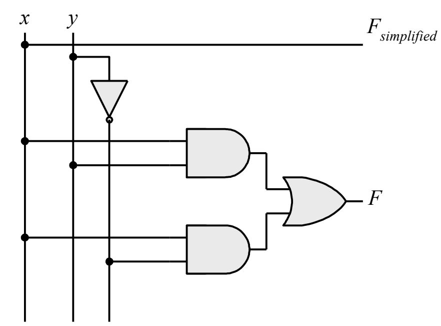
(e) $(yz' + x'w)(xy' + zw') = yz'xy' + yz'zw' + x'wxy' + x'wzw' = 0$

(f) $(x' + z')(x + y' + z') = x'x + x'y' + x'z' + z'x + z'y' + z'z' = x'y' + x'z' + xz' + y'z' = z' + y'(x' + z')$
$= z' + y'z' + x'y' = z' + x'y'$

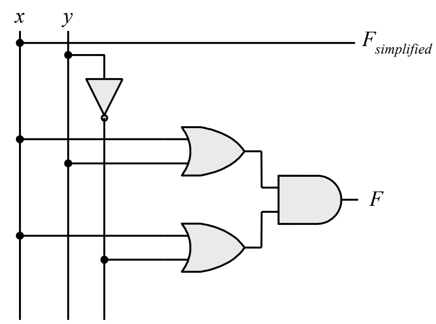
2.4 **(a)** $x'z + xy'$

(b) $w'(x + y'z)$

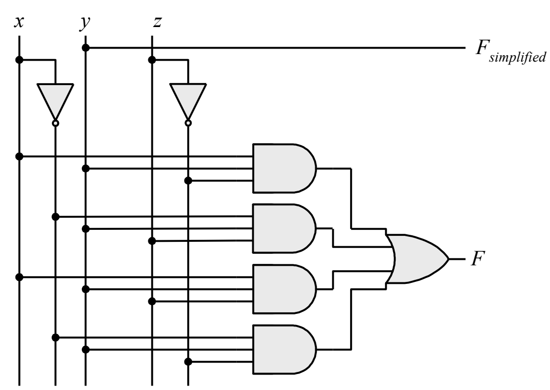
2.5 **(a)**



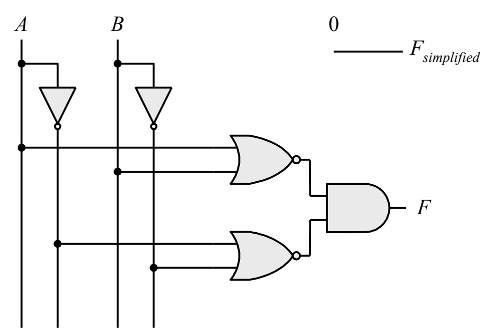
(b)


x
y
$F_{simplified}$
F

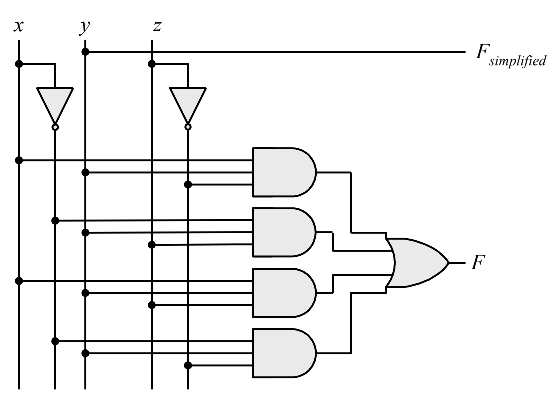
(c)


x
y
z
$F_{simplified}$
F

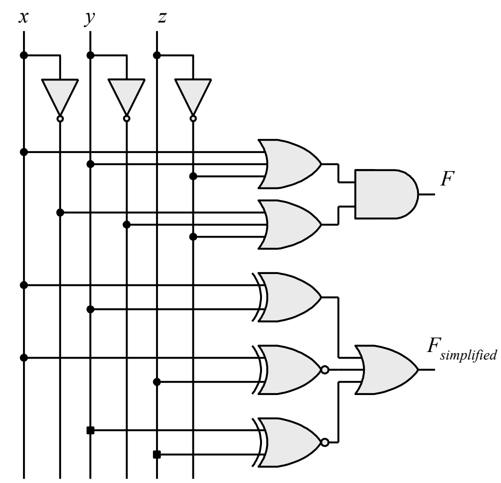
(d)


A
B
0
$F_{simplified}$
F

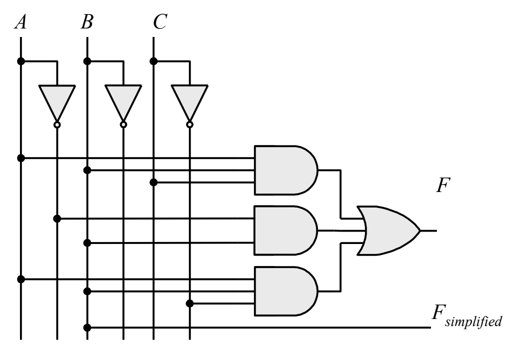
(e)


x
y
z
$F_{simplified}$
F

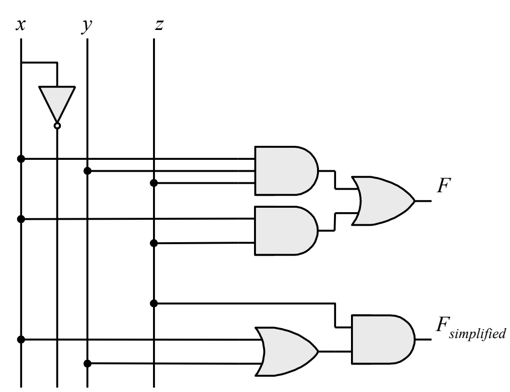
(f)


x
y
z
F
F simplified

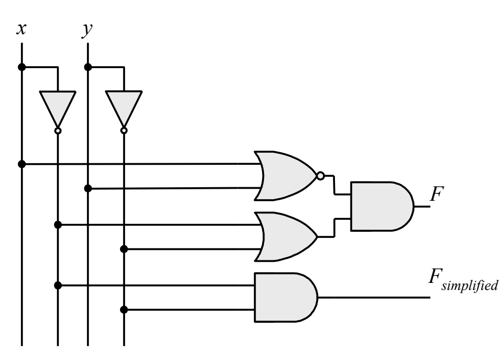
2.6 **(a)**


A
B
C
F
F simplified

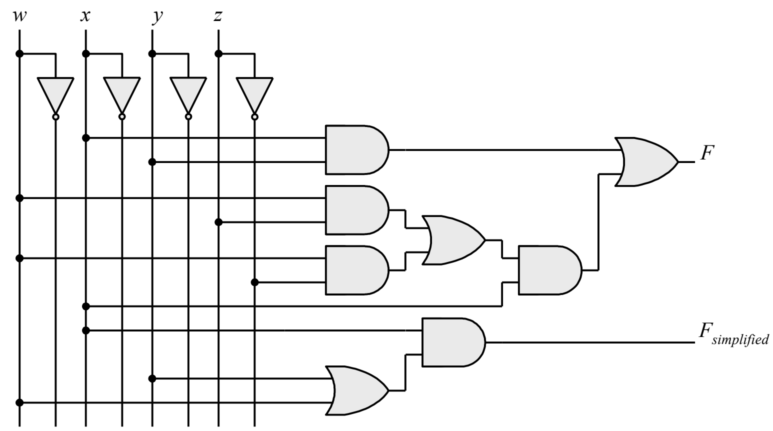
(b)


x
y
z
F
F simplified

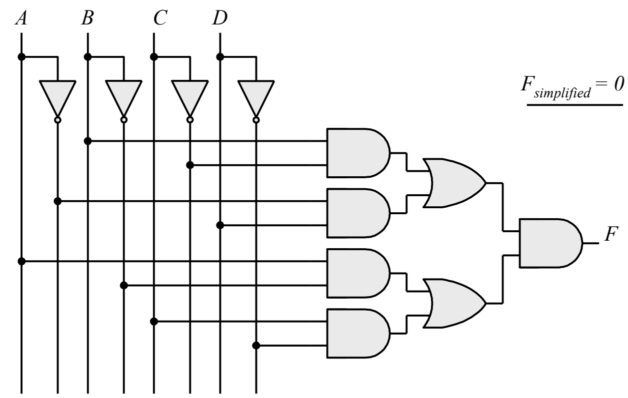
(c)


x
y
F
F simplified

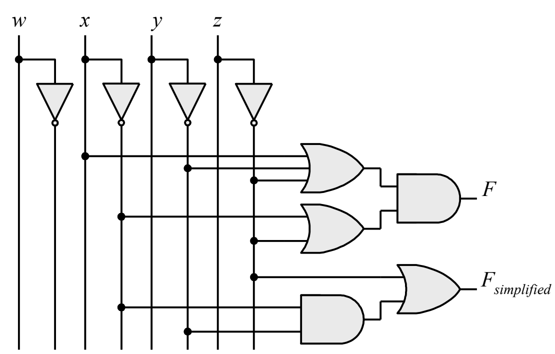
(d)



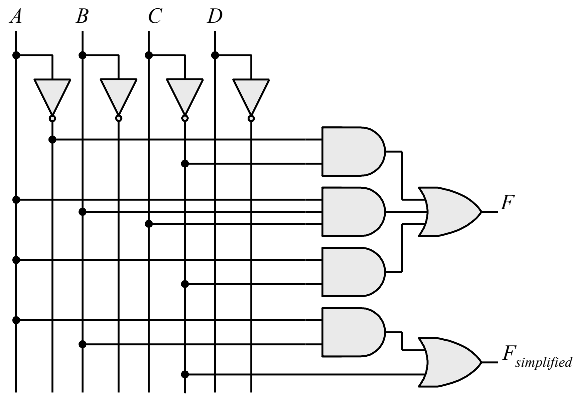
(e)



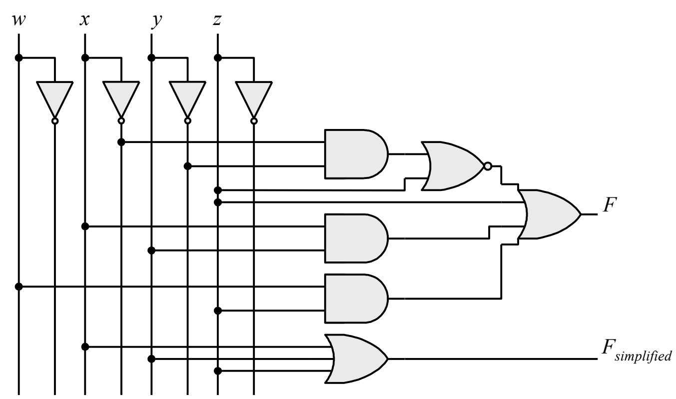
(f)



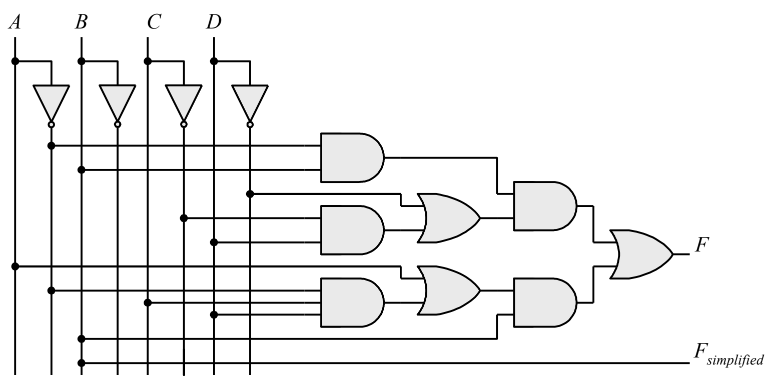
2.7 **(a)**



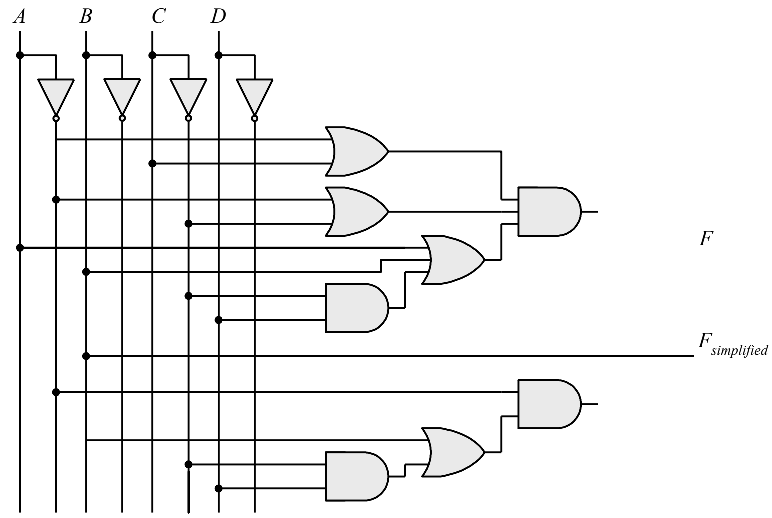
(b)



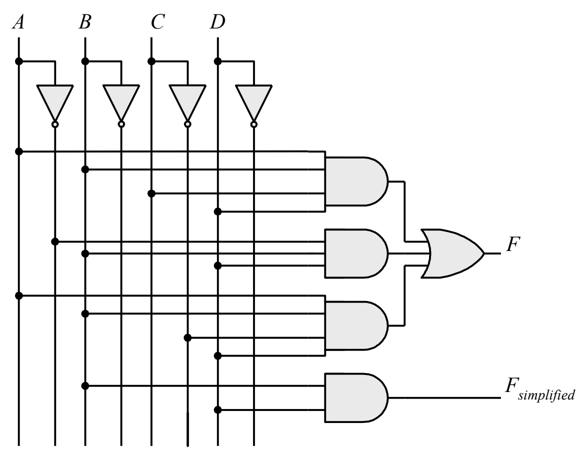
(c)



(d)



(e)



2.8 $F' = (wx + yz)' = (wx)'(yz)' = (w' + x')(y' + z')$

$FF' = wx(w' + x')(y' + z') + yz(w' + x')(y' + z') = 0$
$F + F' = wx + yz + (wx + yz)' = A + A' = 1$ *with* $A = wx + yz$

2.9 **(a)** $x'y' + z'$

(b) $(x' + y)\ z$

(c) $(x' + y)\ (w + z') + (w' + x)\ (y' + z)$

2.10 **(a)** $F_1 + F_2 = \Sigma\ m_{1i} + \Sigma m_{2i} = \Sigma\ (m_{1i} + m_{2i})$

(b) $F1\ F2 = \Sigma\ m_i\ \Sigma m_j$ where $m_i\ m_j = 0$ if $i \neq j$ and $m_i\ m_j = 1$ if $i = j$

2.11 **(a)**

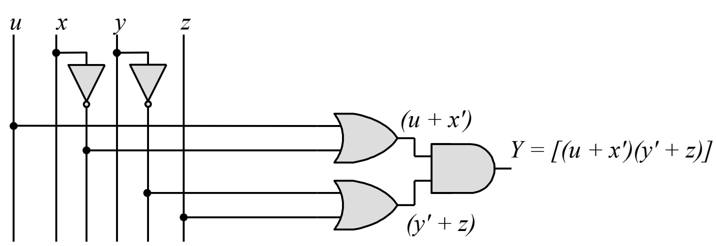
x	y	z	xz	xy'	yz'	F
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	1	1
0	1	1	0	0	0	0
1	0	0	0	1	0	1
1	0	1	1	1	0	1
1	1	0	0	0	1	1
1	1	1	1	0	0	1

(b)

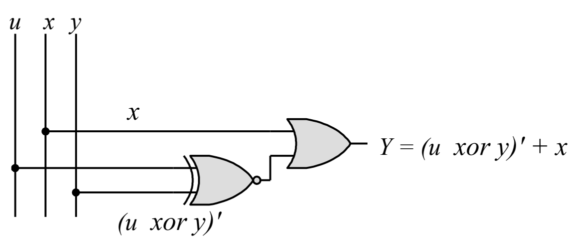
a	b	c	$a'b'c'$	$ab'c$	$a'bc'$	abc'	F
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	0
0	1	0	0	0	1	0	1
0	1	1	0	0	0	0	0
1	0	0	0	0	0	0	0
1	0	1	0	1	0	0	1
1	1	0	0	0	0	1	1
1	1	1	0	0	0	0	0

2.12 **(a)** 10100001
(b) 11111101
(c) 01011100
(d) 01001010
(e) 00010110

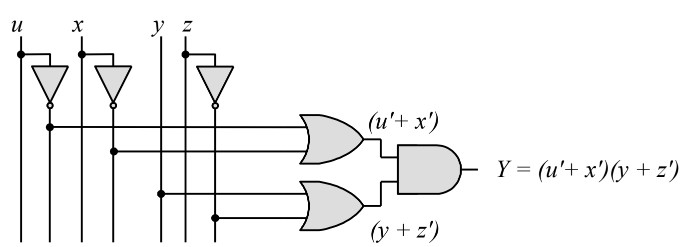
2.13 **(a)**



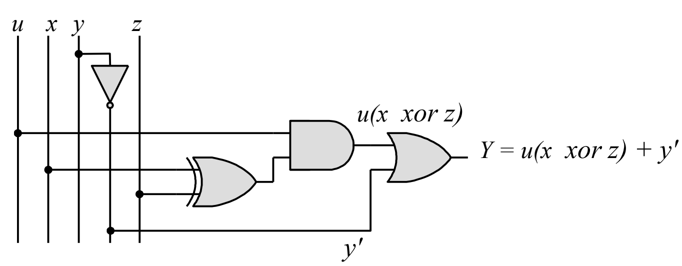
(b)



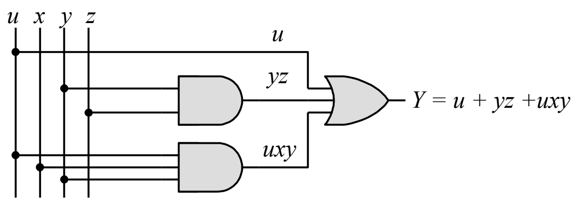
(c)



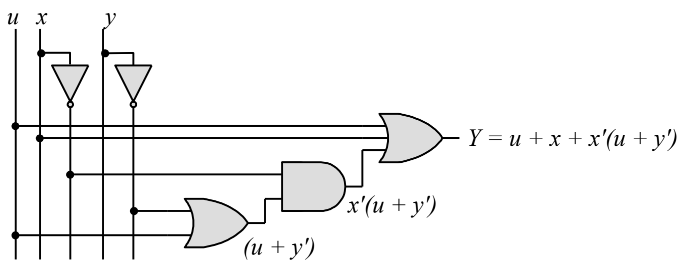
(d)



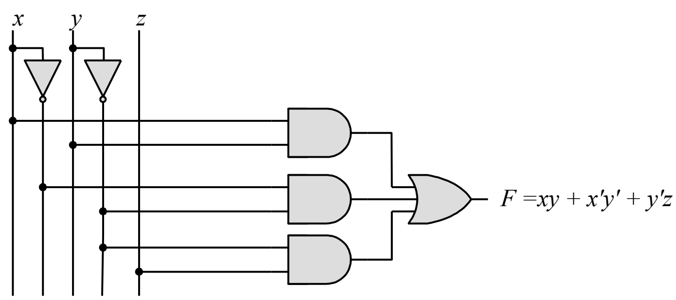
(e)


u x y z
u
yz
uxy
Y = u + yz +uxy

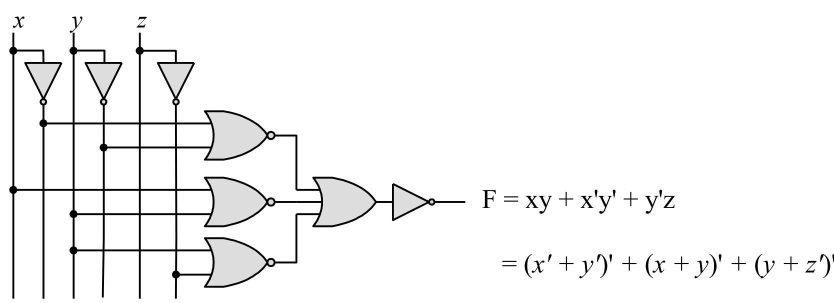
(f)


u x y
Y = u + x + x′(u + y′)
x′(u + y′)
(u + y′)

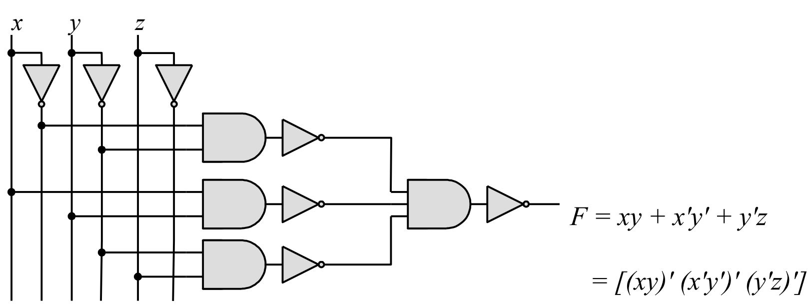
2.14 **(a)**


x y z
F =xy + x'y′ + y′z

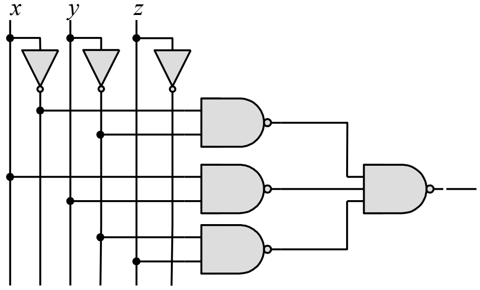
(b)


x y z
F = xy + x'y' + y'z
= (x′ + y′)' + (x + y)' + (y + z′)'

(c)


x y z
F = xy + x′y′ + y′z
= [(xy)′ (x′y′)′ (y′z)′]′

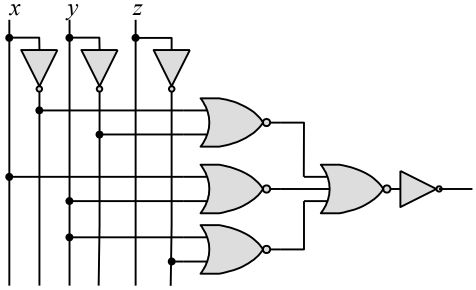
(d)



$F = xy + x'y' + y'z$

$= [(xy)' (x'y')' (y'z)']'$

(e)



$F = xy + x'y' + y'z$

$= (x' + y')' + (x + y)' + (y + z')'$

2.15 $T_1 = A'BC + AB'C' + AB'C + ABC' + ABC = A + BC$
$T_2 = A'B'C' + AB'C' + A'B'C + A'BC' = A' (B' + C') = T_1'$

2.16 **(a)** $F(A, B, C) = A'B'C' + A'B'C + A'BC' + A'BC + AB'C' + AB'C + ABC' + ABC$
$= A'(B'C' + B'C + BC' + BC) + A((B'C' + B'C + BC' + BC)$
$= (A' + A)(B'C' + B'C + BC' + BC) = B'C' + B'C + BC' + BC$
$= B'(C' + C) + B(C' + C) = B' + B = 1$

(b) $F(x_1, x_2, x_3, ..., x_n) = \Sigma m_i$ has $2^n/2$ minterms with x_1 and $2^n/2$ minterms with x'_1, which can be factored and removed as in (a). The remaining 2^{n-1} product terms will have $2^{n-1}/2$ minterms with x_2 and $2^{n-1}/2$ minterms with x'_2, which and be factored to remove x_2 and x'_2. continue this process until the last term is left and $x_n + x'_n = 1$. Alternatively, by induction, F can be written as $F = x_nG + x'_nG$ with $G = 1$. So $F = (x_n + x'_n)G = 1$.

2.17 **(a)** $F = (b + cd)(c + bd) = bc + bd + cd + bcd$
$= \Sigma(3, 5, 6, 7, 11, 13, 14, 15)$
$F' = \Sigma(0, 1, 2, 4, 8, 9, 10, 12)$
$F = \Pi(0, 1, 2, 4, 8, 9, 10, 12)$

a b c d	F
0 0 0 0	0
0 0 0 1	0
0 0 1 0	0
0 0 1 1	1
0 1 0 0	0
0 1 0 1	1
0 1 1 0	1
0 1 1 1	1
1 0 0 0	0
1 0 0 1	0
1 0 1 0	0
1 0 1 1	1
1 1 0 0	0
1 1 0 1	1
1 1 1 0	1
1 1 1 1	1

(b) $(cd + b'c + bd')(b + d) = bcd + bd' + cd + b'cd = cd + bd'$
$= \Sigma (3, 4, 7, 11, 12,14, 15)$
$= \Pi (0, 1, 2, 5, 6, 8, 9, 10, 13)$

a b c d	F
0 0 0 0	0
0 0 0 1	0
0 0 1 0	0
0 0 1 1	1
0 1 0 0	1
0 1 0 1	0
0 1 1 0	0
0 1 1 1	1
1 0 0 0	0
1 0 0 1	0
1 0 1 0	0
1 0 1 1	1
1 1 0 0	1
1 1 0 1	0
1 1 1 0	1
1 1 1 1	1

(c) $(c' + d)(b + c') = bc' + c' + bd + c'd = (c' + bd)$
$= \Sigma (0, 1, 4, 5, 7, 8, 12, 13, 15)$
$= \Pi (2, 3, 6, 9, 10, 11, 14)$

(d) $bd' + acd' + ab'c + a'c' = \Sigma\,(0, 1, 4, 5, 10, 11, 14)$

$F' = \Sigma\,(2, 3, 6, 7, 8, 9, 12, 13, 15)$

$F = \Pi\,(0, 2, 3, 6, 7, 8, 12, 13, 15)$

a b c d	F
0 0 0 0	1
0 0 0 1	1
0 0 1 0	0
0 0 1 1	0
0 1 0 0	1
0 1 0 1	1
0 1 1 0	0
0 1 1 1	0
1 0 0 0	0
1 0 0 1	0
1 0 1 0	1
1 0 1 1	1
1 1 0 0	1
1 1 0 1	0
1 1 1 0	1
1 1 1 1	0

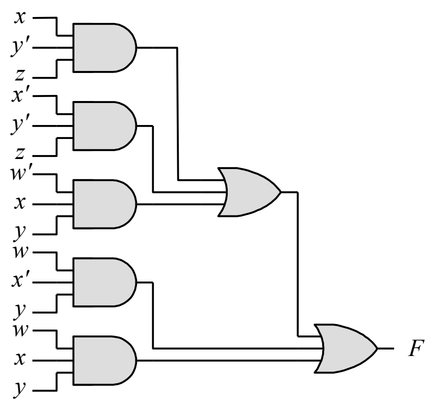
2.18 (a)

w	*x*	*y*	*z*	*F*
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

$F = xy'z + x'y'z + w'xy + wx'y + wxy$

$F = \sum(1, 5, 6, 7, 9, 10, 11, 13, 14, 15)$

(b)



In this logic diagram,

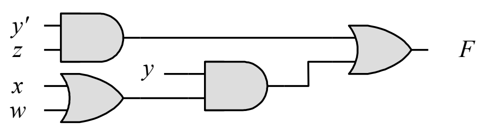
5 - Three-input AND gates
2 - Three-input OR gates
4 - Inverters

are used.

(c) $F = xy'z + x'y'z + w'xy + wx'y + wxy = y'z + xy + wy = y'z + y(w + x)$

(d) $F = y'z + yw + yx = \sum(1, 5, 6, 7, 9, 10, 11, 13, 14, 15)$ same as part (a)

(e)



In this logic diagram,

2 - Two-input AND gates
2 - Two-input OR gates
1 - Inverter

are used.

2.19 $F = B'D + A'D + BD$

ABCD	*ABCD*	*ABCD*
-B'-D	*A'--D*	*-B-D*
0001 = 1	*0001 = 1*	*0101 = 5*
0011 = 3	*0011 = 3*	*0111 = 7*
1001 = 9	*0101 = 5*	*1101 = 13*
1011 = 11	*0111 = 7*	*1111 = 15*

$F = \Sigma(1, 3, 5, 7, 9, 11, 13, 15) = \Pi(0, 2, 4, 6, 8, 10, 12, 14)$

2.20 **(a)** $F'(w, x, y, z) = \sum(2, 4, 6, 7, 10, 11, 12, 14)$

(b) $F'(x, y, z) = \sum(3, 7, 9)$

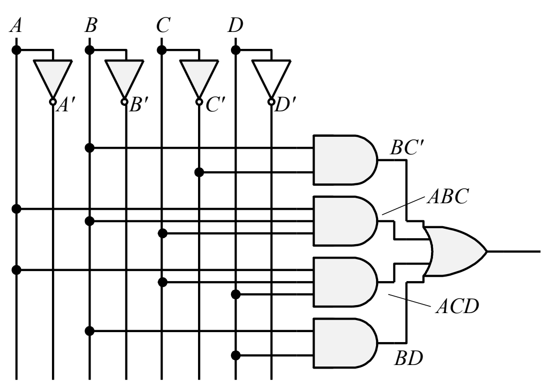
2.21 **(a)** $F(x, y, z) = \sum(1, 2, 4, 5) = \prod(0, 3, 6, 7)$

(b) $F(A, B, C, D) = \prod(0, 1, 3, 4, 7, 11) = \sum(2, 5, 6, 8, 9, 10, 12, 13, 14, 15)$

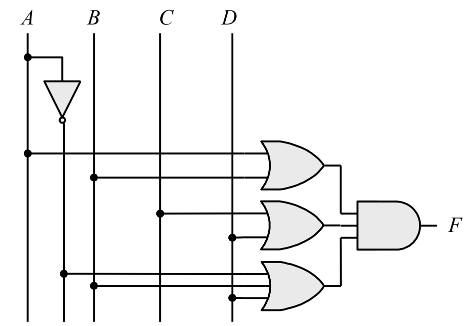
2.22 **(a)** $(u + xw)(x + u'v) = ux + uu'v + xxw + xwu'v = ux + xw + xwu'v$
$= ux + xw = x(u + w)$
$= ux + xw$ (SOP form)
$= x(u + w)$ (POS form)

(b) $x' + x(x + y')(y + z') = x' + x(xy + xz' + y'y + y'z')$
$= x' + xy + xz' + xy'z' = x' + xy + xz'$ *(SOP form)*
$= (x' + y + z')$ *(POS form)*

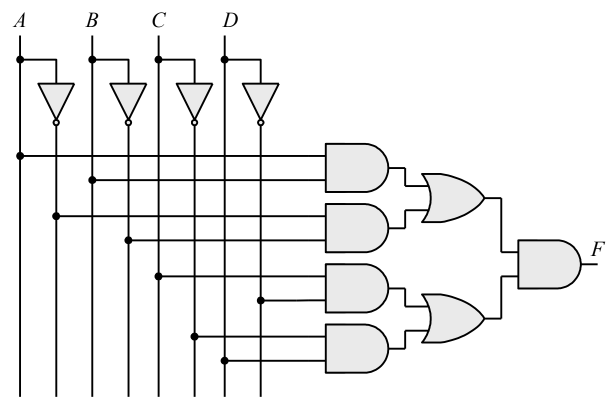
2.23 **(a)** $BC' + ABC + ACD + BD$



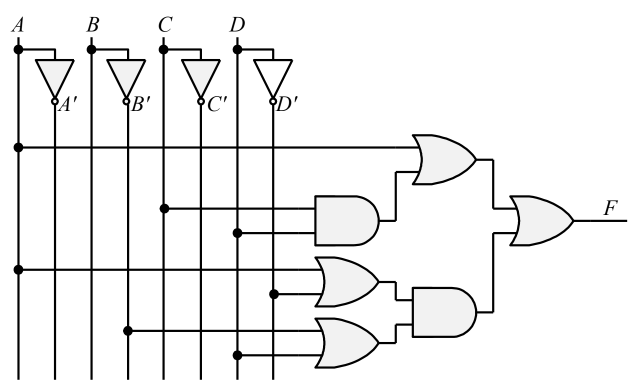
(b) $(A + B)(C + D)(A' + B + D)$



(c) $(AB + A'B')(CD' + C'D)$



(d) $A + CD + (A + D')(B' + D)$



2.24 $(x + y')(yz) + xz' = xyz + xz'$

2.25 **(a)** $x \mid y = xy' \neq y \mid x = x'y$ Not commutative

$(x \mid y) \mid z = xy'z' \neq x \mid (y \mid z) = x(yz')' = xy' + xz$ Not associative

(b) $(x \oplus y) = xy' + x'y = y \oplus x = yx' + y'x$ Commutative

$(x \oplus y) \oplus z = \sum(1, 2, 4, 7) = x \oplus (y \oplus z)$ Associative

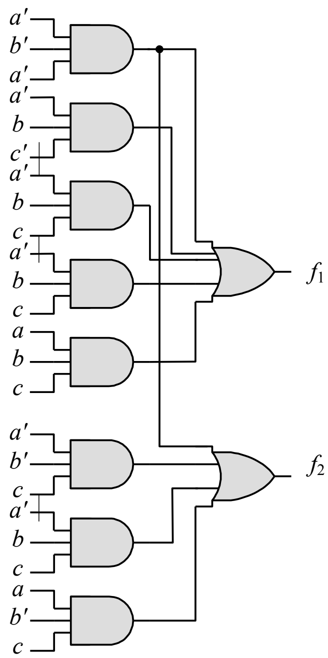
2.26

Gate		NAND (Positive logic)		NOR (Negative logic)	
x y	z	x y	z	x y	z
L L	H	0 0	1	1 1	0
L H	H	0 1	1	1 0	0
H L	H	1 0	1	0 1	0
H H	L	1 1	0	0 0	1

Gate		NOR (Positive logic)		NAND (Negative logic)	
x y	z	x y	z	x y	z
L L	H	0 0	1	1 1	0
L H	L	0 1	0	1 0	1
H L	L	1 0	0	0 1	1
H H	L	1 1	0	0 0	1

2.27 $f_1 = a'b'c + a'bc + abc' + abc$

$f_2 = a'bc' + a'bc + ab'c' + ab'c + abc'$


a′
b′
a′
a′
b
c′
a′
b
c
a′
b
c
a
b
c
f1
a′
b′
c
a′
b
c
a
b′
c
f2

2.28 **(a)** $y = a + b + c' = \Sigma(0, 2, 3, 4, 5, 6, 7)$

a	b	c	y
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

(b) $y = (a + b')(cd + e) = \Sigma(1, 3, 5, 6, 7, 17, 19, 21, 22, 23, 25, 27, 29, 30, 31)$

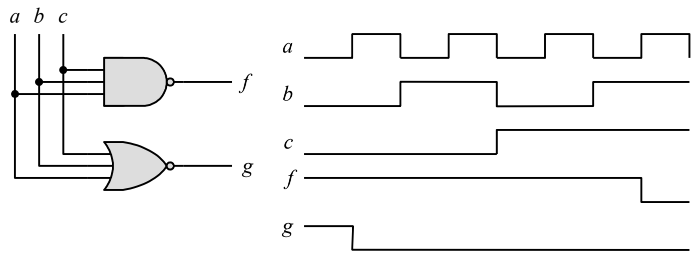
a	b	c	d	e	y	a	b	c	d	e	y
0	0	0	0	0	0	1	0	0	0	0	0
0	0	0	0	1	1	1	0	0	0	1	1
0	0	0	1	0	0	1	0	0	1	0	0
0	0	0	1	1	1	1	0	0	1	1	1
0	0	1	0	0	0	1	0	1	0	0	0
0	0	1	0	1	1	1	0	1	0	1	1
0	0	1	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	0	1	1	1	1
0	1	0	0	0	0	1	1	0	0	0	0
0	1	0	0	1	0	1	1	0	0	1	1
0	1	0	1	0	0	1	1	0	1	0	0
0	1	0	1	1	0	1	1	0	1	1	1
0	1	1	0	0	0	1	1	1	0	0	0
0	1	1	0	1	0	1	1	1	0	1	1
0	1	1	1	0	0	1	1	1	1	0	1
0	1	1	1	1	0	1	1	1	1	1	1

2.29 True

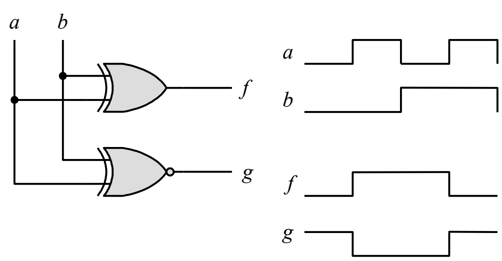
2.30 (b + d)(a' + b' + c) = a'b + bb' + bc + a'd + b'd + cd = a'b + bc + a'd + b'd + cd

2.31 $(a + b + c)\,(a + b + c')\,(a + b' + c)\,(a' + b + c')\,(a' + b' + c')$

2.32



2.33



Chapter 3

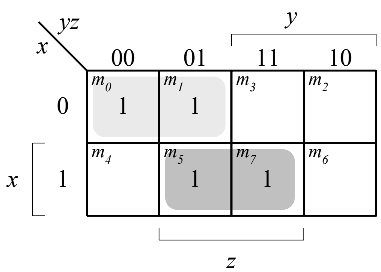
3.1 **(a)** $xy + xz + yz$

(b) $x' + yz$

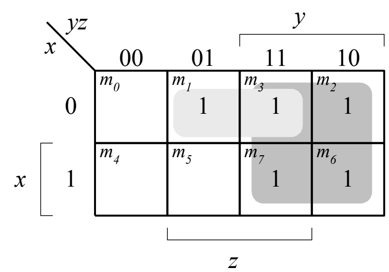
(c) $z' + x'y$

(d) $xy + x'z'$

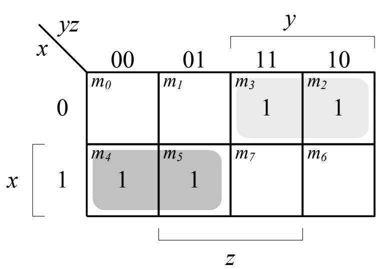
3.2



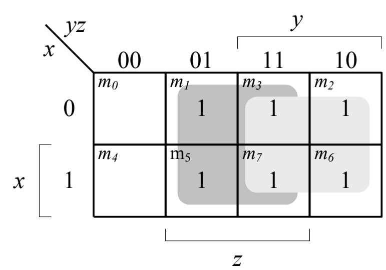
(a) $F = x'y' + xz$



(b) $F = y + x'z$

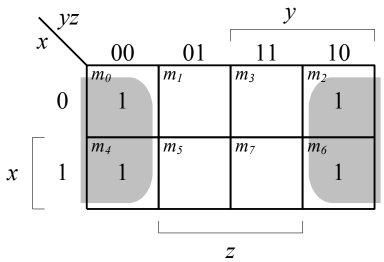


$F = xy' + x'y$



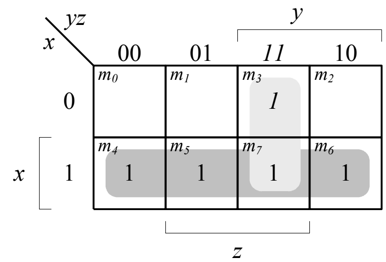
$F = y + z$

(c)



$F = z'$

(d)



$F = x + yz$

(e)

(f)

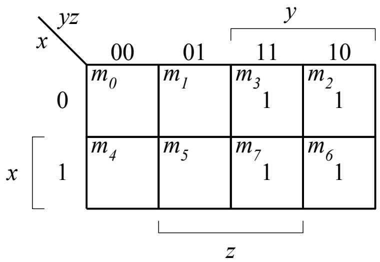
3.3 **(a)** $x'y'z + xy$

(b) $x'y + z'$

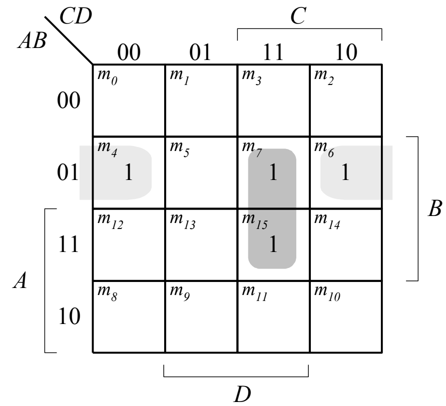
(c) $xy + x'z'$

(d) $x' + yz$

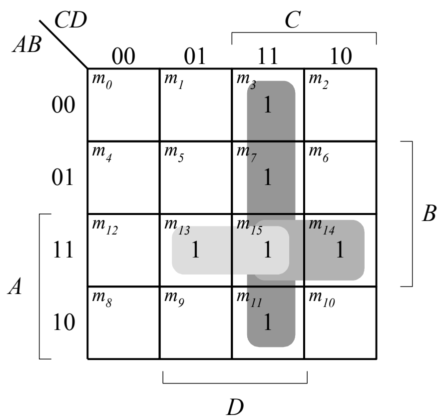
3.4



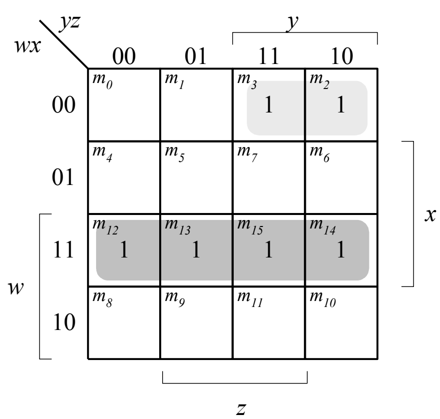
(a) $F = y$



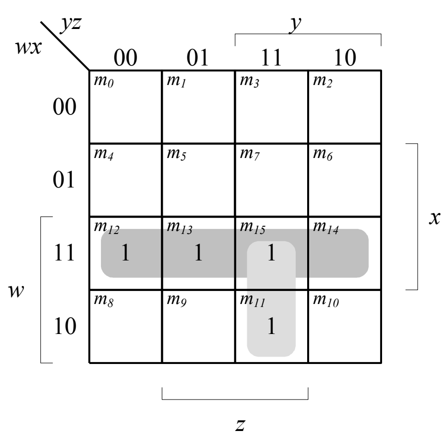
(b) $F = BCD + A'BD'$



(c) $F = CD + ABD + ABC$

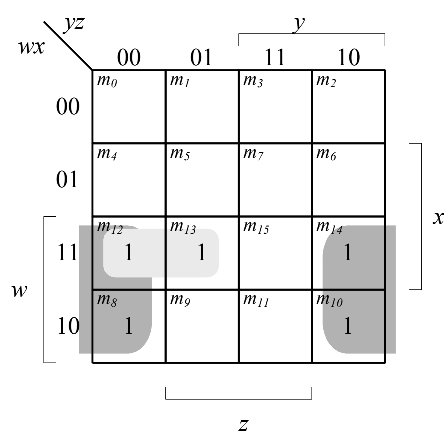


(d) $F = w'x'y + wx$



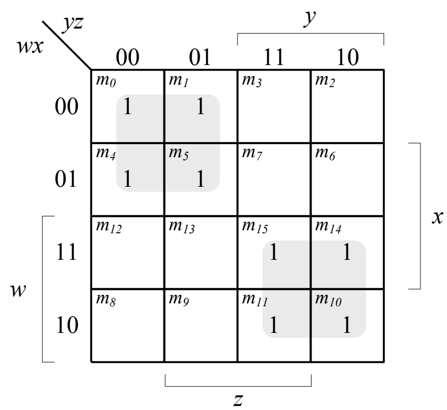
$F = wx + wyz$

(e)



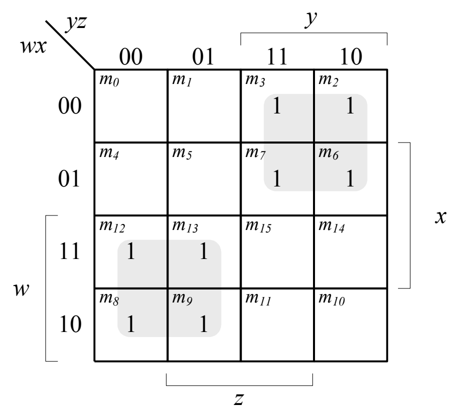
$F = wz' + xy'w$

(f)



$F = w'y' + w'y'$

(g)



$F = wy' + w'y$

(h)

3.5

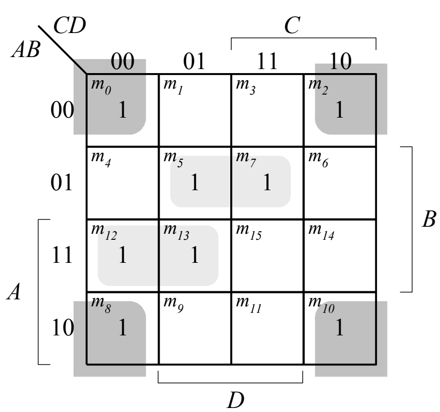
(a) x

(b) $x'y'z' + x'yz + wxy' + wxz$

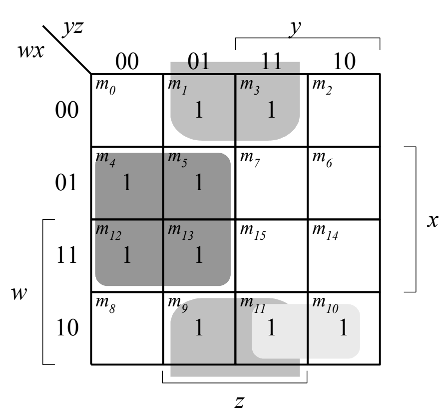
(c) $wxy + wxz + yz$

(d) $w'x'y + wx$

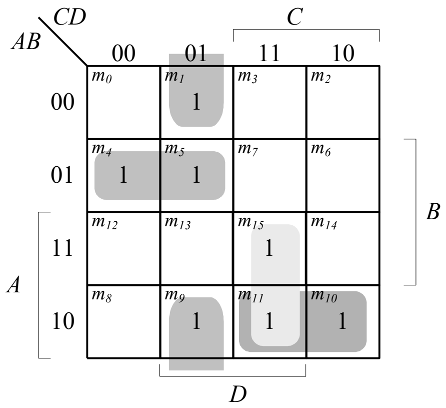
3.6



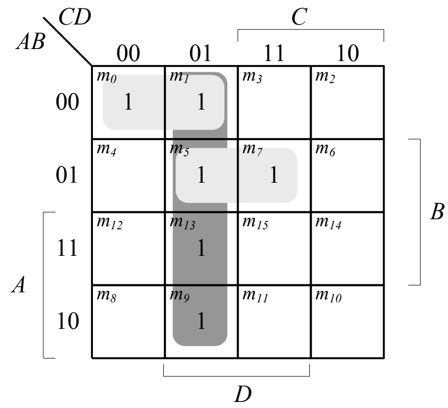
(a) $F = B'D' + A'BD + ABC'$



(b) $F = xy' + x'z + wx'y$



(c) $F = A'BC' + B'C'D + ACD + AB'C$

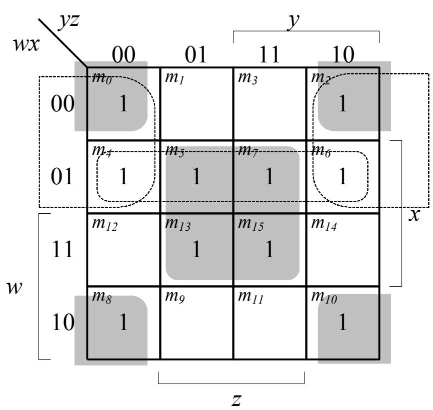


(d) $F = C'D + A'BD + A'B'C'$

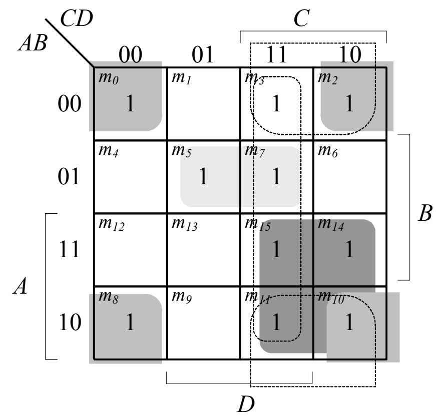
3.7
- **(a)** $x'y + z$
- **(b)** $C'D + B'C + ABC'$
- **(c)** $B'D' + AC + A'BD + CD$ (or $B'C$)
- **(d)** $wx + x'y + yz$

3.8 (a) $F(w, x, y, z) = \Sigma(0, 1, 2, 3, 11, 12, 14, 15)$
(b) $F(A, B, C, D) = \Sigma(1, 3, 5, 9, 12, 13, 14)$
(c) $F(x, y, z) = \Sigma(3, 5, 6, 7)$
(d) $F(A, B, C, D) = \Sigma(3, 4, 5, 7, 11, 12)$

3.9



(a)

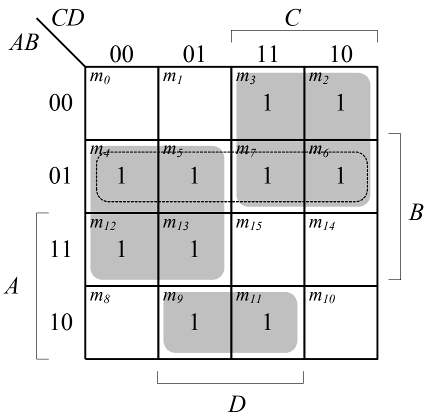


(b)

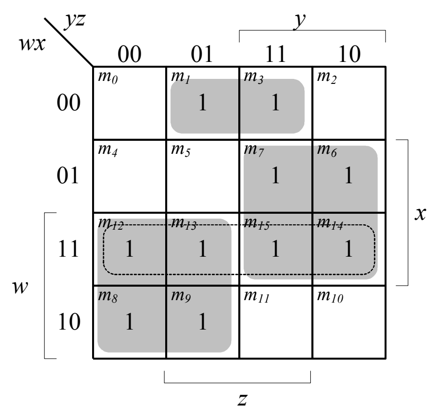
Essential: *xz, x'z'*
Non-essential: *w'x, w'z'*
F = xz + x'z' + (w'x or w'z')

Essential: *B'D', AC, A'BD*
Non-essential: *CD, B'C*
F = B'D' + AC + A'BD + (CD

OR B'C)



(c)

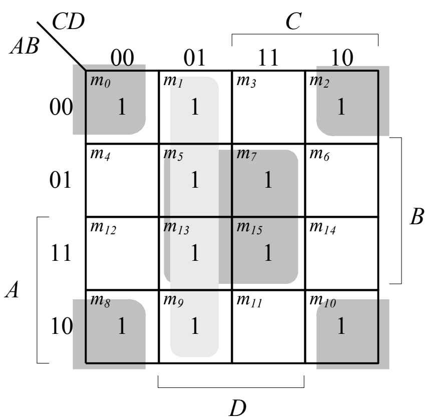


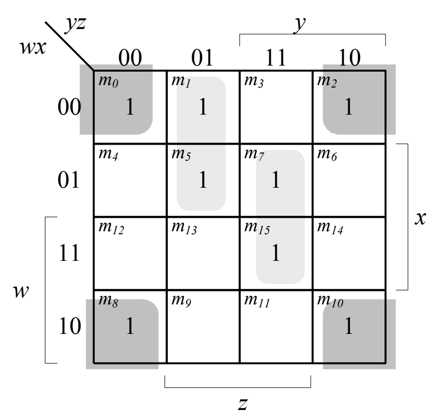
(d)

Essential: $BC', AC, A'B'D$
Non-Essential: $A'B$
$F = BC' + AC + A'B'D$

Essential: $wy', xy, w'x'z$

$F = wy' + xy + w'x'z$

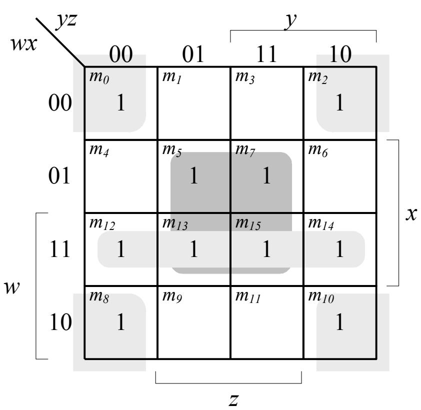




Essential: $BD, B'C', C'D$
$F = BD + B'C' + C'D$

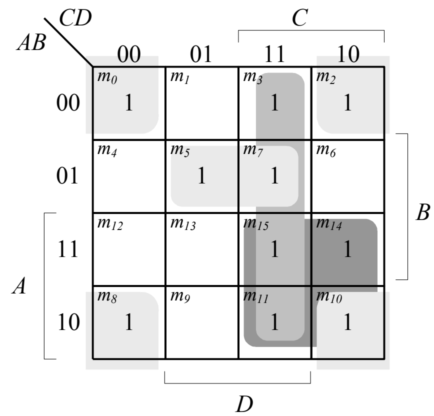
Essential: $x'z', w'y'z, xyz$
$F = x'z' + w'y'z + xyz$

3.10



F = Σ(0, 2, 5, 7, 8, 10, 12, 13, 14, 15)
14, 15)
Essential: *xz, wx, x'z'*
A'BD
F = xz + wx + x'z'

(a)

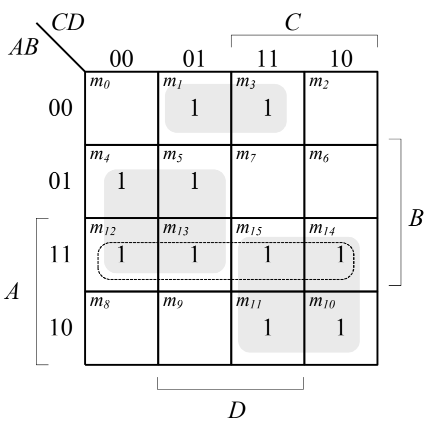


F = Σ(0, 2, 3, 5, 7, 8, 10, 11,

Essential: *AC, B'D', CD,*

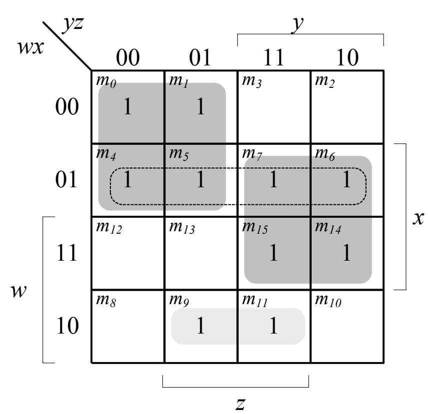
F = AC + B'D' + CD + A'BD

(b)



F = Σ(1, 3, 4, 5, 10, 11, 12, 13, 14, 15)
14, 15)
Essential: AC, BC', A'B'D
Non-essential: *AB, A'B'D, B'CD, A'C'D*
w'wz, w'x'z
F = AC + BC' + A'B'D

(c)

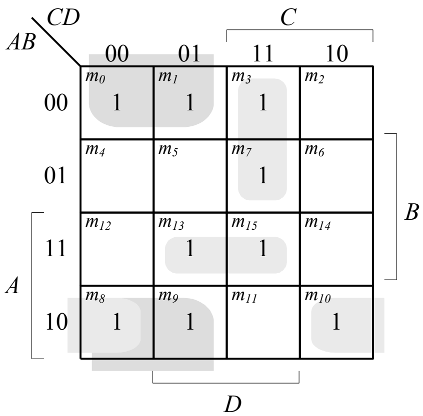


F = Σ(0, 1, 4, 5, 6, 7, 9, 11,

Essential: *w'y', xy, wx'z*
Non-essential: *wx, x'y'z,*

F = *w'y' + xy + wx'z*

(d)

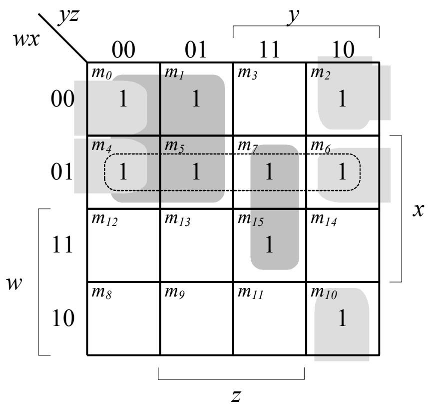


F(A, B, C, D) = S(0, 1, 3, 7 8, 9, 10,13,15)

Essential: *B'C', AB'D'*
Non-essential: *ABD, A'CD, BCD*

F = B'C' + AB'D' +A'CD +ABD

(e)



F = S(0, 1, 2, 4, 5, 6, 7,
10, 15)

Essential: *w'y', w'z', xyz, x'yz'*
Non-Essential: *w'x*

F = w'y' + w'z' + xyz + x'yz'

(f)

3.11 **(a)** $F(w, x, y, z) = \sum (0, 1, 2, 5, 8, 10, 13)$

$wx \backslash yz$	00	01	11	10
00	m_0 1	m_1 1	m_3	m_2 1
01	m_4	m_5 1	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8 1	m_9	m_{11}	m_{10} 1

$F = x'z' + w'x'y' + w'y'z$

$wx \backslash yz$	00	01	11	10
00	m_0	m_1	m_3 1	m_2
01	m_4 1	m_5	m_7 1	m_6 1
11	m_{12} 1	m_{13} 1	m_{15} 1	m_{14} 1
10	m_8	m_9 1	m_{11} 1	m_{10}

$F' = wz + xz' + yz$

$F = (w' + z')(x' + z)(y' + z')$

3.12 **(a)** $F = (B + D') (B + C') (A' + B) (A' + C' + D)$
(b) $F = (A + B' + D) (B' + C + D) (A' + B' + D') (A' + B + C' + D)$

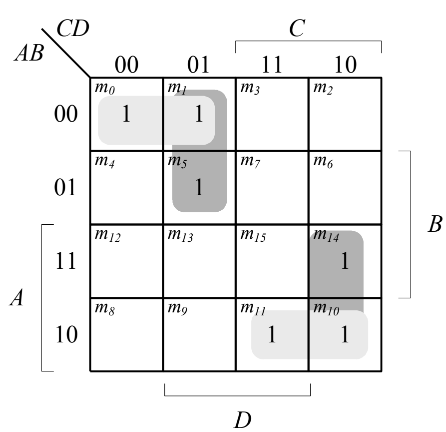
3.13 **(a)** $F = xy + z' = (x + z') (y + z')$

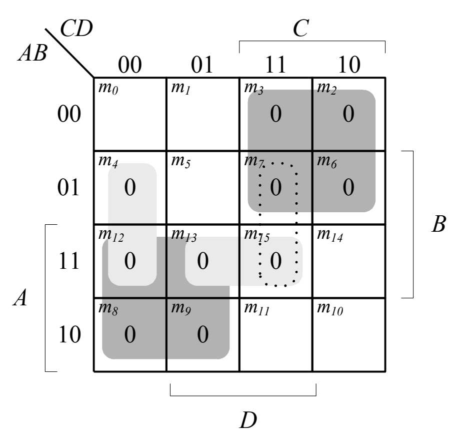
(b) $F = AD + ABC' + BCD = (A + C) (A + B) (C' + D) (B + C + D)$

(c) $F = C'D + AD + AB'C = (C + D) (A + C') (B' + C' + D)$

(d) $F = AC' + AD + C'D + AB'C = (A + D) (A + C') (B' + C' + D)$

3.14





SOP form (using 1s): $F = A'BC'D + AB'CD + A'B'C' + ACD'$

$$F = A'B'C' + A'C'D + AB'C + ACD'$$

POS form (using 0s): $F' = AC' + A'C + A'C'D' + ABD$

$$F = (A' + C)(A + C')(A + C + D)(A' + B' + D')$$

Alternative POS: $F' = AC' + A'C + A'C'D' + BCD$

$$F = (A' + C)(A + C')(A + C + D)(B' + C' + D')$$

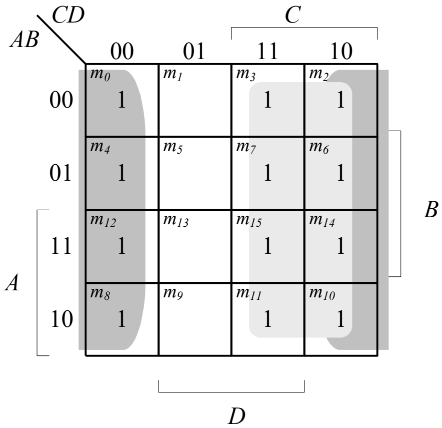
3.15 (a) $F(x, y, z) = 1 = \Sigma(0, 1, 2, 3, 4, 5, 6, 7)$

(b) $F(A, B, C, D) = A' + CD = \Sigma(0, 1, 2, 3, 4, 5, 6, 7, 11, 15)$

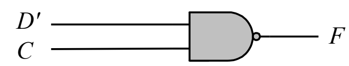
(c) $F(A, B, C, D) = C'D' + CD + AC' = \Sigma(0, 3, 4, 7, 8, 9, 11, 12, 13, 15)$

(d) $F(A, B, C, D) = B = \Sigma(4, 5, 6, 7, 12, 13, 14, 15)$

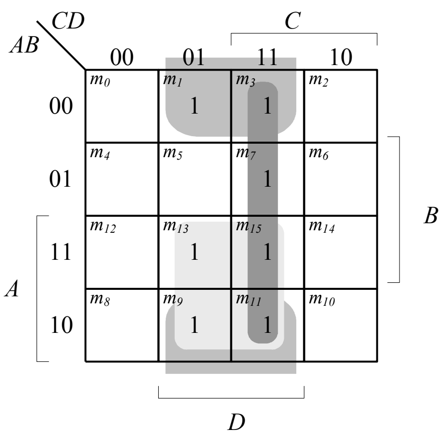
3.16 (a)



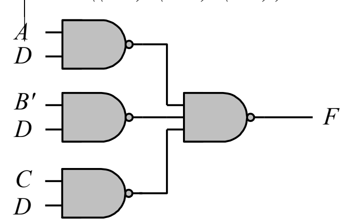
$F = C + D'$
$F = (C'D)'$



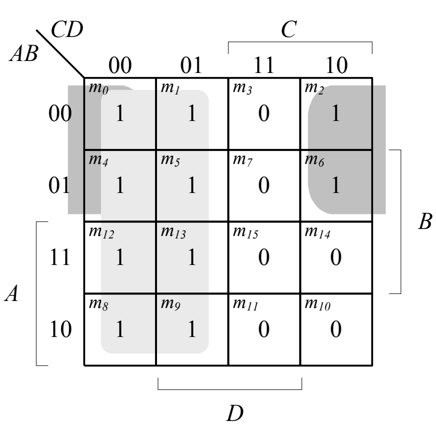
(b)



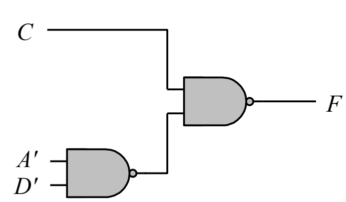
$F = AD + B'D + CD$
$F = ((AD)' (B'D)' (CD)')'$



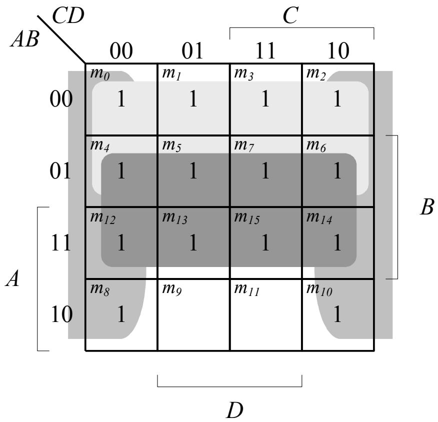
(c) $F = (A' + C' + D')(A' + C')(C' + D')$
$F' = (A' + C' + D')' + (A' + C')' + (C' + D')'$
$F' = ACD + AC + CD$



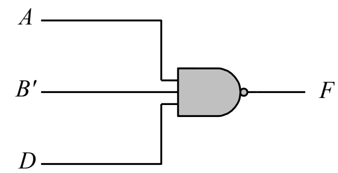
$F = C' + A'D'$
$F = (C(A + D))'$
$F = (C(A'D')')'$



(d)


CD
AB
C
00
01
11
10
m0
m1
m3
m2
1
1
1
1
m4
m5
m7
m6
m12
m13
m15
m14
m8
m9
m11
m10
B
A
D

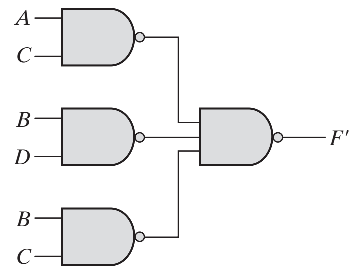
$F = A' + B + D'$
$F = (A(B')D)'$


A
B'
D
F

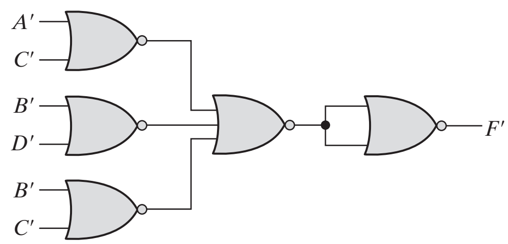
3.17 **(a)** On simplifying using K-maps, we get:

$F = A'B' + C'D' + B'C'$ and $F' = BC + AC + BD$

Logic diagram for F' using NAND gates:

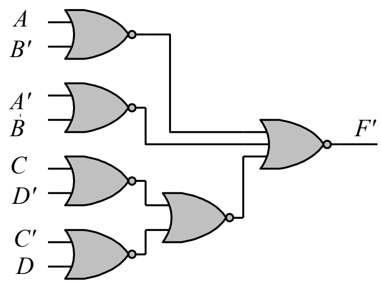


(b) Logic diagram for F' using NOR gates:



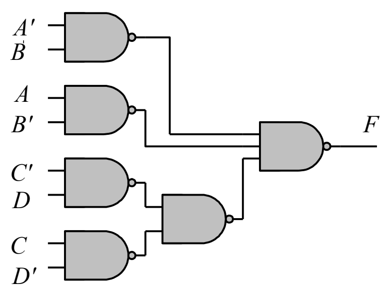
(3.18) (a)

F(A, B, C, D) = (A XOR B)'(C XOR D)
= (A'B + AB')' (C'D + CD')
= ((A'B + AB') + (C'D + CD'))'
=((A + B')' + (A' + B)') + ((C + D')' + (C' + D)') ' (nor gates)

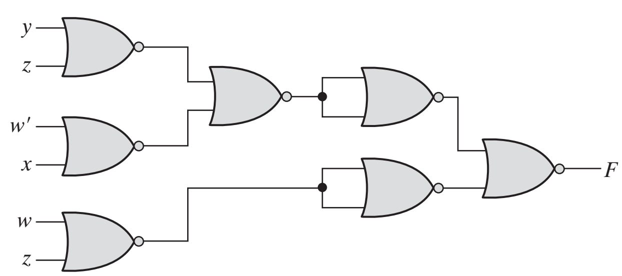


(b)

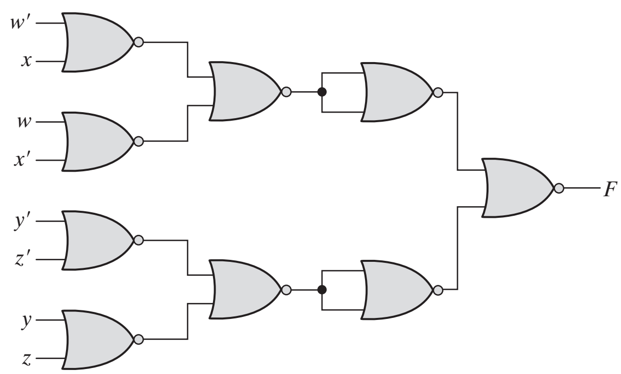
F(A, B, C, D) = (A XOR B)'(C XOR D)
= (A'B + AB')' (C'D + CD')
= (A'B)' (AB')' ((C'D)' (CD')')' (nand gates)



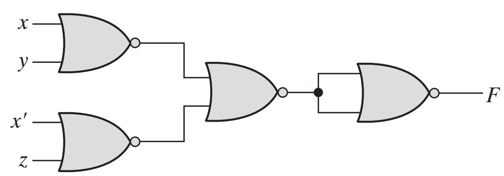
3.19 (a) $F = (w + z')(x + z')(w' + x' + y')$
Simplifying using K-map, we get:
$F = y'z' + wx' + w'z' = [(y + z)' + (w' + x)' + (w + z)']$
$F' = [(y + z)' + (w' + x)' + (w + z)']'$



(b) $F = \Sigma(1, 2, 13, 14)$
$F' = wx' + w'x + y'z' + yz = [(w' + x)(w + x')(y' + z')(y + z)]'$
$F = (w' + x)' + (w + x')' + (y' + z')' + (y + z)$



(c) $F = [(x + y)(x' + z)]' = (x + y)' + (x' + z)'$



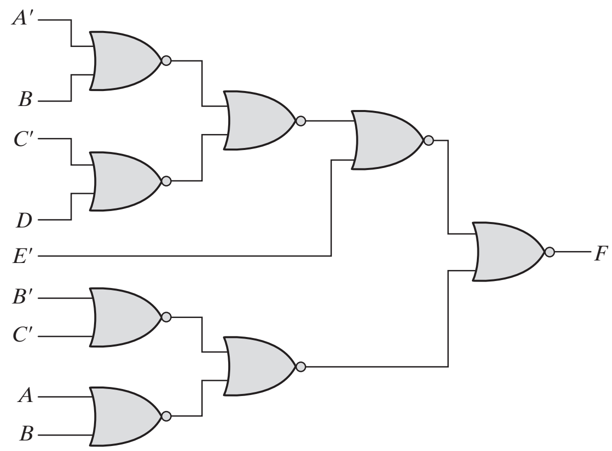
3.20 Multi-level NOR:

$F = (AB' + CD')E + BC(A + B)$

$F' = [(AB' + CD')E + BC(A + B)]'$

$F' = [\ [(AB' + CD')' + E']' + [\ (BC)' + (A + B)']'\ ]'$

$F' = [\ [((A' + B)' + (C' + D)')' + E']' + [\ (B' + C')' + (A + B)']']'$

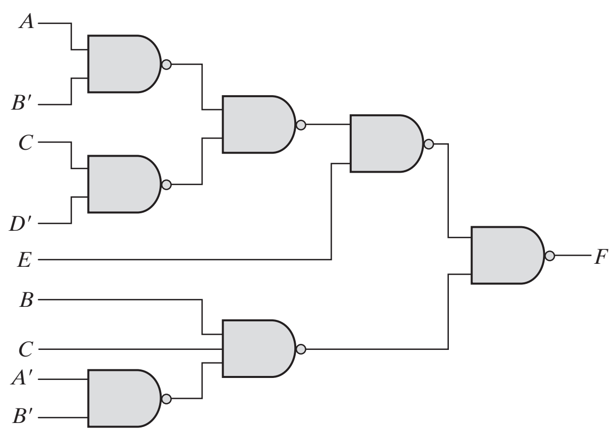


Multi-level NAND:

$F = (AB' + CD')E + BC(A + B)$

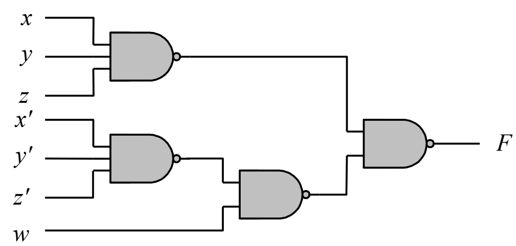
$F' = [(AB' + CD')E]'\ [BC(A + B)]'$

$F' = [(\ (AB')'(CD')')'E]'\ [BC(A'B')']'$

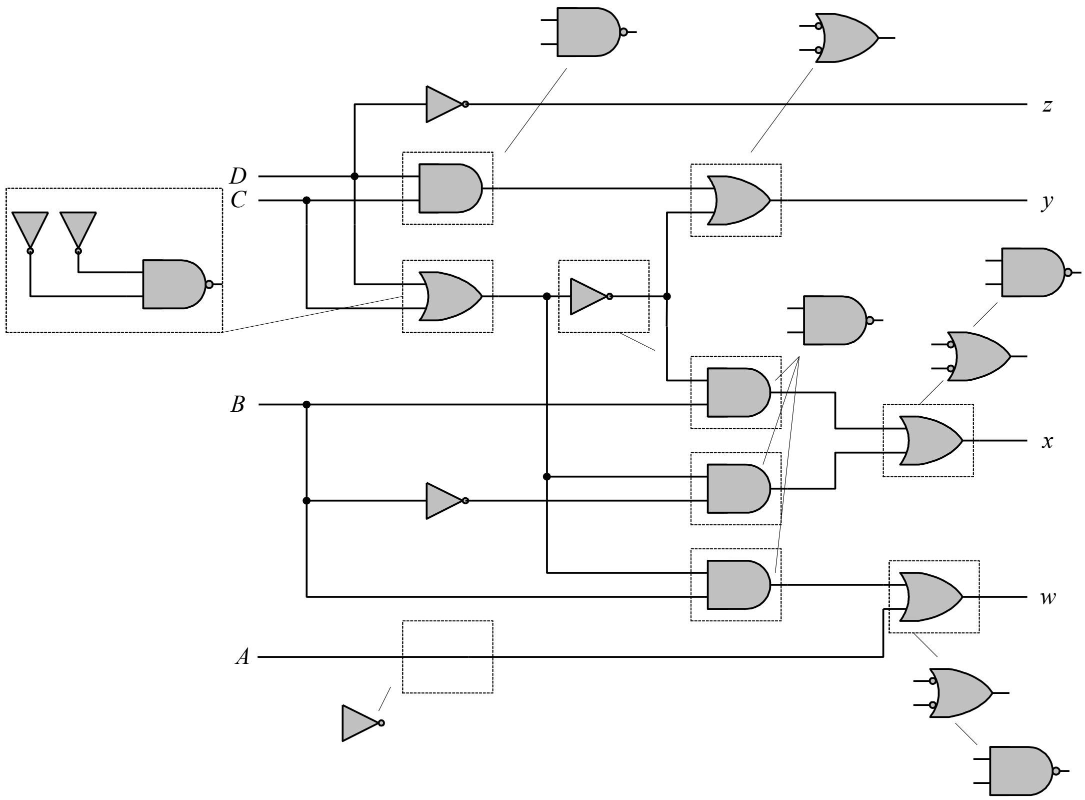


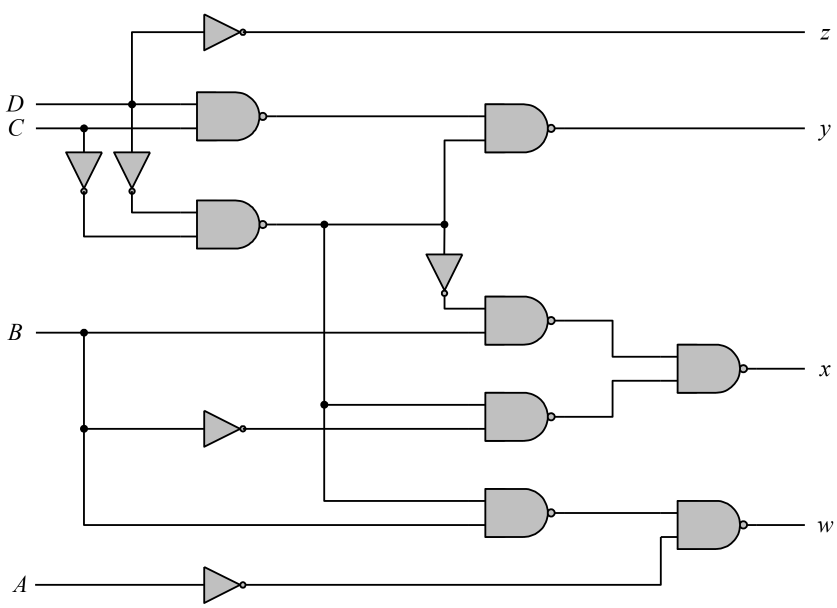
3.21 $F = w(x + y + z) + xyz$

$F' = [w(x + y + z)]'[xyz]' = [w(x'y'z')')]'(xyz)'$


x
y
z
x′
y′
z′
w
F

3.22



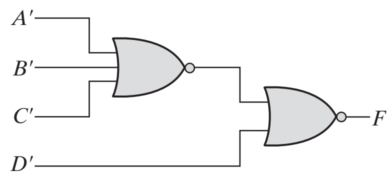


3.23 On simplifying using K-map, we get:

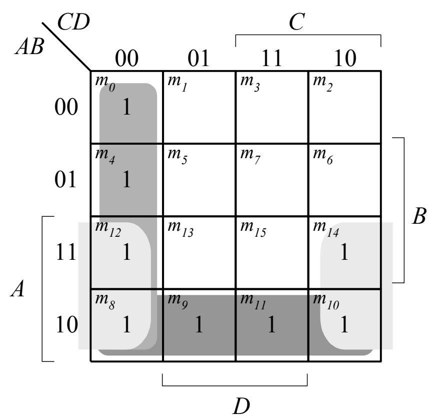
$F = AC' + A'D' + B'CD'$

$F' = D + ABC$

$F = [D + ABC]' = [D + (A' + B' + C')']'$



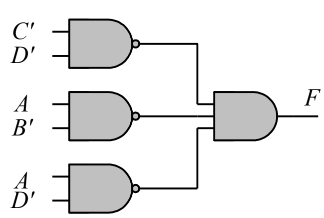
3.24 $F(A, B, C, D) = S(0, 4, 8, 9, 10, 11, 12, 14)$



(a) $F = C'D' + AB' + AD'$

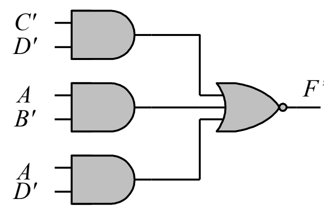
$F' = (C'D')'(AB')'(AD')'$

AND-NAND:



(b) $F' = [C'D' + AB' + AD']'$

AND-NOR:

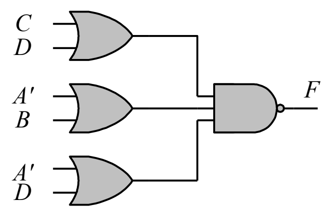


(c) $F = C'D' + AB' + AD' = (C + D)' + (A' + B)' + (A' + D)'$

$F' = (C'D')'(AB')'(AD')' = (C + D)(A' + B)(A' + D)$

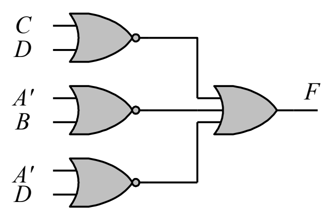
$F = [\ (C + D)(A' + B)(A' + D)\ ]'$

OR-NAND:

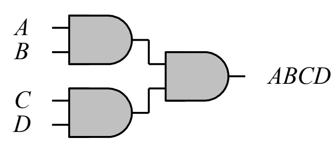


(d) $F = C'D' + AB' + AD' = (C + D)' + (A' + B)' + (A' + D)'$

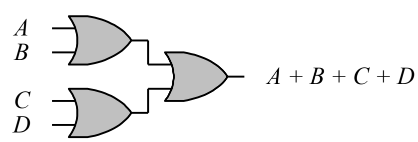
NOR-OR:



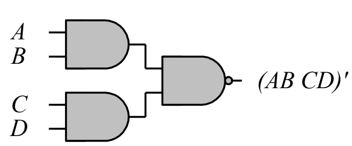
3.25



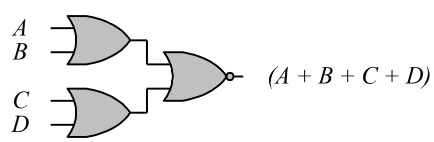
AND-AND → AND



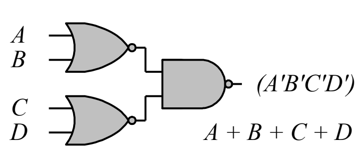
OR-OR → OR



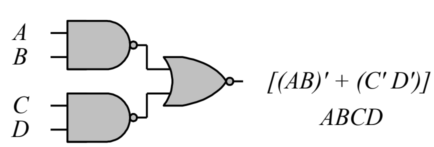
AND-NAND → NAND



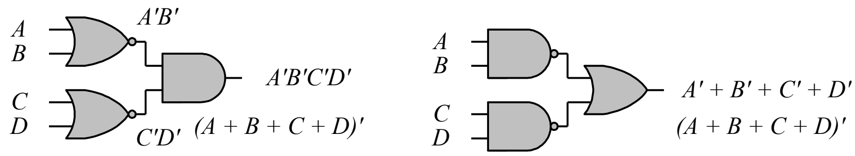
OR-NOR → NOR



NOR-NAND → OR



NAND-NOR → AND



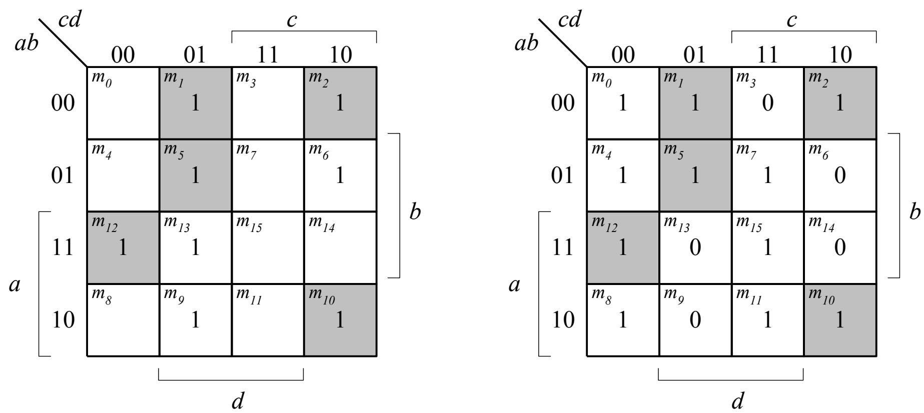
NOR-AND → NOR NAND-OR → NAND

The degenerate forms use 2-input gates to implement the functionality of 4-input gates.

3.26

$f = abc' + c'd + a'cd' + b'cd'$

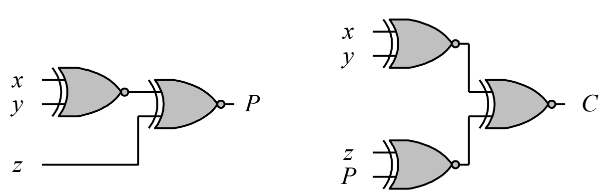
$g = (a + b + c' + d')(b' + c' + d)(a' + c + d')$
$g' = a'b'cd + bcd' + ac'd$



$fg = ac'd + abc'd + b'cd'$

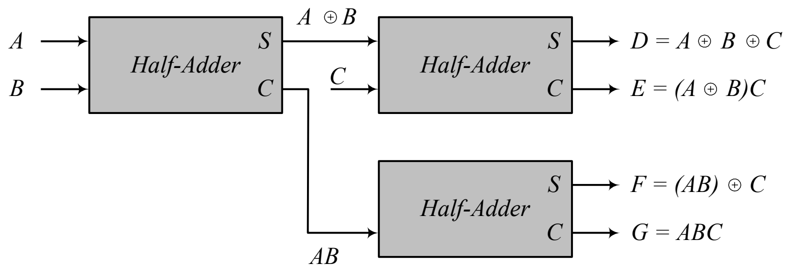
3.27 $(x \oplus y)' = xy + x'y'$;
$Dual = (x + y)(x' + y') = x'y + xy' = (x \oplus y)$, which is the complement of exclusive-NOR.

3.28

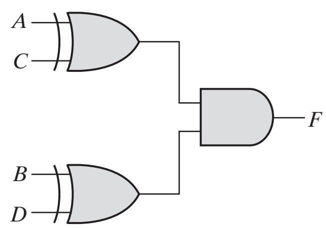


(a) 3-bit odd parity generator *(b) 4-bit odd parity generator*

3.29 $D = A \oplus B \oplus C$
$E = A'BC + AB'C = (A \oplus B)C$
$F = ABC' + (A' + B')C = ABC' + (AB)'C = (AB) \oplus C$
$G = ABC$

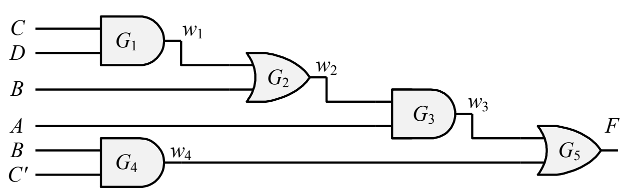


3.30 $F = A'B'CD + A'BCD' + AB'C'D + ABC'D'$
$F = A'C (B'D + BD') + AC' (B'D + BD') = (A \oplus C) (B \oplus D)$



3.31 Note: It is assumed that a complemented input is generated by another circuit that is not part of the circuit that is to be described.

From Fig. 3.20a:



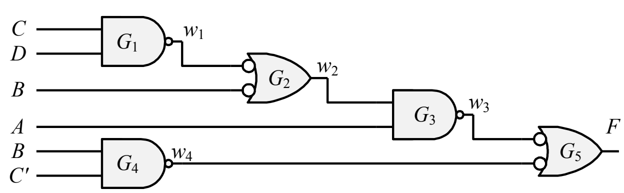
(a) **Verilog/SystemVerilog**

```
module Fig_3_20a_gates (F, A, B, C, C_bar, D);
 output   F;
 input    A, B, C, C_bar, D;
 wire     w1, w2, w3, w4;
 and      (w1, C, D);
 or       (w2, w1, B);
 and      (w3, w2, A);
 and      (w4, B, C_bar);
 or       (F, w3, w4);
endmodule
```

VHDL

```
entity Fig_3_20a_gates is
    port (F: out Std_Logic; A, B, B_bar, C, C_bar, D: in Std_Logic);
end Fig_3_20a_gates;

architecture Gates of Fig_3_20a_gates is
    component and2_gate port (y: out Std_Logic; xin1, xin2: in Std_Logic);
    component or2_gate port (y: out Std_Logic; xin1, xin2: in Std_Logic);
    signal C: Std_Logic;
begin  -- Instantiate components
    G1:  and2_gate port map (w1 => y, C => xin1 =. C, D => xin2);
    G2:  or2_gate port map (w2 => y, w1 => xin1, B => xin2);
    G3:  and2_gate port map (w3 => y, w2 => xin1, A => xin2);
    G4:  and2_gate port map (w4=> y, B => xin1, C_bar => xin2);
    G5:  or2_gate port map (F => y, w3 => xin1, w4 => xin2);
end Gates;

entity and2_gate is
  port (y: out Std_Logic; xin1, xin2: in Std_Logic);
end and2_gate;

architecture Behavioral of and2_gate is
begin
    y <= xin1 and xin2;
end and2_gate;

entity or2_gate is
  port (y: out Std_Logic; xin1, xin2: in Std_Logic);
end or2_gate;

architecture Behavioral of or2_gate is
begin
    y <= xin1 or xin2;
end or2_gate;
```

From Fig. 3.20b:



(b) Verilog/SystemVerilog

```
module Fig_3_20b_gates (F, A, B, B_Bar, C, C_bar, D);
  output   F;
  input    A, B, B_bar, C, C_bar, D;
  wire     w1, w2, w3, w4;
  not      (w1_bar, w1);
  not      (w3_bar, w3);
  not      (w4_bar, w4);
  nand     (w1, C, D);
  or       (w2, w1_bar, B_bar);
```

```
  nand     (w3, w2, A);
  nand     (w4, B, C_bar);
  or       (F, w3_bar, w4_bar);
endmodule
```

VHDL

```
entity Fig_3_20b_gates is
    port (F: out Std_Logic; A, B, B_bar, C, C_bar, D: in Std_Logic);
end Fig_3_20b_gates;

architecture Gates of Fig_3_20b_gates is
    component nand2_gate port (y: out Std_Logic; xin1, xin2: in Std_Logic);
    component inv_or2_gate port (y: out Std_Logic; xin1, xin2: in Std_Logic);
begin
    G1:  nand2_gate port map (w1 => y, C => xin1, D => cin2);
    G2:  inv_or2_gate port map (w2 => y, w1 => xin1, B => xin2);
    G3:  nand2_gate port map w3 => y, w2 => xin1, A => xin2);
    G4:  nand2_gate port map (w4 => y, B => xin1, C_bar => xin2);
    G5:  inv_or2_gate port map (F => y, w3 => xin1, w4 => xin2);
end Gates;

entity inv_or2_gate is
    port (y: out Std_logic; xin1, xin2: in Std_Logic);
end nand2_gate;

architecture Behavior of inv_or2_gate is
begin
    y <= not (not(xin1) and not(xin2));
end Behavioral;

entity nand2_gate is
    port (y: out Std_logic; xin1, xin2: in Std_Logic);
end nand2_gate;

architecture Behavior of nand2_gate is
begin
    y <= not (xin1 and xin2);
end Behavioral;
```

(c) **Verilog**

```
module Fig_3_21a_gates (F, A, A_bar, B, B_bar, C, D_bar);
 output   F;
 input  A, A_bar, B, B_bar, C, D_bar;
 wire       w1, w2, w3, w4;
 and    (w1, A, B_bar);
 and    (w2, A_bar, B);
 or     (w3, w1, w2);
 or     (w4, C, D_bar);
 and    (F, w3, w4);
endmodule
```

VHDL

```
entity Fig_3_21a_gates is
   port (F: out Std_Logic; A, S_bar, B, B_bar, C, D_bar: in Std_Logic);
end Fig_3_

architecture Gates of Fig_3_24 is
   component and2_gate port (y: out Std_Logic; xin1, xin2: in
Std_Logic);
   component or2_gate port (y: out Std_Logic; xin1, xin2: in
Std_Logic);
   signal w1, w2, w3, w4: Std_Logic
begin
   G1: and2_gate port map( w1 => y, A => xin1, B_bar => xin2);
   G2: and2_gate port map( w2 => y; A_bar => xin1, B => xin2);
   G3: or2_gate port map( w3 => y, w1 => xin1, w2 => xin2);
   G4: or2_gate port map( w4 => y, C => xin1, D_bar => xin2);
   G5: and2_gate port map( F => y, w3 => xin1, w4 => xin2);
end Gates;
```

(d) **Verilog**

```
module Fig_3_21b_gates (output F, input A, A_bar, B, B_bar, C_bar,
D);
 wire         w1, w2, w3, w4, F_bar;
 nand       (w1, A, B_bar);
 nand       (w2, A_bar, B);
 inv_or2      (w3, w1, w2);
 inv_or2      (w4, C_bar, D);
 nand       (F_bar, w3, w4);
 not        (F, F_bar);
```

```
endmodule

module inv_or2 (y, xin1, xin2);
   assign y = (!xin1) || (!xin2);
endmodule
```

VHDL

```
entity Fig_3_21b_gates is
   port (F: out Std_Logic; A, A_bar, B, B_bar, C_bar, D: in Std_Logic);
end Fig_3_21b;

architecture Gates of Fig_3_21b is
   component nand2_gate port (y: out: xin1, xin2: in Std_Logic);
   component inv_or2 port (y: out Std_Logic; xin1: xin2: in
Std_Logic);

   component inv_gate port (y: out Std_Logic; x: in Std_logic);
   signal w1, w2, w3, w4, F_bar: Std_Logic;
begin
   G1: nand2_gate port map (w1 => y, A => xin1, B_bar => xin2);
   G2: nand2_gate port map (w2 => y, A_bar => xin1, B => xin2);
   G3: inv_or2 port map (w3 => y, w1 => xin1, w2 => xin2);
   G4: inv_or2 port map (w4 => y, C_ba => x_in1, D => xin2);
   G5: nand2_gate port map (F_bar  => y; => w3 xin1, w4 => xin2);
   G6: inv_gate port map (F => y; F_bar => x);
end Gates;
```

(e) Verilog

```
module Fig_3_24_gates (F, A, A_bar, B, B_bar, C, D, E_bar);
 output    F;
 input  A, B, C, D, E_bar;
 wire       w1, w2;
 nor (w1, A, B);
 nor (w2, C, D);
 inv3_and M0 (F, w1, w2, w3);

endmodule

module  inv3_and (output y, input xin1, xin2, xin3);
   assign y = (!xin1) && (!xin2) && (!xin3);
endmodule
```

VHDL

```
entity Fig_3_24_gates is
   port (F: out Std_Logic; A, A_bar, B, B_bar, C, D, E_bar: in
Std_Logic);
end Fig_3_24_gates;
```

```
architecture Gates of Fig_3_24 is
   component nor2_gate part (y: out Std_Logic; xin1, xin2: in
Std_Logic);
   component inv3_and_gate (y: out Std_Logic; xin1, xin2, xin3: in
Std_Logic);
   signal w1, w2: Std_Logic;
begin
   G1: nor2_gate port map (w1 => y; A => xin1, B => xin2);
   G2: nor2_gate port map (w2=> y, C => xin1, D => xin2);
   G3: inv3_and_gate port map (F => y, w1 => xin1, w2 => xin2, E_bar
=> xin3););
end Gates;

entity inv3_and_gate is
   port (xin1, xin2, xin3: in Std_Logic; y: out Std_Logic);
end inv3_and_gate;

architecture Operators of inv3_and_gate is
begin
   y <= (not xin1) and (not(xin2)) and (not(xin3));
end Operators;
```

(f) **Verilog**

```
module Fig_3_25_gates (F, A, A_bar, B, B_bar, C, D_bar);
 output    F;
 input  A, A_bar, B, B_bar, C, D_bar;
 wire      w1, w2, w3, w4;

 and    (w1, !A_bar, !B);    // alternative would use inverters
 and    (w2, A, !B_bar);
 nor    (w4, C, D_bar);
 nor    (w3, w1, w2);
 and    (F, !w3, !w4);
endmodule
```

VHDL

```
entity Fig_3_25 is
   port (F: out Std_Logic; A, A_bar, B, B_bar, C, D_bar: in Std_logic);
end Fig_3_25;

architecture Gates of Fig_3_25 is
   component inv2_and_gate port (y: out Std_Logic, xin1, xin2: in
Std_Logic);
```

```
    component nor2_gate port (y: out Std_Logic; xin1, xin2: in
Std_Logic);
    signal w1, w2, w3, w4: Std_Logic;
begin
    G1: inv_or2 port map (w1 => y, A_bar => xin1, B => xin2);
    G2: inv_or2 port map (w2 => y, A => xin1, B_bar => xin2);
    G3: nor2_gate port map (w4 => y, C => xin1, D_bar => xin2);
    G4: nor2_gate (w3 => y, w1 => xin1, w2 => xin2);
    G5: inv_and2 port map (F => y, w3 => xin1, w4 => xin2);
end Gates;

entity inv2_and_gate is
    port (y: out Std_Logic; xin1, xin2: in Std_Logic;);
end inv3_and_gate;

architecture Operators of inv3_and_gate is
begin
    y <= (not xin1) and (not xin2);
end Operators;
```

3.32 Note: It is assumed that a complemented input is generated by another circuit that is not part of the circuit that is to be described.

Note: Because the signals here are all scalar–valued, the logical operators (!, &&, and ||) are equivalent to the bitwise operators (~, &, |). If the operands are vectors the bitwise operators produce a vector result; the logical operators would produce a scalar result (i.e., true or false).

(a) **Verilog/SystemVerilog**

```
module Fig_3_20a_CA (F, A, B, C, C_bar, D);
  output    F;
  input  A, B, C, C_bar, D;
  wire      w1, w2, w3, w4;
  assign    w1 = C && D;
  assign    w2 = w1 || B;
  assign    w3 = (w2 && A);
  assign    w4 = (B && C_bar);
  assign    F = w3 || w4);
endmodule
```

VHDL

```
entity Fig_3_20a_SA is
    port (A, B_bar, C, D: in Std_Logic);
end
```

```
architecture Sig_Assign of Fig_3_20a_SA is
   signal w1, w2,w3, w4: Std_Logic;
begin
   w1 <= C and D;
   w2 <= B or w1;
   w3 <= B and A;
   w4 <= B and C_bar;
   F <= w3 or w4;
end Sig_Assign;
```

(b) **Verilog**

```
module Fig_3_20b_CA (F, A, B, C, C_bar, D);
 output   F;
 input    A, B, B_bar, C, C_bar, D;
 wire     w2 = !w1;
 wire     w3 = !B_bar;
 wire     w4, w5, w5_bar, w6, w6_bar;
 assign   w1 = !(C && D);
 assign   w4 = w2 || w3;
 assign   w5 = !(w4 && A);
 assign   w5_bar = !w5;
 assign   w6 = !(B && C_bar);
 assign   w6_bar = !w6;
 assign   F = w5_bar || w6_bar;
endmodule
```

VHDL

```
entity Fig_3_20b_SA is
   port (F: out Std_Logic; A, B, B_bar, C, C_bar, D: in Std_Logic);
end

architecture Sig_Assign of Fig_3_20b_SA is
   signal w1, w2, w3, w4, w5, w5_bar, w6, w6_bar: Std_Logic;
begin
   w1 <= not (C and D);
   w2 <= not w1;
   w3 <= not B_bar;
   w4 <= w2 or w3;
   w5 <= not (w4 and A);
   w5_bar <= not w5;
   w6 <= not (B and C_bar);
   w6_bar <= not w6;
   F <= w5_bar or w6_bar;
end Sig_Assign;
```

(c) **Verilog**

```
module Fig_3_21a_CA (F, A, A_bar, B, B_bar, C, D_bar);
 output   F;
 input    A, A_bar, B, B_bar, C, D_bar;
 wire     w1, w2, w3, w4;
 assign   w1 = A && B_bar;
 assign   w2 = A_bar && B;
 assign   w3 = w1 || w2);
 assign   w4 = C || D_bar;
 assign   F = w3 || w4;
endmodule
```

VHDL

```
entity Fig_3_21a_SA is
   port (F: out Std_Logic; A, A_bar, B, B_bar, C, D_bar: in Std_Logic);
end
architecture Sig_Assign of Fig_3_21a_SA is
   signal w1, w2, w3, w4: Std_Logic);
begin
   w1 <= A and B_bar;
   w2 <= A_bar and B;
   w3 <= w1 or w2;
   w4 <= C or D_bar;
   F <= w3 or w4;
end Sig_Assign;
```

(d)

```
   module Fig_3_21b_CA (F, A, A_bar, B, B_bar, C_bar, D);
 output   F;
 input    A, A_bar, B, B_bar, C_bar, D;
 wire     w1, w2, w1_bar, w2_bar, w3, w4, w5, w6, F_bar;
 assign   w1 = !(A && B_bar);
 assign   w2 = !(A_bar && B);
 assign   w1_bar = !w1;
 assign   w2_bar = !w2;
 assign   w3 = w1_bar || w2_bar;
 assign   w4 = !C_bar;
 assign   w5 = !D;
 assign   w6 = w4 || w5;
 assign   F_bar = !(w3 && w6);
 assign   F = !F_bar;
endmodule
```

VHDL

```
entity Fig_3_21b_SA is
   port (F: out Std_Logic; A, A_bar, B, B_bar, C, D: in Std_Logic);
end
architecture Sig_Assign of Fig_3_21b_SA is
   signal w1, w1_bar, w2, w2_bar, w3, w4, w5, w6, F_bar: Std_Logic);
begin
   w1 <= not (A and B_bar);
   w2 <= not (A_bar and B);
   w2_bar <= !w2;
   w1_bar <= not w1;
   w3 <= w1_bar or w2_bar;
   w4 <= not C_bar;
   w5 <= not D;
   w6 <= w4 or w5;
   F_bar <= not (w3 and w6);
   F <= not F_bar;
end Sig_Assign;
```

(e) **Verilog**

```
module Fig_3_24_CA (F, A, B, C, D, E_bar);
  output    F;
  input     A, B, C, D, E_bar;
  wire      w1, w2, w1_bar, w2_bar, w3_bar;
  assign    w1 = !(A || B);
  assign    w1_bar = !w1;
  assign    w2 = !(C || D);
  assign    w2_bar = !w2;
  assign    w3 = !E_bar;
  assign    F = w1_bar && w2_bar && w3;
endmodule
```

VHDL

```
entity Fig_3_24_SA is
   port (F: out Std_Logic; A, B, C, D, E_bar: in Std_Logic);
end Fig_3_24_SA;

architecture Sig_Assign of Fig_3_24_SA is
  signal w1, w2, w1_bar, w2_bar, w3_bar: Std_Logic;
```

```
begin
   w1 <= not (A or B);
   w1_bar <=  not w1;
   w2 <= not (C or D);
   w2_bar <= not w2;
   w3 <= not E_bar;
   F <= w1_bar and w2_bar and w3;
endmodule
```

(f) **Verilog**

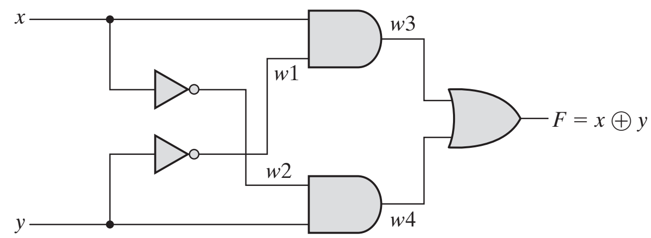
```
module Fig_3_25_CA (F, A, A_bar, B, B_bar, C, D_bar);
 output    F;
 input  A, A_bar, B, B_bar, C, D_bar;
 wire       w1, w2, w3, w4, w5, w6, w7, w8, w9, w10;
 assign    w1 = !A _bar;
 assign    w2 = !B;
 assign    w3 = w1 && w2;
 assign    w4 = !A;
 assign    w5 = !B_bar;
 assign    w6 = w4 && w5;
 assign    w7 = !(C || D_bar);
 assign    w8 = !(w3 || w6);
 assign    w9 = !w8;
 assign    w10 =  !w7;
 assign    F = w9 && w10;
endmodule
```

VHDL

```
entity Fig_3_25_CA is
 port (F: out Std_Logic; A, A_bar, B, B_bar, C, D_bar: in Std_Logic);
end Fig_3_25_CA;

architecture Sig_Assign of Fig_3_25 _SA is
   signal  w1, w2, w3, w4, w5, w6, w7, w8, w9, w10: Std_Logic;
   w1 <= not A _bar;
   w2 <= not B;
   w3 <= w1 and w2;
   w4 <= not A;
   w5 <= not B_bar;
   w6 <= w4 and w5;
   w7 <= not (C or D_bar);
   w8 <= not (w3 or w6);
   w9 <= not w8;
   w10 <=  not w7;
   F <= w9 and w10;
endmodule
```

3.33 **(a)**

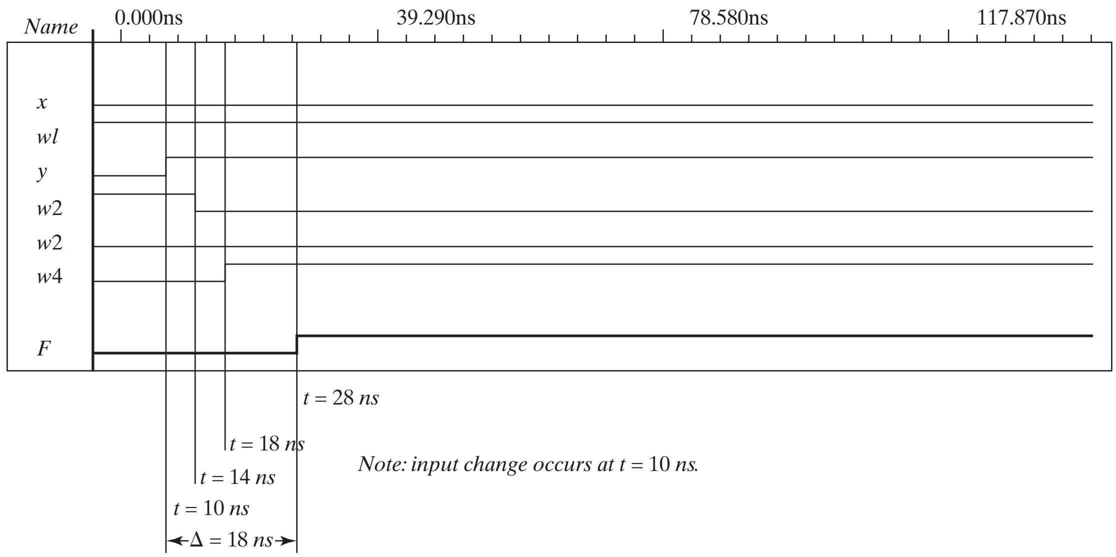
Initially, with $xy = 00$, $w1 = w2 = 1$, $w3 = w4 = 0$ and $F = 0$. $w1$ should change to 0 4 ns after xy changes to 01. $w4$ should change to 18 ns after xy changes to 01. F should change from 0 to 10 ns after $w4$ changes from 0 to 1, i.e., 18 ns after xy changes from 00 to 01.



(b)

```
`timescale 1ns/1ps

module Prob_3_33 (output F, input x, y);
wire   w1, w2, w3, w4;

and #8 (w3, x, w1);
not #4 (w1, x);
and#8 (w4, y, w1);
not #4 (w2, y);
or  #10 (F, w3, w4);

endmodule


module t_Prob_3_33 ();

   reg x, y;

   wire F;

Prob_3_33 M0 (F, x, y);

initial #200 $finish;
initial fork
  x = 0;
  y = 0;
  #20 y = 1;
join
endmodule
```

(c) To simulate the circuit, it is assumed that the inputs xy = 00 have been applied sufficiently long for the circuit to be stable before xy = 01 is applied. The testbench sets xy = 00 at t = 0 ns, and xy = 01 at t = 10 ns. The simulator assumes that xy = 00 has been applied long enough for the circuit to be in a stable state at t = 0 ns, and shows F = 0 as the value of the output at t = 0. the waveforms show the response to xy = 01 applied at t = 10 ns.



3.34

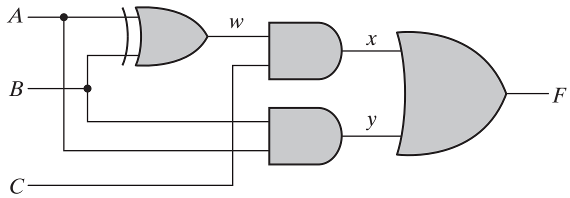
```
module Prob_3_34 (Out_1, Out_2, Out_3, A, B, C, D);
  output      Out_1, Out_2, Out_3;
  input              A, B, C, D;
  wire        A_bar, B_bar, C_bar, D_bar;
  assign      A_bar = ~A;
  assign      B_Bar = ~B;
  assign      C_bar = ~C;
  assign      D_bar = ~D;
  assign      Out_1 = ~( (C | B) & (A_bar | D) & B );
  assign      Out_2 = ((C & B_bar) | (A & B & C) | (C_bar & B) ) & (A | D_bar);
  assign      Out_3 = C & ( (A & D) | B ) | (C & A_bar);
endmodule
```

3.35

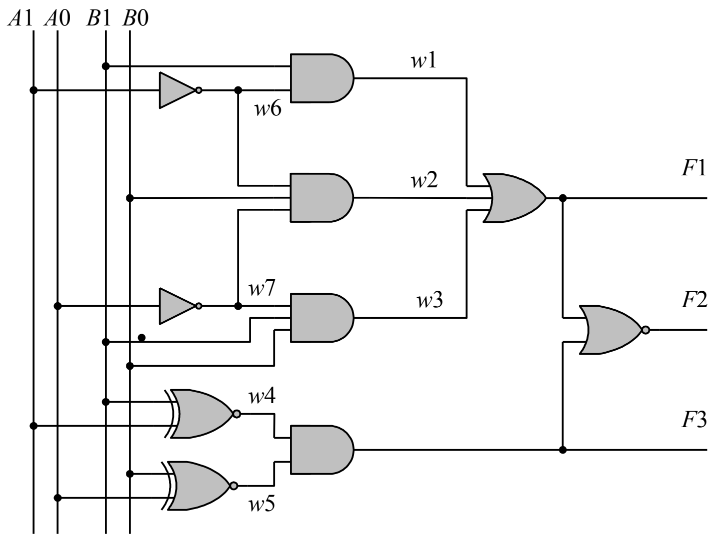
- **Line 1:** Dash not allowed in the name of module and use underscore instead such as Exmpl_3. Terminate the line with semicolon (;).
- **Line 2:** The term **inputs** should be **input** (no 's' at the end). Output is declared but does not appear in the port list and should be followed by a comma if it is intended to be in the list of inputs. If Output is a misspelling of **output** and is to declare output ports, C should be followed by a semicolon (;) and F should be followed by a semicolon (;).
- **Line 3:** C cannot be declared as both input (Line 2) and output (Line 3). Terminate the line with a semicolon (;).
- **Line 5:** A cannot be an output of the primitive as it is declared as an input to the module.
- **Line 6:** Too many inputs for the **not** gate.
- **Line 7: AND** must be in lowercase, change to **and**.
- **Line 8:** Remove semicolon (;) from the end (no semicolon after **endmodule**).

Line 7: Remove semicolon (no semicolon after endmodule).

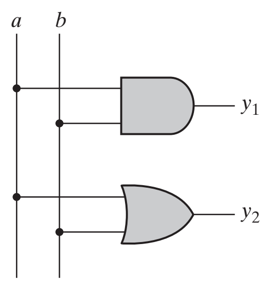
3.36 **(a)**



(b)



(c)



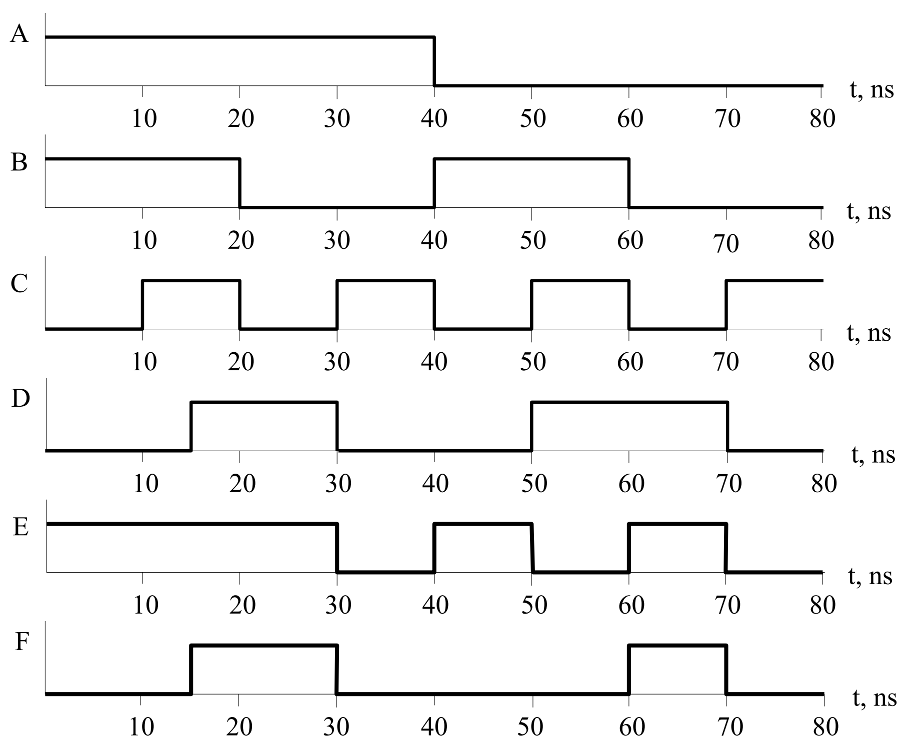
3.37

```
UDP_Majority_4 (y, a, b, c, d);
 output     y;
 input      a, b, c, d;
 table
 // a  b  c  d  :  y
     0  0  0  0  :  0;
     0  0  0  1  :  0;
     0  0  1  0  :  0;
     0  0  1  1  :  0;
     0  1  0  0  :  0;
     0  1  0  1  :  0;
     0  1  1  0  :  0;
     0  1  1  1  :  1;

     1  0  0  0  :  0;
     1  0  0  1  :  0;
     1  0  1  0  :  0;
     1  0  1  1  :  0;
     1  1  0  0  :  0;
     1  1  0  1  :  0;
     1  1  1  0  :  1;
     1  1  1  1  :  1;
  endtable
 endprimitive
```

3.38

```
module t_Circuit_with_UDP_02467;
 wire  E, F;
 reg   A, B, C, D;
 Circuit_with_UDP_02467 m0 (E, F, A, B, C, D);

 initial #100 $finish;
 initial fork
  A = 0; B = 0; C = 0; D = 0;
  #40 A = 1;
  #20 B = 1;
  #40 B = 0;
  #60 B = 1;
  #10 C = 1; #20 C = 0; #30 C = 1; #40 C = 0; #50 C = 1; #60 C = 0; #70 C = 1;
  #20 D = 1;
 join
endmodule
```

```
// Verilog model: User-defined Primitive
primitive UDP_02467 (D, A, B, C);
  output D;
  input A, B, C;
// Truth table for D = f (A, B, C) = S (0, 2, 4, 6, 7);
  table
//    A  B  C  :   D  // Column header comment
      0  0  0  :   1;
      0  0  1  :   0;
      0  1  0  :   1;
      0  1  1  :   0;
      1  0  0  :   1;
      1  0  1  :   0;
      1  1  0  :   1;
      1  1  1  :   1;
  endtable
endprimitive
// Verilog model: Circuit instantiation of Circuit_UDP_02467
module Circuit_with_UDP_02467 (e, f, a, b, c, d);
  output    e, f;
  input     a, b, c, d;

 UDP_02467   M0 (e, a, b, c);
  and            (f, e, d);          //Option gate instance name omitted
endmodule
```



3.39

a	b	s	c
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

s = a'b + ab' = a ^ b
c = ab = a && b

```
module Prob_3_39 (s, c, a, b);
  input a, b;
  output s, c;

  xor (s, a, b);
  and (c, a, b);
endmodule
```

3.40

```
entity Prob_3_39 is
port ( s, c: out Std_Logic; a, b: in Std_Logic);
end Prob_3_39;


architecture Gates of Prob_3_40 is
    component xor2_gate port (y: out Std_Logic; xin1, xin2: in Std_Logic);
    component and2_gate port (y: out Std_Logic; xin1, xin2: in Std_Logic);
begin
    G1: xor2_gate port map (s => y, a => xin1, b => xin2);
    G2: and2_gate port map (c => y, a => xin1, b => xin2);
end Gates;

entity xor2_gate is
    port (y: out Std_logic; xin1, xin2: in Std_Logic);
end xor2_gate;

architecture Behavior of xor2_gate is
begin
    y <= xin1 xor xin2;
end Behavioral;

entity and2_gate is
    port (y: out Std_logic; xin1, xin2: in Std_Logic);
end and2_gate;

architecture Behavior of and2_gate is
begin
    y <= xin1 and xin2;
end Behavioral;
```

CHAPTER 4

4.1

(a)

$T_1 = B'C$

$T_2 = A'B$

$T_3 = A + T_1 = A + B'C$

$T_4 = D \oplus T_2 = D \oplus (A'B) = A'BD' + D(A + B') = A'BD' + AD + B'D$

$F_1 = T_3 + T_4 = A + B'C + A'BD' + AD + B'D$

With $A + AD = A$ and $A + A'BD' = A + BD'$:

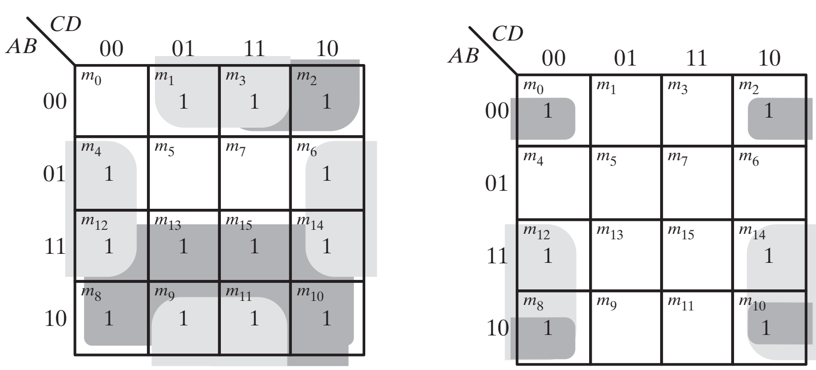
$F_1 = A + B'C + BD' + B'D$

$F_2 = (T_2 + D)' = AD' + B'D'$ $\quad F_2 = AD' + B'D'$

(b)

A	B	C	D	T_1	T_2	T_3	T_4	F_1	F_2
0	0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	1	1	0
0	0	1	0	1	0	1	0	1	1
0	0	1	1	1	0	1	1	1	0
0	1	0	0	0	1	0	1	1	0
0	1	0	1	0	1	0	0	0	0
0	1	1	0	0	1	0	1	1	0
0	1	1	1	0	1	0	0	0	0
1	0	0	0	0	0	1	0	1	1
1	0	0	1	0	0	1	1	1	0
1	0	1	0	1	0	1	0	1	1
1	0	1	1	1	0	1	1	1	0
1	1	0	0	0	0	1	0	1	1
1	1	0	1	0	0	1	1	1	0
1	1	1	0	0	0	1	0	1	1
1	1	1	1	0	0	1	1	1	0

(c)



$F_1 = A + B'C + BD' + B'D$ $\qquad$ $F_2 = AD' + B'D'$

4.2 $F = ABC + AD'$

$G = AB' + AC' + A'D$

4.3 **(a)** $Y = I_0S_1'S_0' + I_1S_1'S_0 + I_2S_1S_0' + I_3S_1S_0$

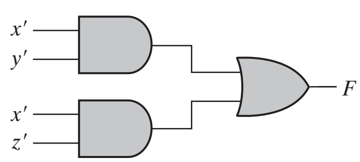
(b) 64 rows and 7 columns; 6 inputs ($2^6 = 64$) and one output.

4.4 **(a)**

x	y	z	F
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

On simplifying, we get:

$F = x'y' + x'z'$

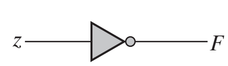


(b)

x	y	z	F
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

On simplifying, we get:

$F = z'$

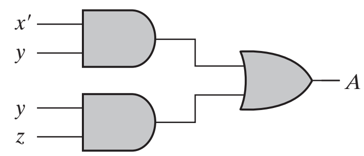


4.5

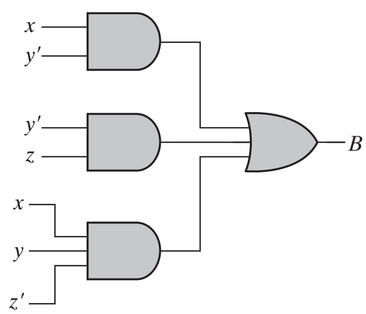
x	y	z	A	B	C
0	0	0	0	1	0
0	0	1	0	1	1
0	1	0	1	0	0
0	1	1	1	0	1
1	0	0	0	0	1
1	0	1	0	1	0
1	1	0	0	1	1
1	1	1	1	0	0

On simplifying, we get:

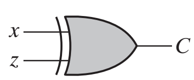
$A = x'y + yz$



$B = x'y' + y'z + xyz'$



$C = x'z + xz'$

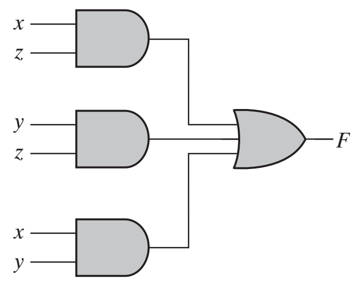


4.6 **(a)**

x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

On simplifying, we get:

$F = xz + yz + xy$



(b)

```
module Prob_4_6 (x, y, z, F);
  input        x, y, z;
  output   F;
  wire         w1, w2, w3;
  and          gate1 (w1, x, z);
  and          gate2 (w2, y, z);
  and          gate3 (w3, x ,y);
  or       gate4 (F, w1, w2, w3);
endmodule
```

4.7 **(a)**

A	B	C	D	w	x	y	z
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

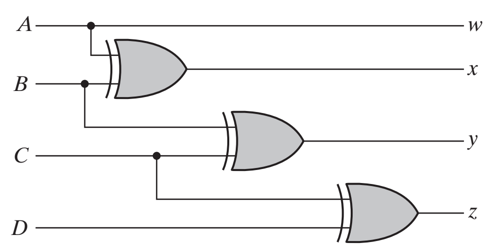
On simplifying, we get:

$w = A$

$x = A \oplus B$

$y = B \oplus C$

$z = C \oplus D$



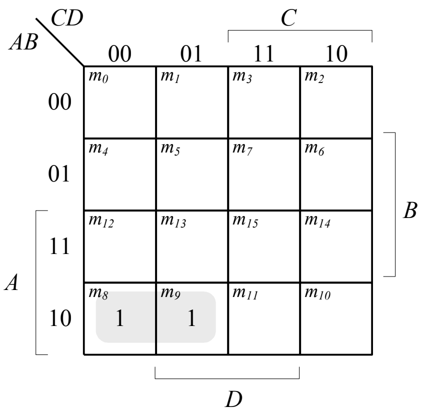
(b)

```
module Prob_4_7(output w, x, y, z, input A, B, C, D);
  assign   w = A;
  assign   x = A ^ B;
  assign   y = B ^ C;
  assign   z = C ^ D;
endmodule
```

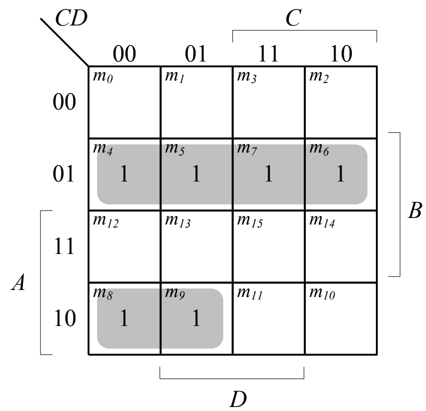
4.8 **(a)** The 8-4-2-1 code (Table 1.5) and the BCD code (Table 1.4) are identical for digits 0 – 9.

(b)

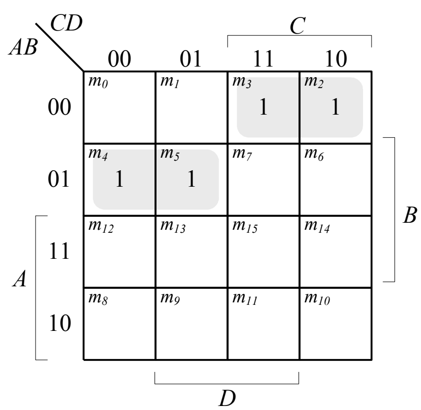
8421 *ABCD*	*Gray* *wxyz*
0000	0000
0001	0001
0010	0011
0011	0010
0100	0110
0101	0111
0110	0101
0111	0100
1000	1100
1001	1101



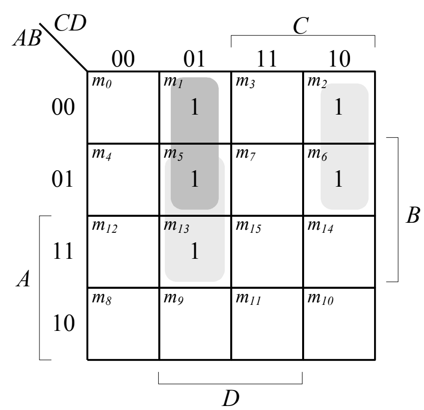
$w = AB'C'$



$x = AB'C' + A'B$



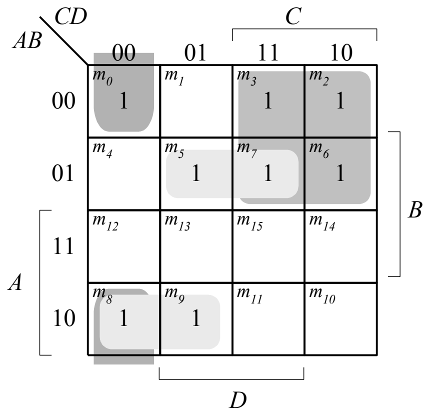
$y = A'BD' + A'B'D$



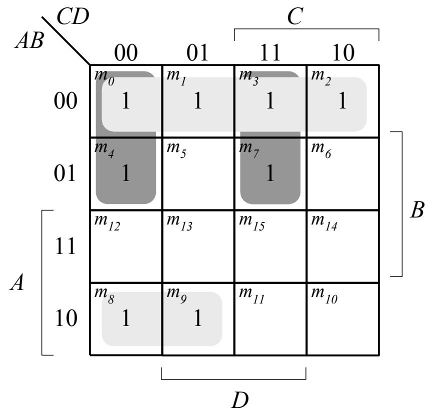
$z = A'C'D + BC'D + A'CD'$

4.9

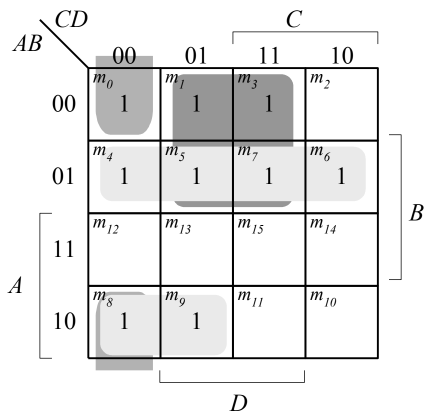
ABCD	*a*	*b*	*c*	*d*	*e*	*f*	*g*
0000	1	1	1	1	1	1	0
0001	0	1	1	0	0	0	0
0010	1	1	0	1	1	0	1
0011	1	1	1	1	0	0	1
0100	0	1	1	0	0	1	1
0101	1	0	1	1	0	1	1
0110	1	0	1	1	1	1	1
0111	1	1	1	0	0	0	0
1000	1	1	1	1	1	1	1
1001	1	1	1	1	0	1	1



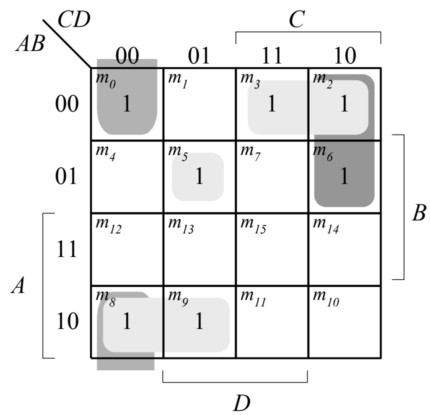
$a = A'C + A'BD + B'C'D' + AB'C'$



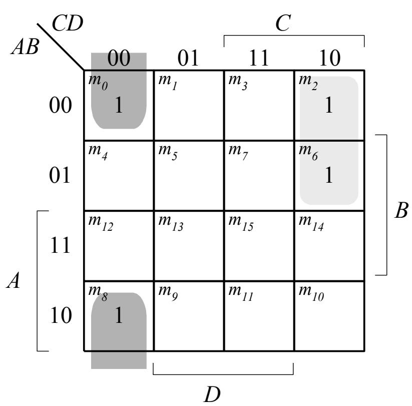
$b = A'B' + A'C'D' + A'CD + AB'C'$



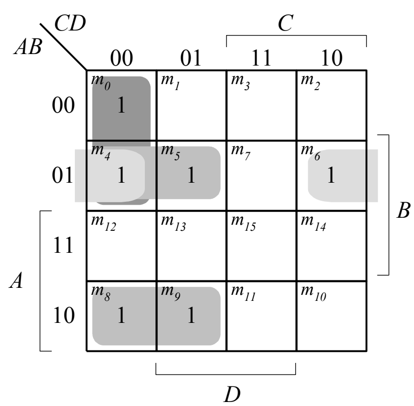
$c = A'B + A'D + B'C'D' + AB'C'$



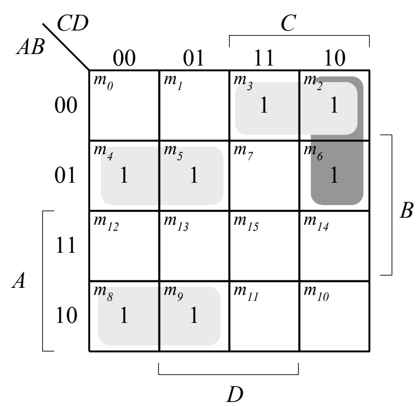
$d = A'CD' + A'B'C + B'C'D' + AB'C' + A'BC'D$



$e = A'CD' + B'C'D'$



$f = A'BC' + A'C'D' + A'BD + AB'C'$



$g = A'CD' + A'B'C + A'BC' + AB'C'$

4.10

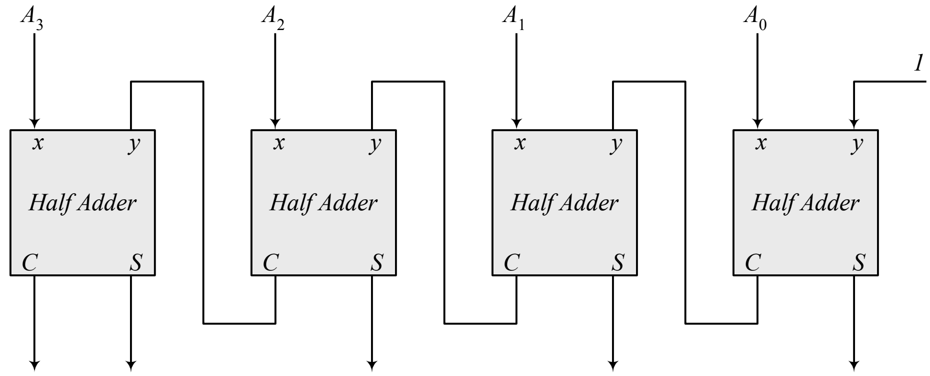
Binary				BCD				
A	*B*	*C*	*D*	*v*	*w*	*x*	*y*	*z*
0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	1
0	0	1	0	0	0	0	1	0
0	0	1	1	0	0	0	1	1
0	1	0	0	0	0	1	0	0
0	1	0	1	0	0	1	0	1
0	1	1	0	0	0	1	1	0
0	1	1	1	0	0	1	1	1
1	0	0	0	0	1	0	0	0
1	0	0	1	0	1	0	0	1
1	0	1	0	1	0	0	0	0
1	0	1	1	1	0	0	0	1
1	1	0	0	1	0	0	1	0
1	1	0	1	1	0	0	1	1
1	1	1	0	1	0	1	0	0
1	1	1	1	1	0	1	0	1

On simplifying, we get:

$v = A(B + C)$
$w = AB'C'$
$x = B(A' + C)$
$y = A'C + ABC'$
$z = D$

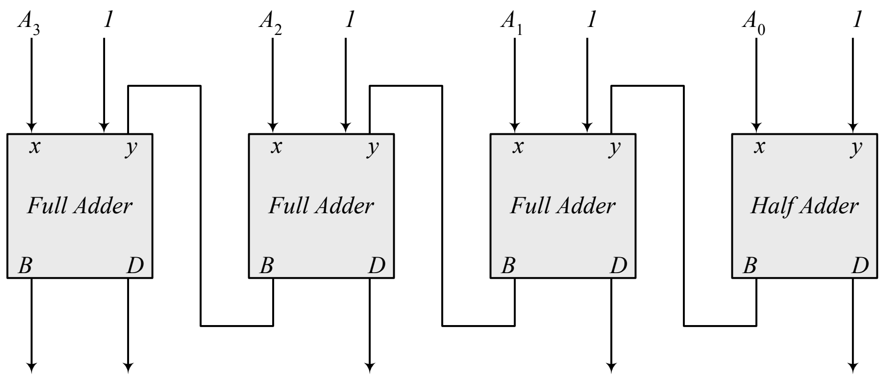
The reason behind the need for 5-bits in the output is that representing the numbers 10 through 15 in BCD requires 5-bits.

4.11 **(a)**



Note: 5-bit output

(b)



Note: To decrement the 4-bit number, add -1 to the number. In 2's complement format (add F_h) to the number. An attempt to decrement 0 will assert the borrow bit. For waveforms, see solution to Problem 4.52.

4.12

(a)

x y	B D
0 0	0 0
0 1	1 1
1 0	0 1
1 1	0 0

$D = x'y + xy'$
$B = x'y$

(b)

x y B_{in}	B D
0 0 0	0 0
0 0 1	1 1
0 1 0	1 1
0 1 1	1 0
1 0 0	0 1
1 0 1	0 0
1 1 0	0 0
1 1 1	1 1

$Diff = x \oplus y \oplus z$
$B_{out} = x'y + x'z + yz$

4.13

	Sum	C	V
(a)	1100	0	1
(b)	0010	1	1
(c)	0100	1	0
(d)	1011	0	1
(e)	1111	0	0

4.14 XOR AND OR XOR

8 + 4 + 4 + 8 = 24 ns

4.15 C_0= Input carry

$C_1 = G_0 + P_0C_0$

$C_2 = G_1 + P_1C_1 = G_1 + P_1G_0 + P_1P_0C_0$

$C_3 = G_2 + P_2C_2 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$

$C_4 = G_3 + P_3C_3 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$

4.16 **(a)**

$(C'G'_i + p'_i)' = (C_i + G_i)P_i = G_iP_i + P_iC_i$
$= A_iB_i(A_i + B_i) + P_iC_i$
$= A_iB_i + P_iC_i = G_i + P_iC_i$
$= A_iB_i + (A_i + B_i)C_i = A_iB_i + A_iC_i + B_iC_i = C_{i+1}$

$(P_iG'_i) \oplus C_i = (A_i + B_i)(A_iB_i)' \oplus C_i = (A_i + Bi)(A'_i + B'_i) \oplus C_i$
$= (A'_iB_i + A_iB'_i) \oplus C_i = A_i \oplus B_i \oplus C_i = S_i$

(b)

Output of NOR gate = $(A_0 + B_0)' = P'_0$
Output of NAND gate = $(A_0B_0)' = G'_0$
$S_1 = (P_0G'_0) \oplus C_0$
$C_1 = (C'_0G'_0 + P'_0)'$ as defined in part (a)

4.17 **(a)**

$(C'_iG'_i + P'_i)' = (C_i + G_i)P_i = G_iP_i + P_iC_i = A_iB_i(A_i + B_i) + P_iC_i$
$= A_iB_i + P_iC_i = G_i + P_iC_i$
$= A_iB_i + (A_i + B_i)C_i = A_iB_i + A_iC_i + B_iC_i = C_{i+1}$

$(P_iG'_i)\oplus C_i = (A_i + B_i)(A_iB_i)'\oplus C_i = (A_i + B_i)(A'_i + B'_i)\oplus C_i$
$= (A'_iB_i + A_iB'_i)\oplus C_i = A_i\oplus B_i\oplus C_i = S_i$

(b)

Output of NOR gate = $(A_0 + B_0)' = P'_0$

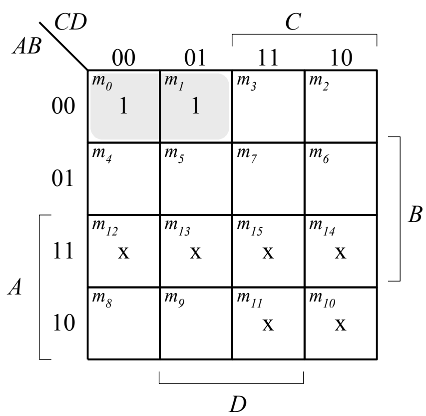
Output of NAND gate = $(A_0B_0)' = G'_0$

$S_0 = (P_0G'_0) \oplus C_0$
$C_1 = (C'_0G'_0 + P'_0)'$ as defined in part (a)

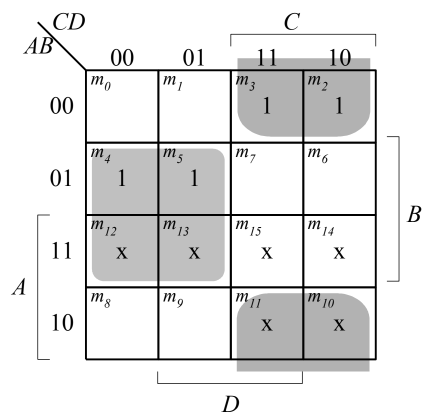
4.18

Inputs $ABCD$	Outputs $wxyz$
0000	1001
0001	1000
0010	0111
0011	0110
0100	0101
0101	0100
0110	0011
0111	0010
1000	0001
1001	0000

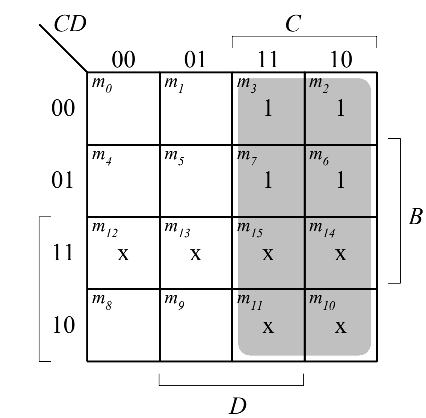
$d(A, b, c, d) = \Sigma(10, 11, 12, 13, 14, 15)$



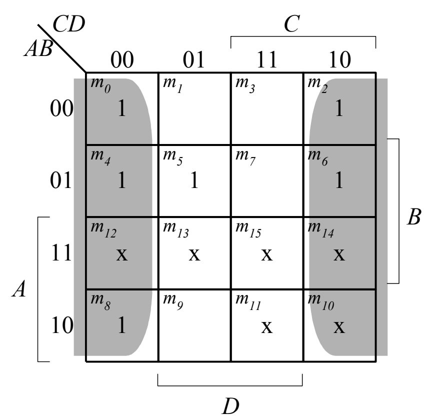
$w = A'B'C'$



$x = BC' + B'C = B \oplus C$



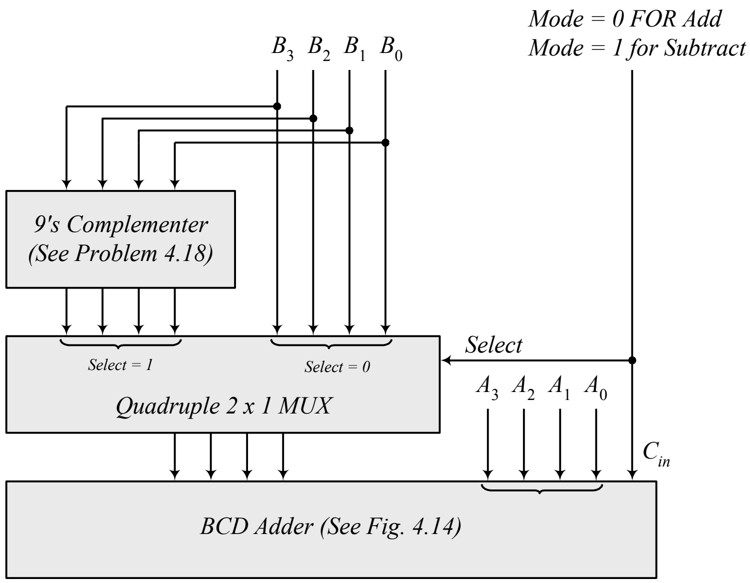
$y = C$



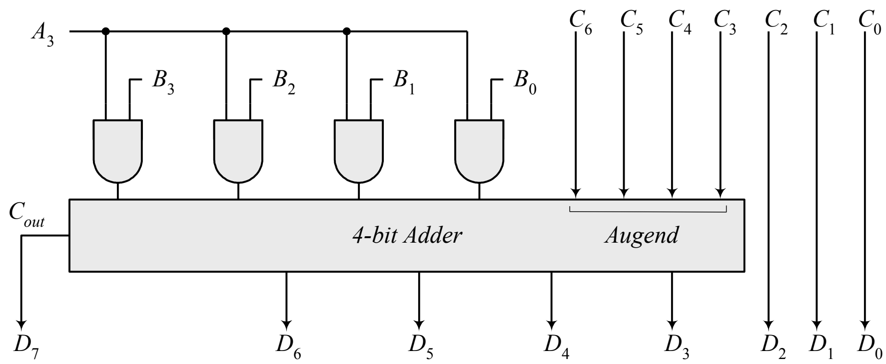
$z = D'$

(b) $w = A'B' + A'C'D'$
$x = B'C'D'$
$y = B'(C'D + CD') = B'(C \oplus D)$
$z = D$

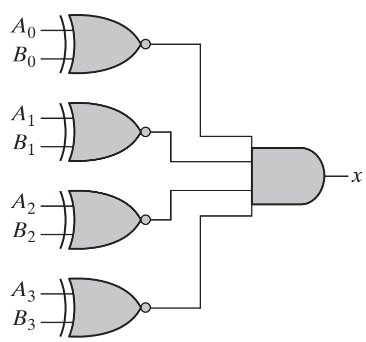
4.19



4.20 Combine the following circuit with the 4-bit binary multiplier circuit of Fig. 4.16.



4.21



$$x = (A_0 \oplus B_0)'\,(A_1 \oplus B_1)'\,(A_2 \oplus B_2)'\,(A_3 \oplus B_3)'$$

4.22

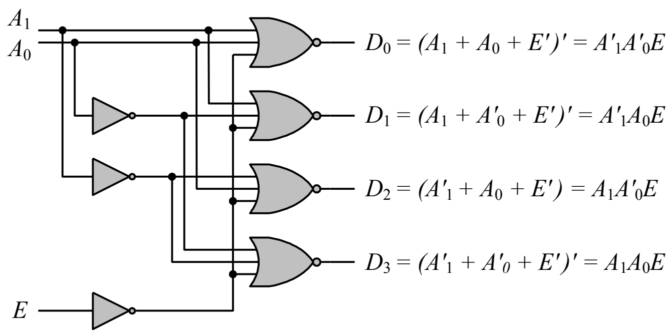
BCD				XS-3			
A	*B*	*C*	*D*	*w*	*x*	*y*	*z*
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

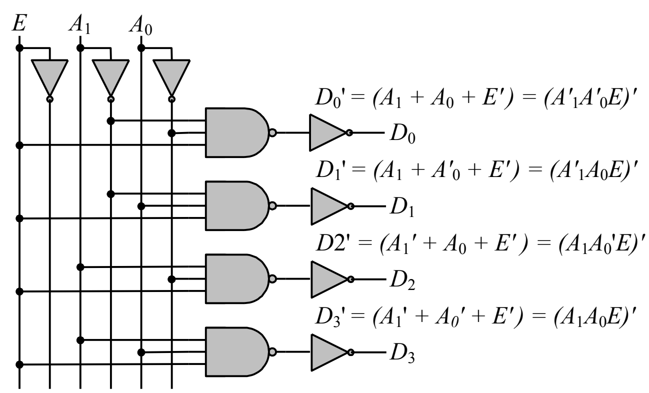
On simplifying, we get:

$w = A + BC + BD$
$x = B'C + B'D + BC'D'$
$y = C'D' + CD$
$z = D'$

4.23

D0 = A1'A0' = (A1 + A0)' (NOR) D0' = (A1'A0')' (NAND)
D1 = A1'A0 = (A1 + A0')' (NOR) D1' = (A1'A0)' (NAND)
D2 = A1A0' = (A1' + A0)' (NOR) D2' = (A1A0')' (NAND)
D3 = A1A0 = (A1' + A0)' (NOR) D0' = (A1A0)' (NAND)





4.24

Inputs: *A, B, C, D*

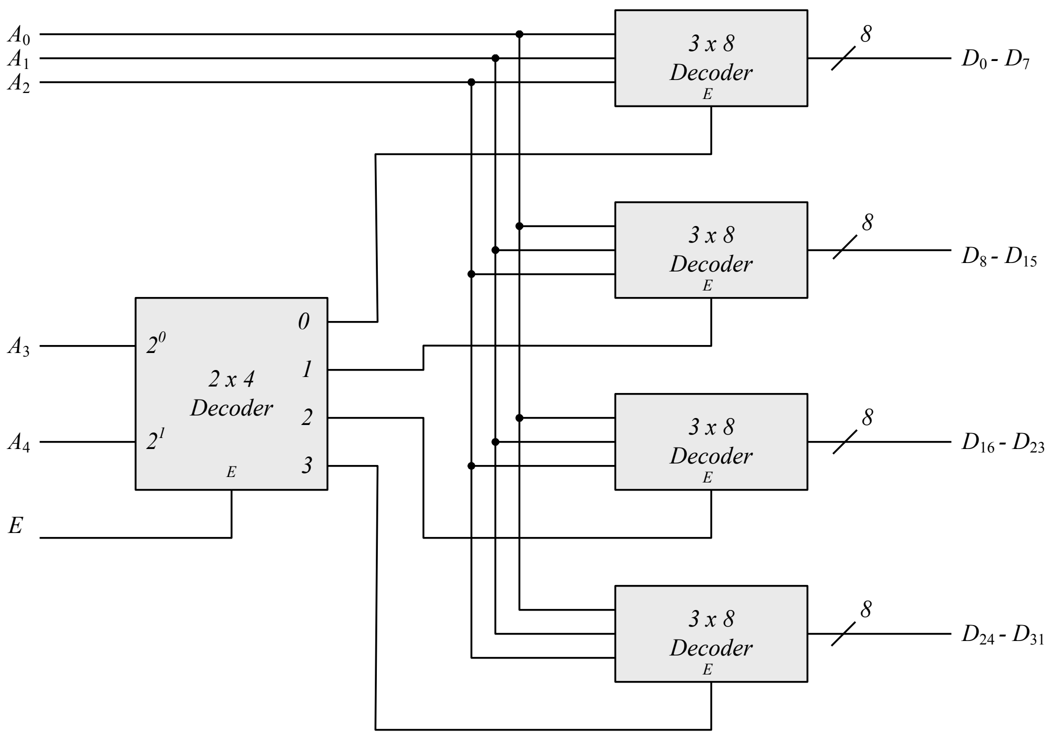
Outputs: $D_0, D_1, \ldots D_9$

$D_0 = A'B'C'D'$
$D_1 = A'B'C'D$
$D_2 = B'CD'$
$D_3 = B'CD$
$D_4 = BC'D'$
$D_5 = BC'D$
$D_6 = BCD'$
$D_7 = BCD$
$D_8 = AD'$
$D_9 = AD$

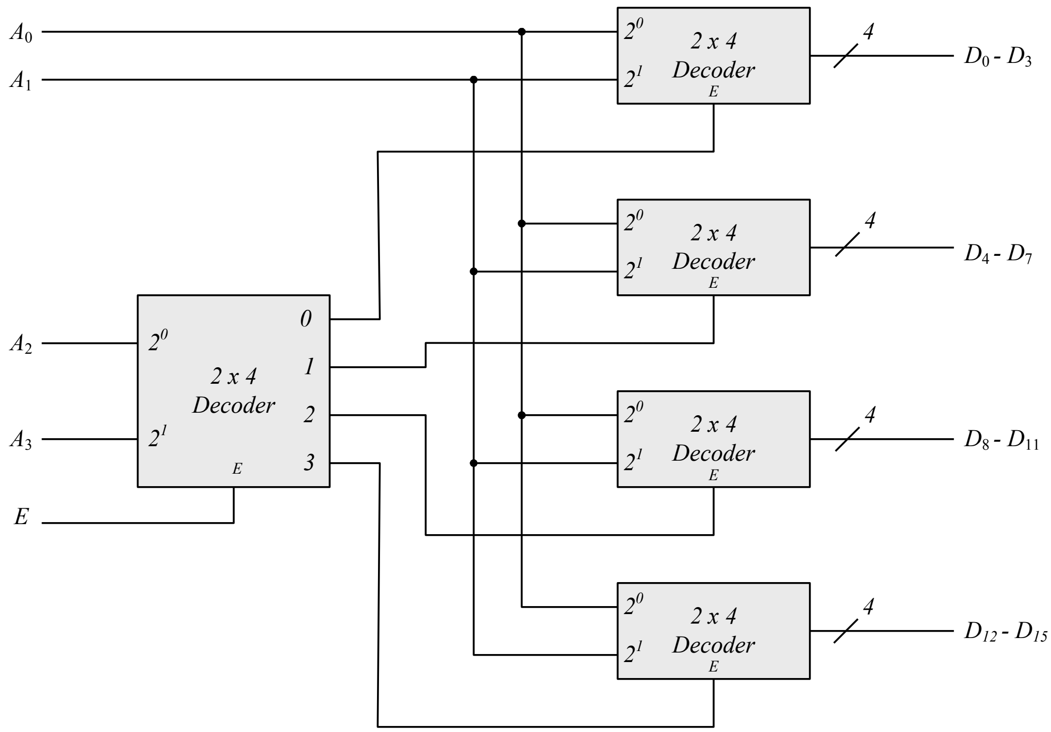
AB \ CD	00	01	11 (C)	10 (C)
00	m_0 D_0	m_1 D_1	m_3 D_3	x D_2
01 (B)	m_4 D_4	m_5 D_5	m_7 D_7	m_6 D_6
11 (A, B)	m_{12} x	m_{13} x	m_{15} x	m_{14} x
10 (A)	m_8 D_8	m_9 D_9	m_{11} x	m_{10} x

D (columns 01, 11)

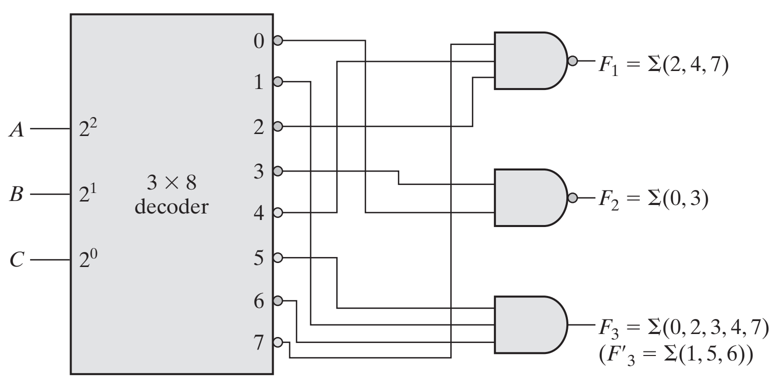
4.25



4.26

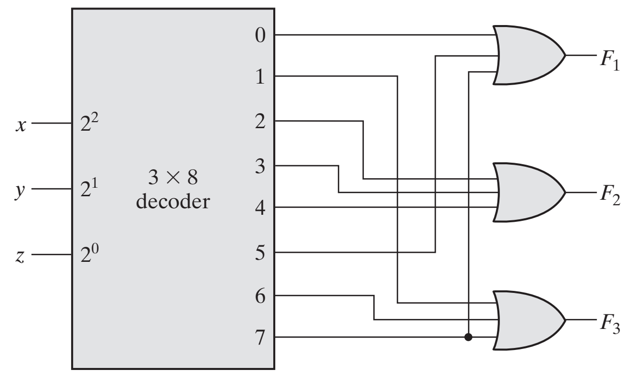

A_0
A_1
A_2
A_3
E
2^0
2^1
2 x 4
Decoder
E
0
1
2
3
4
D_0 - D_3
D_4 - D_7
D_8 - D_{11}
D_{12} - D_{15}

4.27


A
B
C
2^2
2^1
2^0
3 × 8
decoder
0
1
2
3
4
5
6
7
$F_1 = \Sigma(2, 4, 7)$
$F_2 = \Sigma(0, 3)$
$F_3 = \Sigma(0, 2, 3, 4, 7)$
$(F'_3 = \Sigma(1, 5, 6))$

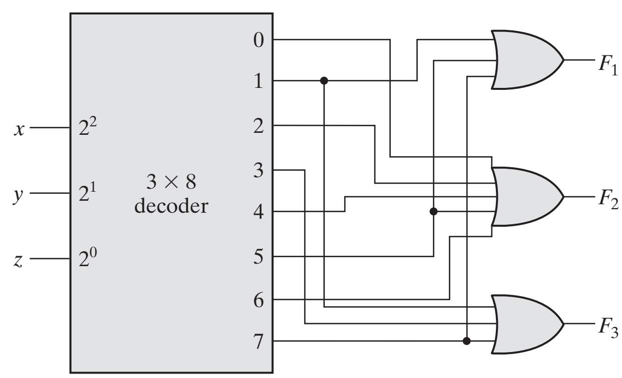
4.28 (a)

$F1 = x'y'z' + x\,(y + y')\,z = \Sigma(0, 5, 7)$
$F2 = xy'z' + x'y\,(z + z') = \Sigma(2, 3, 4)$
$F3 = x'y'z + xy\,(z + z') = \Sigma(1, 6, 7)$



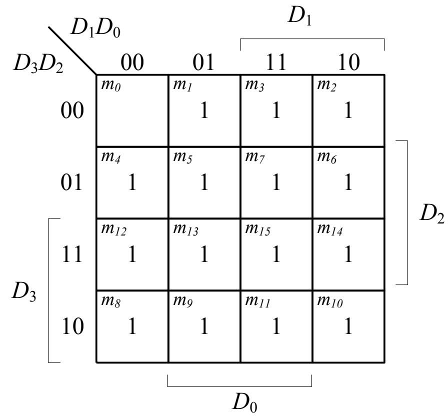
(b)

$F1 = (x + y')\,z = \Sigma(1, 5, 7)$
$F2 = y'z' + xy' + yz' = \Sigma(0, 2, 4, 5, 6)$
$F3 = (x' + y)\,z = \Sigma(1, 3, 7)$

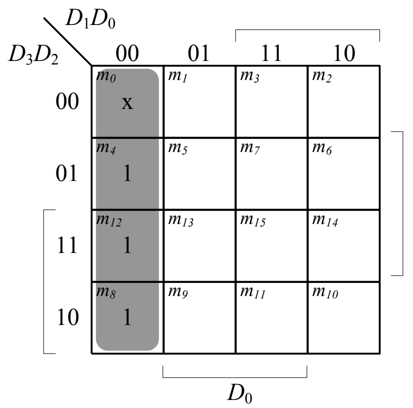


4.29

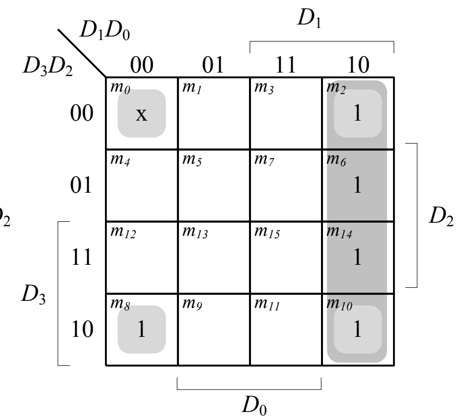
Inputs D_3 D_2 D_1 D_0				*Outputs* x y V		
0	0	0	0	x	x	0
x	x	x	1	0	0	1
x	x	1	0	0	1	1
x	1	0	0	1	0	1
1	0	0	0	1	1	1



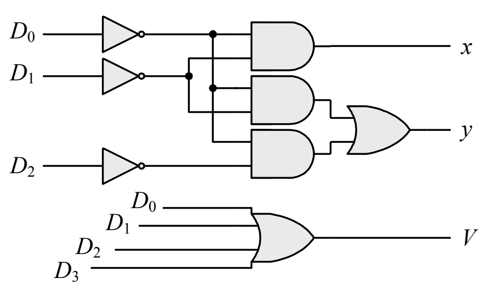
$V = D_0 + D_1 + D_2 + D_3$



$x = D_1'D_0'$



$y = D_0'D_2' + D_1D_0'$

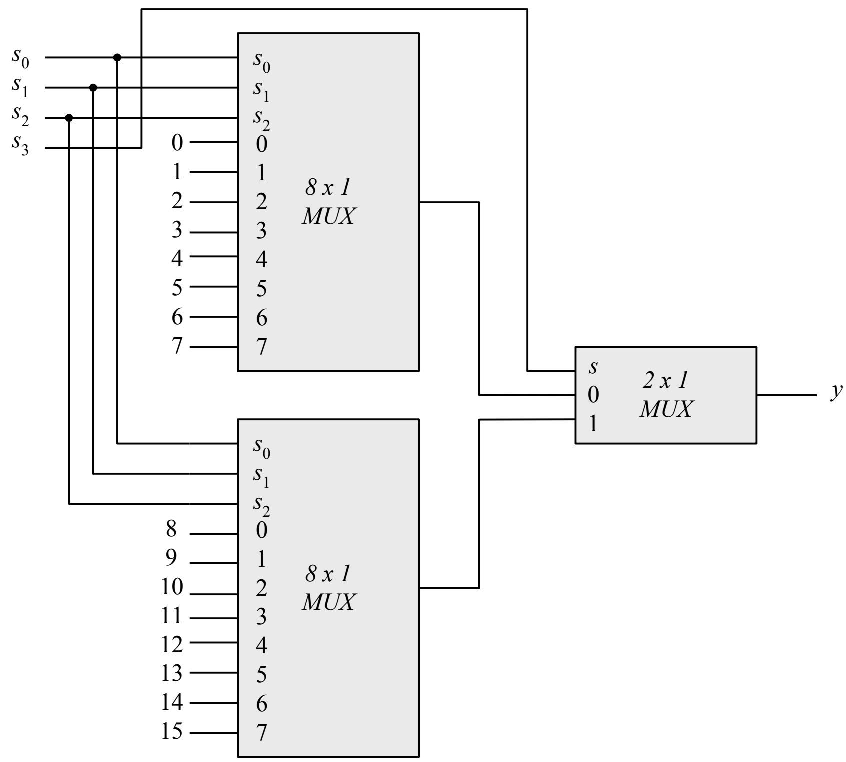


4.30

Inputs								*Outputs*			
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z	V
0	0	0	0	0	0	0	0	x	x	x	0
1	0	0	0	0	0	0	0	0	0	0	1
x	1	0	0	0	0	0	0	0	0	1	1
x	x	1	0	0	0	0	0	0	1	0	1
x	x	x	1	0	0	0	0	0	1	1	1
x	x	x	x	1	0	0	0	1	0	0	1
x	x	x	x	x	1	0	0	1	0	1	1
x	x	x	x	x	x	1	0	1	0	0	1
x	x	x	x	x	x	x	1	1	1	1	1

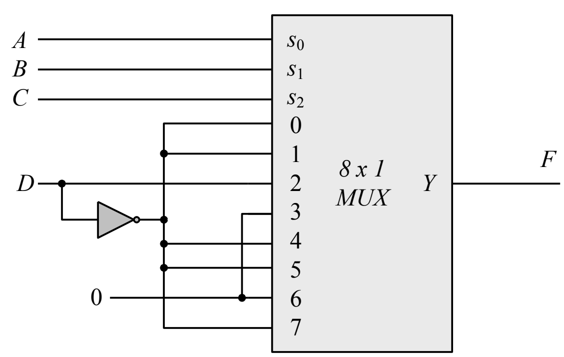
If $D_2 = 1$, $D_6 = 1$, all others = 0
Output xyz = 100 and V = 1

4.31



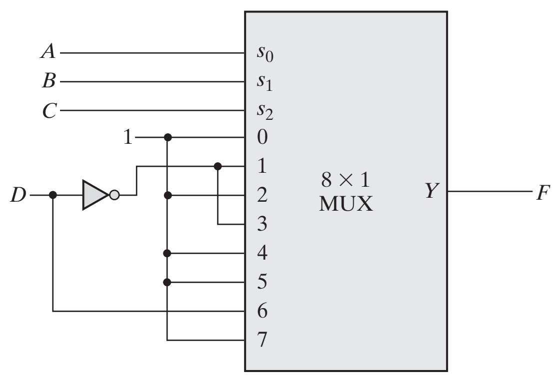
4.32 **(a)** $F(A, B, C, D) = \Sigma\ (0, 3, 5, 7, 13, 14)$

Inputs				Output	
A	B	C	D	F	
0	0	0	0	1	$F =$
0	0	0	1	0	D'
0	0	1	0	0	$F =$
0	0	1	1	1	D
0	1	0	0	0	$F =$
0	1	0	1	1	D
0	1	1	0	0	$F =$
0	1	1	1	1	D
1	0	0	0	0	$F = 0$
1	0	0	1	0	
1	0	1	0	0	$F = 0$
1	0	1	1	0	
1	1	0	0	0	$F =$
1	1	0	1	1	D
1	1	1	0	1	$F =$
1	1	1	1	0	D'



(b) $F(A, B, C, D) = \Pi(3, 8, 12) = (A' + B' + C + D)(A + B' + C' + D')(A + B + C' + D')$
$F' = ABC'D' + A'BCD + A'B'CD = \Sigma(3, 7, 12)$
$F = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 10, 11, 13, 14, 15)$

Inputs	Output	
A B C D	*F*	
0 0 0 0	1	$F = 1$
0 0 0 1	1	
0 0 1 0	1	$F =$
0 0 1 1	0	D'
0 1 0 0	1	$F = 1$
0 1 0 1	1	
0 1 1 0	1	$F =$
0 1 1 1	0	D'
1 0 0 0	1	$F = 1$
1 0 0 1	1	
1 0 1 0	1	$F = 1$
1 0 1 1	1	
1 1 0 0	0	$F =$
1 1 0 1	1	D
1 1 1 0	1	$F = 1$
1 1 1 1	1	



4.33

x	y	z	S	C
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

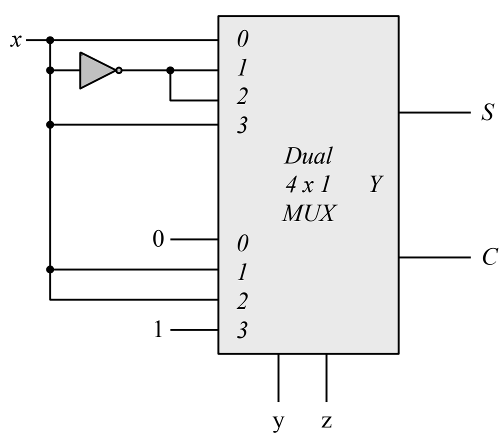
On simplifying, we get:

$S(x, y, z) = \Sigma(1, 2, 4, 7)$

$C(x, y, z) = \Sigma(3, 5, 6, 7)$

S	I_0	I_1	I_2	I_3
x'	0	1	2	3
x	4	5	6	7
	x	x'	x'	x

C	I_0	I_1	I_2	I_3
x'	0	1	2	3
x	4	5	6	7
	0	x	x	1

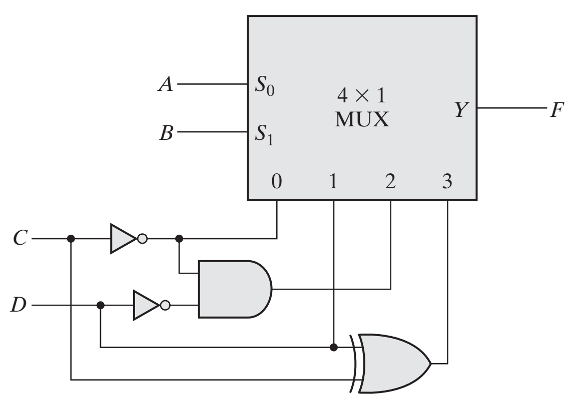


4.34 **(a)** $F(A, B, C, D) = \Sigma(0, 1, 2, 3, 6, 7, 9, 10, 13, 15)$

(b) $F(A, B, C, D) = \Sigma(1, 6, 7, 9, 10, 11, 12)$

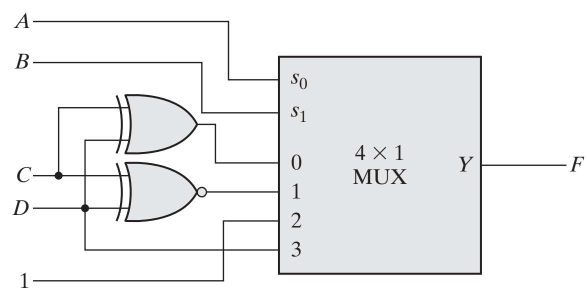
4.35 **(a)**

Inputs				Output	
A	B	C	D	F	
0	0	0	0	1	
0	0	0	1	1	$AB = 00$
0	0	1	0	0	$F = C'$
0	0	1	1	0	
0	1	0	0	0	
0	1	0	1	1	$AB = 01$
0	1	1	0	0	$F = D$
0	1	1	1	1	
1	0	0	0	1	
1	0	0	1	0	$AB = 10$
1	0	1	0	0	$F = C'D'$
1	0	1	1	0	
1	1	0	0	0	$AB = 11$
1	1	0	1	1	$F = C'D + CD' =$
1	1	1	0	0	$C \oplus D$
1	1	1	1	0	



(b)

Inputs				Output	
A	B	C	D	F	
0	0	0	0	0	
0	0	0	1	1	$AB = 00$ $F = C'D + CD' = C \oplus D$
0	0	1	0	1	
0	0	1	1	0	
0	1	0	0	1	
0	1	0	1	0	$AB = 01$ $F = C'D' + CD = (C \oplus D)'$
0	1	1	0	0	
0	1	1	1	1	
1	0	0	0	1	
1	0	0	1	1	$AB = 10$ $F = 1$
1	0	1	0	1	
1	0	1	1	1	
1	1	0	0	0	
1	1	0	1	1	$AB = 11$ $F = D$
1	1	1	0	0	
1	1	1	1	1	



4.36 **(a) (Verilog/SystemVerilog)**

```
module priority_encoder_gates (output x, y, V, input D0, D1, D2, D3);   //
V2001
  wire w1, D2_not;
  not  (D2_not, D2);
  or   (x, D2, D3);
  or   (V, D0, D1, x);
  and  (w1, D2_not, D1);
  or   (y, D3, w1);
endmodule
```

Note: See Problem 4.45 for testbench)

VHDL

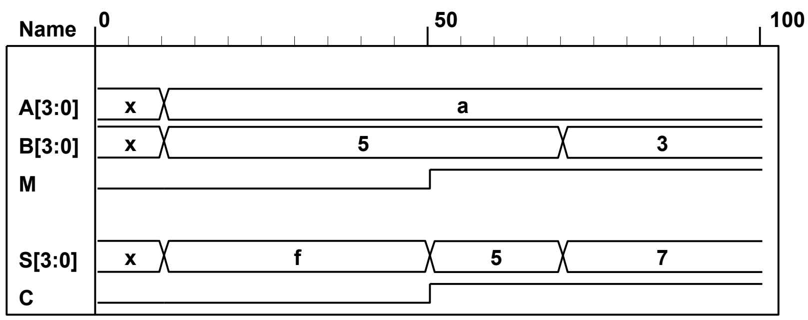
```
entity priority_encoder_gates is
   port ( x, y, V: out Std_Logic; D0, D1, D2, D3 : in Std_Logic);
end priority_encoder_gates;

architecture Gates of Priority_Encoder_Gates is
   component and2_gate  port (y: out Std_Logic; xin1, xin2: in Std_Logic);
   component or2_gate  port (y: out Std_Logic; xin1, xin2: in Std_Logic);
   component not_gate  port (y: out Std_Logic; xin: in Std_Logic);
   signal w1, D2_not: Std_Logic;
begin
   G1: not_gate  port map (D2_not => y, D2 => xin);
   G2: or2_gate port map (x => y, D2 => xin1, D3 => xin2);
   G3: or3_gate port map (V => y, D0 => xin1, D1 => xin2, x => xin3);
   G4: and2_gate port map (w1 => y, D2_not => xin1, D1 => xin2);
   G5: or2_gate port map (y => y, D3=> xin1, w1 => xin2);
end Gates;

entity and2_gate is
```

```
  port (y: out Std_Logic; xin1, xin2: in Std_Logic);
end and2_gate;

architecture Behavioral of and2_gate is
begin
   y <= xin1 and xin2;
end Behavioral;


entity or2_gate is
  port (y: out Std_Logic; xin1, xin2: in Std_Logic);
end or2_gate;

architecture Behavioral of or2_gate is
begin
   y <= xin1 or xin2;
end Behavioral;

entity or3_gate is
  port (y: out Std_Logic; xin1, xin2, xin3: in Std_Logic);
end or3_gate;

architecture Behavioral of or3_gate is
begin
   y <= xin1 or xin2 or xin3;
end Behavioral;
```

4.37 **(a) (Verilog/SystemVerilog)**

```
module Add_Sub_4_bit (
  output [3: 0] S,
  output C,
  input [3: 0] A, B,
  input M
);
  wire  [3: 0] B_xor_M;
  wire  C1, C2, C3, C4;
  assign C = C4;          // output carry
  xor (B_xor_M[0], B[0], M);
  xor (B_xor_M[1], B[1], M);
  xor (B_xor_M[2], B[2], M);
  xor (B_xor_M[3], B[3], M);
  // Instantiate full adders
  full_adder FA0 (S[0], C1, A[0], B_xor_M[0], M);
  full_adder FA1 (S[1], C2, A[1], B_xor_M[1], C1);
  full_adder FA2 (S[2], C3, A[2], B_xor_M[2], C2);
  full_adder FA3 (S[3], C4, A[3], B_xor_M[3], C3);
endmodule

module full_adder (output S, C, input x, y, z);    // See HDL Example 4.2
  wire S1, C1, C2;
  // instantiate half adders
  half_adder HA1 (S1, C1, x, y);
  half_adder HA2 (S, C2, S1, z);
  or G1 (C, C2, C1);
endmodule
```

```
module half_adder (output S, C, input x, y);      // See HDL Example 4.2
  xor   (S, x, y);
  and  (C, x, y);
endmodule

module t_Add_Sub_4_bit ();
  wire [3: 0] S;
  wire C;
  reg [3: 0] A, B;
  reg M;

  Add_Sub_4_bit M0 (S, C, A, B, M);

  initial #100 $finish;
  initial fork
    #10 M = 0;
    #10 A = 4'hA;
    #10 B = 4'h5;
    #50 M = 1;
    #70 B = 4'h3;
  join
endmodule
```



(b) (VHDL)

```
entity Add_Sub_4_bit is
            port (  S: out Std_Logic_Vector (3 downto 0); C: out Std_Logic;
                       A, B: in Std_Logic_Vector (3 downto 0); M: in Std_Logic);
end Add_Sub_4_bit;

architecture Structural of Add_Sub_4_bit is
     signal B_xor_M: Std_Logic_Vector (3 downto 0);
     signal C1, C2, C3, C4: Std_Logic;
     component xor2_gate port (y: out Std_Logic; xin1, xin2: in Std_Logic); end component;
     component full_adder port (sum, cout: out Std_Logic; a, b, cin: in Std_Logic);
         end component;
begin
C <= C4
G1: xor2_gate port map (B_xor_M(0) => y, B(0) => xin1, M => xin2);
G2: xor2_gate port map (B_xor_M(1) => y, B(1) => xin1, M => xin2);
G3: xor2_gate port map (B_xor_M(2) => y, B(2) => xin1, M => xin2);
G4: xor2_gate port map (B_xor_M(3) => y, B(3) => xin1, M => xin2);
FA0: full_adder port map (S(0) => sum, C1 => cout, A(0) => a, B_xor_M(0) => b, M => cin);
```

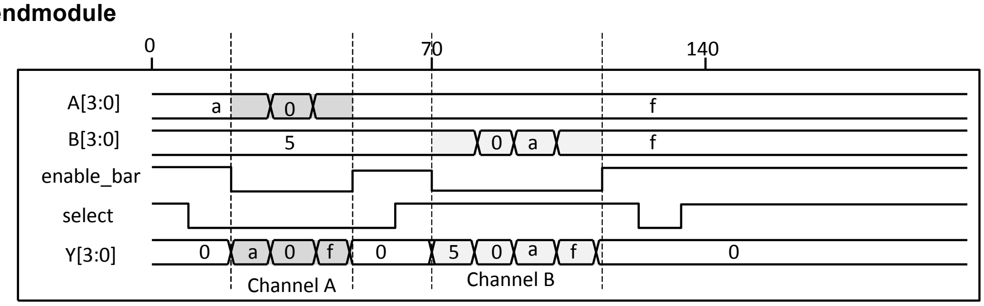
```
            FA1: full_adder port map (S(1) => sum, C2 => cout, A(1) => a, B_xor_M(1) => b, C1 => cin);
            FA2: full_adder port map (S(1) => sum, C3 => cout, A(2) => a, B_xor_M(2) => b, C2 => cin);
            FA3: full_adder port map (S(3) => sum, C4 => cout, A(3) => a, B_xor_M(3) => b, C3 => cin);
            end Structural;

entity full_adder is
   port (s, cout: out Std_Logic; a, b, cin: in Std_Logic);
end full_adder;

architecture Structural of full_adder is
   signal s1, c1, c2;
   component half_adder port (s, c: out Std_logic; a, b: in Std_Logic); end component;
   component or2_gate port (y: out Std_Logic; xin1, xin2: in Std_Logic); end component;
begin
   HA1: half_adder port map (s1 => s, c1 => c, a => a, b => b);
   HA2: half_adder port map (s1 => s, c2 => c, s1=> a, cin => b);
   G1: or2_gate port map (c => y, c2 => xin1, c1 => xin2);
end Structural

entity half_adder is
   port (s, c: out Std_Logic; a, b: in Std_Logic);
end half_adder;

architecture Operators of half_adder is
begin
   s <= x xor y;
   c <= x and y;
end Operators;
```

Test Bench:

```
entity t_Add_Sub_4_bit is
   port ();
end Add_Sub_4_bit;

architecture Behavioral of t_Add_Sub_4_bit is
   signal t_S: Std_Logic_Vector (3 downto 0), t_C: Std_Logic;
   signal t_A, t_B: Std_Logic_Vector (3 downto 0), t_M: Std_Logic;
   component  Add_Sub_4_bit is
                        port (  S: out Std_Logic_Vector (3 downto 0); C: out Std_Logic;
                                A, B: in Std_Logic_Vector (3 downto 0); M: in Std_Logic);

begin

   G1: Add_Sub_4_bit port map(t_S => S, t_C => C, t_A => A, t_B => B, t_M => M);

   t_M <= '0'        after 10 ns;
   t_A <= '1010'     after 20 ns
   t_B <= '0101'     after10 ns;
   t_M <= '1'        after 50 ns;
   t_B <= '0011'     after 70 ns.
end Behavioral;
```

4.38 (a) (Verilog)

```
module quad_2x1_mux (                         // V2001
  input    [3: 0]   A, B,                     // 4-bit data channels
  input             enable_bar, select,       // enable_bar is active-low)
  output   [3: 0]   Y                         // 4-bit mux output
);
  //assign Y = enable_bar ? 0 : (select ? B : A);        // Grounds output
  assign Y = enable_bar ? 4'bzzzz : (select ? B : A);    // Three-state output
endmodule
// Note that this mux grounds the output when the mux is not active.

module t_quad_2x1_mux ();
  reg     [3: 0] A, B, C;                     // 4-bit data channels
  reg             enable_bar, select;         // enable_bar is active-low)
  wire   [3: 0]   Y;                          // 4-bit mux

  quad_2x1_mux M0 (A, B, enable_bar, select, Y);

  initial #200 $finish;
  initial fork
    enable_bar = 1;
    select = 1;
    A = 4'hA;
    B = 4'h5;
    #10 select = 0;       // channel A
    #20 enable_bar = 0;
    #30 A = 4'h0;
    #40 A = 4'hF;
    #50 enable_bar = 1;
    #60 select = 1;       // channel B
    #70 enable_bar = 0;
    #80 B = 4'h0;
    #90 B = 4'hA;
    #100 B = 4'hF;
    #110 enable_bar = 1;
    #120 select = 0;
    #130 select = 1;
    #140 enable_bar = 1;
  join
endmodule
```



(b) (VHDL)

```
entity quad_2x1_mux is
   port (Y: out Std_Logic_Vector (3 downto 0); A, B: in Std_Logic_Vector (3 down to 0);
         enable_bar, select: in Std_Logic_Vector);
end quad_2x1_muxl
```

```
architecture DataFlow of quad_2x1_mux is
begin
    Y <= "0000" when enable_bar = '1';
        else B when (not enable_bar) and (select);
         else A when (not enable_bar) and (not select);
end DataFlow;


entity t_quad_2x1_mux is
    port ();
end t_quad_2x1_mux;

architecture testbench of t_quad_2x1_mux is
begin
    enable_bar <= '1' after 0 ns;
    select <= '1' after 0 ns;
    A <= "1010" after 0 ns;
    B <= "0101" after 0 ns;

    select <= '0' after 10 ns;    -- channel A
    enable_bar <= '0' after 20 ns;
    A <= "0000" after 30 ns;
    A <= "1111" after 40 ns;
    enable_bar <= '1' after 50 ns;
    select <= '1' after 60 ns;    == Channel B
    enable_bar <= '0' after 70 ns;
    B <= "0000" after 80 ns;
    B <= "1010" after 90 ns;
    B <= "1111" after 100 ns;
    enable_bar <= '1' after 110 ns;
    select <= '0' after 120 ns;
    select <= '1' after 130 ns;
    enable_bar <= '1' after 140 ns;
end testbench.
```

4.39

```
module Compare (A, B, Y);
  input          [1: 0]        A, B;   // 2-bit data inputs.
  output  [5: 0]        Y;       // 6-bit comparator output
  reg     [5: 0]        Y;       // EQ, NE, GT, LT, GE, LE

  always @ (A or B)
   if (A==B)     Y = 6'b10_0011;     // EQ, GE, LE
   else if (A < B) Y = 6'b01_0101; // NE, LT, LE
   else                    Y = 6'b01_1010;     // NE, GT, GE
endmodule
```

4.40 Verilog

```
module Prob_4_40 (
output [3: 0]  sum_diff, output carry_borrow,
input [3: 0] A, B, input sel_diff
);

  always @(sel_diff, A, B)    {carry_borrow, sum_diff} = sel_diff ? A - B : A + B;
endmodule

module t_Prob_4_40;
  wire  [3: 0] sum_diff;
  wire carry_borrow;
  reg [3:0] A, B;
  reg sel_diff;

  integer I, J, K;
  Prob_4_40 M0 (  sum_diff, carry_borrow,  A, B, sel_diff);
  initial #4000 $finish;
  initial begin
    for (I = 0; I < 2; I = I + 1) begin
      sel_diff = I;
      for (J = 0; J < 16; J = J + 1) begin
        A = J;
        for (K = 0; K < 16; K = K + 1) begin B = K; #5 ; end
      end
    end
  end
endmodule
```

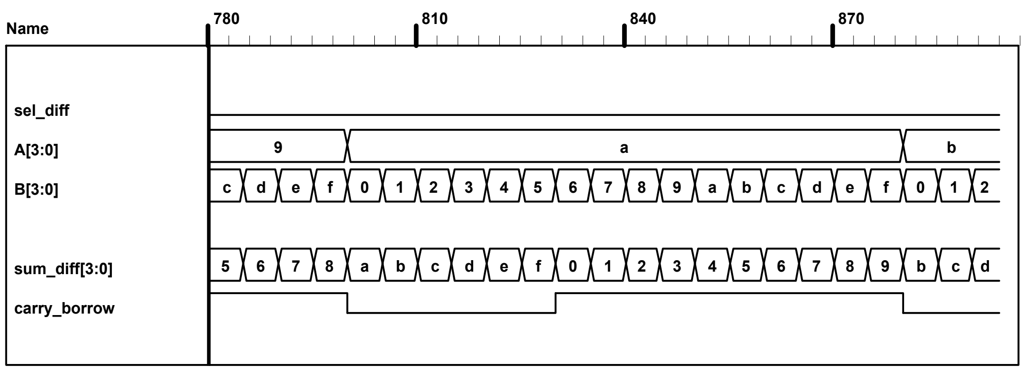
VHDL

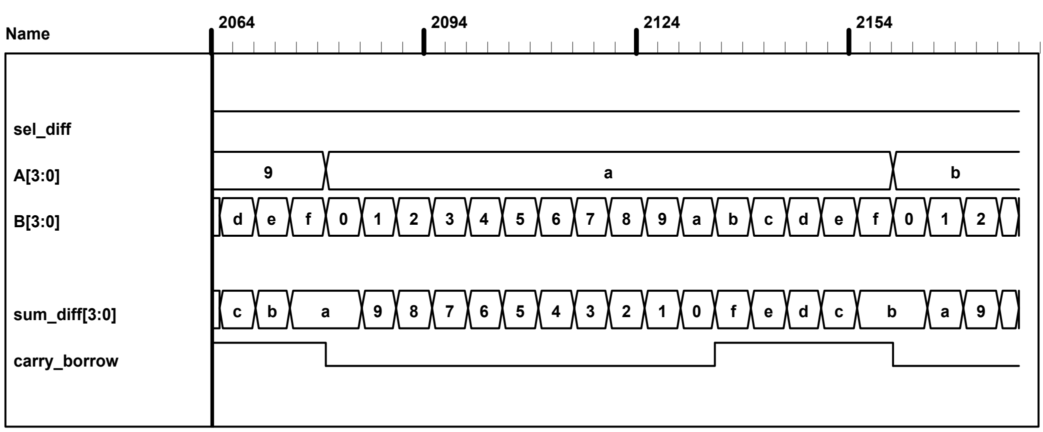
```
entity Prob_4_40 is
port (sum_diff: out Std_Logic_Vector (3 downto 0); carry_borrow: out Std_Logic;
      A, B: in Std_Logic_Vector (3 downto 0); sel_diff: in Std_Logic);
end Prob_4_40;

architecture DataFlow of Prob_4_40 is
begin
   carry_borrow & sum_diff <= A – B when select_diff = 1; else A+ B;
end DataFlow;
```

4.41 (a) Verilog/SystemVerilog

```
module Prob_4_41 (
output reg [3: 0]  sum_diff, output reg carry_borrow,
input [3: 0] A, B, input sel_diff
```

```
);

  always @ (A, B, sel_diff)
  {carry_borrow, sum_diff} <= sel_diff ? ('0' & A) – ('0' & B) : ('0' & A) + ('0' & B);
endmodule


module t_Prob_4_41;
 wire  [3: 0] sum_diff;
 wire carry_borrow;
 reg [3:0] A, B;
 reg sel_diff;

 integer I, J, K;
 Prob_4_46 M0 (  sum_diff, carry_borrow,  A, B, sel_diff);
 initial #4000 $finish;
 initial begin
  for (I = 0; I < 2; I = I + 1) begin
   sel_diff = I;
   for (J = 0; J < 16; J = J + 1) begin
    A = J;
    for (K = 0; K < 16; K = K + 1) begin B = K; #5 ; end
   end
  end
 end
endmodule
```





(b) VHDL

```
entity Prob_4_41 is
port (sum_diff: out Std_Logic_Vector (3 downto 0); carry_borrow: out Std_Logic;
            A, B: in Std_Logic_Vector (3 downto 0), sel_diff: in Std_Logic);
end Prob_4_41;

architecture Behavioral of Prob_4_41 is
   process (A, B, sel_diff)
   begin
      (carrry_borrow & sum_diff) <= ('0' & A) + ('0' & B) when sel_diff; else ('0' & A) – ('0' & B);
   end process;
end Behavioral;
```

4.42 (a)Verilog/SystemVerilog

```
module Xs3_Gates (input A, B, C, D, output w, x, y, z);
 wire  B_bar, C_or_D_bar;
 wire  CD, C_or_D;
 or    (C_or_D, C, D);
 not   (C_or_D_bar, C_or_D);
 not   (B_bar, B);
 and   (CD, C, D);
 not   (z, D);
 or    (y, CD,  C_or_D_bar);
 and   (w1, C_or_D_bar, B);
 and   (w2, B_bar, C_or_D);
 and   (w3, C_or_D, B);
 or    (x, w1, w2);
 or    (w, w3, A);
endmodule
```

VHDL

```
entity Xs3_Converter is
   port (A, B, C, D: in Std_Logic; w, x, y, z: out Std_Logic);
end Xs3_Gates;

architecture Gates of Xs3_Converter is
   signal B_bar, C_or_D_bar, Cd, C_or_D: Std_Logic;
   component ord2_gate port (y: out Std_Logic; xin1, xin2: in Std_Logic);
   component and2_gate port (y: out Std_Logic; xin1, xin2: in Std_Logic);
   component not_gate port (y: out Std_Logic; xin: in Std_Logic);
begin
   G1: or2_gate port map (C_or_D => y, C => xin1, D => xin2);
   G2: not_gate port map (C_or_D_bar => y, C_or_D => xin);
   G3: not_gate port map (B_bar => y, B => xin);
   G4: and2_gate port map (Cd => y, C => xin1, D => xin2);
   G5: not_gate port map (z => y, D => xin);
   G6: or2_gate port map (y => y, CD => xin1, C_or_D_bar => xin2);
   G7: and2_gate port map (w1 => y, C_or_D_bar => xin1, B => xin2);
   G8: and2_gate port map (w2 => y, B_bar => xin1, C_or_D => xin2);
   G9: and2_gate port map (w3 => y, C_or_D => xin1, B => xin2);
   G10: or2_gate port map (x => y, w1 => xin1, w2 => xin2);
   G11: or2_gate port map (x => y, w3 => xin1, A => xin2);
end Gates
```

See declarations of and2_gate, or2_gate and not_gate in previous problems (above).

(b) **Verilog**

```
module Xs3_Dataflow (input A, B, C, D, output w, x, y, z);
assign {w, x, y, z} = {A, B, C, D} + 4'b0011;
endmodule
```

VHDL

```
architecture Behavioral of Xs3_Converter is
begin
    process (A, B, C, D) begin
        {w, x, y, z} <= {A, B, C, D} + 4'b0011;
    end process;
```

`

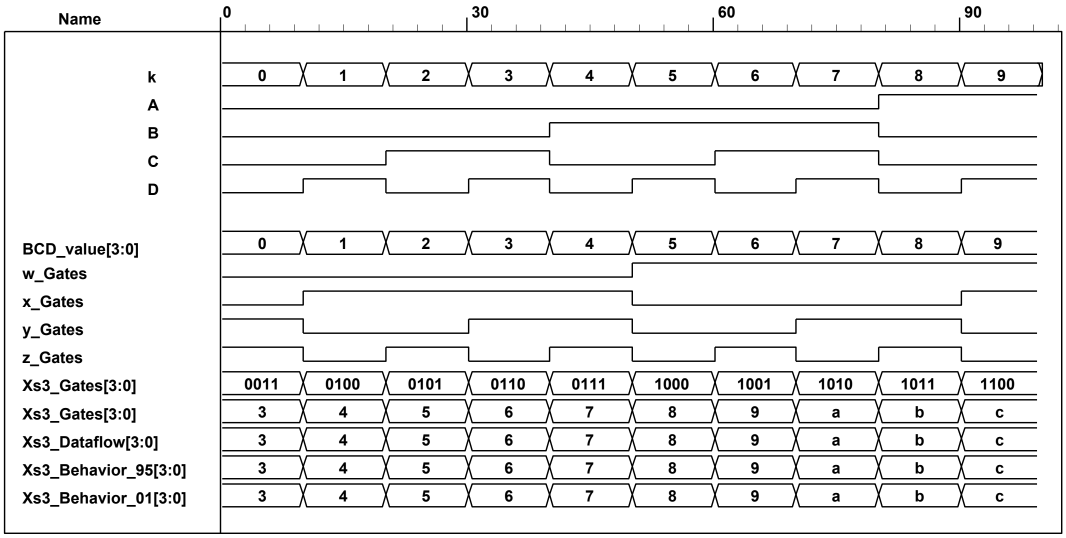
(c) **Verilog/SystemVerilog**

```
module Xs3_Behavior_95 (A, B, C, D, w, x, y, z);
  input     A, B, C, D;
  output    w, x, y, z;
  reg    w, x, y, z;

 always @ (A or B or C or D) begin {w, x, y, z} = {A, B, C, D} + 4'b0011; end
endmodule

module Xs3_Behavior_01 (input A, B, C, D, output reg w, x, y, z);
  always @ (A, B, C, D) begin {w, x, y, z} = {A, B,C, D} + 4'b0011; end
endmodule
```

VHDL

Verilog Testbench

```
module t_Xs3_Converters ();
  reg A, B, C, D;
  wire w_Gates, x_Gates, y_Gates, z_Gates;
  wire w_Dataflow, x_Dataflow, y_Dataflow, z_Dataflow;
  wire w_Behavior_95, x_Behavior_95, y_Behavior_95, z_Behavior_95;
  wire w_Behavior_01, x_Behavior_01, y_Behavior_01, z_Behavior_01;
  integer k;
  wire [3: 0] BCD_value;
  wire [3: 0] Xs3_Gates = {w_Gates, x_Gates, y_Gates, z_Gates};
  wire [3: 0] Xs3_Dataflow = {w_Dataflow, x_Dataflow, y_Dataflow, z_Dataflow};
  wire [3: 0] Xs3_Behavior_95 = {w_Behavior_95, x_Behavior_95, y_Behavior_95, z_Behavior_95};
  wire [3: 0] Xs3_Behavior_01 = {w_Behavior_01, x_Behavior_01, y_Behavior_01, z_Behavior_01};

  assign BCD_value = {A, B, C, D};
  Xs3_Gates   M0 (A, B, C, D, w_Gates, x_Gates, y_Gates, z_Gates);
  Xs3_Dataflow    M1 (A, B, C, D, w_Dataflow, x_Dataflow, y_Dataflow, z_Dataflow);
  Xs3_Behavior_95 M2 (A, B, C, D, w_Behavior_95, x_Behavior_95, y_Behavior_95, z_Behavior_95);
  Xs3_Behavior_01 M3 (A, B, C, D, w_Behavior_01, x_Behavior_01, y_Behavior_01, z_Behavior_01);

  initial #200 $finish;
  initial begin
    k = 0;
    repeat (10) begin {A, B, C, D} = k; #10 k = k + 1; end
  end
endmodule
```



(b)(VHDL)

```
entity Xs3_Behavior_vhdl is
  port (A, B, C, D: in std_logic; w, x, y, z: out std_logic);
end Xs3_Behavior_vhdl;
architecture Behavioral of Xs3_Behavior_vhdl is
begin
  w & x & y & z <= A & B & C & D + '0011';
end Behavioral;
```

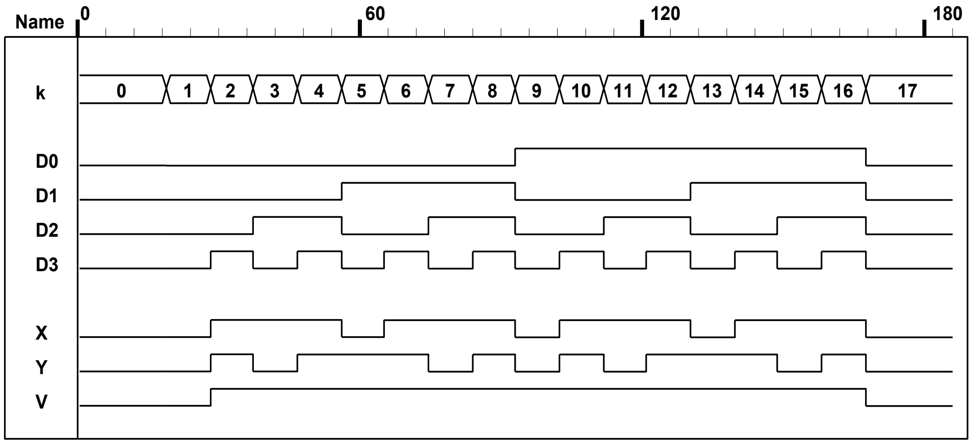
4.43 The circuit is a 4×1 mux with 4-bit input, 2-bit selection input and a single-bit output.

4.44

```
module ALU (output reg [7: 0] y, input [7: 0] A, B, input [2: 0] Sel);
 always     @ (A, B, Sel) begin
 y = 0;
 case (Sel)
  3'b000:    y = 8'b0;
  3'b001:    y = A & B;
  3'b010:    y = A | B;
  3'b011:    y = A + B;
  3'b100:    y = A - B;
  3'b101:    y = A ^ B;
  3'b110:    y = ~A;
  3'b111:    y = 8'hFF;
 endcase
 end
endmodule
```

4.45 **(a) (Verilog)**

```
module priority_encoder_beh (output reg X, Y, V, input D0, D1, D2, D3);   // V2001
  always @ (D0, D1, D2, D3) begin
    X = 0;
    Y = 0;
    V = 0;
    casex ({D0, D1, D2, D3})
      4'b0000:   {X, Y, V} = 3'bxx0;
      4'b1000:   {X, Y, V} = 3'b001;
      4'bx100:   {X, Y, V} = 3'b011;
      4'bxx10:   {X, Y, V} = 3'b101;
      4'bxxx1:   {X, Y, V} = 3'b111;
      default:   {X, Y, V} = 3'b000;
    endcase
  end
endmodule

module t_priority_encoder_beh ();   // V2001
  wire X, Y, V;
  reg D0, D1, D2, D3;
  integer k;

  priority_encoder_beh M0 (X, Y, V, D0, D1, D2, D3);

  initial #200 $finish;
  initial begin
    k = 32'bx;
    #10 for (k = 0; k <= 16; k = k + 1)  #10 {D0, D1, D2, D3} = k;
  end
endmodule
```



VHDL

```
entity priority_encoder is
    port (X, Y, V: out Std_Logic; D0, D1, D2, D3: in Std_Logic);
end priority_encoder;

architecture Behavioral of Priority_Encoder is
begin
  process @ (D0, D1, D2, D3) begin
    X <= 0;
    Y <= 0;
    V <= 0;
    case ({D0, D1, D2, D3})
      when  '0000'  => {X, Y, V} <= 'xx0';       -- Invalid
      when  '1000'  => {X, Y, V} <= '001';
      when  '-100'  => {X, Y, V} = '011';
      when  '—10'   => {X, Y, V} = '101';
      when  '---1'  => {X, Y, V} = '111';
      when others  => {X, Y, V} = '000';
    endcase
  end
end architecture;
```

4.46

```
module Prob_4_46a (output F, input A, B, C, D);
  assign F = (~A&~B&~C&~D) | (~A&~B&C&~D) | (~A&B&~C&D) | (~A&B&C&D) |
  (A&~B&C&D) | (A&B&C&~D);
endmodule

module Prob_4_46b (output F, input A, B, C, D);

  assign F = (~A&~B&~C&~D) | (~A&~B&~C&D) | (~A&~B&C&~D) | (~A&B&~C&~D) |
  (~A&B&~C&D) | (~A&B&C&~D) | (A&~B&~C&~D) | (A&~B&~C&D) | (A&~B&C&~D) |
  (A&~B&C&D) | (A&B&~C&D) | (A&B&C&~D) | (A&B&C&D);
endmodule
```

4.47

```
module Prob_4_47a (output F, input A, B, C, D);
  assign F = (~A&~B&~C&~D) | (~A&~B&~C&D) | (~A&~B&C&~D) | (~A&~B&C&D) |
  (~A&B&C&~D) | (~A&B&C&D) | (A&~B&~C&D) | (A&~B&C&~D) | (A&B&~C&D) |
  (A&B&C&D);
endmodule

module Prob_4_47b (output F, input A, B, C, D);
  assign F = (~A&~B&~C&D) | (~A&B&C&~D) | (~A&B&C&D) | (A&~B&~C&D) |
  (A&~B&C&~D) | (A&~B&C&D) | (A&B&~C&~D);
endmodule
```

4.48

```
module ALU_3state (output reg [7: 0] y_tri, input [7: 0] A, B, input [2: 0] Sel, input En);
  reg        [7: 0] y;
  assign     y_tri = En ? y: 8'bz;
  always     @ (A, B, Sel) begin
  y = 0;
  case (Sel)
    3'b000:   y = 8'b0;
    3'b001:   y = A & B;
    3'b010:   y = A | B;
    3'b011:   y = A + B;
    3'b100:   y = A - B;
    3'b101:   y = A ^ B;
    3'b110:   y = ~A;
    3'b111:   y = 8'hFF;
  endcase
  end
endmodule
```

4.49

```
module Problem_4_49_Gates (output F1, F2, input A, B, C, D);
  wire  A_bar = !A;
  wire  B_bar = !B;
  and   (T1, B_bar, C);
  and   (T2, A_bar, B);
  or    (T3, A, T1);
  xor   (T4, T2, D);
  or    (F1, T3, T4);
  or    (T5, T2, D);
  not   (F2, T5)
endmodule
```

4.50 **(a) 84-2-1 to BCD code converter**

// See Problem 4.8 and Table 1.5.

Verilog

```
// Verilog 1995

// module Prob_4_50a (Code_BCD,  Code84_m2_m1);
// output [3: 0]   Code_BCD;
// input [3:0];
// reg [3: 0]   Code_BCD;
// ...

// Verilog 2001, 2005
module Prob_4_50a (output reg [3: 0] Code_BCD,  input [3: 0] Code_84_m2_m1);

  always @ (Code_84_m2_m1)
    case (Code_84_m2_m1)
      4'b0000: Code_BCD = 4'b0000';       // 0
      4'b0111: Code_BCD = 4'b0001';       // 1
      4'b0110: Code_BCD = 4'b0010';       // 2
      4'b0101: Code_BCD = 4'b0011';       // 3
      4'b0100: Code_BCD = 4'b0100';       // 4
```

```
    4'b1011: Code_BCD = 4'b0101';        // 5
    4'b1010: Code_BCD = 4'b0110';        // 6
    4'b1001: Code_BCD = 4'b0111';        // 7
    4'b1000: Code_BCD = 4'b1000';        // 8
    4'b1111: Code_BCD = 4'b1001';        // 9

    4'b0001: Code_BCD = 4'b1010';        // 10
    4'b0010: Code_BCD = 4'b1011';        // 11
    4'b0011: Code_BCD = 4'b1100';        // 12
    4'b1100: Code_BCD= 4'b1101';         // 13
    4'b1101: Code_BCD = 4'b1110';        // 14
    4'b1110: Code_BCD  = 4'b1111';       // 15
  endcase
endmodule

module t_Prob_4_50a;
 wire [3: 0]  Code_BCD;
 reg [3: 0]; Code_84_m2_m1;
 integer      K;

 Prob_4_50a M0 ( Code_BCD,  Code_84_m2_m1); // Unit under test (UUT)

 initial #100 $finish;
 initial begin
  for (K = 0; K < 16; K = K + 1) begin Code_84_m2_m1 = K; #5 ; end
 end
endmodule
```

```
VHDL
entity Prob_4_50a_vhdl is
port (code_BCD: out std_logic_vector (3 downto 0); Code_84_md_m1: in std_logic_vector (3 downto
0));
end Prob_4_50a_vhdl;

architecture Behavioral of Prob_4_50a is
begin
 process  (Code_84_m2_m1) begin
  case (Code_84_m2_m1)
   when  '0000'  =>  Code_BCD <= '0000';        // 0
   when  '0111'  =>  Code_BCD <= '0001';        // 1
   when  '0110'  =>  Code_BCD <= '0010;     // 2
   when  '0101'  =>  Code_BCD <= '0011;     // 3
   when  '0100'  =>  Code_BCD <= '0100;     // 4
   when  '1011'  =>  Code_BCD <= '0101;     // 5
   when  '1010'  =>  Code_BCD <= '0110;     // 6
   when  '1001'  =>  Code_BCD <= '0111;     // 7
   when  '1000'  =>  Code_BCD <= '1000;     // 8
   when  '1111'  =>  Code_BCD <= '1001;     // 9

   when  '0001'  =>  Code_BCD <= '1010;     // 10
   when  '0010'  =>  Code_BCD <= '1011;     // 11
   when  '0011'  =>  Code_BCD <= '1100;     // 12
   when  '1100'  =>  Code_BCD <= '1101;     // 13
   when  '1101'  =>  Code_BCD <= '1110;     // 14
   when  '1110'  =>  Code_BCD <= '1111;     // 15
  end case;
 end process;
end Behavioral;
```

(b) 84-2-1 to Gray code converter

<u>**Verilog**</u>

```
module Prob_4_50b (output reg [3: 0] Code_BCD,  input [3: 0] Code_84_m2_m1);
always @ (Code_84_m2_m1)
   case (Code_84_m2_m1)
     4'b0000: Code_Gray = 4'b0000;         // 0
     4'b0111: Code_Gray = 4'b0001;         // 1
     4'b0110: Code_Gray = 4'b0011;         // 2
     4'b0101: Code_Gray = 4'b0010;         // 3
     4'b0100: Code_Gray = 4'b0110;         // 4
     4'b1011: Code_Gray = 4'b0111;         // 5
     4'b1010: Code_Gray = 4'b0101;         // 6
     4'b1001: Code_Gray = 4'b0100;         // 7
     4'b1000: Code_Gray = 4'b1100;         // 8
     4'b1111: Code_Gray = 4'b1101;         // 9

     4'b0001: Code_Gray = 4'b1111;         // 10
     4'b0010: Code_Gray = 4'b1110;         // 11
     4'b0011: Code_Gray = 4'b1010;         // 12
     4'b1100: Code_Gray= 4'b1011;          // 13
     4'b1101: Code_Gray = 4'b1001;         // 14
     4'b1110: Code_Gray  = 4'b1000;        // 15
   endcase
endmodule
  module t_Prob_4_50b;
    wire [3: 0]     Code_Gray;
    reg    [3: 0] Code_84_m2_m1;
    integer    K;

    Prob_4_50b M0 (Code_Gray,  Code_84_m2_m1);    // Unit under test (UUT)

    initial #100 $finish;
    initial begin
      for (K = 0; K < 16; K = K + 1) begin Code_84_m2_m1 = K; #5 ; end
    end
  endmodule
```

<u>**VHDL**</u>

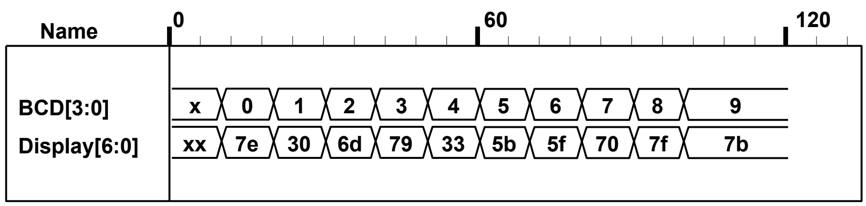
```
arhitecture Behavioral of  Prob_4_50b is
begin
   process (Code_84_m2_m1)
   begin
      case (Code_84_m2_m1)
        when '0000'=>  Code_Gray <= '0000';          // 0
        when '0111'=>  Code_Gray <= '0001';          // 1
        when '0110'=>  Code_Gray <= '0011';          // 2
        when '0101'=>  Code_Gray <= '0010';          // 3
        when '0100'=>  Code_Gray <= '0110';          // 4
        when '1011'=>  Code_Gray <= '0111';          // 5
        when '1010'=>  Code_Gray <= '0101';          // 6
        when '1001'=>  Code_Gray <= '0100';          // 7
        when '1000'=>  Code_Gray <= '1100';          // 8
        when '1111 =>  Code_Gray <= '1101';          // 9

        when '0001'=>  Code_Gray <= '1111';          // 10
        when '0010'=>  Code_Gray <= '1110';          // 11
        when '0011'=>  Code_Gray <= '1010';          // 12
        when '1100'=> Code_Gray <= '1011';           // 13
        when '1101'=>  Code_Gray <= '1001';          // 14
```

```
    when '1110'=> Code_Gray  <= '1000';          // 15
    when OTHERS => Code_Gray  <= BLANK;
  end case;
end process ;
```

4.51 Assume that that the LEDs are asserted when the output is high.

```
module Seven_Seg_Display_V2001 (
 output reg   [6: 0]   Display,
 input        [3: 0]   BCD
);

 //                              abc_defg
 parameter  BLANK       = 7'b000_0000;
 parameter  ZERO        = 7'b111_1110;      // h7e
 parameter  ONE         = 7'b011_0000;      // h30
 parameter  TWO         = 7'b110_1101;      // h6d
 parameter  THREE       = 7'b111_1001;      // h79
 parameter  FOUR        = 7'b011_0011;      // h33
 parameter  FIVE        = 7'b101_1011;      // h5b
 parameter  SIX         = 7'b101_1111;      // h5f
 parameter  SEVEN       = 7'b111_0000;      // h70
 parameter  EIGHT       = 7'b111_1111;      // h7f
 parameter  NINE        = 7'b111_1011;      // h7b

 always @ (BCD)
  case (BCD)
   0:      Display = ZERO;
   1:      Display = ONE;
   2:      Display = TWO;
   3:      Display = THREE;
   4:      Display = FOUR;
   5:      Display = FIVE;
   6:      Display = SIX;
   7:      Display = SEVEN;
   8:      Display = EIGHT;
   9:      Display = NINE;
   default:   Display = BLANK;
 endcase
endmodule

module t_Seven_Seg_Display_V2001 ();
 wire  [6: 0]   Display;
 reg   [3: 0]   BCD;

 parameter  BLANK       = 7'b000_0000;
 parameter  ZERO        = 7'b111_1110;      // h7e
 parameter  ONE         = 7'b011_0000;      // h30
 parameter  TWO         = 7'b110_1101;      // h6d
 parameter  THREE       = 7'b111_1001;      // h79
 parameter  FOUR        = 7'b011_0011;      // h33
 parameter  FIVE        = 7'b101_1011;      // h5b
 parameter  SIX         = 7'b001_1111;      // h1f
 parameter  SEVEN       = 7'b111_0000;      // h70
 parameter  EIGHT       = 7'b111_1111;      // h7f
 parameter  NINE        = 7'b111_1011;      // h7b

 initial #120 $finish;
 initial fork
  #10 BCD = 0;
  #20 BCD = 1;
```

```
   #30 BCD = 2;
   #40 BCD = 3;
   #50 BCD = 4;
   #60 BCD = 5;
   #70 BCD = 6;
   #80 BCD = 7;
   #90 BCD = 8;
   #100 BCD = 9;

  join

  Seven_Seg_Display_V2001 M0 (Display, BCD);
endmodule
```



VHDL

```
 entity Seven_Seg_Display is
    port (Display: out Std_Logic_Vector (6 downto 0);
    BCD: in Std_Logic_Vector (3 downto 0);
end Seven_Seg_Display;

architecture Behavioral of Seven_Seg_Display is
  //                                    abc_defgend Seven_Seg_Display;
  constant     BLANK:    Std_Logic_Vector := '000_0000';
  constant     ZERO:     Std_Logic_Vector := '111_1110';          -- hex 7e
  constant     ONE:      Std_Logic_Vector := '011_0000';          -- hex 30
  constant     TWO:      Std_Logic_Vector := '110_1101';          -- hex 6d
  constant     THREE:    Std_Logic_Vector := '111_1001';          -- hex 79
  constant     FOUR:     Std_Logic_Vector := '011_0011';          -- hex 33
  constant     FIVE:     Std_Logic_Vector := '101_1011';          -- hex 5b
  constant     SIX:      Std_Logic_Vector := '101_1111';          -- hex 5f
  constant     SEVEN:    Std_Logic_Vector := '111_0000';          -- hex 70
  constant     EIGHT:    Std_Logic_Vector := '111_1111';          -- hex 7f
  constant     NINE:     Std_Logic_Vector := '111_1011';          -- hex 7b
  begin
  process (BCD) begin
   case (BCD)
    when  0    =>       Display <= ZERO;
    when  1    =>       Display <= ONE;
    when  2    =>       Display <= TWO;
    when  3    =>       Display <= THREE;
    when  4    =>       Display <= FOUR;
    when  5    =>       Display <= FIVE;
    when  6    =>       Display <= SIX;
    when  7    =>       Display <= SEVEN;
    when  8    =>       Display <= EIGHT;
    when  9    =>       Display <= NINE;
    when others  =>  Display <= BLANK;
   end case
  end process
end Behavioral
```

Alternative with continuous assignments (dataflow):

```
module Seven_Seg_Display_V2001_CA (
 output        [6: 0]   Display,
 input         [3: 0]   BCD
);

 //                      abc_defg
 parameter BLANK    = 7'b000_0000;
 parameter ZERO     = 7'b111_1110;      // h7e
 parameter ONE      = 7'b011_0000;      // h30
 parameter TWO      = 7'b110_1101;      // h6d
 parameter THREE    = 7'b111_1001;      // h79
 parameter FOUR     = 7'b011_0011;      // h33
 parameter FIVE     = 7'b101_1011;      // h5b
 parameter SIX      = 7'b101_1111;      // h5f
 parameter SEVEN    = 7'b111_0000;      // h70
 parameter EIGHT    = 7'b111_1111;      // h7f
 parameter NINE     = 7'b111_1011;      // h7b
 wire       A, B, C, D, a, b, c, d, e, f, g;

 assign A = BCD[3];
 assign B = BCD[2];
 assign C = BCD[1];
 assign D = BCD[0];
 assign Display = {a,b,c,d,e,f,g};
 assign a  = (!A) && C || (!A) && B & D || (!B) && (!C) && (!D) || A && (!B) && (!C);
 assign b = (!A) && (!B) || (!A) && (!C) && (!D) || (!A) && C && D || A && (!B) && (!C);
 assign c = (!A) && B || (!A) && D || (!B) && (!C) && (!D) || A & (!B) & (!C);
 assign d = (!A) && C && (!D) || (!A) && (!B) && C || (!B) && (!C) && (!D)
              || A && (!B) && (!C) || (!A) && B && (!C) && D;
 assign e = (!A) && C && (!D) || (!B) && (!C) && (!D);
 assign f = (!A) && B && (!C) || (!A) && (!C) && (!D) || (!A) && B && (!D) || A && (!B) && (!C);
 assign g = (!A) && C && (!D) || (!A) && (!B) && C || (!A) && B && (!C) || A && (!B) && (C);
endmodule

module t_Seven_Seg_Display_V2001_CA ();
 wire    [6: 0]   Display;
 reg     [3: 0]   BCD;

 parameter      BLANK         = 7'b000_0000;
 parameter ZERO     = 7'b111_1110;      // h7e
 parameter ONE      = 7'b011_0000;      // h30
 parameter TWO      = 7'b110_1101;      // h6d
 parameter THREE    = 7'b111_1001;      // h79
 parameter FOUR     = 7'b011_0011;      // h33
 parameter FIVE     = 7'b101_1011;      // h5b
 parameter SIX      = 7'b001_1111;      // h1f
 parameter SEVEN    = 7'b111_0000;      // h70
 parameter EIGHT    = 7'b111_1111;      // h7f
 parameter NINE     = 7'b111_1011;      // h7b

 initial #120 $finish;
```

```
initial fork
  #10 BCD = 0;
  #20 BCD = 1;
  #30 BCD = 2;
  #40 BCD = 3;
  #50 BCD = 4;
  #60 BCD = 5;
  #70 BCD = 6;
  #80 BCD = 7;
  #90 BCD = 8;
  #100 BCD = 9;

join

Seven_Seg_Display_V2001_CA M0 (Display,   BCD);
endmodule
```

4.52 **(a)** Incrementer for unsigned 4-bit numbers

VERILOG

```
module Problem_4_52a_Data_Flow  (output [3: 0] sum, output carry, input [3: 0] A);
  assign {carry, sum} = A + 1;
endmodule

module t_Problem_4_52a_Data_Flow;
  wire [3: 0]    sum;
  wire           carry;
  reg [3: 0]     A;

  Problem_4_52a_Data_Flow  M0 (sum, carry, A);

  initial # 100 $finish;
  integer K;
  initial begin
    for (K = 0; K < 16; K = K + 1) begin A = K; #5; end
  end
endmodule
```

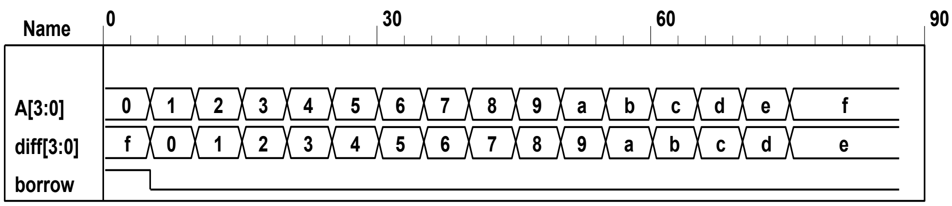
VHDL

```
entity Problem_4_52a_Data_Flow is
 port (sum: out Std_Logic_Vector (3 downto 0); carry: in Std_Logic;
          A: in Std_Logic_Vector (3: 0));
end Problem_4_52;

architecture Data_Flow of Problem_4_52a is
begin
 (carry & sum) <= A + '0001';
endmodule
```

(b) Decrementer for unsigned 4-bit numbers

Verilog

```
module Problem_4_52b_Data_Flow  (output [3: 0] diff, output borrow, input [3: 0] A);
  assign {borrow, diff} = A - 1;
endmodule

module t_Problem_4_52b_Data_Flow;
  wire [3: 0]        diff;
  wire               borrow;
  reg [3: 0]         A;

  Problem_4_52b_Data_Flow  M0 (diff, borrow, A);

  initial # 100 $finish;
  integer K;
  initial begin
    for (K = 0; K < 16; K = K + 1) begin A = K; #5; end
  end
endmodule
```



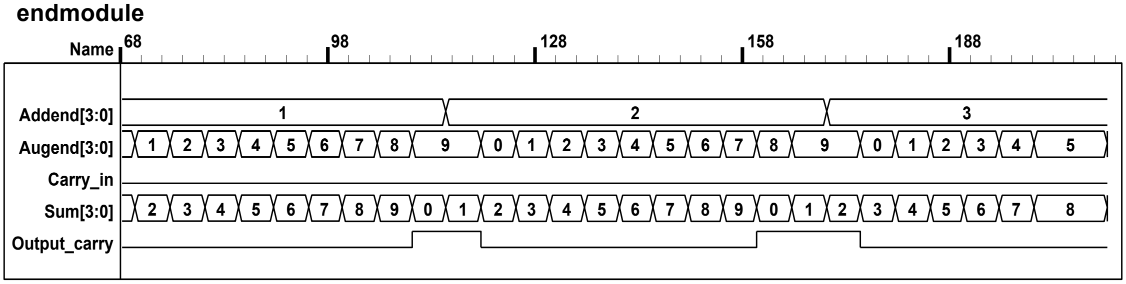
VHDL

```
entity Problem_4_52b is
    port (diff: out Std_Logic_Vector (3 downto 0); borrow: out Std_Logic;


     A: in Std_Logic_Vector (3 downto 0);
 end Problem_42b;

 architecture Data_Flow of Problem_4_52b is
 begin
     (borrow  & diff} <= A – '0001';
 endmodule
```

4.53 // BCD Adder

Verilog

```
module Problem_4_53_BCD_Adder (
  output          Output_carry,
  output [3: 0] Sum,
   input [3: 0]  Addend, Augend,
   input          Carry_in);
  supply0        gnd;
  wire [3: 0]    Z_Addend;
  wire           Carry_out;
  wire           C_out;
  assign         Z_Addend = {1'b0, Output_carry, Output_carry, 1'b0};
  wire [3: 0]    Z_sum;

  and (w1, Z_sum[3], Z_sum[2]);
  and (w2, Z_sum[3], Z_sum[1]);
  or (Output_carry, Carry_out, w1, w2);

  Adder_4_bit M0 (Carry_out, Z_sum, Addend, Augend, Carry_in);
  Adder_4_bit M1 (C_out, Sum, Z_Addend, Z_sum, gnd);
endmodule

module Adder_4_bit (output carry, output [3:0] sum, input [3: 0] a, b, input c_in);
  assign {carry, sum} = a + b + c_in;
endmodule

module t_Problem_4_53_Data_Flow;
  wire [3: 0]    Sum;
  wire           Output_carry;
  reg [3: 0]     Addend, Augend;
  reg            Carry_in;

  Problem_4_53_BCD_Adder M0 (Output_carry, Sum, Addend, Augend, Carry_in);

  initial # 1500 $finish;
  integer i, j, k;
  initial begin
    for (i = 0; i <= 1; i = i + 1) begin Carry_in = i; #5;
      for (j = 0; j <= 9; j = j +1) begin Addend = j; #5;
        for (k = 0; k <= 9; k = k + 1) begin Augend = k; #5;
        end
      end
    end
  end
endmodule
```



VHDL

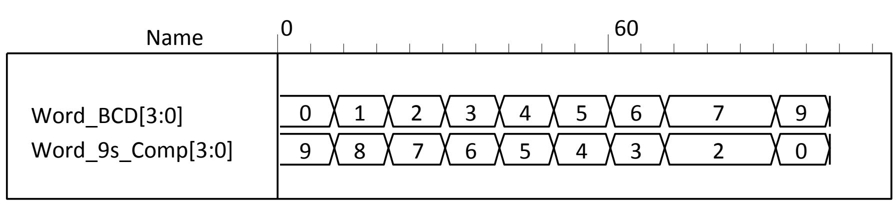
```
entity Problem_4_53_BCD_Adder is
     port(output_carry: out Std_Logic; Sum: out Std_Logic_Vector (3 downto 0);
input [3: 0]     Addend, Augend: in Std_Logic_Vector (3 downto 0);
end Problem_4_53_BCD_Adder
```

```
architecture Gates of Problem_4_53_BDC_Adder is
    signal [3: 0]    Z_Addend: Std_Logic_Vector (3 downto 0);
    signal Carry_out: Std_Logic;
    signal C_out: Std_Logic;
    signal Z_sum: Std_Logic_Vector (3 downto 0);
    constant gnd: Std_Logic := '0';
    component and2_gate port (y: out Std_Logic; xin1, xin2: in Std_Logic);
    component or3_gate port (y: out Std_Logic; xin1, xin2, xin3: in Std_Logic);
    component Adder_4_bit port (c: out Std_Logic; s: out Std_Logic_Vector (3 downto 0);
                    a, b: in Std_Logic_Vector (3 downto 0); cin: in Std_Logic);
begin
    Z_Addend <= ('0' & Output_carry & Output_carry & '0'}
    G1: and2_gate port map (y => w1, xin1 => Z_sum(3), xin2 => Z_sum(2));
    G2: and2_gate port map  (y => w2, xin1 => Z_sum(3), xin2 => Z_sum(1));
    G3: or3_gate    port map (y => Output_carry, xin1 => Carry_out, xin2 => w1, xin3 => w2);

    C0: Adder_4_bit port map (c => Carry_out, s => Z_sum, a => Addend, b => Augend, cin => Carry_in);
    C1: Adder_4_bit (port map (c => C_out, s => Sum, a => Z_Addend, b => Z_sum, cin => gnd);
end gates;

entity Adder_4_bit is
    port (carry out: Std_Logic; sum: Std_Logic_Vector (3 downto 0);
            a, b: in Std_Logic_Vector (3 downto 0); input c_in: in Std_Logic);
end Adder_4_bit

architecture Behavioral of Adder_4_bit is
begin
  (carry & sum <= a + b + ('000' & c_in);
end Behavioral;
```

4.54 **(a)** 9s Complement of BCD

VHDL

```
module Nines_Complementer (              // V2001
  output reg  [3: 0]  Word_9s_Comp,
  input       [3: 0]  Word_BCD
);
  always @ (Word_BCD) begin
    Word_9s_Comp = 4'b0;
    case (Word_BCD)
      4'b0000:  Word_9s_Comp = 4'b1001;      // 0 to 9
      4'b0001:  Word_9s_Comp = 4'b1000;      // 1 to 8
      4'b0010:  Word_9s_Comp = 4'b0111;      // 2 to 7
      4'b0011:  Word_9s_Comp = 4'b0110;      // 3 to 6
      4'b0100:  Word_9s_Comp = 4'b0101;      // 4 to 5
      4'b0101:  Word_9s_Comp = 4'b0100;      // 5 to 4
      4'b0110:  Word_9s_Comp = 4'b0011;      // 6 to 3
      4'b0111:  Word_9s_Comp = 4'b0010;      // 7 to 2
      4'b1000:  Word_9s_Comp = 4'b0001;      // 8 to 1
      4'b1001:  Word_9s_Comp = 4'b0000;      // 9 to 0
      default:  Word_9s_Comp = 4'b1111;      // Error detection
    endcase
  end
endmodule


module t_Nines_Complementer ();
  wire  [3: 0]  Word_9s_Comp;
  reg   [3: 0]  Word_BCD;

  Nines_Complementer M0 (Word_9s_Comp, Word_BCD);

  initial #11$finish;
  initial fork
        Word_BCD = 0;
    #10 Word_BCD = 1;
    #20 Word_BCD = 2;
    #30 Word_BCD = 3;
    #40 Word_BCD = 4;
    #50 Word_BCD = 5;
    #60 Word_BCD = 6;
    #70 Word_BCD = 7;
    #20 Word_BCD = 8;
    #90 Word_BCD = 9;
    #100 Word_BCD = 4'b1100;       // Confirm error detection
  join
endmodule
```



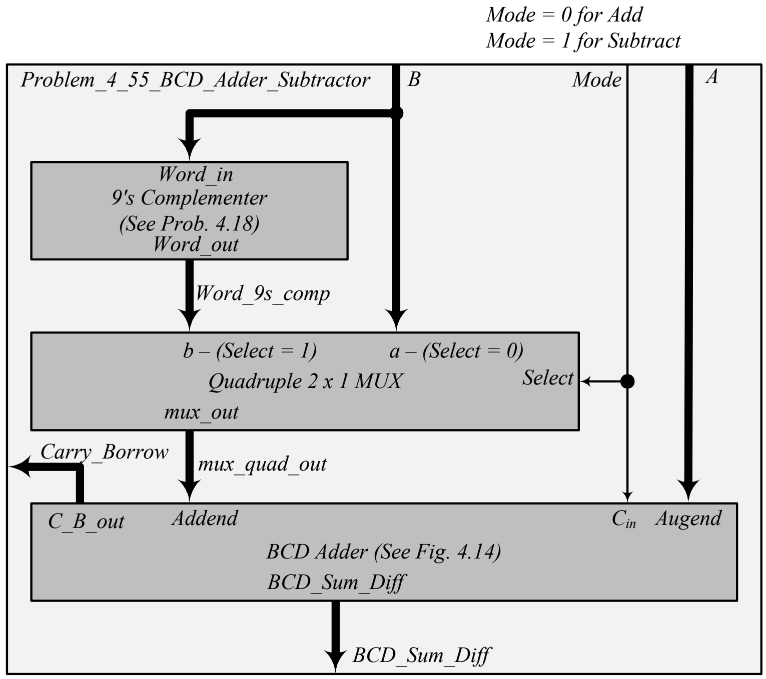
VHDL

```
entity Nines_Complementer is
    port (Word_9s_Comp: out Std_Logic_Vector (3 downto 0); Word_BCD: in Std_Logic_Vector (3
downto 0));
end Nines_Complementer;

architecture Behavioral of Nines_complementer is
  process (Word_BCD) begin
    Word_9s_Comp <= '0000';
    case (Word_BCD)
      when '0000'   => Word_9s_Comp <= '1001';        // 0 to 9
      when '0001'   => Word_9s_Comp <= '1000';
      when '0010'   => Word_9s_Comp <= '0111';        // 2 to 7
      when '0011'   => Word_9s_Comp <= '0110';        // 3 to 6
      when '0100'   => Word_9s_Comp <= '0101';        // 4 to 5
      when '0101'   => Word_9s_Comp <= '0100';        // 5 to 4
      when '0110'   => Word_9s_Comp <= '0011';        // 6 to 3
      when '0111'   => Word_9s_Comp <= '0010';        // 7 to 2
      when '1000'   => Word_9s_Comp <= '0001';        // 8 to 1
      when '1001'   => Word_9s_Comp <= '0000';        // 9 to 0
      whebn others'     =>  Word_9s_Comp <= '1111';        // Error detection
    end case
  end Behavioral
```

(b)

```
module Nine_Complementer (
  output reg [3: 0] Word_9s_Comp,
  input [3: 0] Word_5421
);
always       @ (Word_5421) begin
Word_9s_Comp = 4'b0;
case   (Word_5421)
   4'b0000:    Word_9s_Comp = 4'b1100;              // 0 to 9
   4'b0001:    Word_9s_Comp = 4'b1011;              // 1 to 8
   4'b0010:    Word_9s_Comp = 4'b1010;              // 2 to 7
   4'b0011:    Word_9s_Comp = 4'b1001;              // 3 to 6
   4'b0100:    Word_9s_Comp = 4'b1000;              // 4 to 5
   4'b1000:    Word_9s_Comp = 4'b0100;              // 5 to 4
   4'b1001:    Word_9s_Comp = 4'b0011;              // 6 to 3
   4'b1010:    Word_9s_Comp = 4'b0010;              // 7 to 2
   4'b1011:    Word_9s_Comp = 4'b0001;              // 8 to 1
   4'b1100:    Word_9s_Comp = 4'b0000;              // 9 to 0
  default:     Word_9s_Comp = 4'b1111;              // Error detection
endcase
end
endmodule
```

4.55 From Problem 4.19:



Verilog

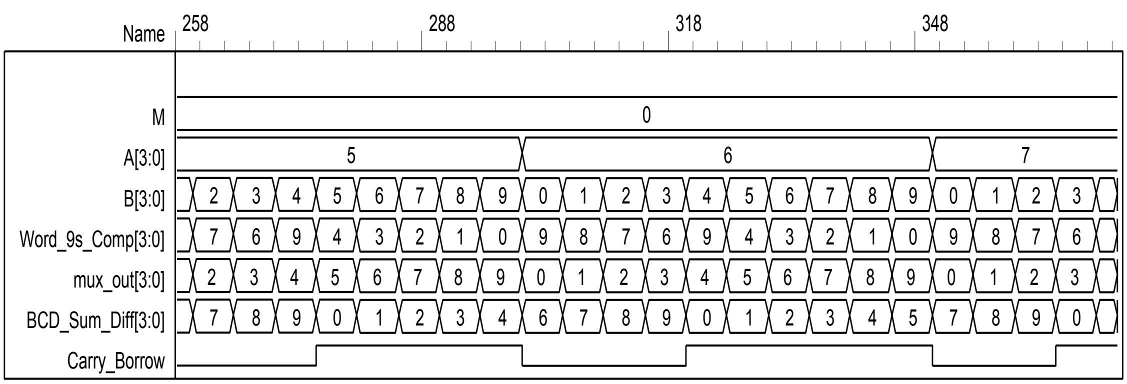
```
// BCD Adder – Subtractor
module Problem_4_55_BCD_Adder_Subtractor (
  output [3: 0]     BCD_Sum_Diff,
  output            Carry_Borrow,
  input [3: 0]      B, A,
  input             Mode

);
  wire [3: 0] Word_9s_Comp, mux_quad_out;
  Nines_Complementer   M0 (Word_9s_Comp, B);
  Quad_2_x_1_mux       M2 (mux_quad_out, Word_9s_Comp, B, Mode);
  BCD_Adder            M1 (Carry_Borrow,  BCD_Sum_Diff, mux_quad_out, A, Mode);
endmodule

module Nines_Complementer (              // V2001
  output reg   [3: 0]  Word_out,
  input        [3: 0]  Word_in
);
  always @ (Word_in) begin
    Word_out = 4'b0;
    case (Word_BCD)
      4'b0000:  Word_out = 4'b1001;    // 0 to 9
      4'b0001:  Word_out = 4'b1000;    // 1 to 8
      4'b0010:  Word_out = 4'b0111;    // 2 to 7
      4'b0011:  Word_out = 4'b0110;    // 3 to 6
      4'b0100:  Word_out = 4'b1001;    // 4 to 5
      4'b0101:  Word_out = 4'b0100;    // 5 to 4
      4'b0110:  Word_out = 4'b0011;    // 6 to 3
      4'b0111:  Word_out = 4'b0010;    // 7 to 2
      4'b1000:  Word_out = 4'b0001;    // 8 to 1
      4'b1001:  Word_out = 4'b0000;    // 9 to 0
      default:  Word_out = 4'b1111;    // Error detection
    endcase
```

```
  end
endmodule

module Quad_2_x_1_mux (output reg [3: 0] mux_out, input [3: 0] b, a,  input select);
  always @ (a, b,  select)
    case (select)
      0:   mux_out = a;
      1:   mux_out = b;
    endcase
endmodule

module BCD_Adder (
  output              Output_carry,
  output    [3: 0]    Sum,
  input     [3: 0]    Addend, Augend,
  input               C_in);
  supply0             gnd;
  wire      [3: 0]    Z_Addend;
  wire                Carry_out;
  wire                C_out;
  assign  Z_Addend = {1'b0, Output_carry, Output_carry, 1'b0};
  wire [3: 0] Z_sum;

  and (w1, Z_sum[3], Z_sum[2]);
  and (w2, Z_sum[3], Z_sum[1]);
  or (Output_carry, Carry_out, w1, w2);

  Adder_4_bit M0 (Carry_out, Z_sum, Addend, Augend, C_in);
  Adder_4_bit M1 (C_out, Sum, Z_Addend, Z_sum, gnd);
endmodule

module Adder_4_bit (output carry, output [3:0] sum, input [3: 0] a, b, input c_in);
  assign {carry, sum} = a + b + c_in;
endmodule

module t_Problem_4_55_BCD_Adder_Subtractor();
  wire  [3: 0]   BCD_Sum_Diff;
  wire           Carry_Borrow;
  reg   [3: 0]   B, A;
  reg            Mode;

  Problem_4_55_BCD_Adder_Subtractor M0 (BCD_Sum_Diff, Carry_Borrow, B, A, Mode);

  initial #1000 $finish;

  integer J, K, M;
  initial begin
    for (M = 0; M < 2; M = M + 1) begin
      for (J = 0; J < 10; J = J + 1) begin
        for (K = 0; K < 10; K = K + 1) begin
          A = J; B = K; Mode = M; #5 ;
        end
      end
    end
  end
endmodule
```

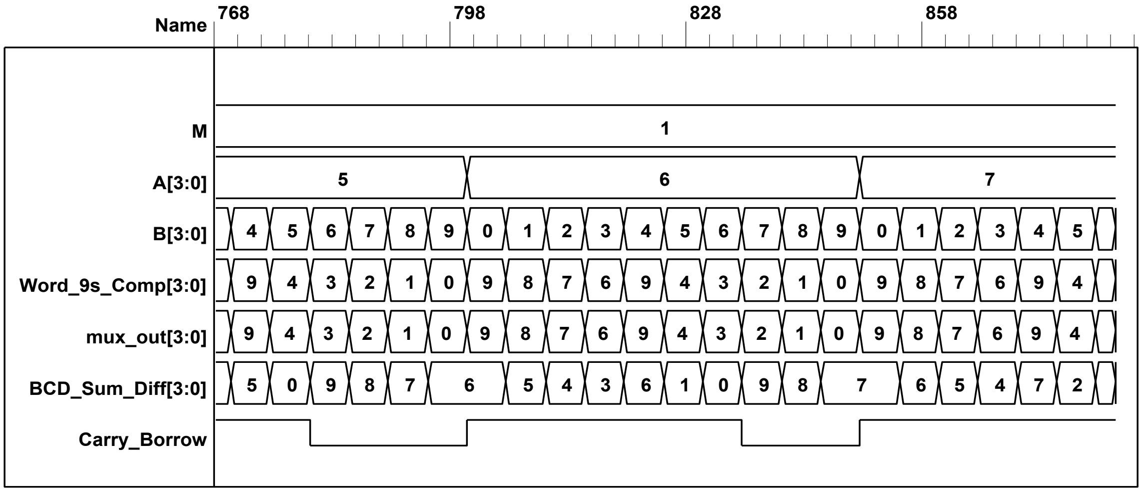
VHDL

```
// BCD Adder – Subtractor
entity Problem_4_55_BCD_Adder_Subtractor is
```

```
    port (BCD_Sum_Diff: out Std_Logic_Vector (3 downto 0);
              Carry_Borrow: out Std_Logic;
              B, A: in Std_Logic_Vector (3:0);
              Mode: in Std_Logic);
end Problem_4_55_BCD_Adder_Subtractor;

architecture Structural of Problem_4_55_BCD_Adder_Subtractor is
  signal Word_9s_comp, mux_quad_out: Std_Logic_Vector (3 downto 0);
  component Nines_Complementer port (Word_out: out Std_Logic_Vector (3 downto 0);
                                                   Word_in: in Std_Logic_Vector);
  component Quad_2_x_1_mux port (mux_out: out Std_Logic_Vector (3 downto 0);
                                             B, A: in Std_Logic_Vector (3 downto 0);
                                             Mode: Std_Logic);
  component BCD_Adder port (C_B_out: out Std_Logic;
                                      BCD_Sum_Diff: out Std_Logic_Vector (3 downto 0);
                                      Addend, Augend: in Std_Logic_Vector (3 downto 0);
                                      Cin: in Std_Logic);
begin

  M1: Nines_Complementer port map (Word_out => Word_9s_Comp, Word_in:  => B);
  M2: Quad_2_x_1_mux port map (mux_out => mux_quad_out, b => Word_9s_comp,
                                            a => B, Select => Mode);
M3: BCD_Adder port map (C_B_out => Carry_Borrow, Addend => mux_quad_out,
                                      Augend => A, cin => Mode,
                                      BCD_Sum_Diff => BCD_Sum_Diff);
end Structural
```



Note: For subtraction, Carry_Borrow = 1 indicates a positive result; Carry_Borrow = 0 indicates a negative result.



4.56

```
assign out = (A[0] & A[1] & A[2]) ;//assumes reg [3:0] A;
```

4.57 <u>Verilog</u>

```
module Prob_4_57(
  input D0, D1, D2, D3,
  output reg x_in, y_in, Valid
);

always @ (D0, D1, D2, D3) begin     // Note: positions numbered left to right
  casex ({D0, D1, D2, D3}
    4'b0000: {x_out, y_out, Valid} = 3'bxx_0;     // Invalid word
    4'b1xxx,: {x_out, y_out, Valid} = 3'b00_1;    // Bit 0, valid word
    4'b01xx: {x_out, y_out, Valid} = 3'b01_1;     // Bit 1, valid word
    4'b001x: {x_out, y_out, Valid} = 3'b10_1;     // Bit 2, valid word
    4'b0001: {x_out, y_out, Valid} = 3'b11_1;     // Bit 3, valid word
  endcase
end
endmodule

module t_Prob_4_57;
 wire    x_out, y_out, Valid;
 reg          D3, D2, D1, D0;
 integer    K;
 Prob_4_57 M0 (D0, D1, D2, D3, x_out, y_out, Valid
);
 initial #100 $finish;
 initial begin
   for (K = 0; K < 16; K = K + 1) begin {D0, D1, D2, D3} = K; #5 ; end
 end
endmodule
```

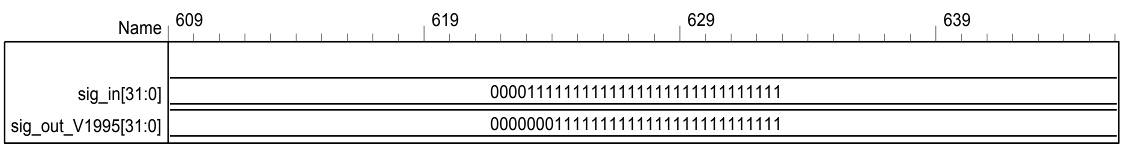
<u>VHDL</u>

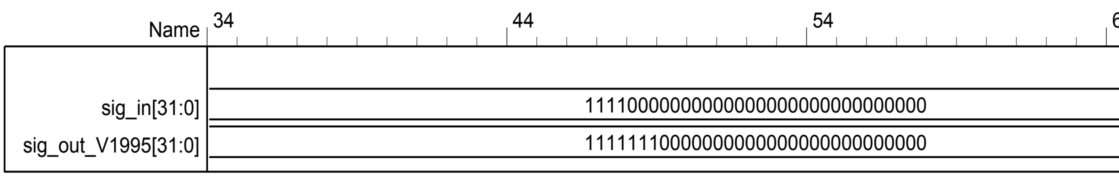
```
entity Prob_4_57 is
  port (D0, D1, D2, D3: in Std_Logic; x_out, y_out: out Std_Logic; Valid: out Std_Logic);
end Prob_4_57;

architecture Behavioral of Prob_4_57 is
begin
   x_out & y_out & Valid <= "000" when D0 & D1 & D2 & D3 = '0000'; else
                            "001" when D0 = 1; else
                            "011" when D0 = 0 and D1 = 1; else
                            "101" when D0 = 0 and D1 = 0 and D2 = 1; else
                            "111" when D0 & D1 & D2 & D3 = '0001'; end if;
  endcase;
end process;
end Behavioral;
```

4.58 **(a)**

Verilog

```
//module shift_right_by_3_V2001 (output [31: 0]  sig_out, input [31: 0] sig_in);
 // assign sig_out = sig_in >>> 3;
//endmodule

module shift_right_by_3_V1995 (output reg [31: 0]  sig_out, input [31: 0] sig_in);
  always @ (sig_in)
    sig_out = {sig_in[31], sig_in[31], sig_in[31],  sig_in[31: 3]};
endmodule


module t_shift_right_by_3 ();
  wire [31: 0]  sig_out_V1995;
  wire [31: 0]  sig_out_V2001;

  reg [31: 0] sig_in;

  //shift_right_by_3_V2001 M0 (sig_out_V2001, sig_in);

  shift_right_by_3_V1995 M1 (sig_out_V1995, sig_in);
  integer k;
  initial #1000 $finish;
  initial begin
    sig_in = 32'hf000_0000;
    #100 sig_in = 32'h8fff_ffff;
    #500 sig_in = 32'h0fff_ffff;
  end
endmodule
```





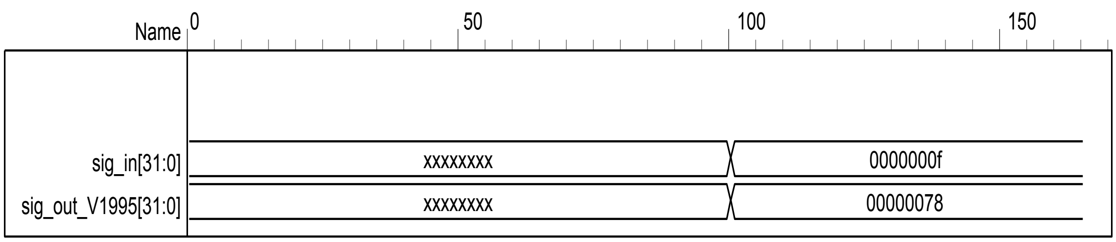
VHDL

```
entity shift_right_by_3 is
     port ( sig_out: out Std_Logic_Vector (31 downto 0), sig_in: in Std_Logic_Vector (31 downto 0));
end shift-right_by_3;

architecture Behavioral of shift_right_by_3 is
     sig_out <= (sig_in[31] & sig_in[31] & sig_in[31] &  sig_in(31: 3));
end Behavioral;
```

(b)

Verilog

```
module shift_left_by_3_V2001 (output [31: 0]  sig_out, input [31: 0] sig_in);
  assign sig_out = sig_in <<< 3;

//module shift_left_by_3_V1995 (output reg [31: 0]  sig_out, input [31: 0] sig_in);
  //always @ (sig_in)
   // sig_out = {sig_in[28: 3], 3'b0};
endmodule


module t_shift_left_by_3 ();
  wire [31: 0]  sig_out_V1995;
  wire [31: 0]  sig_out_V2001;

  reg [31: 0] sig_in;

  shift_left_by_3_V2001 M0 (sig_out_V2001, sig_in);

integer k;
  initial #1000 $finish;
  initial begin
    sig_in = 32'hf000_0000;
    #100 sig_in = 32'h8fff_ffff;
    #500 sig_in = 32'h0fff_ffff;
  end
endmodule
```



VHDL

```
entity  shift_left_by_3 is
     port (sig_out: out Std_Logic_Vector (31 downto 0); sig_in: in Std_Logic_Vector (31 downto 0));
end shift_left_by_3;

architecture Behavioral of shift_left_by_3 is
     sig_out <= (sig_in(28:0) & sig_in '000'); -- sig_out <= sig_in sla 3;
end Behavioral;
```

4.59 **Verilog**

```
module BCD_to_Dec_Decoder (output reg [3: 0] Dec_out, input [3: 0] BCD_in);
  always @ (BCD_in) begin
    Dec_out = 0;
    case (BCD_in)
      4'b0000:   Dec_out = 0;
      4'b0001:   Dec_out = 1;
      4'b0010:   Dec_out = 2;
      4'b0011:   Dec_out = 3;
      4'bx100:   Dec_out = 4;
      4'bx101:   Dec_out = 5;
      4'bx110:   Dec_out = 6;
      4'bx111:   Dec_out = 7;
      4'b1xx0:   Dec_out = 8;
      4'b1xx1:   Dec_out = 9;
      default:   Dec_out = 4'bxxxx;
    endcase
  end
endmodule
```

VHDL

```
entity BCD_to_Dec_Decoder (Dec_out: out integer), BCD_in: in
        Std_Logic_Vector (3 downto 0));
end BCD_to_Dec_Decoder

architecture Behavioral of BCD_to_Dec_Decoder is

  process (BCD_in) begin
     Dec_out <= 0;
     case (BCD_in)
     when '0000'  =>      Dec_out <= 0;
     when '0001'  =>      Dec_out <= 1;
     when '0010'  =>      Dec_out <= 2;
     when '0011'  =>      Dec_out <= 3;
     when 'x100'  =>      Dec_out <= 4;
     when 'x101'  =>      Dec_out <= 5;
     when 'x110'  =>      Dec_out <= 6;
     when 'x111'  =>      Dec_out <= 7;
     when '1xx0'  =>      Dec_out <= 8;
     when '1xx1'  =>      Dec_out <= 9;
     when others  =>      Dec_out <= "xxxx";
    end case;
  end process;
end Behavioral;
```

4.60 **Verilog**

```
module Even_Parity_Checker_4 (output  P, C, input x, y, z);
  xor (w1, x,  y);
  xor (P, w1, z);
  xor (C, w1, w2);
  xor (w2, z, P);
endmodule
```

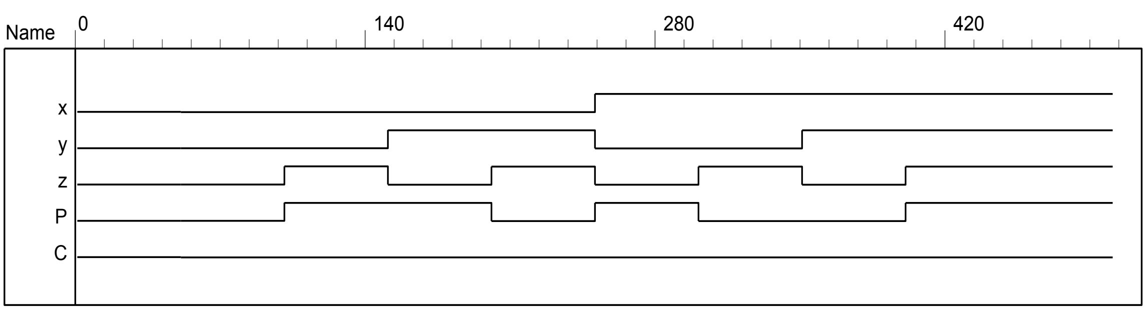
See Problem 4.62 for testbench and waveforms.

VHDL

```
entity Even_Parity_Checker_4 is
    port (P, C: out Std_Logic; x, y, z: in Std_Logic);
end Even_Parity_Checker_4;

architecture Boolean_Eq of Even_Parity_Checker_4 is
    component xor2_gate port (y: out bit; xin1, xin2: out bit);
begin
    G1:    xor2_gate port map (y => w1, xin1 => x, xin2 => y);
    G2:    xor2_gate port map (y => P, xin1 => w1, xin2 => z);
    G3:    xor2_gate port map (C => w1, xin1 => w1, xin2 => w2);
    G4:    xor2_gate port map (y => w2, xin1 => x, xin2 => P);
end Boolean_Eq;
```

4.61 **Verilog**

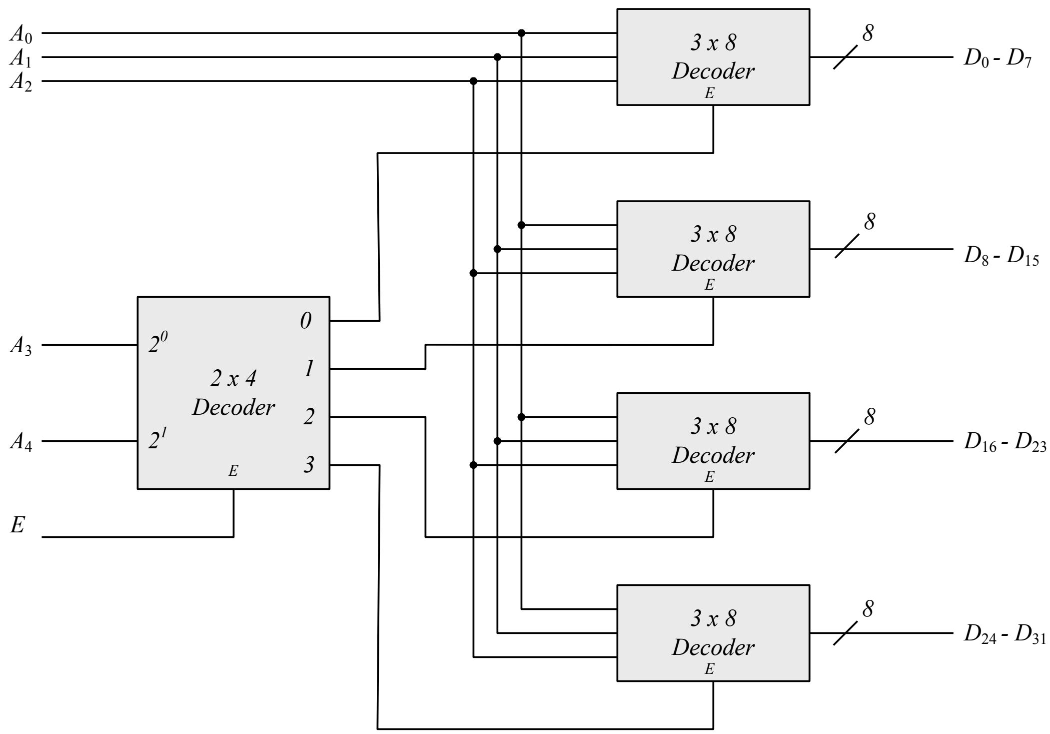
```
module Even_Parity_Checker_4 (output  P, C, input x, y, z);
  assign w1 = x ^ y;
  assign P = w1 ^ z;
  assign C = w1 ^ w2;
  assign w2 = z ^ P;
endmodule
```



VHDL

```
entity Even_Parity_Checker_4 is
    port (P, C: out bit, x, y, z: in bit);
end Even_Parity_Checker_4;

architecture Boolean_Eq of Even_Parity_Checker_4 is
    w1 <= x xor y;
    P <= w1 xor z;
    C <= w1 xor w2;
    w2 <= z xor P;
end Boolean_Eq;
```

4.62



Verilog

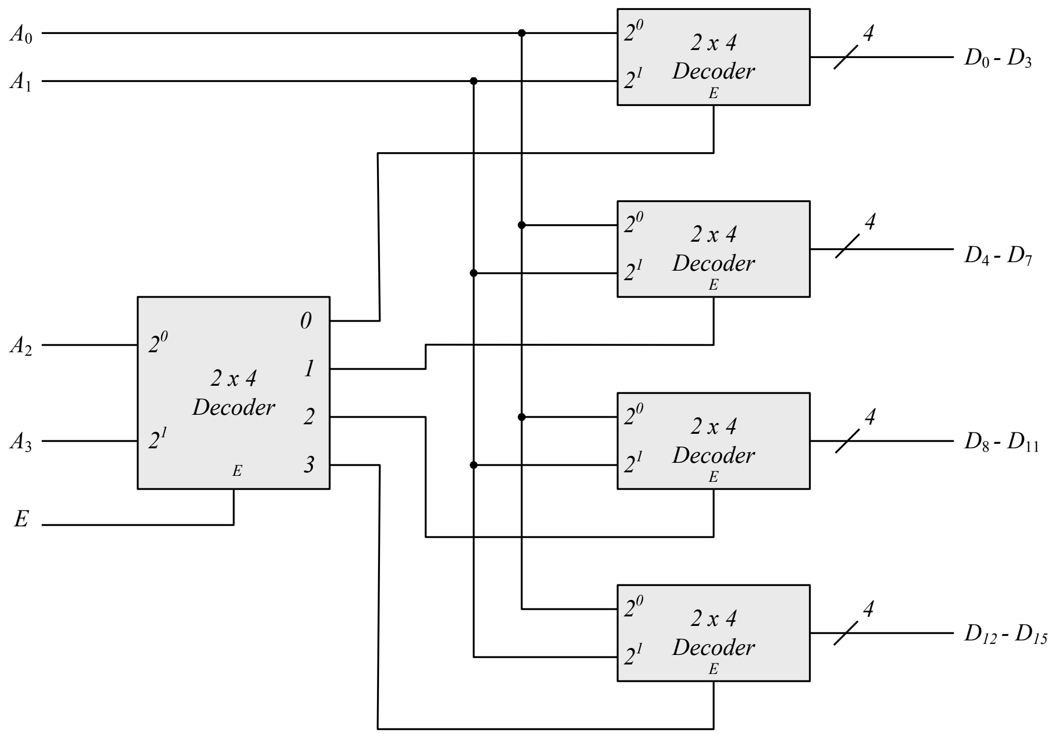
```
module Decoder_5x32 (
  output D31, D30, D29, D28, D27, D26, D25, D24, D23, D22, D21, D20, D19, D18, D17, D16,
         D15, D14, D13, D12, D11, D10, D9, D8, D7, D6, D5, D4, D3, D2, D1, D0,
  input A4, A3, A2, A1, A0, E;
  wire E3, E2, E1, E0;
  Decoder_3x8 M0 (D7, D6, D5, D4, D3, D2, D1, D0, A2, aA1, A0, E0);
  Decoder_3x8 M1 (D15, D14, D13, D12, D11, D10, D9, D8, A2, A1, A0, E1);
  Decoder_3x8 M2 (D23, D22, D21, D20, D19, D18, D17, D16, in2, in1, in0, E2);
  Decoder_3x8 M3 (D31, D30, D29, D28, D27, D26, D25, D24, A2, A1, A0, E3);
  Decoder_2x4 M4 (E3, E2, E1, E0, A4, A3, E);
endmodule

module Decoder_3x8 (output D7, D6, D5, D4, D3, D2, D1, D0, input in2, in1, in0, E);
  not (in2_bar, in2);
  not (in1_bar, in1);
  not (in0_bar, in0);
  and (D0, in2_bar, in1_bar, in0_bar, E);
  and (D1, in2_bar, in1_bar, in0, E);
  and (D2, in2_bar, in1, in0_bar, E);
  and (D3, in2_bar, in1, in0, E);
  and (D4, in2, in1_bar, in0_bar, E);
  and (D5, in2, in1_bar, in0, E);
  and (D6, in2, in1, in0_bar, E);
  and (D7, in2, in1, in0, E);
endmodule
```

VHDL

```
entity Decoder_5x32 is
   port (D31, D30, D29, D28, D27, D26, D25, D24, D23, D22, D21, D20, D19, D18, D17, D16,
         D15, D14, D13, D12, D11, D10, D9, D8, D7, D6, D5, D4, D3, D2, D1, D0: out Std_Logic;
```

```
        A4, A3, A2, A1, A0, E: in Std_Logic);

end Decoder_5x32;

architecture Gates of Decoder_5x32 is
    signal E3, E2, E1, E0: Std_Logic;
    component Decoder_3x8 port (D7, D6, D5, D4, D3, D2, D1, D0: out Std_Logic;
        in2, in1, in0, E: in Std_Logic);
begin
G1: Decoder_3x8 port map (D7, D6, D5, D4, D3, D2, D1, D0, A2, A1, A0, E0);
G2: Decoder_3x8 (D15, D14, D13, D12, D11, D10, D9, D8, A2, A1, A0, E1);
G3: Decoder_3x8 (D23, D22, D21, D20, D19, D18, D17, D16, in2, in1, in0, E2);
G4: Decoder_3x8 (D31, D30, D29, D28, D27, D26, D25, D24, A2, A1, A0, E3);
G5: Decoder_2x4 (E3, E2, E1, E0, A4, A3, E);
end Gates;

entity Decoder_3x8 is
    port (D7, D6, D5, D4, D3, D2, D1, D0: out Std_Logic; in2, in1, in0, E: in Std_Logic);
end Decoder_3x8;

architecture Boolean_Eq of Decoder_3x8 is
    signal in0_bar, in1_bar, in2_bar: Std_Logic;
    component not_gate port (y: out Std_Logic; xin: in Std_Logic);
    component and3_gate port (y: out Std_Logic; xin1, xin2, xin3: in Std_Logic);
    component and4_gate port (y: out Std_Logic; xin1, xin2, xin3, xin4: in Std_Logic);
begin
    G1: not_gate port map (y => in2_bar, xin => in2);
    G2: not_gate port map (y => in1_bar, xin => in1);
    G3: not_gate port map (y => in0_bar, xin => in0);
    G4: and4_gate port map (y => D0, xin1 => in2_bar, xin2 => in1_bar, xin3 => in0_bar, xin4 => E);
    G5: and4_gate port map (y => D1, xin1 => in2_bar, xin2 => in1_bar, xin3 => in0, xin4 => E);
    G6: and4_gate port map (y => D2, xin1 => in2_bar, xin2 => in1, xin3 => in0_bar, xin4 => E);
    G7: and4_gate port map (y => D3, xin1 => in2_bar, xin2 => in1, xin3 => in0, xin4 => E);
    G8: and4_gate port map (y => D4, xin1 => in2, xin2 => in1_bar, xin3 => in0_bar, xin4 => E);
    G9: and4_gate port map (y => D5, xin1 => in2, xin2 => in1_bar, xin3 => in0_bar, xin4 => E);
    G9: and4_gate port map (y => D6, xin1 => in2, xin2 => in1, xin3 => in0_bar, xin4 => E);
    G9: and4_gate port map (y => D7, xin1 => in2, xin2 => in1, xin3 => in0, xin4 => E);
end Boolean_Eq;

entity Decoder_2x4 is
    port (y0, y1, y2, y3: out Std_Logic; x0, x1, E: in Std_Logic);
end Decoder_2x4;

architecture Boolean_Eq of Decoder_2x4 is
    signal x0_bar, x1_bar: Std_Logic;
begin
    y0 <= '0' when E = 'x'; else '1' when E and x0 and x1;
    y1 <= '0' when E = 'x'; else '1' when and x0 and x1_bar;
    y2 <= '0' when E = 'x'; else '1' when and x0_bar and x1;
    y3 <= '0' when E = 'x'; else '1' when and x0_bar and x1_bar;
end Boolean_Eq;
```

4.63



Verilog

```
module Decoder_2x4 (output D3, D2, D1, D0, input in1, in0, E);
  not (in1_bar, in1);
  not (in0_bar, in0);
  and (D0, in1_bar, in0_bar, E);
  and (D1, in1_bar, in0, E);
  and (D2, in1, in0_bar, E);
  and (D3, in1, in0, E);
endmodule

module Decoder_4x16 (
  output D15, D14, D13, D12, D11, D10, D9, D8, D7, D6, D5, D4, D3, D2, D1, D0,
  input A3, A2, A1, A0, E);
  wire E3, E2, E1, E0;
  Decoder_2x4 M0 (output D3, D2, D1, D0, input in1, in0, E0);
  Decoder_2x4 M1 (output D7, D6, D5, D4, input in1, in0, E1);
  Decoder_2x4 M2 (output D11, D10, D9, D8, input in1, in0, E2);
  Decoder_2x4 M3 (output D15, D14, D13, D12, input in1, in0, E3);
  Decoder_2x4 M4 (output E3, E2, E1, E0, input A3, A2, E);
endmodule
```

VHDL

```
entity Decoder_2x4
    port (D3, D2, D1, D0: out Std_Logic; in1, in0, E: in Std_Logic);
end Decoder+2x4;

architecture Boolen Eq of Decoder_2x4 is
    signal in1_bar, in0_bar: Std_Logic;
begin
  in1_bar <= not in1);
  in0_bar <= not in0);
  D0 <= in1_bar and in0_bar and E);
  D1 <= in1_bar and in0 and E);
```

```
  D2 <= in1 and in0_bar and E);
  D3 <=  in1 and in0 and E);
end Boolean_Eq;

entity Decoder_4x16  is
    port (D15, D14, D13, D12, D11, D10, D9, D8, D7, D6, D5, D4, D3, D2, D1, D0: Std_Logic;
          A3, A2, A1, A0, E: in Std_Logic);
end Decoder_4x16;

architecture Gates of Decoder 4x16 is
    signal E3, E2, E1, E0: Std_Logic;
    component Decoder_2x4 port (D3, D2, D1, D0: out Std_Logic; in1, in0, E0: in Std_Logic);
begin
    M0: Decoder_2x4 port map (D3, D2, D1, D0, in1, in0, E0);
    M1: Decoder_2x4 port map (D7, D6, D5, D4, in1, in0, E1);
    M2: Decoder_2x4 port map (D11, D10, D9, D8, in1, in0, E2);
    M3: Decoder_2x4 port map (D15, D14, D13, D12, in1, in0, E3);
    M4: Decoder_2x4 port map (E3, E2, E1, E0, A3, A2, E);
endmodule
```

4.64

Inputs								*Outputs*	
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	$x\ y\ z$	V
0	0	0	0	0	0	0	0	x x x	0
1	0	0	0	0	0	0	0	0 0 0	1
x	1	0	0	0	0	0	0	0 0 1	1
x	x	1	0	0	0	0	0	0 1 0	1
x	x	x	1	0	0	0	0	0 1 1	1
x	x	x	x	1	0	0	0	1 0 0	1
x	x	x	x	x	1	0	0	1 0 1	1
x	x	x	x	x	x	1	0	1 0 0	1
x	x	x	x	x	x	x	1	1 1 1	1

If $D_2 = 1$, $D_6 = 1$, all others = 0
Output xyz = 100 and V = 1

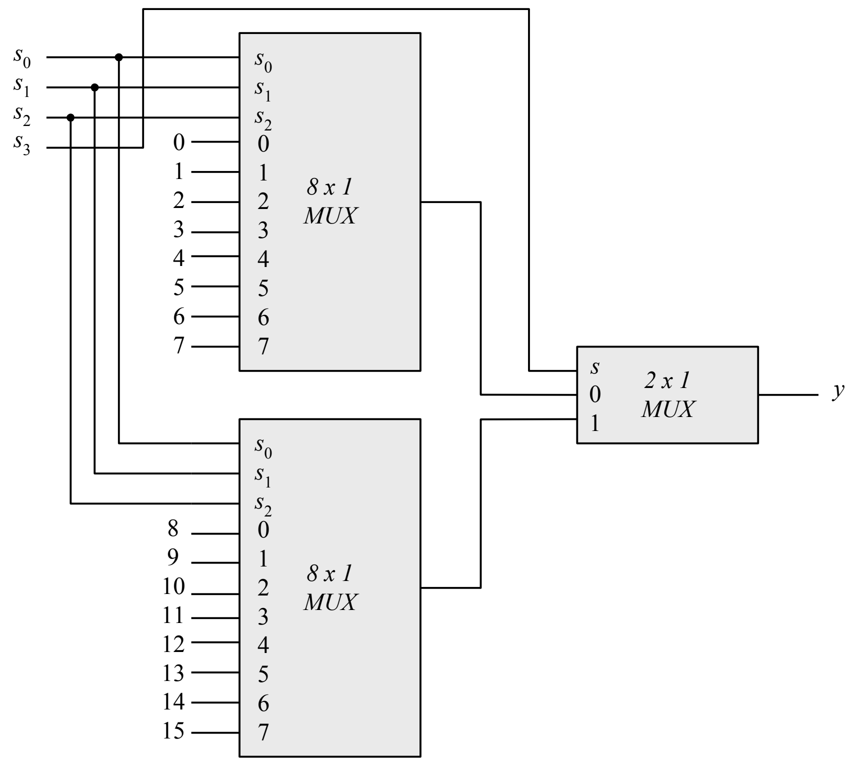
Verilog

```
module Prob_4_64 (output x, y, x, V, input, D0, D1, D2, D3, D4,D5 D6, D7);

  always @( D0, D1, D2, D3, D4,D5 D6, D7)
   case({D0, D1, D2, D3, D4,D5 D6, D7})
     8'b0000_0000:    {x, y, x, V} = 4'bxxx0;
     8'b1000_0000:    {x, y, x, V} = 4'b0001;
     8'b0100_0000:    {x, y, x, V} = 4'b0011;
     8'b0010_0000:    {x, y, x, V} = 4'b0101;

     8'b0001_0000:    {x, y, x, V} = 4'b0111;
     8'b0000_1000:    {x, y, x, V} = 4'b1001;
     8'b0000_0100:    {x, y, x, V} = 4'b1011;
     8'b0000_0010:    {x, y, x, V} = 4'b1001;
     8'b0000_0001:    {x, y, x, V} = 4'b1111;
     default:         {x, y, x, V} = 4'b1010;         // Use for error detection
   endcase
endmodule
```

VHDL

```
entity Prob_4_64 is
   port (x, y, x, V: out Std_Logic, D0, D1, D2, D3, D4,D5 D6, D7: in Std_Logic);
end Prob_4_64;

architecture Behavioral is
begin
   process (D0  D1, D2, D3, D4,D5, D6, D7)
   case (D0 & D1 & D2 & D3 & D4 & D5 & D6 & D7)
      when '0000_0000'    =>  (x & y & x & V) <= 'xxx0';
      when '1000_0000'    =>  (x & y & x & V) <= '0001';
      when '1000_0000'    =>  (x & y & x & V) <= '0011';
      when '0010_0000'    =>  (x & y & x & V) <= '0101';
      when '0001_0000'    =>  (x & y & x & V) <= '0111';
      when '0000_1000'    =>  (x & y & x & V) <= '1001';
      when '0000_0100'    =>  (x & y & x & V) <= '1011';
      when '0000_0010'    =>  (x & y & x & V) <= 1001';
      when '0000_0001'    =>  (x & y & x & V) <= '1111';
      when others:        => (x & y & x & V) <= '1010';         // Use for error detection
   endcase
end Behavioral;
```

4.65



Verilog

```
module Mux_2x1 (
  output y_out,
  input in1, in0, sel);
  not (sel_bar, sel);
  and (y0, in0, sel);
  and (y1, in1, sel);
  or (y_out, in0, in1, sel_bar
);
endmodule

module Mux_4x1 (
  output y_out,
  input in3, in2, in1, in0, sel1, sel0);
  not (sel_1_bar, sel1);
  and (s0, sel_1_bar, sel0);
  and (s1, sel[1], sel0);
  Mux_2x1 M0 (y_M0, in0, in1, s0);
  Mux_2x1   M1 (y_M1, in2, in3, s1);
  or (y_out, y_M0, y_M1
);
endmodule

module Mux_8x1 (
  output y_out,
  input in7, in6, in5, in4, in3, in2, in1, in0, sel2, sel1, sel0
);
  Mux_4x1 M0 (y_M0,  in3, in2, in1, in0, sel1, sel0);
  Mux_4x1 M1 (y_M1, in7, in6, in5, in4, sl1, sel0);
  Mux_2x1 M2 (y_out, y_M0, y_M1, sel2);
endmodule

module Mux_16x1 (
```

```
  output y_out,
  input in15, in14, in13, in12, in11, in10, in9, in8, in7, in6, in5, in4, in3, in2, in1, in0, sel3, sel2, sel1, sel0
);
  Mux_8x1 M0 (y_M0, in7, in6, in5, in4, in3, in2, in1, in0, sel2, sel1, sel0);
  Mux_8x1 M1 (y_M1, in15, in14, in13, in12, in11, in10, in9, in8, sel2, sel1, sel0);
  Mux_2x1 M2 (y_out, y_M0, y_M1, sel3);
endmodule
```

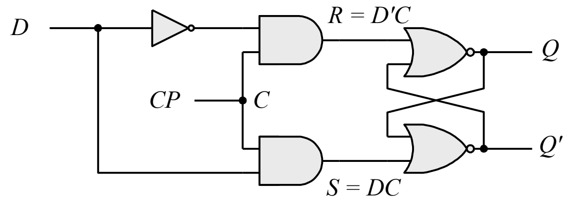
VHDL

```
entity Mux_2x1 is
    port ( y_out: out Std_Logic; in1, in0, sel: in Std_Logic);
end Mux_2x1;

architecture Gates of Mux_2x1 is
    component not_gate port (y: out, Std_Logic; xin: in Std_Logic);
    component and2_gate port (y: out, Std_Logic; xin1, xin2: in Std_Logic);
    component or2_gate port (y: out, Std_Logic; xin1, xin2: in Std_Logic);
begin
    G1:    not_gate port map  (y => sel_bar, xin => sel);
    G2:    and2_gate port map  (y => y0, xin1 => in0, xin2 => sel);
    G3:    and2_gate port map  (y => y1, xin1 => in1, xin2 => sel);
    G4:    or2_gate port map  (y => y1, in1 => in1, in2 => sel);
    G5:    or (y_out, in0, in1, sel_bar
end Gates;

entity Mux_4x1 is
    port (y_out: out Std_Logic; in3, in2, in1, in0, sel1, sel0: in Std_Logic);
end Mux_4x1;

architecture Gates of Mux_4x1 is
    component not_gate   port (y: out, Std_Logic; xin: in Std_Logic);
    component and2_gate port (y: out, Std_Logic; xin1, xin2: in Std_Logic);
    component or2_gate port (y: out, Std_Logic; xin1, xin2: in Std_Logic);
    component Mux_2x1 port ( y: out Std_Logic; xin1, xin2, xin3: in Std_Logic);

begin
    G1: not_gate port map (y => sel_1_bar, xin => sel1);
    G2: and2_gate port map (y =: s0, xin1: sel_1_bar, xin2 => sel0);
    G3: and2_gate port map (y > s1, xin1 => sel[1], xin2 => sel0);
    G4: Mux_2x1 port map ( y => y_out, xin1 => in0, xin2 => in1, xin3 => s0);
    G5: Mux_2x1 port map ( y => y_out, xin1 => in2, xin2 => in3, xin3 => s1);
end Gates;

entity Mux_8x1 is
    port (y: out Std_Logic; in7, in6, in5, in4, in3, in2, in1, in0, sel2, sel1, sel0: in Std_Logic);
end Mux_bx1;

architecture Structural of Mux_8x1 is
    component MUX_4x1 port (y_out: out Std_Logic; in3, in2, in1, in0, sel1, sel0: in Std_Logic);
    component Mux_2x1 port ( y_out: out Std_Logic; in1, in0, sel: in Std_Logic);
end Mux_2x1;


    G1: Mux_4x1 port map (y_out => y_M0,  in3 => in3, in2 => in2, in1 => in1, in0 => in0,
            sel1 => sel1, sel0 => sel0);
    G2: Mux_4x1 (y _out => y_M1, in3 => in7, in2 => in6, in1 => in5, in0 => in4,
            sel1 => sel1, sel0 => sel0);
    G3: Mux_2x1 M2 (y_out, in2 => y_M0, in1 => y_M1, sel => sel2);
end Structural;
```

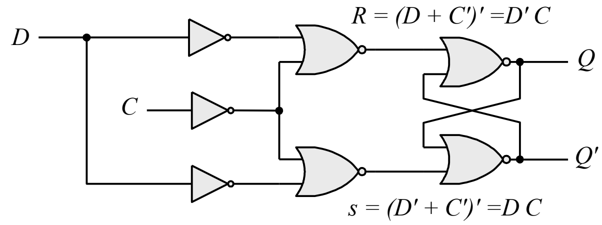
```
entity Mux_16x1 is
    port (y_out: out Std_Logic; in15, in14, in13, in12, in11, in10, in9, in8, in7, in6, in5, in4,
          in3, in2, in1, in0, sel3, sel2, sel1, sel0: in Std_Logic);
end Mux_16x1;

architecture Structural of Mux_16x1 is
    component   Mux_8x1 port (y: out Std_Logic; in7, in6, in5, in4, in3, in2, in1, in0,
                sel2, sel1, sel0: in Std_Logic);
    component Mux_2x1 port ( y_out: out Std_Logic; in1, in0, sel: in Std_Logic);
begin
  G0: Mux_8x1 port map (> y_M0, in7 => in7, in6 => in6, in5 => in5, in4 => in4, in3 => in3,
        in2 => in2, in1 => in1, in0 => in0, sel2 => sel2, sel1 => sel1, sel0 => sel0);
  G1: Mux_8x1 port map (y => y_M1, in7: in15, in6 => in14, in5 => in13, in4 => in12, in3 => in11,
        in2 => in10, in1 => in9, in0 => in8, sel2 => sel2, sel1 => sel1, sel0 => sel0);
  G2: Mux_2x1 port map (y => y_out, in1 => y_M0, in0 => y_M1, sel => sel3);
end Structural;
```

CHAPTER 5

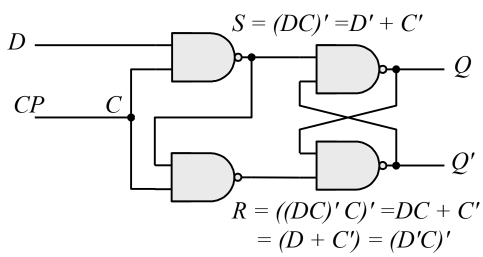
5.1 **(a)**



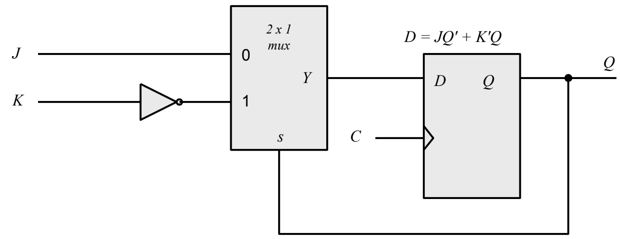
(b)



(c)



5.2



5.3 $Q'(t+1) = (JQ' + K'Q)' = (J' + Q)(K + Q') = J'K + KQ + J'Q'$

On simplifying, we get:

$Q'(t+1) = J'Q' + KQ$

5.4

(a)

A	B	$Q(t+1)$
0	0	0
0	1	$Q(t)$
1	0	$Q'(t)$
1	1	1

(b)

A	B	$Q(t)$	$Q(t+1)$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

$Q(t+1) = \Sigma(3, 4, 6, 7) = AQ' + BQ$

(c)

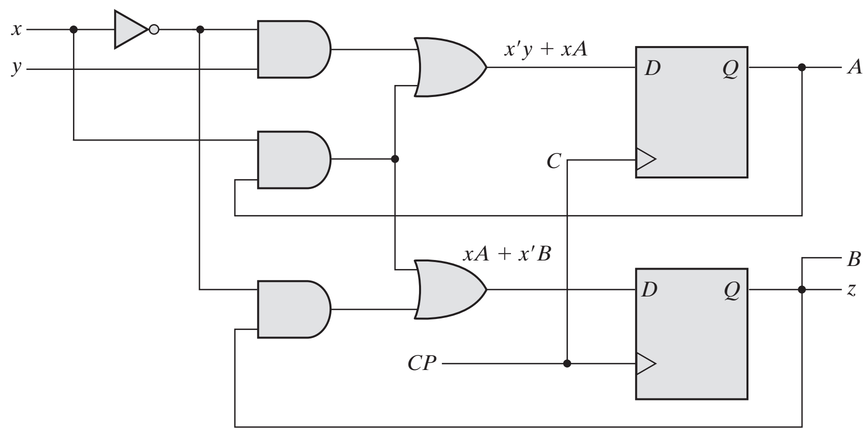
$Q(t)$	$Q(t+1)$	A	B
0	0	0	×
0	1	1	×
1	0	×	0
1	1	×	1

(d) The flip-flop can be converted to a D flip-flop by connecting inputs A and B together.

5.5

The truth table describes a combinational circuit.
The state table describes a sequential circuit.
The characteristic table describes the operation of a flip-flop.
The excitation table gives the values of flip-flop inputs for a given state transition.
The four equations correspond to the algebraic expression of the four tables.

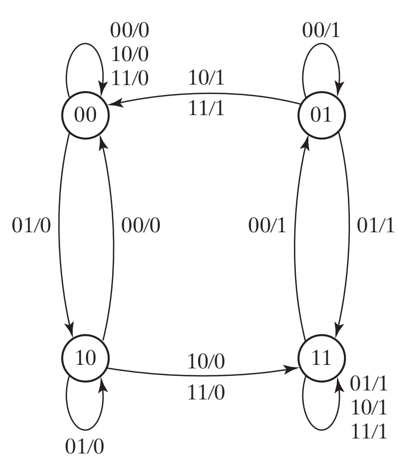
5.6 **(a)**



(b)

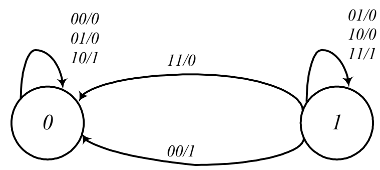
Present state		Inputs		Next state		Output
A	B	x	y	A	B	z
0	0	0	0	0	0	0
0	0	0	1	1	0	0
0	0	1	0	0	0	0
0	0	1	1	0	0	0
0	1	0	0	0	1	1
0	1	0	1	1	1	1
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	0	0	0
1	0	0	1	1	0	0
1	0	1	0	1	1	0
1	0	1	1	1	1	0
1	1	0	0	0	1	1
1	1	0	1	1	1	1
1	1	1	0	1	1	1
1	1	1	1	1	1	1

(c)



5.7

Present state Q	Inputs x	y	Next state Q	Output S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



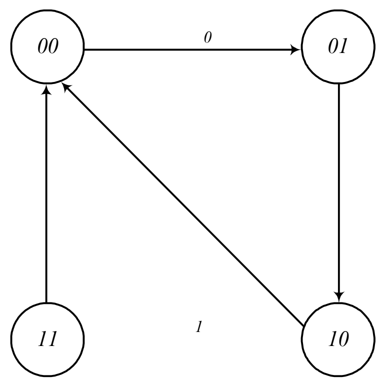
$$S = x \oplus y \oplus Q$$
$$Q(t + 1) = xy + xQ + yQ$$

5.8 A counter with a repeated sequence of 00, 01, 10.

Present State A	B	Next State A	B	FF Inputs T_A	T_B
0	0	0	1	0	1
0	1	1	0	1	1
1	0	0	0	1	0
1	1	0	0	1	1

$T_A = A + B$
$T_B = A' + B$

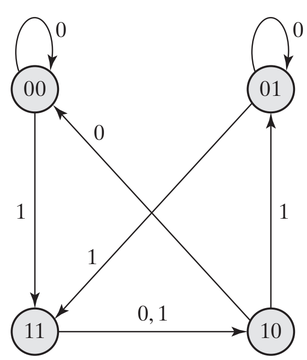
Repeated sequence: $\rightarrow 00 \rightarrow 01 \rightarrow 10$



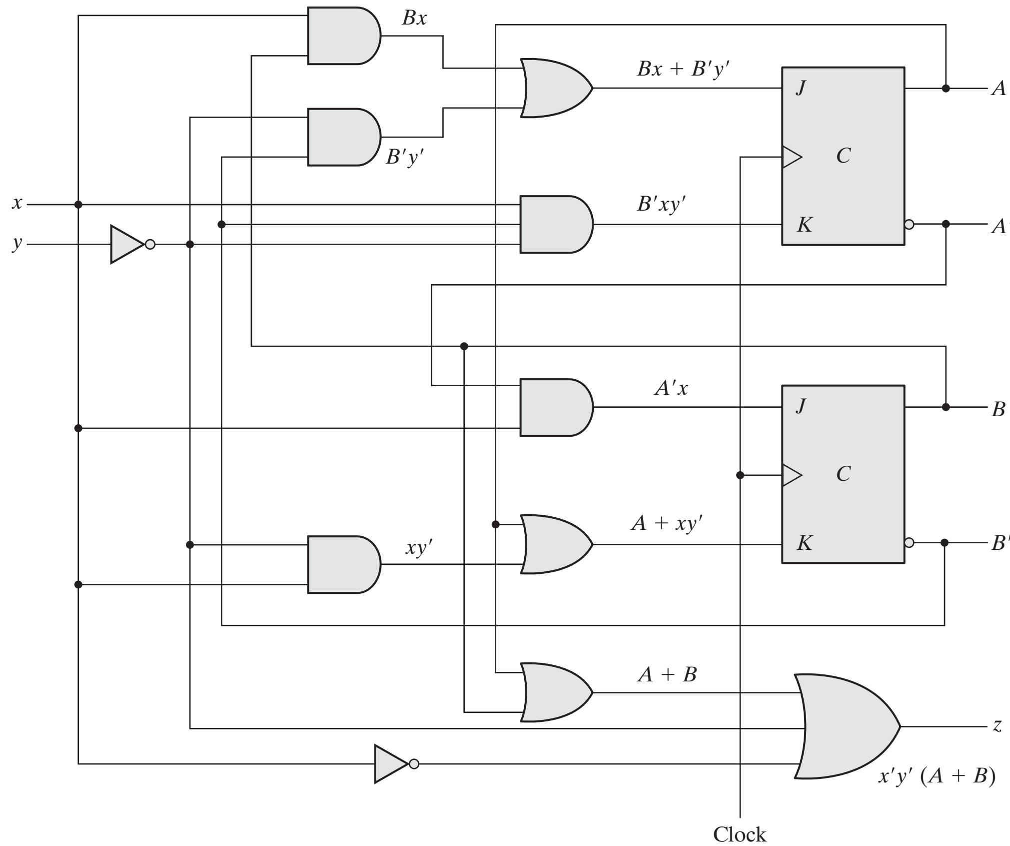
5.9 **(a)** $A(t + 1) = J_AA' + K_A'A = xA' + BA$

$B(t + 1) = J_BB' + K_B'B = xB' + A'B$

(b)



5.10 **(a)**



(b)

Present state		Inputs		Next state		Output	Flip-flop inputs			
A	B	x	y	A	B	z	J_A	K_A	J_B	K_B
0	0	0	0	1	0	0	1	0	0	0
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	1	0	1	1	1	1
0	0	1	1	0	1	0	0	0	1	0
0	1	0	0	0	1	1	0	0	0	0
0	1	0	1	0	1	0	0	0	0	0
0	1	1	0	1	0	0	1	0	1	1
0	1	1	1	1	1	0	1	0	1	0
1	0	0	0	1	0	1	1	0	0	1
1	0	0	1	1	0	0	0	0	0	1
1	0	1	0	0	0	0	1	1	0	1
1	0	1	1	1	0	0	0	0	0	1
1	1	0	0	1	0	1	0	0	0	1
1	1	0	1	1	0	0	0	0	0	1
1	1	1	0	1	0	0	1	0	0	1
1	1	1	1	1	0	0	1	0	0	1

(c) From the state table, the state equations are derived as:

$A(t + 1) = \Sigma(0, 2, 6, 7, 8, 9, 11, 12, 13, 14, 15)$

$B(t + 1) = \Sigma(2, 3, 4, 5, 7)$

On simplifying, we get the simplified state equations as:

$A(t + 1) = Ax' + Bx + Ay + A'B'y'$

$B(t + 1) = A'B'x + A'B\,(x' + y)$

5.11 **(a)**

Present state	00	00	01	00	01	11	00	01	11	10	00	01	11	10	10
Input	0	1	0	1	1	0	1	1	1	0	1	1	1	1	0
Next state	00	01	00	01	11	00	01	11	10	00	01	11	10	10	00
Output	0	0	1	0	0	1	0	0	0	1	0	0	0	0	1

Output sequence: 001001000100001

(b) State table after reduction:

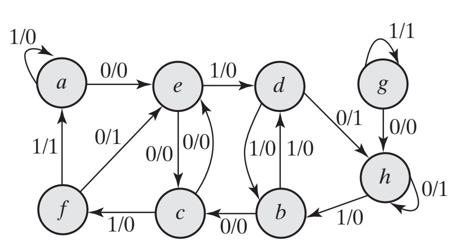
Present state		Input	Next state		Output
A	B	x	A	B	Y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	0	1	0

(c)

Present state	00	00	01	00	01	01	00	01	01	01	00	01	01	01	01
Input	0	1	0	1	1	0	1	1	1	0	1	1	1	1	0
Next state	00	01	00	01	01	00	01	01	01	00	01	01	01	01	00
Output	0	0	1	0	0	1	0	0	0	1	0	0	0	0	1

Output sequence: 001001000100001 same as (a)

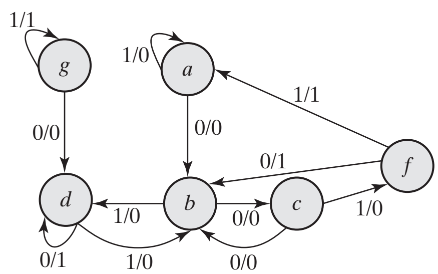
5.12 **(a)**



(b) State table after reduction:

Present state	Next state $x = 0$	Next state $x = 1$	Output $x = 0$	Output $x = 1$
a	*b*	*a*	0	0
b	*c*	*d*	0	0
c	*b*	*f*	0	0
d	*d*	*b*	1	0
f	*b*	*a*	1	1
g	*d*	*g*	0	1

(c)



5.13 **(a)** Using the state table of Problem 5.12, we get:

Present state	*a*	*e*	*d*	*h*	*h*	*b*	*c*	*f*	*e*	*c*	*e*
Input	0	1	0	0	1	0	1	0	0	0	1
Next state	*e*	*d*	*h*	*h*	*b*	*c*	*f*	*e*	*c*	*e*	*d*
Output	0	0	1	1	0	0	0	1	0	0	0

Output sequence: 00110001000

(b) Using the reduced state table obtained in Problem 5.12 (b), we get:

Present state	*a*	*b*	*d*	*d*	*d*	*b*	*c*	*f*	*b*	*c*	*b*
Input	0	1	0	0	1	0	1	0	0	0	1
Next state	*b*	*d*	*d*	*d*	*b*	*c*	*f*	*b*	*c*	*b*	*d*
Output	0	0	1	1	0	0	0	1	0	0	0

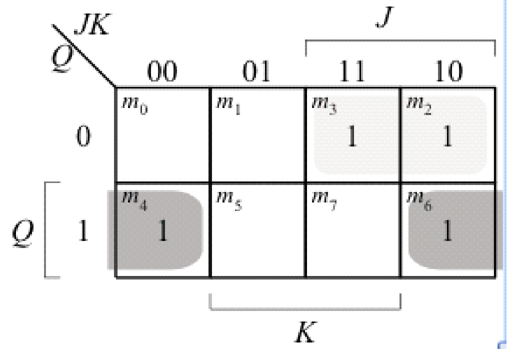
Output sequence: 00110001000 same as (a)

5.14

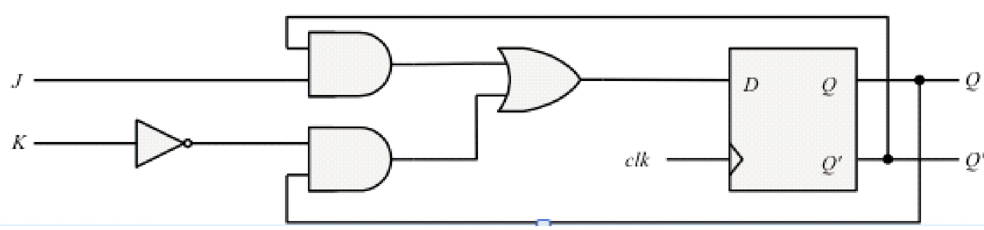
Present state	Next state $x = 0$	Next state $x = 1$	Output $x = 0$	Output $x = 1$
00001	00001	00010	0	0
00010	00100	01000	0	0
00100	00001	01000	0	0
01000	10000	01000	0	1
10000	00001	01000	0	1

5.15 $D_Q = Q'J + QK'$

Present state Q	*Inputs* J	K	*Next state* Q	
0	0	0	0	No change
0	0	1	0	Reset to 0
0	1	0	1	Set to 1
0	1	1	1	Complement
1	0	0	1	No change
1	0	1	0	Reset to 0
1	1	0	1	Set to 1
1	1	1	0	Complement



$Q(t+1) = DQ = Q'J + K'Q$



5.16 **(a)**

Present state A	Present state B	Input x	Next state A	Next state B
0	0	0	0	0
0	0	1	1	0
0	1	0	0	1
0	1	1	0	0
1	0	0	1	0
1	0	1	1	1
1	1	0	1	1
1	1	1	0	1

On simplifying, we get:

$D_A = Ax' + B'x$

$D_B = Ax + Bx'$

(b)

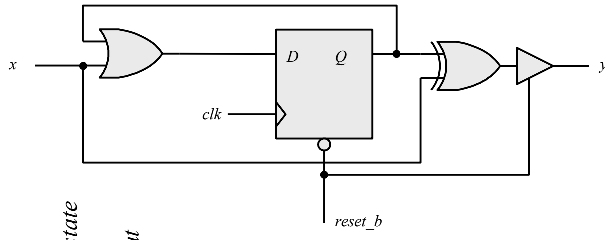
Present state A	Present state B	Input x	Next state A	Next state B
0	0	0	0	0
0	0	1	1	1
0	1	0	0	1
0	1	1	0	0
1	0	0	1	0
1	0	1	0	1
1	1	0	1	1
1	1	1	1	0

On simplifying, we get:

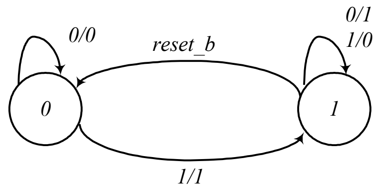
$D_A = A'B'x + Ax' + AB$

$D_B = B'x + Bx'$

5.17 The output is 0 for all 0 inputs until the first 1 occurs, at which time the output is 1. Thereafter, the output is the complement of the input. The state diagram has two states. In state 0: output = input; in state 1: output = input'.



Present state	Input	Next state	Output
A	x	A	y
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	0



$D_A = A + x$
$y = Ax' + A'x$

5.18

Present state		Inputs		Next state		Flip-flop inputs			
A	B	E	F	A	B	J_A	K_A	J_B	K_B
0	0	0	0	0	0	0	×	0	×
0	0	0	1	0	0	0	×	0	×
0	0	1	0	0	1	0	×	1	×
0	0	1	1	1	1	1	×	1	×
0	1	0	0	0	1	0	×	×	0
0	1	0	1	0	1	0	×	×	0
0	1	1	0	1	0	1	×	×	1
0	1	1	1	0	0	0	×	×	1
1	0	0	0	1	0	×	0	0	×
1	0	0	1	1	0	×	0	0	×
1	0	1	0	1	1	×	0	1	×
1	0	1	1	0	1	×	1	1	×
1	1	0	0	1	1	×	0	×	0
1	1	0	1	1	1	×	0	×	0
1	1	1	0	0	0	×	1	×	1
1	1	1	1	1	0	×	0	×	1

On simplifying, we get:

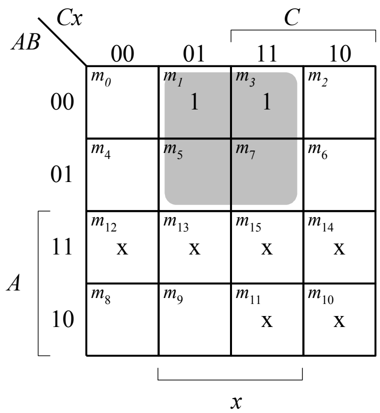
$J_A = K_A = (B'F + BF')\,E$

$J_B = K_B = E$

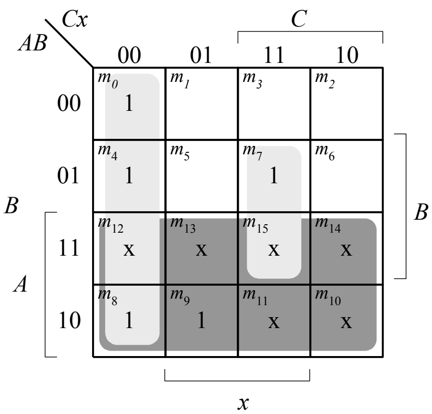
5.19 **(a)** Unused states (see Fig. P5.19): 101, 110, 111.

Present state ABC	*Input x*	*Next state ABC*	*Output y*
000	0	011	0
000	1	100	1
001	0	001	0
001	1	100	1
010	0	010	0
010	1	000	1
011	0	001	0
011	1	010	1
100	0	010	0
100	1	011	1

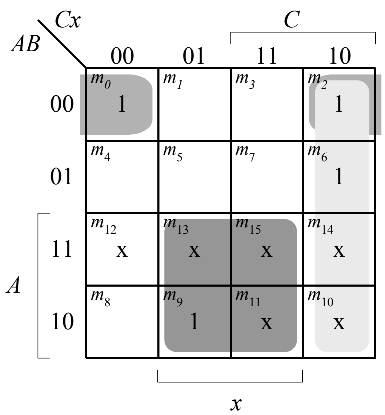
$d(A, B, C, x) = \Sigma\ (10, 11, 12, 13, 14, 15)$



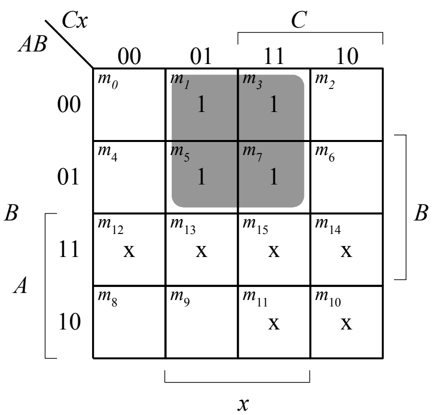
$D_A = A'B'x$



$D_B = A + C'x' + BCx$

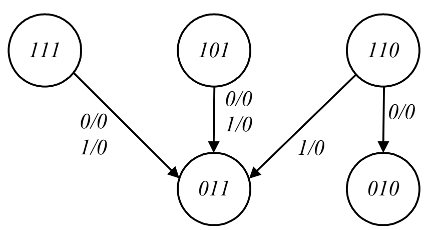


$D_C = Cx' + Ax + A'B'x'$



$y = A'x$

The machine is self-correcting, i.e., the unused states transition to known states.



(b) With JK flip=flops, the state table is the same as in (a).

Flip-flop inputs

J_A	K_A	J_B	K_B	J_C	K_C
0	x	1	x	1	x
1	x	0	x	0	x
0	x	0	x	x	0
1	x	0	x	x	1
0	x	x	0	0	x
0	x	x	1	0	x
0	x	x	1	x	0
0	x	x	0	x	1
x	1	1	x	0	x
x	1	1	x	1	x

$J_A = B'x$ $\quad K_A = 1$
$J_B = A + C'x'$ $\quad K_B = C'x + Cx'$
$J_C = Ax + A'B'x'$ $\quad K_C = x$
$y = A'x$

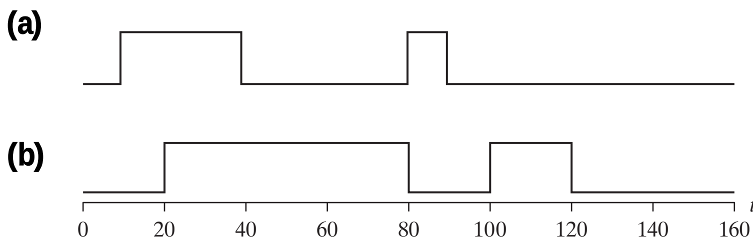
The machine is self-correcting because $K_A = 1$.

5.20 $T_A = A'B + Bx'$

$T_B = A'Bx + Ax' + B'x'$

5.21 Blocking assignments use the symbol (=) as the assignment operator and are executed sequentially in the order they are listed in a block of statements. On the other hand, nonblocking assignments use (<=) as the operator and are executed concurrently by evaluating the set of expressions on the right-hand side of the list of statements; they do not make assignments to their left-hand sides until all of the expressions are evaluated.

5.22 **(a)**, **(b)**



5.23 **(a)** $RegA = 15$, $RegB = 15$

(b) $RegA = 15$, $RegB = 30$

5.24

```
module DFF (output reg Q, input D, clk, preset, clear);
  always @ (posedge clk)
  if (preset == 0) Q <= 1'b1;
  else if (clear == 0) Q <= 1'b0;
  else Q <= D;
endmodule
```

5.25

```
module Dual_Input_DFF (output reg Q, input D1, D2, select, clk, reset_b);
  always @ (posedge clk, negedge reset_b)
  if (reset_b == 0) Q <= 0;
  else Q <= select ? D2 : D1;
endmodule
```

5.26 **(a)**

$Q(t + 1) = JQ' + K'Q$

When $Q = 0$, $Q(t + 1) = J$
When $Q = 1$, $Q(t + 1) = K'$

(b)

Verilog

```
module JK_Behavior_a (output reg Q, input J, K, CLK, reset_b);
  always @ (posedge CLK, negedge reset_b)
    if (reset_b == 0) Q <= 0; else
      if (Q == 0)       Q <= J;
      else              Q <= ~K;
endmodule
```

Alternative:

```
module JK_Behavior_b (output reg Q, input J, K, CLK, reset_b);
  always @ (posedge CLK, negedge reset_b)      // Asynch reset_b
    if (reset_b == 0) Q <= 0;
    else
    case ({J, K})
      2'b00:  Q <= Q;
      2'b01:  Q <= 0;
      2'b10:  Q <= 1;
      2'b11:  Q <= ~Q;
    endcase
endmodule

module t_Prob_5_26  ();
  wire Q_a, Q_b;
  reg J, K, clk, reset_b;
   JK_Behavior_a M0 (Q_a, J, K, clk, reset_b);
   JK_Behavior_b M1 (Q_b, J, K, clk, reset_b);

   initial #100 $finish;
   initial begin clk = 0; forever #5 clk = ~clk; end
   initial fork
     #2 reset_b = 1;
     #3 reset_b = 0;           // Initialize to s0
     #4 reset_b = 1;
    J =0; K = 0;
    #20 begin J= 1; K = 0;  end
    #30 begin J = 1; K = 1; end
    #40 begin J = 0; K = 1; end
    #50 begin J = 1; K = 1; end
  join
 endmodule
```

VHDL

```
entity JK_Behavior_a is
  port (Q: out Std_Logic; J, K, CLK, reset_b: in Std_Logic);
end JK_Behavior_a;

architecture Behavioral of JK_Behavior is
```

```
begin
process (CLK)
    if (CLK'event and reset_b = '0') then Q <= 0;
    elsif (Q = 0)   then    Q <= J;
    else Q <= not K; end if;
end Behavioral;
```

Alternative:

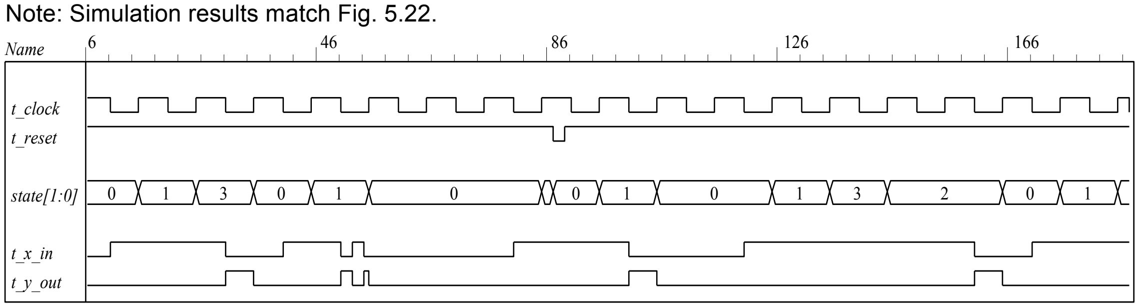
```
entity JK_Behavior_b
    port (Q: out Std_Logic; J, K, CLK, reset_b: in Std_Logic);
end JK_Behavior_b;

architecture Behavioral of JK_Behavior_b is
  begin
    process (CLK, reset_b)        // Asynch reset_b
    begin
        if (reset_b = '0') Q <= '0';
        else if (CLK'event and CLK = '1') then
            case (J & K) is
                when  '00'     =>  Q <= Q;
                when  '01'     =>  Q <= 0;
                when  '10'     =>  Q <= 1;
                when  '11'     =>  Q <= not Q;
            endcase
    end process;
endmodule
```

5.27 <u>Verilog</u>

```
// Mealy FSM zero detector (See Fig. 5.16)
module Mealy_Zero_Detector (
  output reg y_out,
  input x_in, clock, reset
);
  reg [1: 0] state, next_state;
  parameter    S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;

  always @ (posedge clock, negedge reset)   // state transition
    if (reset == 0) state <= S0;
    else state <= next_state;

  always @ (state, x_in) // Form the next state
    case (state)
      S0:begin y_out = 0;   if (x_in)   next_state = S1; else next_state = S0; end
      S1:      begin y_out = ~x_in; if (x_in)       next_state = S3; else next_state = S0; end
      S2:begin y_out = ~x_in; if (~x_in) next_state = S0; else next_state = S2; end
      S3:      begin y_out = ~x_in; if (x_in)   next_state = S2; else next_state = S0; end
    endcase

endmodule

module t_Mealy_Zero_Detector;
  wire  t_y_out;
  reg    t_x_in, t_clock, t_reset;

Mealy_Zero_Detector M0 (t_y_out, t_x_in, t_clock, t_reset);


initial #200 $finish;
initial begin t_clock = 0; forever #5 t_clock = ~t_clock; end
```

```
initial fork
      t_reset = 0;
  #2 t_reset = 1;
  #87 t_reset = 0;
  #89 t_reset = 1;
  #10 t_x_in = 1;
  #30 t_x_in = 0;
  #40 t_x_in = 1;
  #50 t_x_in = 0;
  #52 t_x_in = 1;
  #54 t_x_in = 0;
  #70 t_x_in = 1;
  #80 t_x_in = 1;
  #70 t_x_in = 0;
  #90 t_x_in = 1;
  #100 t_x_in = 0;
  #120 t_x_in = 1;
  #160 t_x_in = 0;
  #170 t_x_in = 1;
join
endmodule
```

Note: Simulation results match Fig. 5.22.

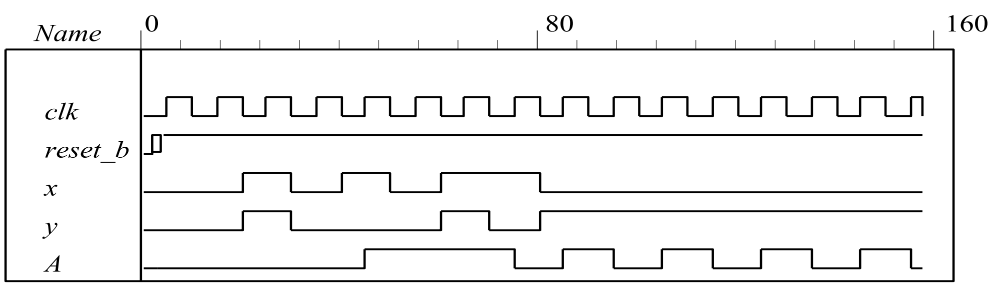


VHDL

```
entity Mealy_Zero_Detector is
    port (y_out: out Std_Logic; x_in, clock, reset: in Std_Logic);
end Mealy_Zero_Detector;

 architecture Behavioral of Mealy_Zero_Detector is
    type state_type is S0, S1, S2, S3;
    signal state, next_state: state_type;
begin
    process (clock, reset)     // state transition
    begin
        if (reset'event and reset = '0') then state <= S0;
        else if (clock'event and clock = '1') then state <= next_state;
    end process;

    process (state, x_in)      // Form the next state
    begin
      y_out <= S0;
      case (state) is
        when S0   =>  if x_in = '0' next_state <= S1; else next_state <= S0;
        when S1   =>  y_out <= not x_in; if (x_in = '1') then next_state = S3; else next_state = S0;
        when S2   =>  y_out = not x_in; if (x_in = '0') then next_state = S0; else next_state = S2;
        when S3   =>  y_out = not x_in; if (x_in = '1') then next_state = S2; else next_state = S0;
      end case;
    end process;
end Behavior;
```

5.28 Verilog (a)

```
module Prob_5_28a  (output A, input x, y, clk, reset_b);
 parameter s0 = 0, s1 = 1;
 reg state, next_state;
 assign A = state;

 always @ (posedge clk, negedge reset_b)
   if (reset_b == 0) state <= s0; else state <= next_state;

 always @ (state, x, y) begin
   next_state = s0;
   case (state)
     s0:      case ({x, y})
                2'b00, 2'b11: next_state = s0;
                2'b01, 2'b10: next_state = s1;
              endcase
     s1:      case ({x, y})
                2'b00, 2'b11: next_state = s1;
                2'b01, 2'b10: next_state = s0;
              endcase
    endcase
  end
endmodule


module t_Prob_5_28a  ();
 wire A;
 reg x, y, clk, reset_b;
  Prob_5_28a M0 (A, x, y, clk, reset_b);
  initial #350 $finish;
  initial begin clk = 0; forever #5 clk = ~clk; end
  initial fork
    #2 reset_b = 1;
    #3 reset_b = 0;     // Initialize to s0
    #4 reset_b = 1;
   x =0; y = 0;
   #20 begin x= 1; y = 1;  end
   #30 begin x = 0; y = 0; end
   #40 begin x = 1; y = 0; end
   #50 begin x = 0; y = 0; end
   #60 begin x = 1; y = 1; end
   #70 begin x = 1; y = 0; end
   #80 begin x = 0; y = 1; end
 join
endmodule
```



VHDL

```
entity Prob_5_28a  is
      port A: out Std_Logic; x, y, clk, reset_b: in Std_Logic);
end Prob_5_28a;

architecture Behavioral of Prob_5_28a is
```

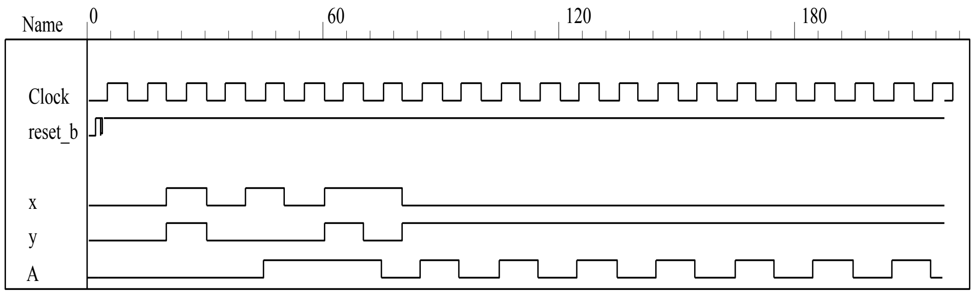
```
      type state_type is (S0, s1);
 begin
      A <= state;
 then
 process (clk, reset_b)
      if (reset_b = '0') state <= s0;
      else if (clk'event and clk = '1') then state <= next_state;
      end if;
end process;

process (state, x, y)
begin
   next_state <= s0;
   case (state) is
        when s0    =>  case (x & y}
                            when '00', '11' =>  next_state <= s0;
                            when '01', '10' =>  next_state <= s1;
                       end case;

        when s1    =>  case (x & y}
                            when '00', '11' =>  next_state <= s1;
                            when '01', '10' =>  next_state <= s0;
                       end case;
      end case;
 end Behavioral;
```

(b) Verilog

```
 module Prob_5_28b  (output A, input x, y, Clock, reset_b);
  xor (w1, x, y);
  xor (w2, w1, A);
  DFF M0 (A, w2, Clock, reset_b);
 endmodule

 module DFF (output reg Q, input D, Clock, reset_b);
  always @ (posedge Clock, negedge reset_b)
    if (reset_b == 0) Q <= 0;
    else Q <= D;
 endmodule
```

Testbench – Verilog

```
 module t_Prob_5_28b  ();
  wire A;
  reg x, y, clk, reset_b;
   Prob_5_28b M0 (A, x, y, clk, reset_b);
   initial #350 $finish;
   initial begin clk = 0; forever #5 clk = ~clk; end
   initial fork
     #2 reset_b = 1;
     #3 reset_b = 0;          // Initialize to s0
     #4 reset_b = 1;
    x =0; y = 0;
    #20 begin x= 1; y = 1;  end
    #30 begin x = 0; y = 0; end
    #40 begin x = 1; y = 0; end
    #50 begin x = 0; y = 0; end
    #60 begin x = 1; y = 1; end
    #70 begin x = 1; y = 0; end
    #80 begin x = 0; y = 1; end
  join
 endmodule
```



VHDL

```
entity Prob_5_28b is
     port(A: out Std_Logic; x, y, Clock, reset_b: in Std_Logic);
end Prob_5_28b;

architecture Structural of Prob_5_28b is
     component xor2_gate port (y: out Std_Logic; xin1, xin2: in Std_Logic);
     component DFF port (Q: out Std_Logic; D, Clock, reset_b: in Std_Logic);
begin
     G1: xor2_gate port map (y => w1, x => xin1; y => xine2)
     G2: xor2_gate port map (y => w2, xin1 => w1, xin2 => A);
     G3: DFF port map (Q => A, D => w2, Clock => Clock, reset_b => reset_b);
end Structural;

entity DFF is
     port (Q: out Std_Logc; D, Clock, reset_b: in Std_Logic);
end DFF;

architecture Behavioral of DFF is
begin
     process (Clock, reset_b)
          if (reset_b'event and  = '0') Q <= 0;
          elsif (Clock'event and Clock = '1') then Q <= D;
          end if;
     end process;
end Behavioral;

entity xor2_gate is
     port (y: out Std_Logic; xin1, xin2: in Std_Logic);
end xor2_gate;

architecture Boolean_eq of xor2gate is
begin
     y <= xin1 xor xin2;
end Boolean_eq;
```

Testbench - VHDL

```
entity t_Prob_5_28b  () is;
end t_Prob_5.28b;

architecture TestBench of t_Prob_5_28b is
     signal t_A, t_x, t_y, t_clk, t_reset_b: Std_Logic;
     signal  A;
     component Prob_5_28b port (A: out Std_Logic; x, y, Clock, reset_b: in Std_Logic);
begin
```

```
M0: Prob_5_28b M0 (t_A =>A, t_x => x, t_y => y, t_clk => clk, t_reset_b => reset_b);

process – clock for testbench
begin
    t_clk <= '0';
    wait for 5 ns;
    t_clk <= '1';
    wait for 5 ns;
end process;

process
begin
    t_x <= '0';
    t_y <= '0'
   t_reset_b = '1' after 2ns;
   t_reset_b = '0' after 3ns;        // Initialize to s0
   t_reset_b = '1' after 4 ns;

    t_x <= '1' after 20 ns;
    t_y <= '1' after 20 ns;

    t_x <= '0' after 30 ns
    t_y <= '0' after 30 ns;

    t_x <= '1' after 40 ns;

   t_x <= '0' after 50 ns;

    t_x <= '1' after 60 ns;
    t_y <= '1' after 60 ns;
    t_y <= '0' after 70 ns;

   t_x <= '0' after 80 ns;
   t_y <= '1' after 80 ns;
end process;
end TestBench;
```

(c) Use results of (b) and (c).

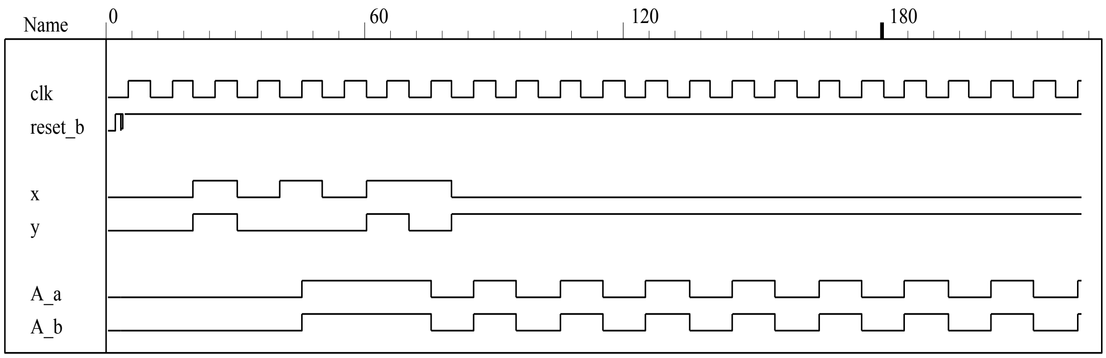
Verilog

```
module t_Prob_5_28c  ();
  wire A_a, A_b;
  reg x, y, clk, reset_b;
   Prob_5_28a M0 (A_a, x, y, clk, reset_b);
   Prob_5_28b M1 (A_b, x, y, clk, reset_b);

   initial #350 $finish;
   initial begin clk = 0; forever #5 clk = ~clk; end
   initial fork
     #2 reset_b = 1;
     #3 reset_b = 0;           // Initialize to s0
     #4 reset_b = 1;
    x =0; y = 0;
    #20 begin x = 1; y = 1;  end
    #30 begin x = 0; y = 0; end
    #40 begin x = 1; y = 0; end
    #50 begin x = 0; y = 0; end
    #60 begin x = 1; y = 1; end
    #70 begin x = 1; y = 0; end
    #80 begin x = 0; y = 1; end
  join
 endmodule
```

VHDL

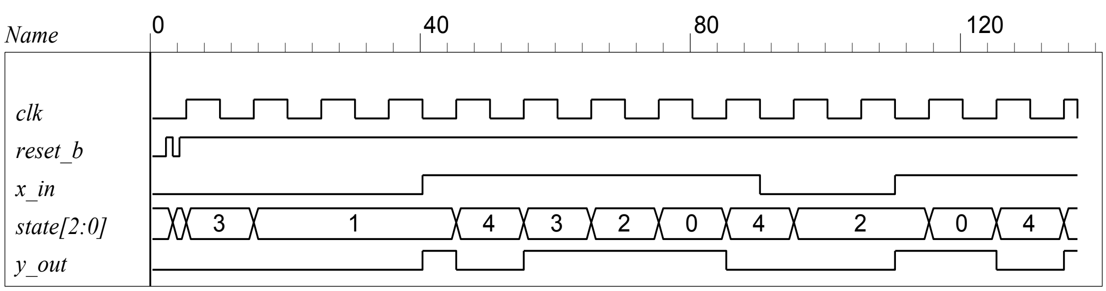
```
entity t_Prob_5_28c  ();
end t_Prob_5_28c;

architecture TestBench of t_Prob_5_28c is
     signal A_a, A_b;
     signal t_x, t_y, t_clk, t_reset_b;
    component Prob_5_28a port(A: out Std_Logic; x, y, Clock, reset_b: in Std_Logic);
    component Prob_5_28b port(A: out Std_Logic; x, y, Clock, reset_b: in Std_Logic);
begin
     M_a: Prob_5_28a (A_a => A, t_x => x, t_y => y, t_clk => clk, t_reset_b => reset_b);
     M_b: Prob_5_28b (A_b => A, t_x => x, t_y => y, t_clk => clk, t_reset_b => reset_b);

process – clock for testbench
begin
     t_clk <= '0';
     wait for 5 ns;
     t_clk <= '1';
     wait for 5 ns;
end process;

 process
     begin
          t_x <= '0';
          t_y <= '0'
          t_reset_b = '1' after 2ns;
          t_reset_b = '0' after 3ns;        // Initialize to s0
          t_reset_b = '1' after 4 ns;

               t_x <= '1' after 20 ns;
               t_y <= '1' after 20 ns;

               t_x <= '0' after 30 ns
               t_y <= '0' after 30 ns;

               t_x <= '1' after 40 ns;
```

```
        t_x <= '0' after 50 ns;

        t_x <= '1' after 60 ns;
        t_y <= '1' after 60 ns;
        t_y <= '0' after 70 ns;

        t_x <= '0' after 80 ns;
        t_y <= '1' after 80 ns;
    end process;
end TestBench;
```



5.29 **Verilog**

```
module Prob_5_29 (output reg y_out, input x_in, clock, reset_b);
  parameter s0 = 3'b000, s1 = 3'b001, s2 = 3'b010, s3 = 3'b011, s4 = 3'b100;
  reg [2: 0] state, next_state;

  always @ (posedge clock, negedge reset_b)
    if (reset_b == 0) state <= s0;
    else state <= next_state;

  always @ (state, x_in) begin
    y_out = 0;
    next_state = s0;
    case (state)
      s0:     if (x_in) begin next_state = s4; y_out = 1; end else begin next_state = s3; y_out = 0; end
      s1:     if (x_in) begin next_state = s4; y_out = 1; end else begin next_state = s1; y_out = 0; end
      s2:     if (x_in) begin next_state = s0; y_out = 1; end else begin next_state = s2; y_out = 0; end
      s3:     if (x_in) begin next_state = s2; y_out = 1; end else begin next_state = s1; y_out = 0; end
      s4:     if (x_in) begin next_state = s3; y_out = 0; end else begin next_state = s2; y_out = 0; end
      default:    next_state = 3'bxxx;
    endcase
  end
endmodule


module t_Prob_5_29 ();
  wire y_out;
  reg x_in, clk, reset_b;

  Prob_5_29 M0 (y_out, x_in, clk, reset_b);

  initial #350$finish;
  initial begin clk = 0; forever #5 clk = ~clk; end
  initial fork
    #2 reset_b = 1;
    #3 reset_b = 0;      // Initialize to s0
    #4 reset_b = 1;
                         // Trace the state diagram and monitor y_out
     x_in = 0;           // Drive from s0 to s3 to  S1 and park
    #40 x_in = 1;        // Drive to s4 to s3 to s2 to s0 to s4 and loop
    #90 x_in = 0;        // Drive from s0 to s3 to s2 and part
    #110  x_in = 1;      // Drive s0 to s4  etc
   join
endmodule
```

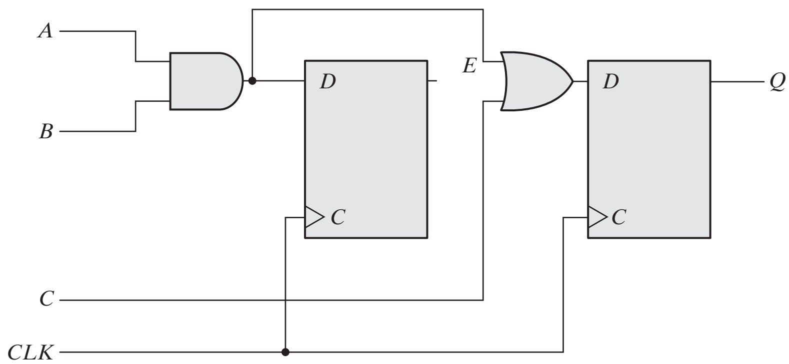


VHDL

```
entity Prob_5_29
     port (y_out: out Std_Logic; x_in, clock, reset_b: in Std_Logic);
end Prob_5_29;

     architecture Behavioral of Prob_5_29 is;
```

```
    type state_type is  integer range 0 to 4;
    state_type state, next_state;
    constant s0 = 1, s1 = 1, s2 = 2; s3 = 3; s4 = 4;

    process (clock, reset_b)
    begin
        if (reset_b'event and reset_b = '0') then state <= s0;
        elsif (clockc
'event and clock = '1') then   state <= next_state;
        end if;

    process  (state, x_in) begin
        y_out <= '0';
        next_state <= s0;
        case (state)
    when s0   => if (x_in) next_state <= s4; y_out <= '1'; else next_state <= s3; y_out <= '0'; end if;
    when s1   => if (x_in) next_state <= s4; y_out <= '1'; else next_state <= s1; y_out <= '0'; end if;
    when s2   => if (x_in) next_state <= s0; y_out <= '1; else next_state <= s2; y_out <= '0'; end if;
    when s3   => if (x_in) next_state <= s2; y_out = '1'; else next_state = s1; y_out = '0'; end if;
    when s4   => if (x_in) next_state <= s3; y_out = '0'; else next_state = s2; y_out = '0'; end if;
    default:   next_state = s0;
  endcase;
 end
end Behavioral;

module t_Prob_5_29 ();
    signal t_y_out;
    signal t_x_in, t_clk, t_reset_b;
    component Prob_5_29 port (y_out: out Std_Logic; x_in, clk, reset_b: in Std_Logic);
begin
    M0: Prob_5_29 port map (t_y_out => y_out; t_x_in => x_in, t_clk => clk, t_reset_b => reset_b);

 process clk = 0; forever #5 clk = ~clk; end
 initial fork
   #2 reset_b = '1';
   #3 reset_b = '0';          // Initialize to s0
   #4 reset_b = '1';
                         // Trace the state diagram and monitor y_out
    x_in = '0';          // Drive from s0 to s3 to  S1 and park
   #40 x_in = '1';       // Drive to s4 to s3 to s2 to s0 to s4 and loop
   #90 x_in = '0';       // Drive from s0 to s3 to s2 and part
   #110  x_in = '1';          // Drive s0 to s4  etc
  join
endmodule
```

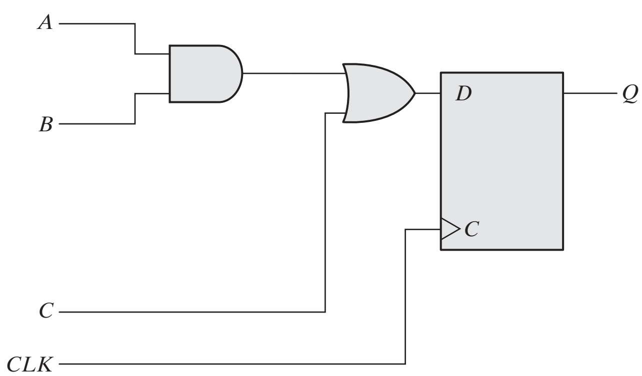
5.30



5.31 (a)

```
module Seq_Ckt (input A, B, C, CLK, output reg Q);
reg E;
always @ (posedge CLK)
begin
Q = E | C;
E = A & B;
end
endmodule
```

(b)



5.32 **Verilog**

```
initial begin
  enable = 0; A = 1; B = 0; C = 0; D = 1; E = 1; F = 1;
  #10   A = 0; B = 1; C = 1;
  #10   A = 1; B = 0; D = 1;    E = 0;
  #10   B = 1; E = 1; F = 0;
  #10   enable = 1;
        B = 0; D= 0; F =1;
  #10   B = 1;
  #10   B = 0; D = 1;
  #10   B = 1;
end

initial fork
  enable = 0; A = 1; B = 0; C = 0; D = 1; E = 1; F = 1;
#10 begin  A = 0; B = 1; end
#20 begin  A = 1; B = 0; D = 1; E = 0; end
#30 begin  B = 1; E = 1; F = 0; end
#40 begin  B = 0; D = 0; F = 1; end
#50 begin  B = 1; end
#60 begin  B = 0; D = 1; end
#70 begin  B = 1; end
join
```

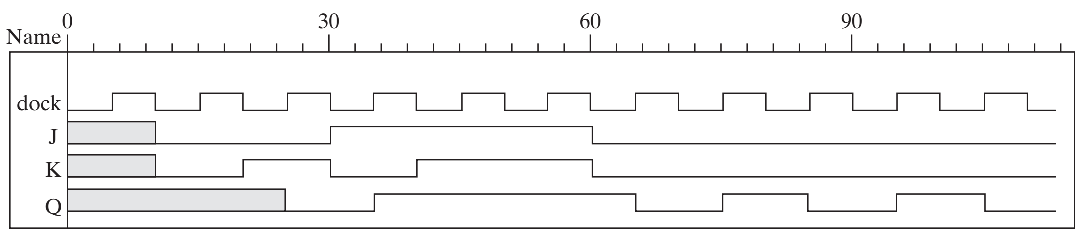
VHDL

```
process ()
   enable <= '0'; A <= '1'; B <= '0'; C <= '0'; D <= '0'; E <= '1'; F <= '1';
   wait for 10 ns;
   a <= '0'; B <= '1'; C <= '1';
   wait for 10 ns;
   A <= '1'; B <= '0'; D <= '1'; E <= '0'; B <= '1'; E <= '1'; F <= '0';
   wait for 10 ns;
   enable <= '1';
   B <= '0'; D <= 0; F <= '1';
   wait for 10 ns;
   B <= '1';
   wait for 10 ns;
   B <= '0';
   D <= '1';
   wait for 10 ns;
   B <= '1';
end process;
```

5.33 All models of sequential circuits must include a reset (or preset) signal; otherwise, the initial state of the flip-flops of the sequential circuit cannot be determined. A sequential circuit cannot be tested with HDL simulation unless an initial state can be assigned by an external input signal.

5.34

```
module JK_flop_Prob_5_34 (output Q, input J, K, clk);
  wire K_bar;
  D_flop M0 (Q, D, clk);
  Mux M1 (D, J, K_bar, Q);
  Inverter M2 (K_bar, K);
endmodule

module D_flop (output reg Q, input D, clk);
  always @ (posedge clk) Q <= D;
endmodule

module Inverter (output y_bar, input y);
  assign y_bar = ~y;
endmodule

module Mux (output y, input a, b, select);
  assign y = select ? a: b;
endmodule

module t_JK_flop_Prob_5_34 ();
  wire Q;
  reg J, K, clock;
  JK_flop_Prob_5_34 M0 (Q, J, K, clock);
  initial #500 $finish;
  initial begin clock = 0; forever #5 clock = ~clock; end
  initial fork
    #10 begin J = 0; K = 0; end // toggle Q unknown
    #20 begin J = 0; K = 1; end // set Q to 0
    #30 begin J = 1; K = 0; end // set q to 1
    #40 begin J = 1; K = 1; end // no change
    #60 begin J = 0; K = 0; end // toggle Q
join
endmodule
```



5.35

```
module Prob_5_35a (output reg z, input x, y, clk, reset_b);
 parameter s0 = 2'b00, s1 = 2'b01, s2 = 2'b10, s3 = 2'b11;
 reg [2: 0] state, next_state;

 always @ (posedge clk, negedge reset_b)
  if (reset_b == 0) state <= s0;
  else state <= next_state;

  always @ (state, x, y) begin
   z = 0;
   next_state = s0;
   case (state)
    s0: begin
       z = 0;
       case ({x, y})
      2'b00, 2'b10, 2'b11: next_state = s0;
      2'b01: next_state = s2;
      endcase
    end
    s1: begin
      z = 1;
     case ({x, y})
       2'b10, 2'b11: next_state = s0;
       2'b00: next_state = s1;
       2'b01: next_state = s3;
      endcase
    end
    s2: begin
       z = 0;
       case ({x, y})
      2'b10, 2'b11: next_state = s3;
      2'b00: next_state = s0;
       2'b01: next_state = s2;
      endcase
    end
    s3: begin
      z = 1;
     case ({x, y})
       2'b01, 2'b10, 2'b11: next_state = s3;
```

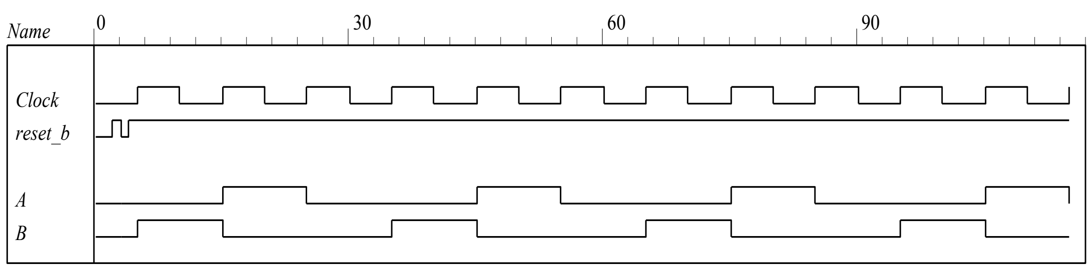


```
      2'b00: next_state = s1;
      endcase
    end
  endcase
 end
endmodule
```

5.36 Note: See Problem 5.8 (counter with repeated sequence: (A, B) = 00, 01, 10, 00 ....
// See Fig. P5.8

VERILOG

```
module Problem_5_36 (output A, B, input Clock, reset_b);
  wire T_A, T_B, A_b, B_b;
  or (T_A, A, B);
  or (T_B, A_b, B);
  T_flop M0 (A, A_b, T_A, Clock, reset_b);
  T_flop M1 (B, B_b, T_B, Clock, reset_b);
endmodule

module T_flop (output reg Q, output QB, input T, Clock, reset_b);
  assign QB = ~ Q;
  always @ (posedge Clock, negedge reset_b)
    if (reset_b == 0) Q <= 0;
    else if (T) Q <= !Q;
endmodule

module t_Problem_5_36 ();
  wire A, B;
  reg  Clock, reset_b;

  Problem_5_36 M0 (A, B, Clock, reset_b);

  initial #350$finish;
  initial begin Clock = 0; forever #5 Clock = ~Clock; end
  initial fork
    #2 reset_b = 1;
    #3 reset_b = 0;
    #4 reset_b = 1;
  join
endmodule
```



VHDL

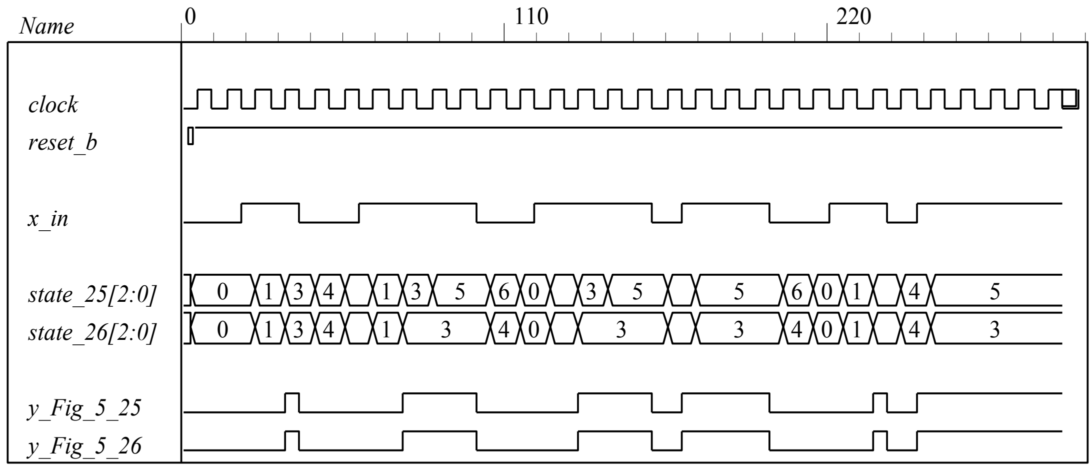
```
entity Problem_5_36_vhdl (output A, B: out bit; input Clock, reset_b: out bit) is
end Problem_5_36_vhdl;

architecture Structural of Problem_5_36_vhdl is
     component or2_gate port (y: out, bit; xin1, xin2: in bit);
     component T_flop port (Q, Q_b: in bit, T, Clock, reset_b: in bit);

begin
     G1: or2_gate port map (y => T_A, xin1 => A, xin2 => B);
     G2: or2_gate port map (y => T_B, xin1 => A_b, xin2 => B);
     G3: T_flop port map (Q => A, Q_b => A_b, T => T_A, Clock => Clock, reset_b => reset_b);
     G4: T_flop port map (Q => B, Q_b => B_b, T => T_B, Clock => Clock, reset_b => reset_b);
end Structural;

entity T_flop is
     port (Q, QB: out bit; T, Clock, reset_b: in bit);
end T_flop;
```

```
architecture Behavioral of T_flop is
    QB <= not Q;
  process  (Clock, reset_b) is
    beginnot
        if (reset_b '0') then Q <= 0;
        elsif (T) Q <= Q;
end process;
end Behavioral;

module t_Problem_5_36 ();
  wire A, B;
  reg  Clock, reset_b;

  Problem_5_36 M0 (A, B, Clock, reset_b);

  initial #350$finish;
  initial begin Clock = 0; forever #5 Clock = ~Clock; end
  initial fork
    #2 reset_b = 1;
    #3 reset_b = 0;
    #4 reset_b = 1;
  join
endmodule
```

5.37 Verilog

```
module Prob_5_37_Fig_5_25 (output reg y, input x_in, clock, reset_b);

  parameter a = 3'b000, b = 3'b001, c = 3'b010, d = 3'b011, e = 3'b100, f = 3'b101, g = 3'b110;
  reg [2: 0] state, next_state;

  always @ (posedge clock, negedge reset_b)
    if (reset_b == 0) state <= a;
    else state <= next_state;

  always @ (state, x_in) begin
    y = 0;
    next_state = a;
    case (state)
      a:     begin y = 0; if (x_in == 0) next_state = a; else next_state = b; end

      b:     begin y = 0; if (x_in == 0) next_state = c; else next_state = d; end

      c:     begin y = 0; if (x_in == 0) next_state = a; else next_state = d; end

      d:     if (x_in == 0) begin y = 0; next_state = e; end
             else begin y = 1; next_state = f; end

      e:     if (x_in == 0) begin y = 0;  next_state = a; end
             else begin y = 1; next_state = f; end

      f:     if (x_in == 0) begin y = 0;  next_state = g; end
             else begin y = 1; next_state = f; end

      g:     if (x_in == 0) begin y = 0; next_state = a; end
             else begin y = 1; next_state = f; end

      default:   next_state = a;
    endcase
  end
endmodule
```

```
module Prob_5_37_Fig_5_26 (output reg y, input x_in, clock, reset_b);
  parameter a = 3'b000, b = 3'b001, c = 3'b010, d = 3'b011, e = 3'b100;
  reg [2: 0] state, next_state;

  always @ (posedge clock, negedge reset_b)
    if (reset_b == 0) state <= a;
    else state <= next_state;
 always @ (state, x_in) begin
    y = 0;
    next_state = a;
    case (state)
      a:       begin y = 0; if (x_in == 0) next_state = a; else next_state = b; end

      b:       begin y = 0; if (x_in == 0) next_state = c; else next_state = d; end

      c:       begin y = 0; if (x_in == 0) next_state = a; else next_state = d; end

      d:       if (x_in == 0) begin y = 0; next_state = e; end
               else begin y = 1; next_state = d; end

      e:       if (x_in == 0) begin y = 0;  next_state = a; end
               else begin y = 1; next_state = d; end

      default:    next_state = a;
    endcase
  end
endmodule


module t_Problem_5_37 ();
  wire y_Fig_5_25, y_Fig_5_26;
  reg x_in, clock, reset_b;

  Problem_5_37_Fig_5_25 M0 (y_Fig_5_25, x_in, clock, reset_b);
  Problem_5_37_Fig_5_26 M1 (y_Fig_5_26, x_in, clock, reset_b);

  wire [2: 0] state_25 = M0.state;
  wire [2: 0] state_26 = M1.state;

  initial #350 $finish;
  initial begin clock = 0; forever #5 clock = ~clock; end
  initial fork
    x_in = 0;
    #2 reset_b = 1;
    #3 reset_b = 0;
    #4 reset_b = 1;
    #20 x_in = 1;
    #40 x_in = 0;    // abdea, abdea

    #60 x_in = 1;
    #100 x_in = 0;  // abdf....fga, abd ... dea

    #120 x_in = 1;
    #160 x_in = 0;
    #170 x_in = 1;
    #200 x_in = 0; // abdf....fgf...fga, abd ...ded...ea

    #220 x_in = 1;
    #240 x_in = 0;
    #250 x_in = 1;  // abdef...  // abded...
  join
endmodule
```



VHDL

```
 entity Prob_5_37_Fig_5_25_vhdl is
 port (y: out bit; xin, clock, reset_b: out bit);
end Prob_5_37_Fig_5_25_vhdl;

architecture Behavioral of Prob_5_37_Fig_5_25_vhdl is
  constant a: Bit_Vector (2 downto 0) := "000";
  constant b: Bit_Vector (2 downto 0) := "001";
  constant c: Bit_Vector  (2 downto 0) := "010";
  constant d: Bit_Vector (2 downto 0) := "011";
  constant e: Bit_Vector (2 downto 0) := "100";
  constant f:  Bit_Vector (2 downto 0) := "101";
  constant g: Bit_Vector (2 downto 0) := "110";
  signal state, next_state: Bit_Vector (2 downto 0);
begin

  process (clock, reset_b) begin
    if (reset_b = '0') then state <= a;
    elsif (clock'event and clock = '1') then state <= next_state;
    end if;
  end process;

  process (state, x_in) begin
    y <= '0';
    next_state <= a;
    case (state) is
      when a =>     y <= '0'; if (x_in = '0') then next_state <= a; else next_state <= b; end if;

      when b =>     y <= '0'; if (x_in = '0') then next_state <= c; else next_state <= d; end if;

      when c =>     y <= '0'; if (x_in = '0') then next_state <= a; else next_state <= d; end if;

      when d =>     if (x_in = '0') then y <= '0'; next_state <= e;
                          else y <= '1'; next_state <= f; end if;

      when e =>     if (x_in = '0') then y <= '0';  next_state <= a;
                          else y <= '1'; next_state <= f; end if;

      when f => if (x_in = '0') then y <= '0';  next_state <= g;
                          else y <= '1'; next_state <= f; end if;
```

```
      when g =>     if (x_in = '0') then y <= '0'; next_state <= a;
                         else y <= '1'; next_state <= f; end if;

      when others =>  next_state <= a;
    end case;
  end process;
end Behavioral;


entity Prob_5_37_Fig_5_26_vhdl is
   port (y: out bit;  x_in, clock, reset_b: in bit);
end Prob_5_37_Fig_5_26_vhdl;

architecture Behavioral of Prob_5_37_Fig_5_26_vhdl is
   constant a: Bit_Vector := "000";
   constant b: Bit_Vector :=  "001";
   constant c: Bit_Vector :=  "010";
   constant d: Bit_Vector :=  "011";
   constant e: Bit_Vector :=  "100";
   signal state, next_state: Bit_Vector (2 downto 0);
begin

   process (clock, reset_b) begin
      if reset_b = '0' then state <= a;
      elsif clock'event and clock = '1' then  state <= next_state; end if;
   end process;

   process (state, x_in) begin
      y <= '0';
      next_state <= a;
      case (state)  is
         when a =>     y <= '0'; if x_in = '0' then next_state <= a; else next_state <= b; end if;

         when b =>     y <= '0'; if x_in = '0') then next_state <= c; else next_state <= d; end if;

         when c =>     y <= '0'; if x_in = '0' then next_state <= a; else next_state <= d; end if;

         when d =>     if  x_in = '0' then y <= '0'; next_state <= e;
                       else y <= '1'; next_state <= d; end if;

         when e =>     if x_in = '0' then y <= '0';  next_state <= a;
                       else y <= '1'; next_state <= d; end if;

         when others =>  next_state <= a;
     end case;
   end process;
end Behavioral;
```

5.38 (a)

```
module Prob_5_38a (input x_in, clock, reset_b);

 parameter s0 = 2'b00, s1 = 2'b01, s2 = 2'b10, s3 = 2'b11;

 reg [1: 0] state, next_state;

 always @ (posedge clock, negedge reset_b)
```

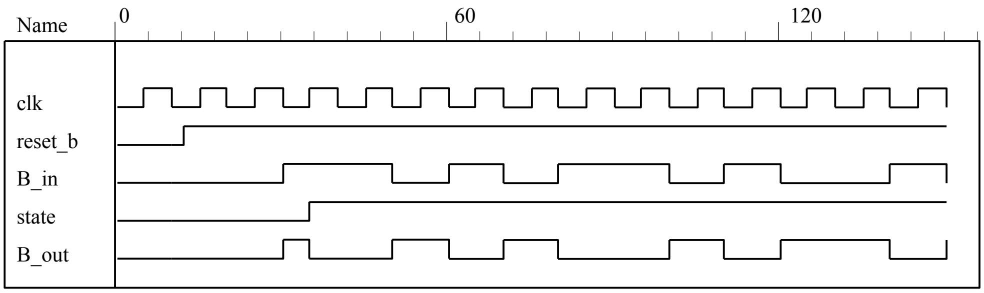
```
    if (reset_b == 0) state <= s0;
    else state <= next_state;
  always @ (state, x_in) begin
  next_state = s0;
  case (state)
    s0: if (x_in == 0) next_state = s0;
          else if (x_in == 1) next_state = s2;
    s1: if (x_in == 0) next_state = s1;
         else if (x_in == 1) next_state = s0;
    s2: if (x_in == 0) next_state = s2;
          else if (x_in == 1) next_state = s3;
    s3: if (x_in == 0) next_state = s3;
          else if (x_in == 1) next_state = s1;
    default: next_state = s0;
 endcase
end
endmodule
```

(b)

```
module Prob_5_38b (input x_in, clock, reset_b);
  parameter s0 = 2'b00, s1 = 2'b01, s2 = 2'b10, s3 = 2'b11;
  reg [1: 0] state, next_state;
  always @ (posedge clock, negedge reset_b)
    if (reset_b == 0) state <= s0;
    else state <= next_state;
  always @ (state, x_in) begin
    next_state = s0;
  case (state)
    s0: if (x_in == 0) next_state = s0;
          else if (x_in == 1) next_state = s3;
    s1: if (x_in == 0) next_state = s1;
         else if (x_in == 1) next_state = s0;
    s2: if (x_in == 0) next_state = s2;
         else if (x_in == 1) next_state = s1;
    s3: if (x_in == 0) next_state = s3;
         else if (x_in == 1) next_state = s2;
    default: next_state = s0;
  endcase
end
endmodule
```

5.39 Verilog

```
module Serial_2s_Comp (output reg B_out, input B_in, clk, reset_b);
// See problem 5.17
  parameter S_0 = 1'b0, S_1 = 1'b1;
  reg state, next_state;
  always @ (posedge clk, negedge reset_b) begin
    if (reset_b == 0) state <= S_0;
    else state <= next_state;
  end

  always @ (state, B_in) begin
    B_out = 0;
    case (state)
      S_0:    if (B_in == 0) begin next_state = S_0; B_out = 0; end
           else if (B_in == 1) begin next_state = S_1;  B_out = 1; end

      S_1:    begin next_state = S_1; B_out = ~B_in; end
     default:     next_state = S_0;

    endcase
  end
endmodule

module t_Serial_2s_Comp ();
  wire B_in, B_out;
  reg clk, reset_b;
  reg [15: 0] data;
  assign B_in = data[0];

  always @ (negedge clk, negedge reset_b)
    if (reset_b == 0) data <= 16'ha5ac; else data <= data >> 1;  // Sample bit stream

  Serial_2s_Comp M0 (B_out, B_in, clk, reset_b);

  initial #150 $finish;
  initial begin clk = 0; forever #5 clk = ~clk; end
  initial fork
    #10 reset_b = 0;
    #12 reset_b = 1;
  join
endmodule
```



VHDL

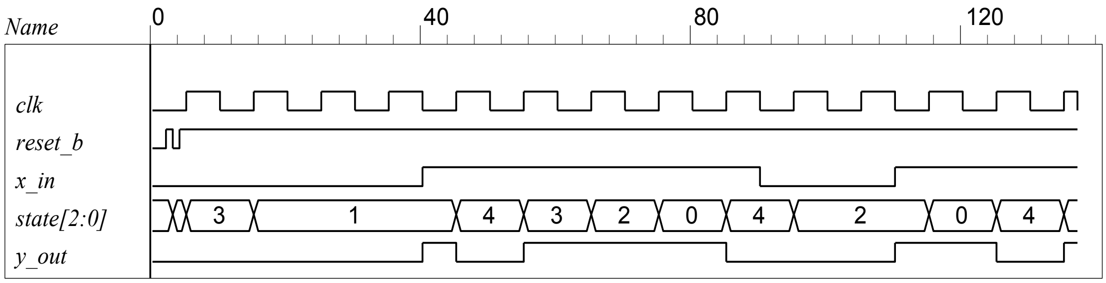
```
// See problem 5.17
entity Prob_5_39_Serial_2s_Comp_vhdl is
      port (B_out: out bit; B_in, clk, reset_b: in bit);
end Prob_5_39_Serial_2s_Comp_vhdl;

architecture Behavioral of Prob_5_39_Serial_2s_Comp_vhdl is
     constant S_0: bit := '0';
     constant S_1: bit := '1';
     signal  state, next_state: bit;
begin
  process (clk, reset_b) begin
    if reset_b = '0' then state <= S_0;
    elsif clock'event and clock = '1' then state <= next_state; end if;
  end process;

  process (state, B_in) begin
    B_out <= '0';
    case (state) is
          when S_0 => if  B_in = '0' then next_state <= S_0; B_out <= '0';
          elsif B_in = '1' then next_state <= S_1;  B_out <= '1'; end if;

          when S_1 =>  next_state <= S_1; B_out <= not B_in;

          when others =>  next_state <= S_0;
     end case;
  end process;
end Behavioral;
```

5.40

```
module Prob_5_40 (input E, F, clock, reset_b);
  parameter s0 = 2'b00, s1 = 2'b01, s2 = 2'b10, s3 = 2'b11;
  reg [1: 0] state, next_state;

  always @ (posedge clock, negedge reset_b)
    if (reset_b == 0) state <= s0;
    else state <= next_state;
  always @ (state, E, F) begin
    next_state = s0;
    case (state)
      s0: if (E == 0) next_state = s0;
           else if (F == 1) next_state = s3; else next_state = s1;
      s1: if (E == 0) next_state = s1;
           else if (F == 1) next_state = s0; else next_state = s2;
      s2: if (E == 0) next_state = s2;
            else if (F == 1) next_state = s1; else next_state = s3;
      s3: if (E == 0) next_state = s3;
           else if (F == 1) next_state = s2; else next_state = s0;
     default: next_state = s0;
 endcase
end
endmodule
```

5.41 Verilog

```
module Prob_5_41 (output reg y_out, input x_in, clock, reset_b);
  parameter s0 = 3'b000, s1 = 3'b001, s2 = 3'b010, s3 = 3'b011, s4 = 3'b100;
  reg [2: 0] state, next_state;

  always @ (posedge clock, negedge reset_b)
    if (reset_b == 0) state <= s0;
    else state <= next_state;

  always @ (state, x_in) begin
    y_out = 0;
    next_state = s0;
    case (state)
      s0:     if (x_in) begin next_state = s4; y_out = 1; end else begin next_state = s3; y_out = 0; end
      s1:     if (x_in) begin next_state = s4; y_out = 1; end else begin next_state = s1; y_out = 0; end
      s2:     if (x_in) begin next_state = s0; y_out = 1; end else begin next_state = s2; y_out = 0; end
      s3:     if (x_in) begin next_state = s2; y_out = 1; end else begin next_state = s1; y_out = 0; end
      s4:     if (x_in) begin next_state = s3; y_out = 0; end else begin next_state = s2; y_out = 0; end
      default:    next_state = 3'bxxx;
    endcase
  end
endmodule

module t_Prob_5_41 ();
  wire y_out;
  reg x_in, clk, reset_b;

  Prob_5_41 M0 (y_out, x_in, clk, reset_b);

  initial #350$finish;
  initial begin clk = 0; forever #5 clk = ~clk; end
  initial fork
    #2 reset_b = 1;
    #3 reset_b = 0;      // Initialize to s0
    #4 reset_b = 1;
                // Trace the state diagram and monitor y_out
     x_in = 0;         // Drive from s0 to s3 to  S1 and park
    #40 x_in = 1;         // Drive to s4 to s3 to s2 to s0 to s4 and loop
    #90 x_in = 0;         // Drive from s0 to s3 to s2 and part
    #110  x_in = 1;       // Drive s0 to s4  etc
   join
endmodule
```

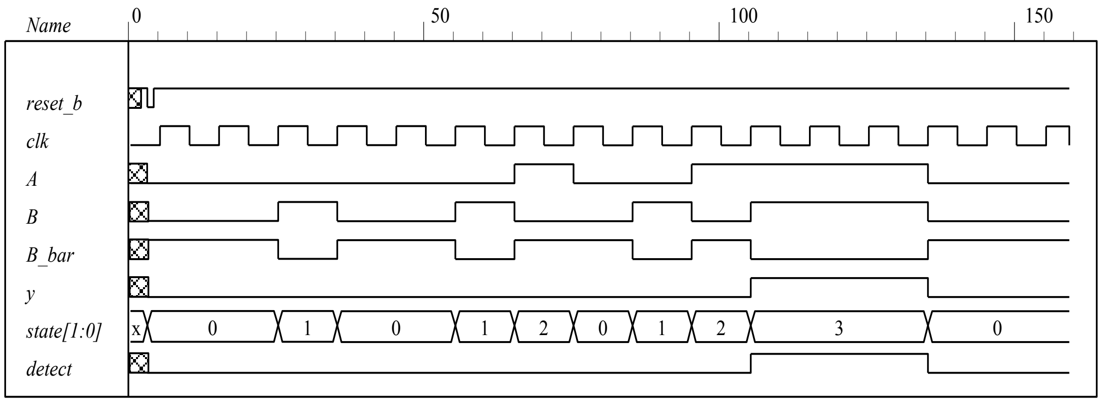


VHDL

```
library IEEE;
use IEEE_Std_logic_1164.all
entity Prob_5_41_vhdl is
    port  (y_out: out bit; x_in, clock, reset_b: in bit);
end Prob_5_41_vhdl;

architecture Behavioral of Prob_5_41_vhdl is
    constant s0: Std_Logic_vector (2 downto 0) := "000";
    constant s1: Std_Logic it_vector (2 downto 0) := "001";
    constant s2: Std_Logic _vector (2 downto 0) := "010";
    constant s3: Std_Logic _vector (2 downto 0) := "011";
    constant s4: Std_Logic _vector (2 downto 0) := "100";
    signal state, next_state: Std_Logic_vector (2 downto 0);
begin
    process (clock, reset_b) begin
        if reset_b = '0' then state <= s0;
        elsif clock'event and clock = '1' then state <= next_state; end if;
    end process;

    process (state, x_in) begin
        y_out <= '0';
        next_state <= s0;
        case (state) is
            when s0 =>    if x_in = '1' then next_state <= s4; y_out <= '1';
                          else next_state <= s3; y_out <= '0'; end if;

            when s1 =>    if x_in = '1' then next_state <= s4; y_out <= '1';
                          else next_state <= s1; y_out <= '0'; end if;

            when s2 =>    if x_in = '1' then next_state <= s0; y_out <= '1';
                          else next_state <= s2; y_out <= '0'; end if;

            when s3 =>    if x_in = '1' then next_state <= s2; y_out <= '1';
                          else next_state <= s1; y_out <= '0'; end if;

            when s4 =>    if x_in = '1' then next_state <= s3; y_out <= '0';
                          else next_state <= s2; y_out <= '0'; end if;

            when others =>  next_state <= "xxx";
        end case;
    end process;
end  Behavioral;
```

5.42 Verilog

```
module Prob_5_42 (output A, B, B_bar, y, input x, clk, reset_b);
// See Fig. 5.29
  wire w1, w2, w3, D1, D2;
  and (w1, A, x);
  and (w2, B, x);
  or (D_A, w1, w2);

  and (w3, B_bar, x);
  and (y, A, B);
  or (D_B, w1, w3);
  DFF M0_A (A, D_A, clk, reset_b);
  DFF M0_B (B, D_B, clk, reset_b);
  not (B_bar, B);
endmodule

module DFF (output reg Q, input data, clk, reset_b);
  always @ (posedge clk, negedge reset_b)
  if (reset_b == 0) Q <= 0; else Q <= data;
endmodule

module t_Prob_5_42 ();
  wire A, B, B_bar, y;
  reg bit_in, clk, reset_b;
  wire [1:0] state;
  assign state = {A, B};
  wire detect = y;

  Prob_5_42 M0 (A, B, B_bar, y, bit_in, clk, reset_b);

  // Patterns from Problem 5.45.

  initial #350$finish;
  initial begin clk = 0; forever #5 clk = ~clk; end
  initial fork
    #2 reset_b = 1;
    #3 reset_b = 0;
    #4reset_b = 1;
                      // Trace the state diagram and monitor detect (assert in S3)
         bit_in = 0;  // Park in S0
    #20 bit_in = 1;   // Drive to S0
    #30 bit_in = 0;   // Drive to S1 and back to S0 (2 clocks)
    #50 bit_in = 1;
    #70 bit_in = 0;   // Drive to S2 and back to S0 (3 clocks)
    #80 bit_in = 1;
    #130 bit_in = 0;// Drive to S3, park, then  and back to S0
   join
endmodule
```



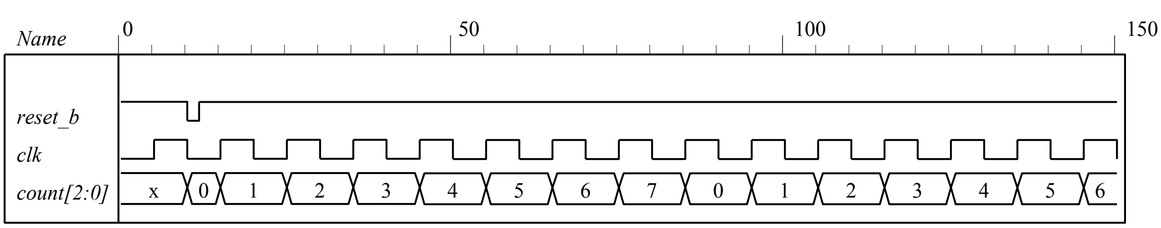
VHDL

```
entity Prob_5_42_vhdl is          -- See Fig. 5.29
   port (A, B: buffer bit; A_bar, B_bar: buffer bit; y: out bit; xin, clk, reset_b: in bit);
end Prob_5_42_vhdl;

architecture Structural of Prob_5_42_vhdl is
   signal w1, w2, w3, D_A, D_B: bit;
   component and2_gate port (y: out bit; xin1, xin2: in bit); end component;
   component or2_gate port (y: out bit; xin1, xin2: in bit); end component;
   component not_gate port (y: out bit; xin: in bit); end component;
   component D_FF port (Q: out bit; data, clk, reset_b: in bit); end component;

begin
  G1:   and2_gate     port map (y => w1, xin1 => A, xin2 => xin);
  G2:   and2_gate     port map (y => w2, xin1 => B, xin2 => xin);
  G3:   or2_gate      port map (y => D_A, xin1 => w1, xin2 => w2);
  G4:   and2_gate     port map (y => w3, xin1 => B_bar, xin2 => xin);
  G5:   and2_gate     port map (y => y, xin1 => A, xin2 => B);
  G6:   or2_gate      port map (y => D_B, xin1 => w1, xin2 =>w3);
  G7:   not_gate      port map (y => B_bar, xin => B);
  M0_A:    D_FF       port map (Q => A, data => D_A, clk => clk, reset_b => reset_b);
  M0_B:    D_FF       port map (Q => B, data => D_B, clk => clk, reset_b => reset_b);
end Structural;

entity and2_gate is
  port (y: out bit; xin1, xin2: in bit);
end and2_gate;

architecture Behavioral of and2_gate is
begin
   y <= xin1 and xin2;
end Behavioral;

entity or2_gate is
  port (y: out bit; xin1, xin2: in bit);
end or2_gate;

architecture Behavioral of or2_gate is
begin
   y <= xin1 or xin2;
end Behavioral;

entity not_gate is
  port (y: out bit; xin: in bit);
end not_gate;
```

```
architecture Behavioral of not_gate is
begin
    y <= not xin;
end Behavioral;

entity D_FF is
    port (Q: out bit; data, clk, reset_b: in bit);
end D_FF;

architecture Behavioral of D_FF is
    begin
        process  (clk, reset_b) begin
            if (reset_b = '0') then Q <= '0';
            elsif clk'event and clk = '1' then Q <= data;
            end if;
        end process;
end Behavioral;
```

5.43 Verilog

```
module Binary_Counter_3_bit (output [2: 0] count, input clk, reset_b)
  always @ (posedge clk) if (reset_b == 0) count <= 0; else count <= next_count;
  always @ (count) begin
    case (state)
      3'b000:     count = 3'b001;
      3'b001:     count = 3'b010;
      3'b010:     count = 3'b011;
      3'b011:     count = 3'b100;
      3'b100:     count = 3'b001;
      3'b101:     count = 3'b010;
      3'b110:     count = 3'b011;
      3'b111:     count = 3'b100;
     default:     count = 3'b000;
    endcase
  end
endmodule

module t_Binary_Counter_3_bit ()
  wire [2: 0] count;
  reg clk, reset_b;
  Binary_Counter_3_bit M0 ( count, clk, reset_b)

  initial #150 $finish;
  initial begin clk = 0; forever #5 clk = ~clk; end
  initial fork
    reset = 1;
    #10 reset = 0;
    #12 reset = 1;
endmodule
```

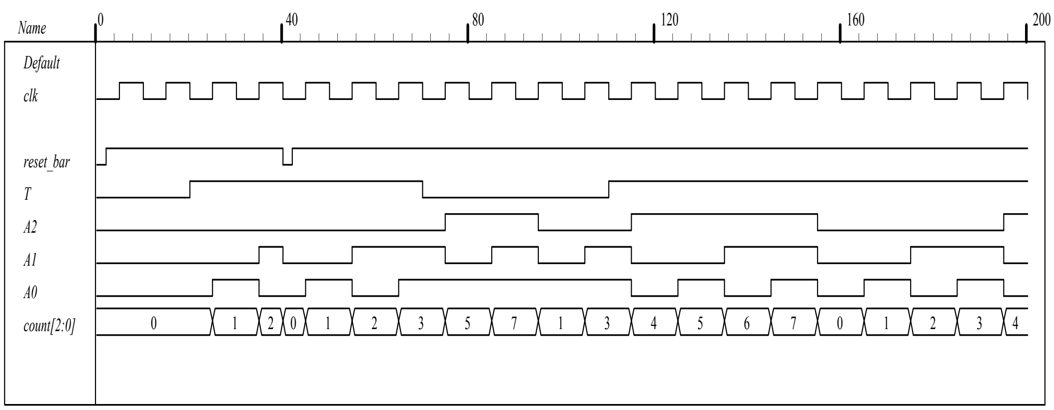


Alternative: structural model.

```
module Prob_5_41 (output A2, A1, A0, input T, clk, reset_bar);
```

```
  wire   toggle_A2;

  T_flop M0 (A0, T, clk, reset_bar);
  T_flop M1 (A1, A0, clk, reset_bar);
  T_flop M2 (A2, toggle_A2, clk, reset_bar);
  and (toggle_A2, A0, A1);
endmodule

module T_flop (output reg Q, input T, clk, reset_bar);
  always @ (posedge clk, negedge reset_bar)
    if (!reset_bar) Q <= 0; else if (T) Q <= ~Q; else Q <= Q;
endmodule

module t_Prob_5_41;
  wire A2, A1, A0;
  wire [2: 0] count = {A2, A1, A0};
  reg T, clk, reset_bar;
  Prob_5_41 M0 (A2, A1, A0, T, clk, reset_bar);

  initial #200 $finish;
  initial begin clk = 0; forever #5 clk = ~clk; end
  initial fork reset_bar = 0; #2 reset_bar = 1; #40 reset_bar = 0; #42 reset_bar = 1; join
  initial fork T = 0; #20 T = 1; #70 T = 0; #110 T = 1; join
endmodule
```

If the input to $A0$ is changed to 0 the counter counts incorrectly. It resumes a correct counting sequence when T is changed back to 1.



VHDL

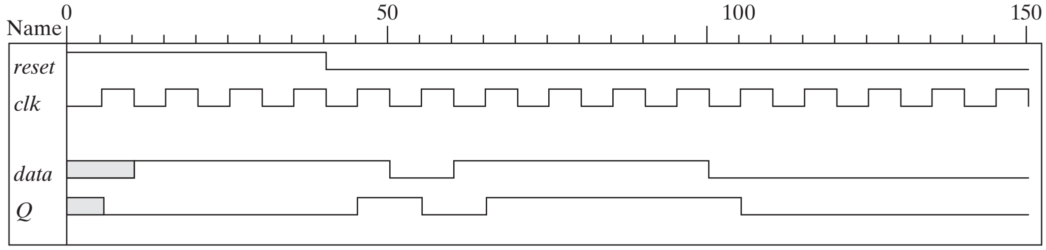
```
library IEEE;
use IEEE_Std_Logic_1164.all;

entity Prob_5_43_Binary_Counter_3_bit_vhdl is
    port (count: out bit_vector (2 downto 0; clk, reset_b: in bit);
end Prob_5_43_Binary_Counter_3_bit_vhdl;

architecture Behavioral of Prob_5_43_Binary_Counter_3_bit_vhdl is
begin
    process (clk) begin
        if reset_b = '0' then count <= "000";
        elsif clk'event and clk = '1' then  count <= next_count; end if;
    end process;
```

.*Digital Design With An Introduction to the Verilog HDL, VHDL, and SystemVerilog, Sixth Edition – Solution Manual*.

```
    process  (count) begin
        case (state)
            when "000" =>     count <= "001";
            when "001" =>     count <= "010";
            when "010" =>     count <= "011";
            when "011" =>     count <= "100";
            when "100" =>     count <= "001";
            when "101" =>     count <= "010";
            when "110" =>     count <= "011";
            when "111" =>     count <= "100";
            when others =>  count <= "000";
        end case;
    end process;
end Behavioral;
```

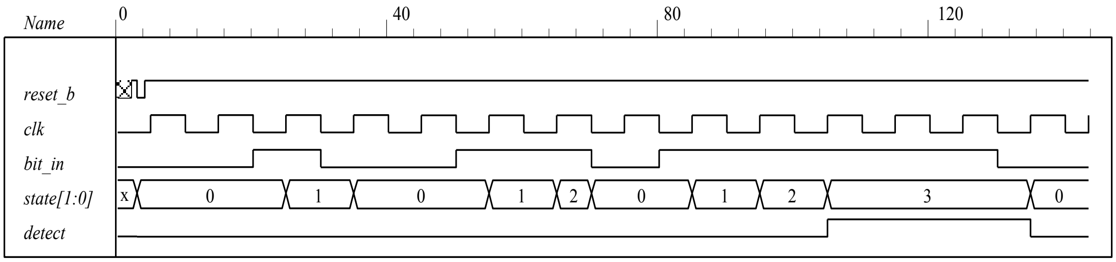
5.44

```
module DFF_synch_reset (output reg Q, input data, clk, reset);
 always @ (posedge clk)
 if (reset) Q <= 0; else Q <= data;
endmodule


module t_DFF_synch_reset ();
 reg data, clk, reset;
 wire Q;


DFF_synch_reset M0 (Q, data, clk, reset);
 initial #150 $finish;
 initial begin clk = 0; forever #5 clk = ~clk; end
 initial fork
 reset = 1;
 #20 reset = 1;
 #40 reset = 0;
 #10 data = 1;
 #50 data = 0;
 #60 data = 1;
 #100 data = 0;
 join
endmodule
```



5.45 Verilog

```
module Prob_5_45_Seq_Detector (output detect, input bit_in, clk, reset_b);
parameter S0 = 0, S1 = 1, S2 = 2, S3 = 3;
reg [1: 0] state, next_state;

assign detect = (state == S3);
always @ (posedge clk, negedge reset_b)
if (reset_b == 0) state <= S0; else state <= next_state;

  always @ (state, bit_in) begin
    next_state = S0;
    case (state)
      0:        if (bit_in) next_state = S1; else state = S0;
      1:        if (bit_in) next_state = S2; else next_state = S0;
      2:        if (bit_in) next_state = S3; else state = S0;
      3:        if (bit_in) next_state = S3; else next_state = S0;
      default:    next_state = S0;
    endcase
  end
endmodule

module t_ Prob_5_45_Seq_Detector ();
  wire detect;
  reg bit_in, clk, reset_b;

  Prob_5_45_Seq_Detector M0 (detect, bit_in, clk, reset_b);

  initial #350$finish;
  initial begin clk = 0; forever #5 clk = ~clk; end
  initial fork
    #2 reset_b = 1;
    #3 reset_b = 0;
    #4reset_b = 1;
                        // Trace the state diagram and monitor detect (assert in S3)
          bit_in = 0;   // Park in S0
    #20 bit_in = 1;     // Drive to S0
    #30 bit_in = 0;     // Drive to S1 and back to S0 (2 clocks)
    #50 bit_in = 1;
    #70 bit_in = 0;     // Drive to S2 and back to S0 (3 clocks)
    #80 bit_in = 1;
    #130 bit_in = 0;    // Drive to S3, park, then  and back to S0
   join
endmodule
```

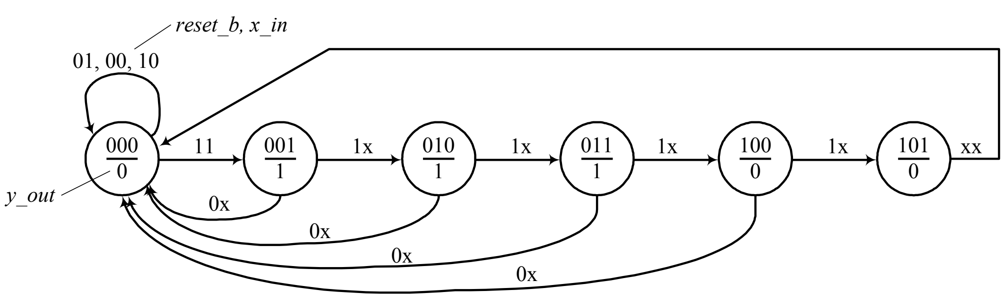


VHDL

```
library IEEE;
use IEEE_Std_Logic_1164.all;

entity Seq_Detector_vhdlis
    port (detect: out bit; bit_in, clk, reset_b: in bit);
end Prob_5_45_Seq_Detector_vhdl;

architecture Behavioral of Prob_5_45_Seq_Detector_vhdl is
    constant S0: Std_Logic_vector (1 downto 0) := "00";
    constant S1: Std_Logic_vector (1 downto 0) := "01";
    constant S2: Std_Logic_vector (1 downto 0) := "10";
    constant S3: Std_Logic_vector (1 downto 0) := "11";
    signal state, next_state: Std_Logic_vector (1 downto 0);
begin
    detect <= '1' when state = S3;

    process  (clk, reset_b) begin
        if reset_b = '0' then state <= S0;
        elsif clk'event and clk = '1' then state <= next_state; end if;
    end process;

    process (state, bit_in) begin
        next_state <= S0;
        case (state) is
            when S0 =>   if bit_in = '1' then next_state <= S1; else state <= S0; end if;
            when S1 =>   if bit_in = '1' then next_state <= S2; else next_state <= S0; end if;
            when S2 =>   if bit_in = '1' then nex t_state <= S3; else state <= S0; end if;
            when S3 =>   if bit_in = '1' then next_state <= S3; else next_state <= S0; end if;
            when others =>  next_state <= S0;
        end case;
    end process;
end Behavioral;
```

5.46 Pending simulation results

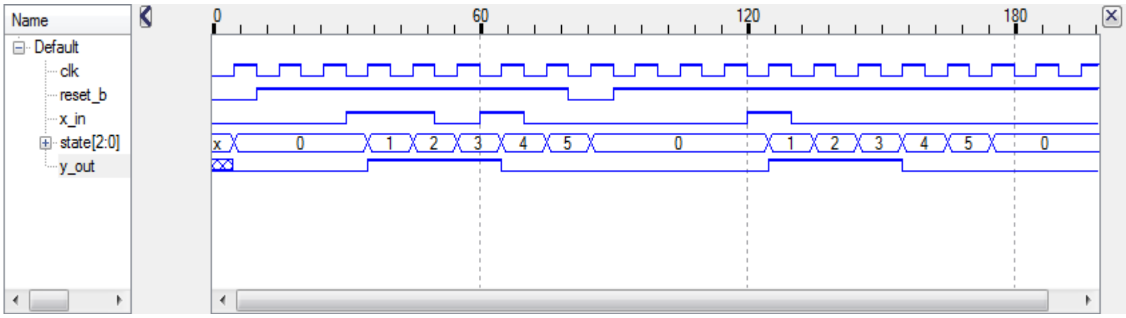
Assumption: Synchronous active-low reset
Moore machine



Verify that machine remains in state 000 while reset_b is asserted, independently of x_in.
Verify that machine makes transition from 000 to 001 if not reset_b and if x_in is asserted.
Verify that state transitions from 000 through 101 are correct.
Verify reset_b "on the fly."
Verify that y_out is asserted correctly.

Verilog

```
module Prob_5_46 (output y_out, input x_in, clk, reset_b);
  reg [2:0] state, next_state;

  assign y_out = (state == 3'b001)||(state == 3'b010) || (state == 3'b011);
  always @ (posedge clk)
    if (reset_b == 1'b0) state <= 3'b000; else state <= next_state;

  always @ (x_in, state) begin
    next_state = 3'b000;
    case (state)
      3'b000:    if (x_in) next_state = 3'b001; else next_state = 3'b000;
      3'b001:    next_state = 3'b010;
      3'b010:    next_state = 3'b011;
      3'b011:    next_state = 3'b100;
      3'b100:    next_state = 3'b101;
      3'b101:    next_state = 3'b000;
      default:   next_state = 3'b000;
    endcase
  end
endmodule


module t_Prob_5_46 ();
  reg x_in, clk, reset_b;
  wire y_out;

  Prob_5_46 M0 (y_out, x_in, clk, reset_b);

  initial #200 $finish;
  initial begin clk = 0; forever #5 clk = !clk; end
  initial fork
    reset_b = 0;
    #10 reset_b  = 1;
    #80 reset_b = 0;
    #90 reset_b = 1;

    x_in = 0;
    #30 x_in = 1;
    #40 x_in = 1;
    #50 x_in = 0;
    #60 x_in = 1;
    #70 x_in = 0;
    #120 x_in = 1;
    #130 x_in = 0;
  join
endmodule
```



<u>VHDL</u>

```
entity Prob_5_46 is
     port (output y_out, input x_in, clk, reset_b);
end Prob_5_46;

architecture Behavioral is
  signal state, next_state: bit_vector (2 downto 0);
begin
     y_out <= '1' when (state = "001") or (state = "010") or (state = "011");
     process  (clk) begin
          if reset_b = '0' then state <= "000";
          else if clk'event and clk = '1' then state <= next_state; end if;
     end process;

     process (x_in, state) begin
          next_state <= "000";
          case (state) is
               when 3'b000 =>  if x_in = '1' then next_state <= "001";
                               else next_state = 3"000"; end if;
               when "001" =>   next_state <= "010";
               when "010" =>   next_state <= "011";
               when "011" =>   next_state <= "100";
               when "100" =>   next_state <= "101";
               when "101" =>   next_state <= "000";
               when others  => next_state <= "000";
          end case;
     end process;
end Behavioral;
```

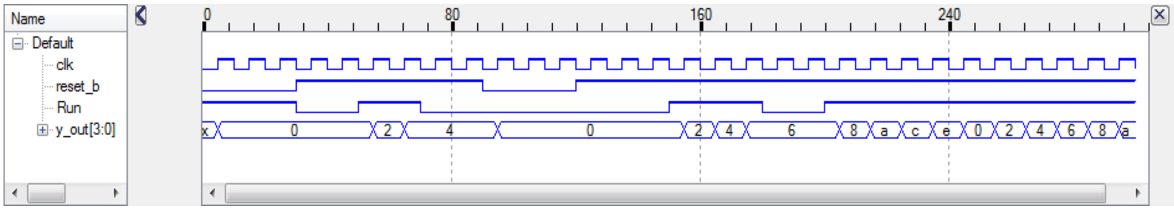
5.47

Assume synchronous active-low reset.

```
Verilog
module Prob_5_47 (output reg [3:0] y_out, input Run, clk, reset_b);
  always @ (posedge clk)
    if (reset_b == 1'b0) y_out <= 4'b0000;
    else if (Run && (y_out < 4'b1110)) y_out <= y_out + 2'b10;
    else if (Run && (y_out == 4'b1110)) y_out <= 4'b0000;
    else y_out <= y_out;         // redundant statement and may be omitted
endmodule
```

// Verify that counting is prevented while reset_b is asserted, independently of Run
// Verify that counting is initiated by Run if reset_b is de-asserted
// Verify reset on-the-fly
// Verify that deasserting Run suspends counting
// Verify wrap-around of counter.

```
module t_Prob_5_47 ();
  reg Run, clk, reset_b;
  wire [3:0] y_out;

  Prob_5_47 M0 (y_out, Run, clk, reset_b);

  initial #300 $finish;
  initial begin clk = 0; forever #5 clk = !clk; end
  initial fork
    reset_b = 0;
    #30 reset_b = 1;

    Run = 1;              // Attempt to run is overridden by reset_b
    #30 Run = 0;
    #50 Run = 1;          // Initiate counting
    #70 Run = 0;          // Pause
    #90 reset_b = 0;      // reset on-the-fly
    #120 reset_b = 1;  // De-assert reset_b
    #150 Run = 1;         // Resume counting
    #180 Run = 0;         // Pause counting
    #200 Run = 1;         // Resume counting
  join
endmodule
```



```
VHDL
entity Prob_5_47_vhdl is
    port  (y_out: out bit_vector (3 downto 0); input Run, clk, reset_b: in bit);
end Prob_5_47_vhdl;

architecture Behavioral of Prob_5_47_vhdl is
begin
  process (clk)
    if reset_b = '0'      then y_out <= "0000";
    elsif clk'event and clk = '1' then
        if (Run = '1' and (y_out < "1110")) then y_out <= y_out + "10";
        elsif (Run '1' and (y_out = "1110")) then y_out <= "0000";
        else y_out <= y_out;       // redundant statement and may be omitted
        end if;
end Behavioral;

// Verify that counting is prevented while reset_b is asserted, independently of Run
// Verify that counting is initiated by Run if reset_b is de-asserted
// Verify reset on-the-fly
// Verify that deasserting Run suspends counting
// Verify wrap-around of counter.


entity t_Prob_5_47_vhdl is
    port ();
end t_Prob_5_47_vhdl;

architecture Behavioral of t_Prob_5_47_vhdl is
    signal t_Run, t_clk, t_reset_b: bit;
    signal y_out: bit_vector (3 downto 0);
    component  Prob_5_47 M0
        port map (y_out => t_y_out, Run => t_Run, clk => _clk, reset_b => t_reset_b);
    end component;

    begin
    G0: Prob_5_47_vhdl
            port map (y_out => t_y_out, Run => t_Run, clk => t_clk, reset_b => t_reset_b);

    process – clock for testbench
    begin
        t_clk <= '0';
         wait for 5 ns;
         t_clk <= '1';
         wait for 5 ns;
    end process;

    t_reset_b <= '0';                   -- concurrent signal assignments
    t_reset_b <= '1' afer 30 ns;

    t_Run <= '1' after 30 ns;           // Attempt to run is overridden by reset_b
    t_Run <= '0' after 30 ns;;
    t_Run <= '1'; after 50 ns;          // Initiate counting
    t_Run <= '0' after 70 ns;           // Pause
    t_reset_b <= '0' after 90 ns;       // reset on-the-fly
    t_reset_b <= '1' after 120 ns;      // De-assert reset_b
    t_Run <= '1' after 150 ns;          // Resume counting
    t_Run <= '0' after 180 ns;          // Pause counting
    t_Run <= '1' after 200 ns;          // Resume counting

end Behavioral;
```

5.48

Assume "a" is the reset state.

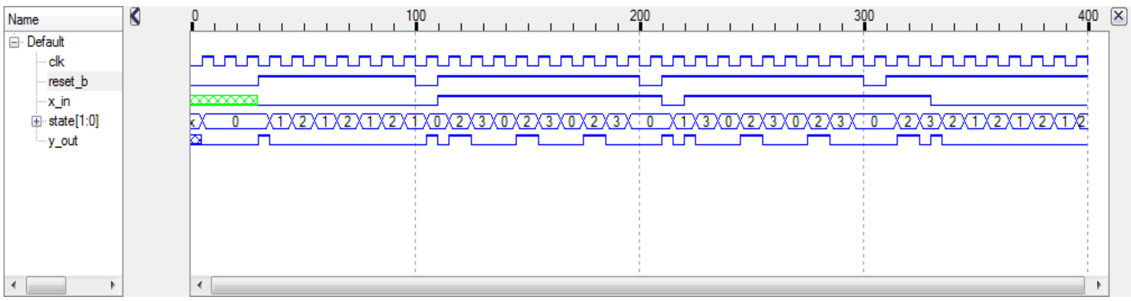
Verilog

```
module Prob_5_48 (output reg y_out, input x_in, clk, reset_b);
  parameter s_a = 2'd0;
  parameter s_b = 2'd1;
  parameter s_c = 2'd2;
  parameter s_d = 2'd3;
  reg [1: 0] state, next_state;

  always @ (posedge clk)
    if (reset_b == 1'b0) state <= s_a;
    else state <= next_state;

  always @ (state, x_in) begin
    next_state = s_a;
    y_out = 0;
    case (state)
      s_a:   if (x_in == 1'b0) begin next_state = s_b; y_out = 1; end
             else begin next_state = s_c; y_out = 0; end
      s_b:   if (x_in == 1'b0) begin next_state = s_c; y_out = 0; end
             else begin next_state = s_d; y_out = 1; end
      s_c:   if (x_in == 1'b0) begin next_state = s_b; y_out = 0; end
             else begin next_state = s_d; y_out = 1; end
      s_d:   if (x_in == 1'b0) begin next_state = s_c; y_out = 1; end
             else begin next_state = s_a; y_out = 0; end
      default:   begin next_state = s_a; y_out = 0; end
    endcase
  end
endmodule
```

Verify reset action.
Verify state transitions.
  Transition to a; hold x_in = 0 and get loop bc…
  Transition to a; hold x_in = 1 and get loop acda…
  Transitons to b; hold x_in = 1 and get loop bdacd…
  Transition to d; hold x_in = 0 and get loop dcbc…
Confirm Mealy outputs at each state/input pair
Verify reset on-the-fly.

```
module t_Prob_5_48 ();
  reg x_in, clk, reset_b;
  wire y_out;

  Prob_5_48 M0 (y_out, x_in, clk, reset_b);

  initial #400 $finish;
  initial begin clk = 0; forever #5 clk = !clk; end
  initial fork
    reset_b = 0;
    #30 reset_b = 1;
    #30 x_in = 0;              // loop abcbcbc…

    #100 reset_b = 0;
    #110 reset_b = 1;
    #110 x_in = 1;             // loop acdacda…

    #200 reset_b = 0;
```

```
    #210 reset_b = 1;
    #210 x_in = 0;
    #220 x_in = 1;          // loop bdacdacd…

    #300 reset_b = 0;
    #310 reset_b = 1;
    #310 x_in = 1;
    #330 x_in = 0;          // loop acdcbcbc….
join
endmodule
```



5.49

Assume "a" is the reset state.

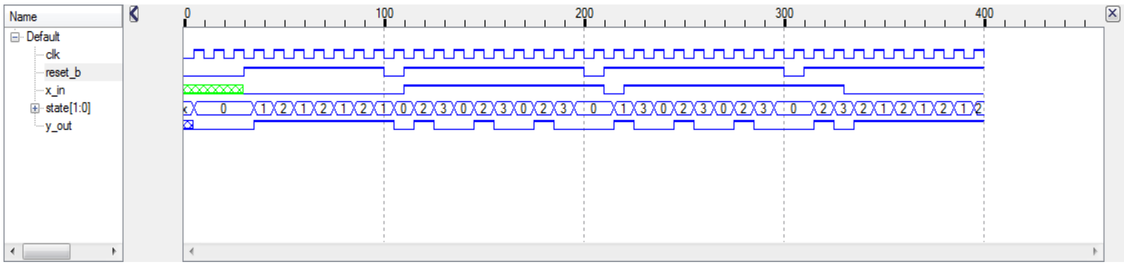
Verilog

```
module Prob_5_49 (output reg y_out, input x_in, clk, reset_b);
  parameter s_a = 2'd0;
  parameter s_b = 2'd1;
  parameter s_c = 2'd2;
  parameter s_ =d 2'd3;
  reg [1: 0] state, next_state;

  always @ (posedge clk)
    if (reset_b == 1'b0) state <= s_a;
    else state <= next_state;

  always @ (state, x_in) begin
    next_state = s_a;
    y_out = 1'b0;
    case (state)
      s_a:    if (x_in == 1'b0) next_state = s_b;
              else next_state = s_c;
      s_b:    begin y_out = 1'b1; if (x_in == 1'b0) next_state = s_c;
              else  next_state = s_d; end
      s_c:    begin y_out = 1'b1; if (x_in == 1'b0) next_state = s_b;
              else next_state = s_d; end
      s_d:    if (x_in == 1'b0) next_state = s_c;
              else next_state = s_a;
      default:    next_state = s_a;
    endcase
  end
endmodule
```

// Verify reset action.
// Verify state transitions.
// Transition to a; hold x_in = 0 and get loop abcbc…

// Transition to a; hold x_in = 1 and get loop acda…
// Transitons to b; hold x_in = 1 and get loop bdacd…
// Transition to d; hold x_in = 0 and get loop dcbc…
// Confirm Moore outputs at each state
// Verify reset on-the-fly.

```
module t_Prob_5_49 ();
 reg x_in, Run, clk, reset_b;
 wire y_out;

 Prob_5_49 M0 (y_out, x_in, clk, reset_b);

 initial #400 $finish;
 initial begin clk = 0; forever #5 clk = !clk; end
 initial fork
  reset_b = 0;
  #30 reset_b = 1;
  #30 x_in = 0;          // loop abcbcbc…

  #100 reset_b = 0;
  #110 reset_b = 1;
  #110 x_in = 1;         // loop acdacda…

  #200 reset_b = 0;
  #210 reset_b = 1;
  #210 x_in = 0;
  #220 x_in = 1;         // loop bdacdacd…

  #300 reset_b = 0;
  #310 reset_b = 1;
  #310 x_in = 1;
  #330 x_in = 0;         // loop acdcbcbc….
join
endmodule
```



VHDL

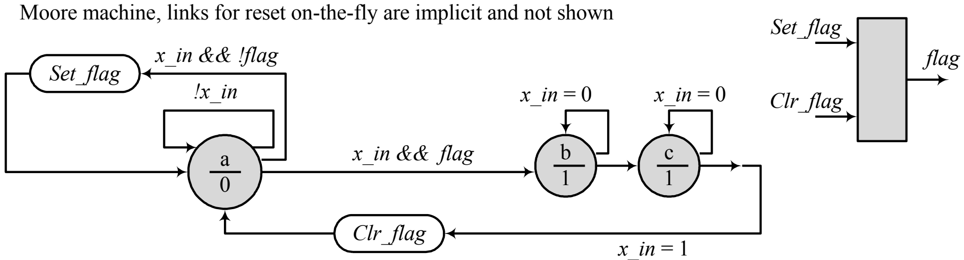
```
entity Prob_5_49_vhdl is
    port (y_out: out bit; x_in, clk, reset_b: in bit);
end Prob_5_49_vhdl;

architecture Behavioral of Prob_5_49_vhdl is
    constant s_a: bit_vector (1 downto 0)  := "00";
    constant s_b: bit_vector (1 downto 0)  := "01";
    constant s_c: bit_vector (1 downto 0)  := "10";
    constant s_d: bit_vector (1 downto 0)  := "11";
    signal state, next_state: bit_vector (1 downto 0);
begin
    process (clk) begin
        if (reset_b = '0') then state <= s_a;
```

```
        elsif clk'event and clk = '1' then  state <= next_state; end if;
    end process;

    process  (state, x_in) begin
        next_state <= s_a;
        y_out <= '0';
        case (state) is
            when s_a =>  if x_in = '0' then next_state <= s_b;
                         else next_state <= s_c; end if;

            when s_b =>  y_out <= '1'; if x_in = '0' then next_state <= s_c;
                         else  next_state <= s_d; end if;

            when s_c =>  y_out <= '1'; if x_in = '0' then next_state <= s_b;
                         else next_state <= s_d; end if;

            when s_d =>  if x_in = '0' then next_state <= s_c;
                         else next_state <= s_a;  end if;

            when others =>  next_state <= s_a;
        end case;
    end process;
end Behavioral;
```

5.50

The machine is to remain in its initial state until a second sample of the input is detected to be 1. A flag will be set when the first sample is obtained. This will enable the machine to detect the presence of the second sample while being in the initial state. The machine is to assert its output upon detection of the second sample and to continue asserting the output until the fourth sample is detected.

Assumption: Synchronous active-low reset
Moore machine, links for reset on-the-fly are implicit and not shown

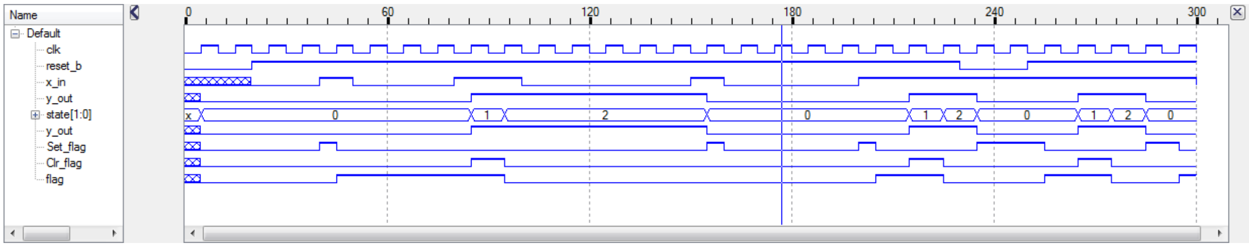


Note: the output signal *y_out* is a Moore-type output. The control signals *Set_flag* and *Clr_flag* are not.

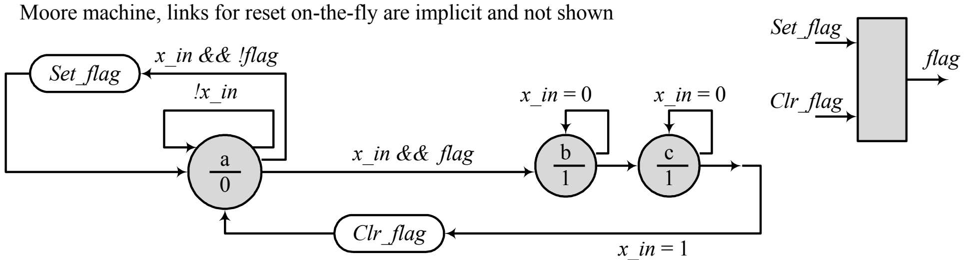
Verilog

```
module Prob_5_50 (output y_out, input x_in, clk, reset_b);
  parameter s_a = 2'd0;
  parameter s_b = 2'd1;
  parameter s_c = 2'd2;

  reg Set_flag;
  reg Clr_flag;
  reg [1:0] state, next_state;
  assign y_out = (state == s_b) || (state == s_c) ;
  always @ (posedge clk)
    if (reset_b == 1'b0) state <= s_a;
    else state <= next_state;
```

```
  always @ (state, x_in, flag) begin
    next_state = s_a;
    Set_flag = 0;     // Assign by exception
    Clr_flag = 0;
    case (state)
      s_a:    if ((x_in == 1'b1) && (flag == 1'b0))
              begin next_state = s_a;  Set_flag = 1; end
              else if ((x_in == 1'b1) && (flag == 1'b1))
              begin next_state = s_b; Set_flag = 0; end
              else if (x_in == 1'b0) next_state = s_a;
      s_b:    if (x_in == 1'b0) next_state = s_b;
              else begin next_state = s_c; Clr_flag = 1; end
      s_c:    if (x_in == 1'b0) next_state = s_c;
              else next_state = s_a;
      default:    begin next_state = s_a; Clr_flag = 1'b1; end
endcase
  end

  always @ (posedge clk)
    if (reset_b == 1'b0) flag <= 0;
    else if (Set_flag) flag <= 1'b1;
    else if (Clr_flag) flag <= 1'b0;
endmodule

// Verify reset action
// Verify detection of first input
// Verify wait for second input
// Verify transition at detection of second input
// Verify with between detection of input
// Verify transition to s_d at fourth detection of input
// Verify return to s_a and clearing of flag after fourth input
// Verify reset on-the-fly

module t_Prob_5_50 ();
  wire y_out;
  reg x_in, clk, reset_b;

  Prob_5_50 M0 (y_out, x_in, clk, reset_b);

  initial #500 $finish;
  initial begin clk = 0; forever #5 clk = !clk; end
  initial fork
    reset_b = 1'b0;
    #20 reset_b = 1;
    #20 x_in = 1'b0;
    #40 x_in = 1'b1;
    #50 x_in = 1'b0;
    #80 x_in = 1'b1;
    #100 x_in = 0;
    #150 x_in = 1'b1;
    #160 x_in = 1'b0;
    #200 x_in = 1'b1;
    #230 reset_b = 1'b0;
    #250 reset_b = 1'b1;
    #300 x_in = 1'b0;
  join
endmodule
```



Assumption: Synchronous active-low reset
Moore machine, links for reset on-the-fly are implicit and not shown

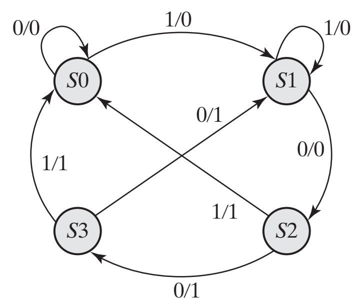


VHDL

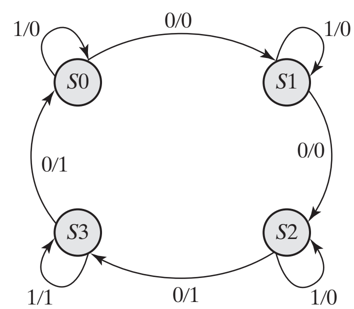
```
entity Prob_5_50_vhdl is
    port  (y_out: out bit; x_in, clk, reset_b: in bit);
    end Prob_5_50_vhdl;


architecture Behavioral of Prob_5_50_vhdl is
    constant s_a: bit_vector  := "00";
    constant s_b: bit_vector  := "01";
    constant s_b: bit_vector  := "10";
    signal Set_flag: bit;
    signal Clr_flag: bit;
    signal state, next_state: bit_vector (1 downto 0);
begin
    y_out <= '1' when state = s_b or state = s_c;

    process (clk) begin
        if (reset_b = '0') then state <= s_a;
        else state <= next_state; end if;
    end process;

    process (state, x_in, flag) begin
        next_state <= s_a;
        Set_flag <= 0;
        Clr_flag <= 0;
        case (state) is
            when s_a =>  if x_in = '1' and flag = '0'
                         then next_state <= s_a;  Set_flag <= '1';
                         elsif x_in = '1' and flag = '1'
                         then next_state <= s_b; Set_flag <= 0;
                         elsif x_in = '0' then next_state <= s_a;
                         end if;

            when s_b =>  if x_in = '0' then next_state <= s_b;
                         else next_state <= s_c; Clr_flag <= '1';
                         end if;

            when s_c =>  if x_in = '0' then next_state <= s_c;
                         else Clr_flag <= '1'; next_state <= s_a; end if;
        end case;
```

```
        end process;

      process (clk) begin
        if reset_b = '0'  then flag <= '0';
        elsif Set_flag = '1' then flag <= '1';
        elsif Clr_flag='1' then  flag <= '0';
        end if;
      end process;
    end Behavioral;
```

5.51

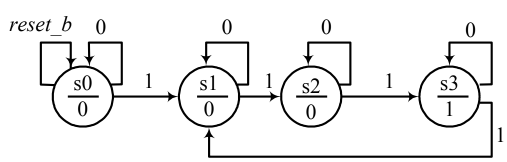


5.52



5.53

Assumption: Synchronous active-low reset
Moore machine, links for reset on-the-fly are implicit and not shown

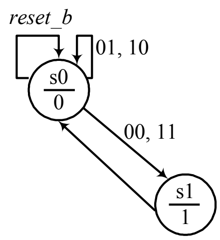


Verilog

```
module Prob_5_53 (output reg y, input clk, reset_b, x_in);
parameter s0 = 2'd0;
parameter s1 = 2'd1;
parameter s2 = 2'd2;
parameter s3 = 2;d3;
reg [1:0] state, next_state;

always @ (posedge clk, negedge reset_b)
    if (reset_b == 1'b0) then state <= s0;
    else state <= next_state;

always @ (state, xin) begin
    y = 0;
    next_state = s0;
    case (state)
        s0:     begin y = 1'b0; if (xin) next_state = s1; else next_state = s0; end
        s1:     begin y = 1'b0; if (xin) next_state = s2; else next_state = s1; end
        s2:     begin y = 1'b0; if (xin) next_state = s3; else next_state = s2; end
        s3:     begin y = 1'b1; if (xin) next_state = s1; else next_state = s3; end
```

```
        default:    begin y = 1'b0; next_state = s0; end
      endcase
endmodule
```

5.54



Verilog

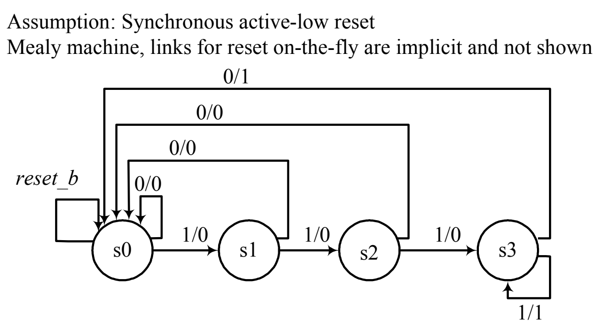
```
module Prob_5_54 (input xin1, xin2, clk, reset_b, output reg y_out);
   parameter s0 = 1'b0;
   parameter s1 = 1'b1;
   reg [1: 0] state, next_state;

   always @ (posedge clk, negedge reset_b)     // state transitions
      if (reset_b == 1'b0) state <= s0;
      else state <= next_state;

   always @ (state, xin1, xin2) begin       // next state
      next_state = s0; ;
      case (state)
         s0: if (xin1 == xin2) next_state = s1; else next_state = s0;
         s1: next_state = s0;
         default:    next_state = s0;
      endcase
   end

always @ (state, xin1, xin2) begin       //output
      y_out = 1'b0;
      case (state)
         s0: y_out = 1'b0;
         s1: y_out = 1'b1;
         default:    y_out = 1'b0;
      endcase
   end
endmodule
```

VHDL

```
entity Prob_5_54_vhdl is
   port (xin1, xin2, clk, reset_b: in bit; y_out: out bit);
end Prob_5_54_vhdl;

architecture Behavioral of Prob_5_54_vhdl is
   constant s0: bit  := '0';
   constant s1: bit := '1';
   signal state, next_state: bit_vector (1 downto 0);
begin
   process (clk, reset_b) begin      // state transitions
      if (reset_b = '0') then state <= s0;
      elsif clk'event and clk = '1' then state <= next_state; end if;
   end process;
```

```
    process (state, xin1, xin2) begin          // next state
        next_state <= s0; ;
        case (state) is
            s0: if (xin1 = xin2) then next_state <= s1; else next_state <= s0;
            s1: next_state <= s0;
            default:    next_state <= s0;
        end case;
    end process;

process (state, xin1, xin2) begin          //output
        y_out <= 1'b0;
        case (state)
            when s0:   y_out <= '0';
            when s1:   y_out <= '0';
            when others: y_out <= '0';
        end case;
    end process;
end Behavioral;
```

5.55



Verilog

```
module Prob_5_55 (input x_in, clk, reset_b, output reg y);
    parameter s0 = 1'd0;
    parameter s1 = 1'd1;
    parameter s2 = 1'd2;
    parameter s3 = 1'd3;
    reg [1: 0] state, next-state;

    always @ (posedge clk, negedge reset_b)
        if (reset_b == 1'b0) state <= s0;
        else state <= next_state;

    always @ (state, x_in) begin
        y = 1'b0;
        next_state = s0;

        case (state)         // Mealy machine
        s0:     if (x_in) begin y = 1'b0; next_state = s1; end
                else begin y = 1'b0; next_state = s0; end
        s1:     if (x_in) begin y = 1'b0; next_state = s2; end
                else y = 1'b0; next_state = s0; end
        s2:     if (x_in) begin y = 1'b0; next_state = s3; end
                else begin y = 1'b0; next_state = s0; end
        s3:     if (x_in) begin y = 1'b1; next_state = s3; end
                else begin y = 1'b0; next_state = s0; end
        default:    begin y = 1'b0; next_state = s0; end
    endcase
```

```
    end
endmodule
```

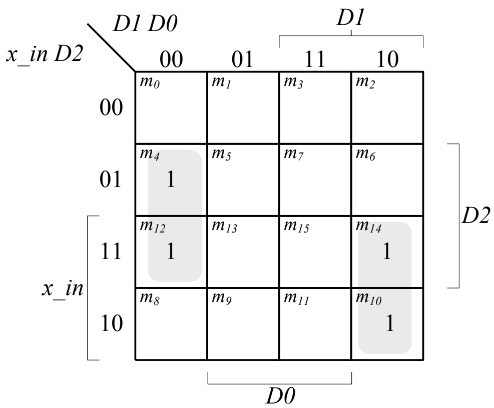
VHDL

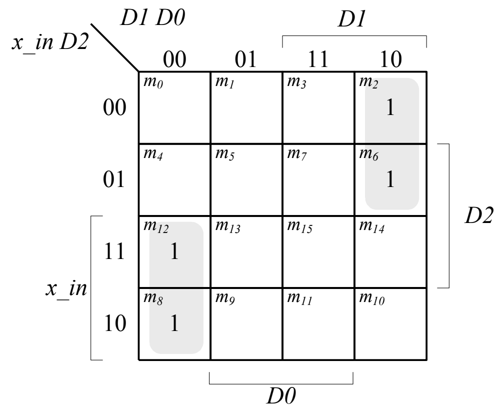
```
entity Prob_5_55_vhdl is
    port (x_in, clk, reset_b: out bit; y: out bit);
end Prob_5_55_vhdl;

architecture Behavioral of Prob_5_55_vhdl is
    constant s0: bit_vector (1 downto 0)  := "00";
    constant s1: bit_vector (1 downto 0)  := "01";
    constant s2: bit_vector (1 downto 0)  := "10";
    constant s3: bit_vector (1 downto 0)  := "11";
    signal state, next_state: bit_vector (1 downto 0);
begin
    process (clk, reset_b)
        if (reset_b = '0') then state <= s0; end if;
        else if clk'event and clk = '1' then state <= next_state; end if;

    process ( clk, reset_b) begin
        y <= '0';
        next_state <= s0;

        case (state)
            when s0:        y <= 1'b0; if (x_in = '1') then next_state <= s1; else next_state <= s0; end if;

            when s1:        y <= 1'b0; if (x_in = '1') then next_state <= s2; else next_state <= s1; end if;

            when s2:        y <= 1'b0; if (x_in = '1') then next_state <= s3; else next_state <= s2; end if;

            when s3:        y <= 1'b1; if (x_in = '1') then next_state <= s1; else next_state <= s3; end if;

            when others: y <= 1'b0; next_state <= s0;
        endcase
    end process;
end Behavioral;
```

5.56

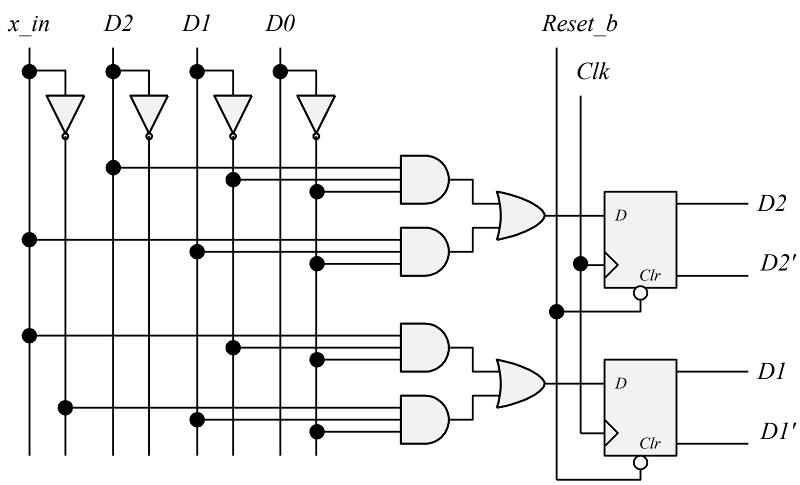
reset_b	x_in	D2	D1	D0	nD2	nD1	nD0
0	x	0	0	0	0	0	0
0	x	0	0	1	0	0	0
0	x	0	1	0	0	0	0
0	x	0	1	1	0	0	0
0	x	1	0	0	0	0	0
0	x	1	0	1	0	0	0
0	x	1	1	0	0	0	0
0	x	1	1	1	0	0	0
1	0	0	0	0	0	0	0
1	1	0	0	0	0	1	0
1	x	0	0	1	0	0	0
1	0	0	1	0	0	1	0
1	1	0	1	0	1	0	0
1	x	0	1	1	0	0	0
1	0	1	0	0	1	0	0
1	1	1	0	0	1	1	0
1	x	1	0	1	0	0	0
1	0	1	1	0	1	1	0
1	1	1	1	0	0	0	0
1	x	1	1	1	0	0	0

For reset_b = 1:
nD2 = (x_in D2'D1D0') || (x_in' D2 D1' D0') || (x_in D2 D1' D0') || (x_in D2 D1 D0')
nD1 = (x_in D2' D1' D0') || (x_in' D2' D1 D0') || (x_in D2 D1' D0') || (x_in' D2 D1 D0')





nD2 = D2 D1'D0' + x_in D1 D0' *nD1 = x_inD1' D0' + x_in' D1 D0'*



5.57

Assume synchronous active-low reset. Assume that the counter is controlled by assertion of *Run*.

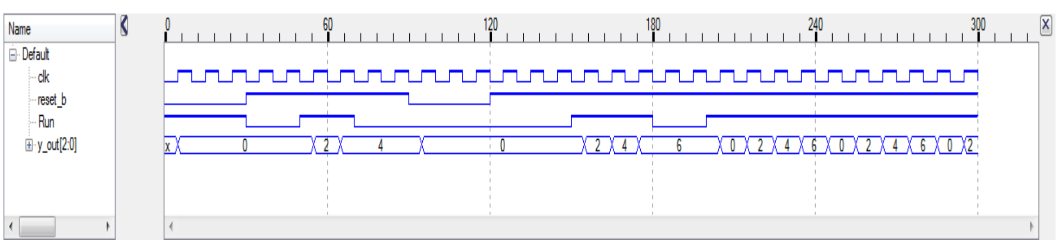
Verilog (Simple version below increments the count by 2, but fails if count is ever not 0, 2, 4, or 6)

```
module Prob_5_57 (output reg [2:0] y_out, input Run, clk, reset_b);
  always @ (posedge clk)
    if (reset_b == 1'b0) y_out <= 3'b000;
    else if (Run && (y_out < 3'b110)) y_out <= y_out + 3'b010;
    else if (Run && (y_out == 3'b110)) y_out <= 3'b000;
    else y_out <= y_out;         // redundant statement and may be omitted
endmodule
```

Safer alternative:

```
module Prob_5_57_safe (output reg [2:0] y_out, input Run, clk, reset_b);
    reg [2: 0] next_ y_out;

  always @ (posedge clk)
    if (reset_b == 1'b0) y_out <= 3'b000; else y_out <= next_ y_out;

    always @ (y_out, Run) begin
      y_out = 3'd0;   // assign by exception
      case (y_out)
```

```
          3'd0:   if (Run) next_ y_out = 3'd2;
          3'd1:   next_ y_out = 3'd0;
          3'd2:   if (Run) next_ y_out = 3'd4;
          3'd3:   next_ y_out = 3'd0;
          3'd4:   if (Run) next_ y_out = 3'd6;
          3'd5:   next_ y_out = 3'd0;
          3'd6:   if (Run) next_count = 3'd0;
          3'd7:   next_ y_out = 3'd0;  // Can be omitted – covered by default
          default: if (Run) next_ y_out = 3'd0;
        endcase;
    end;
endmodule

// Verify that counting is prevented while reset_b is asserted, independently of Run
// Verify that counting is initiated by Run if reset_b is de-asserted
// Verify reset on-the-fly
// Verify that deasserting Run suspends counting
// Verify wrap-around of counter.

module t_Prob_5_57 ();
  reg Run, clk, reset_b;
  wire [2:0] y_out;

  Prob_5_57 M0 (y_out, Run, clk, reset_b);

  initial #300 $finish;
  initial begin clk = 0; forever #5 clk = !clk; end
  initial fork
    reset_b = 0;
    #30 reset_b = 1;

    Run = 1;              // Attempt to run is overridden by reset_b
    #30 Run = 0;
    #50 Run = 1;          // Initiate counting
    #70 Run = 0;          // Pause
    #90 reset_b = 0;      // reset on-the-fly
    #120 reset_b = 1;  // De-assert reset_b
    #150 Run = 1;         // Resume counting
    #180 Run = 0;         // Pause counting
    #200 Run = 1;         // Resume counting
  join
endmodule
```



VHDL

```
entity Prob_5_57_vhdl is
    port (y_out: buffer bit_vector (2 downto 0); Run, clk, reset_b: in bit);
end Prob_5_57_vhdl;

architecture Behavioral of Prob_5_57_vhdl is
    signal next_y_out: bit_vector (2 downto 0);
```

```
begin
  process (clk) begin
    if  clk'event and clk = '1' and (reset_b = '0') then y_out <= "000";       -- Priority: synch reset
    elsif clk'event and clk = '1' and Run = '1' then y_out <= next_y_out; end if;
  end process;

  process (y_out) begin
    if  clk'event and clk = '1' and Run = '1 then
    next_ y_out = "000";  -- assign by exception
    case (y_out)
      when "000" =>     next_ y_out <= "010";      -- 2
      when "001" =>     next_ y_out <= "000";
      when "010" =>     next_ y_out <= "100";      -- 4
      when "011" =>     next_ y_out <= "000";
      when "100" =>     next_ y_out <= "110";      -- 6
      when "101" =>     next_ y_out <= "000";
      when "110" =>:    next_ y_out <= "000";
      when "111" =>     next_ y_out <= "000";-- Can be omitted – covered by default
      when others =>  next_ y_out <= "000";
    endcase;
  end process;
end Behavioral;
```

5.58 Verilog

```
module Prob_5_58 (output reg y_out, input x_in, clk, reset_b)
  parameter s0 = 2'b00;
  parameter s1 = 2'b01;
  parameter s2= 2'b10;
  parameter s3 = 2'b11;
  reg [1:0] state, next_state;

  always @ (posedge clk, negedge reset_b)
    if (reset_b == 1'b0) state <= s0;
    else state <= next_state;

  always @(state, x_in) begin
    y_out = 0;
    next_state = s0;
    case(state)
      s0: if (x_in == 1'b0) next_state = s0; else if (x_in == 1'b1) next_state = s1;
      s1: if (x_in == 1'b0) next_state = s0; else if (x_in == 1'b1) next_state = s2;
      s2: if (x_in == 1'b0) next_state = s0; else if (x_in == 1'b1) next_state = s3;
      s3:  begin y_out = 1; if (x_in == 1'b0) next_state = s0;
                                  else if (x_in == 1'b1) next_state = s3; end;
      default:    begin next_state = s0; y_out = 0; end
    endcase;
  end
endmodule

module t_Prob_5_58 ();
  wire y_out;
  reg x_in, clk, reset_b;

  Prob_5_58 M0 (y_out, x_in, clk, reset_b)

  initial begin clk = 0; forever #5 clk = !clk; end
  initial fork
    reset_b = 0;
    x_in = 0;
    #20 reset_b = 1;
    #40 reset_b = 1;
    #50 x_in = 1;
    #60 x_in = 0;
    #80 x_in = 1;
    #90 x_in = 0;
    #110 x_in = 1;
    #120 x_in = 1;
    #150 x_in = 0;
    #200 x_in = 1;
    #210 reset_b = 0;
    #240 reset_b = 1;
  join
endmodule
```

VHDL

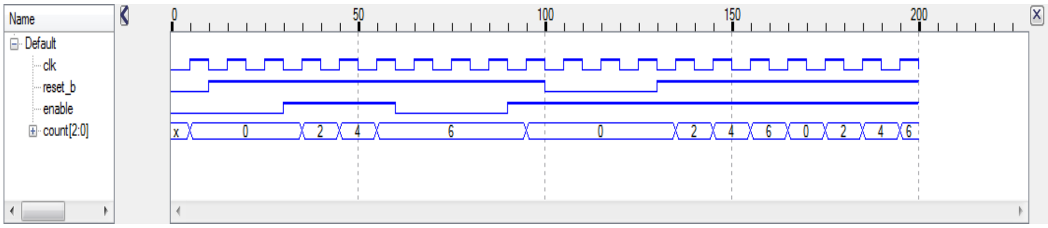
```
entity Prob_5_58_vhdl is
    port (y_out: out bit; x_in, clk, reset_b: in bit);
end Prob_5_58_vhdl;

architecture Behavioral of Prob_5_58_vhdl is
    constant s0: bit_vector (1 downto 0) := "00";
    constant s1: bit_vector (1 downto 0) := "01";
    constant s2: bit_vector (1 downto 0) := "10";
    constant s3: bit_vector (1 downto 0) := "11";
```

```
    signal state, next_state: bit_vector (1 downto 0);
begin

process (clk, reset_b) begin
    if (reset_b = '0') then state <= s0; -- Asynchronous, active-low reset
    elsif clk'event and clk = '1' then state <= next_state;
end process;

process (state, x_in) begin
   y_out <= '0';
   next_state <= s0;
   case(state) is
   when s0 =>     if (x_in = '0') then next_state <= s0;
                  elsif (x_in = '1') then next_state = s1; end if;

    when s1 =>    if (x_in = '0') then next_state <= s0;
         elsif (x_in = '1') then next_state = s2; end if;

    when s2 =>    if (x_in = '1') then next_state <= s0;
         elsif (x_in = '1') then next_state <= s3;

    when s3 =>     y_out <= 1; if (x_in = '0') then next_state <= s0;
                  elsif (x_in = '1') then next_state <= s3;

    when others =>  next_state <= s0; y_out <= 0;
   endcase
  end process;
end Behavioral;
```

5.59 Verilog

```
module Prob_5_59 (output reg [2: 0] count, input enable, clk, reset_b);
  always @ (posedge clk)
   if (reset_b == 1'b0) count <= 3'b000;
   else if (enable) case (count)
     3'b000:     count <= 3'b010;       // 2
     3'b010:     count <= 3'b100;       // 4
     3'b100:     count <= 3'b110;       // 6
     3'b110:     count <= 3'b000;
     default:    count <= 3'b111;       // Use for error detection
   endcase
endmodule

module t_Prob_5_59 ();
  wire [2:0] count;
  reg enable, clk, reset_b;

  Prob_5_59 M0 (count, enable, clk, reset_b);

  initial #200 $finish;
  initial begin clk = 0; forever #5 clk = ~clk; end
  initial fork
   reset_b  = 0;
   #10 reset_b = 1;
   #100 reset_b = 0;
   #130 reset_b = 1;
   enable = 0;
   #30 enable = 1;
   #60 enable = 0;
   #90 enable = 1;
  join
```

```
endmodule
```



VHDL

```
entity Prob_5_59_vhdl is
    port (count: out bit_vector (2 downto 0); enable, clk, reset_b: in bit);
end Prob_5_59_vhdl;

architecture Behavioral of Prob_5_59_vhdl is
begin
    process (clk)
        if clk'event and clk = '1' then if (reset_b = '0') then count <= "000";
        elsif (enable = '1') then
            count<= '0';
            case (count) is
                when "000" =>     count <= "010";
                when "000" =>     count <= "100";
                when "100" =>     count <= "'110";
                when "110" =>     count <= "000";
                when others =>  count <= "000";         // Use for error detection
            end case;
    end process;
end  Behavioral;
```

5.60

Assume synchronous active-low reset. Assume that counting is controlled by Run.

Verilog

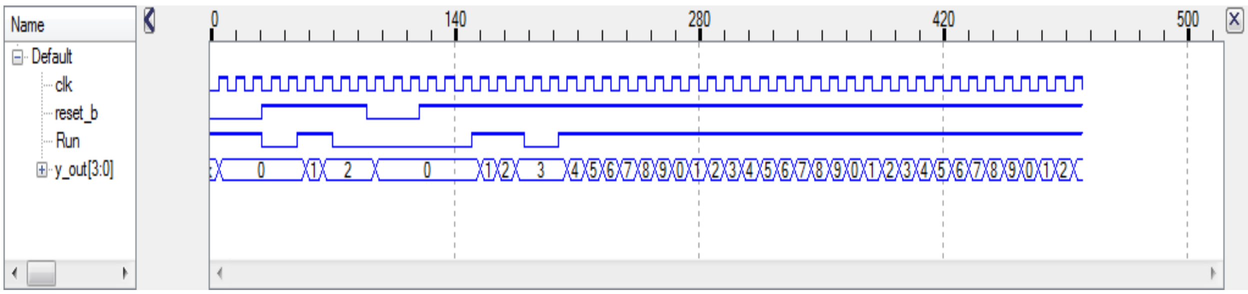
```
module Prob_5_60 (output reg [3:0] y_out, input Run, clk, reset_b);
  always @ (posedge clk)
   if (reset_b == 1'b0) y_out <= 4'b000;
   else if (Run && (y_out < 4'b1001)) y_out <= y_out + 4'b0001;
   else if (Run && (y_out == 4'b1001)) y_out <= 4'b0000;500
   else y_out <= y_out;         // redundant statement and may be omitted
endmodule
```

// Verify that counting is prevented while reset_b is asserted, independently of Run
// Verify that counting is initiated by Run if reset_b is de-asserted
// Verify reset on-the-fly
// Verify that deasserting Run suspends counting
// Verify wrap-around of counter.

```
module t_Prob_5_60 ();
  reg Run, clk, reset_b;
  wire [3:0] y_out;

  Prob_5_60 M0 (y_out, Run, clk, reset_b);
```



```
initial #500 $finish;
initial begin clk = 0; forever #5 clk = !clk; end
initial fork
  reset_b = 0;
  #30 reset_b = 1;

  Run = 1;              // Attempt to run is overridden by reset_b
  #30 Run = 0;
  #50 Run = 1;          // Initiate counting
  #70 Run = 0;          // Pause
  #90 reset_b = 0;      // reset on-the-fly
  #120 reset_b = 1;  // De-assert reset_b
  #150 Run = 1;         // Resume counting
  #180 Run = 0;         // Pause counting
  #200 Run = 1;         // Resume counting
 join
endmodule
```



VHDL

```
entity Prob_5_60_vhdl is
    port  (y_out: buffer bit_vector (3 downto 0); Run, clk, reset_b: in bit);
end Prob_5_60_vhdl;

architecture Behavioral of Prob_5_60_vhdl is
begin
process (clk) begin
    if clk'event and clk = '1' then
        if (reset_b = '0') then y_out <= "0000";
        elsif  (Run = '1')  then
            case (y_out) is
                when "0000" =>   y_out <= "0001";
                when "0001" =>   y_out <= "0010";
                when "0010" =>   y_out <= "0011";
                when "0011" =>   y_out <= "0100";
                when "0100" =>   y_out <= "0101";
                when "0101" =>   y_out <= "0110";
                when "0110" =>   y_out <= "0111";
                when "0111" =>   y_out <= "1000";
                when "1000" =>   y_out <= "1001";
                when "1001" =>   y_out <= "0000";
                when others =>   y_out <= "0001";
            end case;
    end if;
end process;
end Behavioral;
```